

POLARON

Language Reference

VERSION 1.0.179

João Victor Pereira Tavares

September 2026

Contents

- 1 Introduction & Philosophy
- 2 Program Structure & Modules
- 3 Expressions, Statements & Method Calls
- 4 Values & the Type System
- 5 Memory & Ownership
- 6 The Region Binder
- 7 Object-Oriented Programming
- 8 Control Flow
- 9 Errors, Results & Contracts
- 10 Concurrency
- 11 Compile-Time, Reflection & Prefixes
- 12 Systems Programming
- 13 Keyword Reference
- 14 Diagnostics
- 15 Commands, Method References & Tuples
- 16 The Toolchain & Projects
- 17 Testing
- 18 How the Pieces Fit Together
- 19 Building Real Things
- 20 Final Considerations
- 21 The Mark, the Colour and the Icon
- 22 Standard Library — Concurrency & Core
- 23 Standard Library — Collections

24 Standard Library — Data Structures & ECS

25 Standard Library — Text, Encoding & Crypto

26 Standard Library — Parsing, Time & JSON

27 Standard Library — Math, Net & Misc

28 Standard Library — Networking and TLS

29 Standard Library — The System

30 Standard Library — Reflection-Driven

31 Standard Library — Science and Units

1. Introduction & Philosophy

Welcome to the Polaron Language Reference. This book describes **Polaron** — *Linguagem De Programação 3* — as it exists today: a real compiler that turns source text into native machine code. Every example in these pages is written in the language the compiler actually accepts, and the features described are the features it actually implements. Where the design reaches further than the current implementation, the text says so plainly.

Polaron was created by **João Victor Pereira Tavares**. This reference tracks toolchain version **1.0.12**.

1.1 What Polaron is

Polaron is a **systems programming language** with three commitments that are rarely held together at the same time:

- It is **object-oriented by mandate**. There are no free functions and no loose statements at the top of a file. All behavior lives inside a type — a class, interface, record, struct, or enum — which in turn lives inside a namespace, inside a bundle, inside a program. If you come from Java or C#, this hierarchy will feel familiar; the difference is that Polaron takes it all the way down to the systems layer.
- It is **manually memory-managed, with no garbage collector**. You decide where each object lives — on the stack or on the heap — and when it is released. Destructors run deterministically, at a point in the program you can name and reason about, not whenever a collector decides to wake up.
- It **compiles to fast native code** through LLVM. There is no interpreter and no virtual machine between your program and the processor. Because Polaron shares the LLVM backend with Clang, code written in Polaron reaches the same machine instructions that equivalent C or C++ would, and runs at the same speed.

Put together, these commitments aim at a specific gap in the landscape: a language that reads with the clarity and structure of a managed, object-oriented language, but that gives you the control, predictability, and raw performance of a low-level one — without a garbage collector sitting in the hot path.

The reference implementation is written in C++20 and targets Windows x86-64 first, with a native Linux x86-64 port. It emits LLVM IR, which the toolchain hands to `clang / ld` to produce a real executable.

1.2 The design pillars

Polaron is verbose in places, and the verbosity is deliberate. The language's guiding conviction is that *explicit beats implicit* whenever the explicit form eliminates ambiguity, documents intent, or prevents a whole category of bugs. Everything else follows from a small set of pillars.

Pillar 1 — Object orientation is mandatory

There is no such thing as a top-level function in Polaron. You cannot drop a `main` on its own into a file. Behavior belongs to types, and types belong to namespaces. The rationale is uniformity: when *every* piece of logic has a home in the type system, tooling, encapsulation, visibility, and the module graph all work the same way everywhere, with no special cases for "the loose bits outside the classes." A codebase becomes a forest of well-labeled types instead of a mix of objects and free-floating helpers.

The one concession to pragmatism is data: a namespace may hold global *variables*, but never global *functions*. Logic is always a method on something.

Pillar 2 — Manual memory with value semantics

Polaron has no garbage collector. You allocate, and you free. An object is created with `new` and placed explicitly `on stack` or `on heap`; heap objects are released with `delete`. Stack objects are released automatically when their scope ends, and their destructor runs at exactly that point (RAII).

Crucially, **assignment is a deep copy**, even for class instances. Writing `b = a` gives `b` its own independent copy of `a`, not a shared reference to it. This is the opposite of the default in most object-oriented languages, and it is a deliberate choice: value semantics make ownership obvious and aliasing bugs rare. When you *do* want two names to refer to the same object, you say so explicitly with a pointer (`T*`) or a reference (`T&`).

```
import System.IO.Console;
program ValueCopy;

public bundle main {
    public namespace app {
        public class Point {
            public mutable int x;
            public constructor Point(int x) { this.x = x; }
        }

        public class Main {
            public static method main(string[] args) returns void {
                Point a = new Point(1) on stack;
                Point b = a;           // deep copy -- b is independent of a
                b.x = 99;
                Point* p = &a;         // a pointer shares the instance
                p.x = 7;
                System.IO.Console.println($"a={a.x} b={b.x}");    // a=7 b=99
                return;
            }
        }
    }
}
```

The assignment `b = a` copied the point, so changing `b.x` left `a` untouched. The pointer `p`, by contrast, aliased `a`, so `p.x = 7` changed `a` itself.

Pillar 3 — No exploitable undefined behavior

In C and C++, signed integer overflow, out-of-bounds indexing, and dereferencing a null pointer are *undefined behavior*: the compiler is free to assume they never happen, and the program is free to do anything at all when they do. This is the source of a large fraction of security vulnerabilities in systems software. Polaron refuses that bargain.

Every operation that would be undefined in C has a **defined** outcome in Polaron — it either wraps predictably, saturates, traps, or is checked deterministically. The unifying promise is not "everything traps" but "nothing is undefined":

- Integer overflow has a **defined two's-complement wrap** by default — the zero-overhead choice, and defined rather than undefined, so it can never be steered into corruption. When you want overflow to be an *error* instead, wrap the expression in `checked( ... )`, and signed `+` `/` `-` `*` trap deterministically; the standard library also offers per-operation `x.wrappingAdd(y)`, `x.saturatingAdd(y)` (clamp to the type's max/min), and the `unchecked` variants.
- Array indexing is **bounds-checked**. An out-of-range access halts with a defined `array index out of bounds` panic instead of corrupting memory.
- Division by zero **traps** rather than producing an undefined result.
- Dereferencing a null pointer produces a **deterministic trap**, not a wander into arbitrary memory.

The point is not to make Polaron slower than C — where the compiler can prove a check is unnecessary, it removes it — but to make sure that when something *does* go wrong, the failure is a clean, predictable stop, never a silent exploitable corruption. Safety here is a property of the language's definition, not an optional linter.

Pillar 4 — No ceremony in the common case, a cannon when you need one

Verbosity in Polaron is not verbosity for its own sake. The everyday path is meant to be lightweight, and the heavy machinery is meant to be *available* rather than *mandatory*. You do not have to specify where an object lives every single time — `new Rectangle(3, 4)` defaults to the stack for objects and the heap for arrays — but when placement matters, `on stack` and `on heap` are right there to say it. You do not have to reach for pointers to pass data around, because value semantics handle the common case; but when you need shared, aliased state, `T*` and `T&` give you exactly that.

This is the "sensible default, explicit opt-in" principle applied throughout the language. The 90% case should read cleanly. The 10% case — the region allocator, the ownership discipline, the freestanding kernel — should be *expressible*, precisely and without fighting the language, when you finally need the cannon.

Pillar 5 — Immutability by default

Bindings and fields are **immutable unless you say otherwise**. A variable you never reassign needs no annotation; a variable you *do* intend to reassign must be marked `mutable`. This turns the more dangerous choice — mutable state — into the one that is visible in the source, and makes the safer choice the effortless one.

```
int side = 4;           // immutable: side can never be reassigned
mutable int count = 0;  // mutable: count is expected to change
count = count + 1;      // fine
// side = 5;           // compile error: side is not mutable
```

The same rule governs fields. A field set once in the constructor and never touched again is left immutable; only a field that is reassigned during the object's life is marked `mutable`.

Pillar 6 — Native performance as a first-class goal

Performance is not an afterthought bolted on with an optimizing pass; it is a design constraint. Polaron has no implicit boxing, no hidden allocations behind ordinary syntax, no runtime type soup, and no garbage collector to introduce pauses. It compiles straight through LLVM to native instructions. Because it

targets the same backend as Clang, a Polaron program and its C or C++ equivalent that express the same computation end up at parity — there is no inherent overhead imposed by the language itself. Polaron is intended to be usable for the domains where that matters: game engines, simulations, audio processing, and systems software.

1.3 Who Polaron is for

Polaron is aimed at programmers who need control and speed but are tired of the accidental complexity that usually comes with them:

- **Java and C# developers** who love the structure of object orientation but hit a wall with garbage-collection pauses in latency-sensitive work.
- **C++ developers** who want a modern, manually-managed language without decades of accreted legacy and undefined behavior.
- **Game and engine developers, simulation and audio authors** — anyone for whom a GC pause at the wrong millisecond is unacceptable.
- **Systems and kernel programmers**, who can drop into the freestanding mode described below and write bare-metal code while keeping full object orientation.

It is deliberately *not* trying to be your web-backend, data-science, or quick-scripting language. For those, reach for Go, Python, or TypeScript. Polaron competes with **C, C++, Rust, and Zig** — the languages you choose when the machine is close and every cycle counts. Against C and C++ it offers defined behavior and value-semantic clarity without giving up native speed. Against Rust it trades the borrow checker for a simpler, more familiar object model plus explicit ownership tools you opt into. Against Zig it keeps a rich, structured type system and mandatory OOP rather than a minimal procedural core.

1.4 A first taste

Here is the canonical Polaron "hello, world." Read it from the outside in.

```
import System.IO.Console;
program HelloWorld;

public bundle main {
  public namespace app {
    public class Main {
      public static method main(string[] args) returns void {
        System.IO.Console.println("Hello, world!");
        return;
      }
    }
  }
}
```

Even the smallest program shows the shape of the language:

- The `import` line brings the standard-library console into scope. Polaron requires imports to be explicit — nothing from the standard library is available implicitly, so the dependency on `System.IO.Console` is stated up front.

- `program HelloWorld`; names the program.
- A **program** contains one or more **bundles**, a bundle contains one or more **namespaces**, and a namespace contains **types**. Every layer here is marked `public` because the entry point must be reachable from the outside.
- The entry point is a `public static method main(string[] args) returns void` inside a `public class Main`. The compiler looks for exactly this chain — a public bundle, a public namespace, a public `Main`, and a public static `main` — to decide where execution starts.
- Output goes through `System.IO.Console`. Its `println` prints a line; `printf` takes a format string and arguments; `print` writes without a trailing newline.

Now a slightly richer program — a real class with a private field, a constructor, and a method — to show how the everyday code you will write actually looks:

```
import System.IO.Console;
program Geometry;

public bundle main {
    public namespace shapes {

        public class Rectangle {
            private int width;
            private int height;

            public constructor Rectangle(int width, int height) {
                this.width = width;
                this.height = height;
            }

            public method area() returns int {
                return this.width * this.height;
            }
        }

        public class Main {
            public static method main(string[] args) returns void {
                Rectangle r = new Rectangle(3, 4);    // default placement: on stack
                System.IO.Console.println($"area = {r.area()}");    // area = 12
                return;
            }
        }
    }
}
```

Several conventions, all enforced by the compiler, are on display:

- **Every block is braced.** There is no brace-less `if` or single-statement body; a block always has `{ }`.
- **Member access always goes through `this`.** Inside `Rectangle`, the fields are `this.width` and `this.height`, never bare `width`. This makes it unmistakable when you are touching object state versus a local.
- **Visibility is explicit on everything** — `public`, `private`, `protected`, or `internal`. There is no implicit default, so a member's reach is always written down.

- **The fields are immutable.** `width` and `height` are set once in the constructor and never reassigned, so neither needs `mutable`.
- **`new Rectangle(3, 4)` defaults to the stack.** Because `r` is a plain object, it lives on the stack and its destructor (if any) runs when `main` returns — no `delete` required. Had we asked for `on heap`, we would owe a matching `delete r`;
- **`$" ... "` is string interpolation.** The expression `{r.area()}` is evaluated and spliced into the string, which is a convenient way to build console output.
- **`var` is only for locals.** Here the type is written out, but inside a method body you may write `var r = new Rectangle(3, 4)`; and let the compiler infer it. Fields, parameters, and return types always require an explicit type — inference stops at the method boundary.

To build and run a program with the current toolchain, the compiler (`polc`) emits LLVM IR and `clang` links it:

```
polc geometry.pol -o geometry.ll
clang geometry.ll -o geometry.exe
geometry.exe
```

1.5 Two execution modes

Polaron comes in two flavors, and it is worth knowing they exist before you meet them in detail later in the reference.

Managed (normal) mode is what every example above uses. It is the full language: the standard library, the console, dynamic memory, exceptions, reflection, and the runtime services that make ordinary application code pleasant to write.

Freestanding mode is the systems subset, declared with `program X freestanding`; (or on an individual `bundle X freestanding`). It strips away the parts that assume an operating system and a runtime — no managed heap services, no reflection, no exceptions — so you can write a kernel or bare-metal code while keeping classes, inheritance, and the rest of the object model. The compiler enforces the boundary: use a forbidden feature in a freestanding program and it is a compile error, not a mysterious crash at runtime.

1.6 How to read this reference

This book is organized to be read front to back the first time and dipped into by topic thereafter. The chapters build on one another:

1. **Introduction & Philosophy** (*this chapter*) — what Polaron is, why it is shaped the way it is, and a first taste of real code.
2. **Program Structure & Modules** — programs, bundles, namespaces, imports, visibility, and the entry point in depth.
3. **Expressions, Statements & Method Calls** — the everyday mechanics of code inside a method: declaring and assigning variables, the operators, forming expressions, the statement kinds, and how you call methods (on an object, on `this`, on a class, and chained).
4. **Values & the Type System** — the primitive families, `boolean`, `char`, `String` / `string`, literals and explicit conversion, plus records, structs, unions, enums, and generics.

5. **Memory & Ownership** — stack versus heap, `new` / `delete`, deep-copy assignment, pointers and references, deterministic destruction, and the ownership and region tools you opt into.
6. **Object-Oriented Programming** — fields, constructors and destructors, instance and static methods, `this`, encapsulation, inheritance, interfaces, `override`, and virtual dispatch.
7. **Control Flow** — conditionals, loops, ranges, `switch` / `match`, and the "chaos tetrad".
8. **Errors, Results & Contracts** — exceptions, `Result` / `Option`, contracts, and defined-behavior traps.
9. **Concurrency** — threads, channels, mutexes, and `async` / `await`.
10. **Compile-Time, Reflection & Prefixes** — `comptime`, reflection, the universal prefixes, and `unimport`, which needs the managed runtime. Persistents do **not** — see §5.9.
11. **Systems Programming** — freestanding mode, raw pointers, the low-level `Memory` API, and FFI.
12. **Keyword Reference** — every reserved word, its status, and a short example.
13. **Diagnostics** — how to read a Polaron error, its structure, and `polaron explain`.
14. **Commands** — `command` and its `carries` list, command types as roles, method references.
15. **Toolchain** — how to build, run, package, and test Polaron programs.

Six **Standard Library** chapters follow the guide: concurrency & core, collections, data structures, text/encoding/crypto, parsing/time/JSON, and math/net/misc.

Throughout, code appears in fenced ````polaron` blocks and reflects what the compiler accepts today. When a feature is planned but not yet fully implemented, the text will say so, so that you are never left guessing whether an example will actually run. This reference is the canonical description of the language, cross-checked against the compiler; it exists to *explain* the language and teach it to a human reader, not merely to enumerate it.

With the philosophy in hand, turn to Chapter 2 to see how a program is assembled from bundles and namespaces.

2. Program Structure & Modules

Every language has to answer a deceptively simple question before you can write a single line of logic: *where does code live?* In some languages the answer is "in a file"; in others it is "in a package" or "in a module." Polaron takes a firm position. Code lives inside a strict, four-level hierarchy — **program** → **bundle** → **namespace** → **type** — and that hierarchy is not optional decoration you can skip for small programs. It is the shape of every Polaron program, from a one-line "Hello, world!" to an operating-system kernel.

This chapter walks that hierarchy from the outside in. We start with the `program` declaration that opens every source file, descend through bundles and namespaces, arrive at the entry point where execution begins, and then look at the two mechanisms that control what can see what: **visibility modifiers** and **imports**. Along the way we cover how several files compose into one program, and we close with a glance at freestanding mode.

The compilation unit, top to bottom

Open any Polaron source file and you will find the same skeleton. Here is the smallest complete, runnable program:

```
import System.IO.Console;
program HelloWorld;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                System.IO.Console.printf("Resultado: %d\n", 42);
            }
        }
    }
}
```

Read it from the top. First come the **imports** — one per line, each ending in a semicolon, each naming exactly one thing you intend to use. Then comes the **program declaration**, which names the whole compilation and is terminated by a semicolon. Everything after that is the body: a `bundle`, which contains a `namespace`, which contains a `class`, which contains the `method` where the work happens.

Notice that even this trivial program pays the full structural tax. There is no "script mode," no top-level statements, no free functions floating outside a class. Polaron is object-oriented by mandate: the only place executable code can live is inside a method, and the only place a method can live is inside a type. The scaffolding you see here is the *minimum*, not ceremony you have grown by accident.

Why this rigidity?

The hierarchy exists because each level answers a different question:

- The **program** is the whole deliverable — the thing you compile and ship. It has a name and, ultimately, an identity at runtime.

- A **bundle** is a *unit of independent compilation*. Bundles can be compiled separately, distributed as their own binary artifact, swapped in and out of a build, and even shared between running programs. They are Polaron's answer to libraries, plugins, and build variants all at once.
- A **namespace** is a naming and visibility scope *inside* a bundle. It keeps type names from colliding and draws the line that imports have to cross.
- A **type** (`class`, `interface`, `record`, `struct`, or `enum`) is where data and behavior finally live.

The first three levels are purely organizational — they hold no fields and run no code of their own. They exist to give every type a precise address and to make the reach of every name explicit rather than accidental.

The shape of the declarations

The outer declarations follow one fixed shape, and it pays to have it exact:

- The **program declaration is always terminated by a semicolon** — `program Name;`. It never takes a brace block. It names the whole compilation and then stands alone on its line.
- Every **bundle and namespace is always a brace block** — `public bundle X { ... }`, `public namespace Y { ... }`. They are never written with a trailing semicolon.

So a file is a single `program Name;` line, followed by one or more braced bundles, each holding one or more braced namespaces:

```
program GameEngine;

public bundle audio {
    public namespace mixers {
        public class StereoMixer { /* ... */ }
    }
    public namespace effects {
        public class Reverb { /* ... */ }
    }
}
```

A program may declare **several bundles**, each its own brace block, one after the other under the same `program` line:

```
program GameEngine;

public bundle audio {
    public namespace mixers { public class StereoMixer { /* ... */ } }
}

public bundle physics {
    public namespace world { public class RigidBody { /* ... */ } }
}
```

There is deliberately no second syntax to remember: `program Name;` opens the file, and everything below it — bundles, namespaces, types — nests with braces. This is the same skeleton as the Hello, world program above, just with more than one namespace or bundle inside it.

The entry point

An executable program must have exactly one place for execution to begin. Polaron finds that place by looking for a specific, fully public chain of declarations — there is no configuration file and no naming magic beyond the rule itself. The entry point is:

a `public bundle`, containing a `public namespace`, containing a `public class` named `Main`, containing a `public static method main(string[] args)` that returns `void` (or `int`).

Every word of that sentence is enforced. The bundle, the namespace, and the class must all be `public`; the class must be named exactly `Main`; the method must be named exactly `main`, must be `static`, must take a single parameter, and that parameter must be an array of `string`. The return type must be either `void` or `int` — nothing else, and it may not itself be an array.

```
import System.IO.Console;
program Greeter;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                System.IO.Console.println("Hello from Polaron");
                return;
            }
        }
    }
}
```

The `string[] args` parameter is the program's command-line arguments — the words that followed the executable's name on the command line, delivered as a dynamic array of strings. A program that ignores its arguments simply never touches `args`, but the parameter must still appear in the signature: it is part of the shape the compiler matches against.

A `void main` may end with a bare `return;` or simply run off the end of the method; both are fine. If you would rather signal an exit status to the operating system, declare `returns int` instead and return a number.

The compiler is strict in both directions. If **no** valid entry-point chain exists, compilation of an executable fails with a diagnostic telling you to provide a public `Main`. If **more than one** valid `main` exists across the program, that is an ambiguity error — you must disambiguate by renaming, narrowing visibility, or changing a signature so that exactly one candidate remains. Libraries are the deliberate exception: a bundle compiled as a library (see multi-file compilation below) needs no entry point at all, because it is meant to be consumed, not launched.

Visibility

Visibility is how Polaron controls reach: which code is allowed to name, touch, or call which other code. Every declaration states its visibility explicitly — there is no "package-private by omission" default to memorize. If you leave the modifier off where one is expected, that is a mistake to be corrected, not a silent default.

At the **bundle and namespace level**, three modifiers apply, and they describe how far across *program boundaries* a bundle or namespace reaches:

- **public** — reachable by *other running programs* at runtime (Polaron lets one program call into another's public bundles). This is the visibility the entry point requires.
- **internal** — reachable anywhere inside the *same program*, across any of its bundles, but never from outside.
- **private** — reachable only inside the *owning bundle*.

At the **member level** — the fields and methods inside a class — four modifiers apply, and they describe reach within the type hierarchy in the way you would expect from any class-based language:

- **public** — visible to all code that can see the type.
- **protected** — visible inside the class and its subclasses.
- **private** — visible only inside the declaring class.
- **internal** — visible within the same program.

Here **protected** lets a subclass reach a base-class field while keeping it hidden from unrelated code:

```
public class Account {
    protected mutable int balance;

    public constructor Account(int initial) {
        this.balance = initial;
    }
}

public class Savings extends Account {
    public constructor Savings(int initial) {
        super(initial);
    }
    public method addInterest() returns void {
        this.balance = this.balance + this.balance / 100;    // protected field, reachable in s
    }
}
```

Two habits from the language's conventions are worth restating because they interact with visibility on every line: member access always goes through **this**. (there is no implicit receiver), and values are immutable unless marked **mutable**. The **balance** field above is both **protected** *and* **mutable**, because a subclass reads it and the class reassigns it.

Imports

If visibility decides what *could* be seen, imports decide what a given file actually *chooses* to see. Polaron's rule here is uncompromising and central to the language's character: **nothing is implicit**. There is no ambient prelude of globally visible names, no automatically imported standard library. If a file uses a type, that type must either live in the same namespace or be imported by name.

An import names one symbol and ends with a semicolon:

```
import System.IO.Console;
import System.Collections.ArrayList;
```

The path is the symbol's full address, and it is always complete: the **bundle** first, then the **namespace(s)**, then the symbol itself as the last component. `import System.IO.Console;` brings the `Console` type from bundle `System`, namespace `IO`, into this file's scope. The standard library lives in the bundle `System`; a plugged-in library is imported by its own full path in exactly the same shape (`import Opengl.GL.GL;`). There is no short form and no bundle-less spelling — the first segment must name a real bundle, and the compiler rejects a path whose bundle or namespace does not match where the symbol actually lives.

Wildcards are not allowed in any form — you cannot write `import System.IO.*;`. Each thing you use is spelled out, one import per line. This is verbose on purpose: reading the top of a file tells you exactly where every dependency comes from, with no guessing. (By convention bundle and namespace names are PascalCase; the compiler warns, but does not error, when they are not.)

Because the rule admits no exceptions, even the most everyday facilities require an import. Printing to the console needs `import System.IO.Console;`; there is no built-in `print`. A program that forgets the import and then calls `System.IO.Console.println( ... )` will not compile. The verbosity is the intended cost of having no hidden magic.

Naming a type the standard library also names

Because a type's address is its full path, a simple name is not reserved. Your program may declare `class Regex`, `class Scanner`, `class Files` — the standard library keeps meaning its own, yours means yours, and neither has to know about the other. What decides which one a file gets is the same thing as always: the namespace it is written in, and what it imports.

The exception is the handful of types the **compiler** answers for itself rather than declaring in the library — `File`, `Time`, `Net`, `Memory`, `Process` and their neighbours (§11). There is no class to shadow, so taking one of those names takes it for the whole program, including the standard library's own calls to it; the compiler rejects that at the declaration. `Math` is the deliberate exception to the exception: it is a virtual type precisely so that a program with its own `Math` compiles. Freestanding code is exempt from the whole rule, having no standard library to disturb.

How imports gate cross-namespace access

The purpose of an import is to cross a namespace boundary. Within a single namespace, types see each other freely — no import is needed for one class in `app` to use another class in `app`. The moment a type in one namespace refers to a type in *another* namespace, though, an import (or a fully qualified name) is required, and the compiler enforces this on **every** place a type name can appear: local variables, field declarations, method parameters, return types, and expressions alike.

Consider a program with two namespaces where `app` wants to use a helper from `lib`:

```

import System.IO.Console;
import main.Lib.LibHelper;           // bundle main, namespace lib, type LibHelper
program TwoNamespaces;

public bundle main {

    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                LibHelper* h = new LibHelper() on heap;    // LibHelper is visible thanks to the
                System.IO.Console.printf("weight=%d\n", h.weight());
                delete h;
                return;
            }
        }
    }

    public namespace lib {
        public class LibHelper {
            public constructor LibHelper() {}
            public method weight() returns int { return 9; }
        }
    }
}

```

Drop the `import lib.LibHelper;` line and the compiler objects that `LibHelper` lives in namespace `lib` and must be imported to be used here. The diagnostic even suggests the exact line to add. The same barrier applies if `LibHelper` appeared only as a parameter type or a return type — the import gates the *name*, wherever it is written.

Import-prefix validation

An import is not merely a spelling of a name; it is a *claim* about where that name lives, and the compiler checks the claim. The prefix — everything before the final component — must be the symbol's real, declared namespace. If it is not, compilation fails.

```

import lib.LibHelper;      // OK: LibHelper really is declared in namespace 'lib'
import app.LibHelper;      // error: 'LibHelper' is in namespace 'lib', not 'app'
import lib.Nonexistent;    // error: import of unknown symbol 'lib.Nonexistent'

```

This turns imports into checked documentation. You cannot accidentally import the wrong `Widget` from the wrong namespace and have it silently resolve to something unexpected: the path either matches the truth or the build stops.

The standard library is internally cohesive, so a `System.*` type may lean on another `System.*` type without an explicit import between them — `System.Json` can use a `StringBuilder` from `System.Text` freely. But that cohesion is a convenience *inside* the standard library. Your code always imports what it uses from `System`, every symbol, every time.

Programs that span several files

Real programs outgrow a single file, and Polaron lets one program be assembled from many `.pol` files. There are two distinct mechanisms, and it is worth keeping them separate in your mind.

The first is **compiling several files together as one program**. You hand the compiler more than one source file at once:

```
polc core.pol ui.pol net.pol -o app.ll
```

Every file must open with the *same* `program` name — the first file establishes the name, and any file that disagrees is rejected with an error. The compiler then merges all the bundles and all the file-level imports from every file into a single program and analyzes them as a whole. Because file-level imports are gathered together, the split across files is purely for your own organization; the resulting program behaves exactly as if you had written it all in one file. Each file still carries its own imports at the top, so each file remains readable on its own.

The second mechanism is **separate bundle compilation**. Because a bundle is a unit of independent compilation, you can build one on its own into a distributable binary — a `.polb` file (the compiled implementation) paired with a `.polh` header (its public declarations, used for type-checking consumers). A library bundle is compiled in library mode and needs no entry point:

```
polc calc_lib.pol --lib -o calc.polb      # produces calc.polb + calc.polh
```

A separate program then *consumes* that bundle. It imports the bundle's public types by name exactly as if they were part of its own source, and points the compiler at the compiled bundle so the implementation can be linked in:

```
import System.IO.Console;
import math.Calc;           // a public type from the separately-compiled 'calc' bundle
program CalcApp;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                int r = Calc.square(7);
                System.IO.Console.printf("square = %d\n", r);
                return;
            }
        }
    }
}
```

```
polc calc_app.pol --use calc.polb -o app.ll
```

Here the consumer type-checks against `calc.polh` and links the real code from `calc.polb`. The bundle boundary is thus both a compilation boundary and a distribution boundary — the foundation for shipping

libraries, plugins, and different build variants of the same program without maintaining parallel source trees.

Cross-program access via IPC

Two separate Polaron programs can talk to each other as if they shared one object graph. A **server** program exports a class; a **client** program holds a proxy for it and calls its methods, and each call is serialized across a pipe/socket named after the server program. The object's state lives only in the server process.

The server calls `Program.serve`, naming itself and supplying a **capability policy** — a lambda consulted for every capability a client requests:

```
import System.Ipc.Program;

// Grant "mixdown", refuse everything else.
Program.serve("GameEngine", lambda(String capability) returns boolean {
    return capability.equals("mixdown");
});
```

Any method on the exported class is callable remotely. A method that takes a `BundleAccessToken` parameter is **privileged**: the dispatcher runs it only after checking the token is one this program actually issued (see the client side below).

The client imports the remote type's header with `import from program`, connects by name, and builds a proxy. `Program.connect` returns a nullable `ProgramHandle*` (null if the server isn't running); the proxy's every method call becomes IPC:

```
import from program GameEngine bundle audio.StereoMixer;
import System.Ipc.Program;
import System.Ipc.IpcError;

nullable ProgramHandle* a = Program.connect("GameEngine");
if (a == null) {
    // the engine is not running
    return;
}

StereoMixer mixer = a.bundle("audio").namespace("mixers").type<StereoMixer>().instantiate();
try {
    int n = mixer.play("boom.wav"); // arguments are serialized; the result comes back
    mixer.setVolume(9);             // state persists in the server across calls

    // Capabilities: ask the server's policy, then use the token for a privileged call.
    BundleAccessToken* mixdown = a.requestAccess("mixdown");
    if (mixdown.granted()) {
        int level = mixer.mixdown(mixdown);
    }
} catch (IpcError e) {
    // a call failed on the wire
}
a.close();
```

The type is known to the client only from the server's header (its *body* lives in the other process), so the compiler hands the client a proxy whose methods are wire frames. Capability tokens make the trust boundary explicit: the server decides, per client, what is allowed.

A forward pointer: freestanding mode

Everything above assumes the managed Polaron runtime — the machinery behind exceptions, the standard `Console`, reflection, and more. For systems programming that runs with no runtime beneath it (a kernel, a bootloader, bare metal), a program or an individual bundle can be declared **freestanding** by adding the keyword to its declaration: `program Kernel freestanding;` or `public bundle main freestanding { ... }`. In that mode the compiler removes the managed runtime and actively *rejects* any feature that would depend on it — `async/await`, exceptions, reflection, the managed `Console` and standard library — leaving the systems core of structs, regions, raw pointers, the low-level `Memory` API, and FFI. Freestanding mode is a subject of its own and is covered in a later chapter; it is mentioned here only so that its declaration syntax has a place in the structural picture drawn by this one.

3. Expressions, Statements & Method Calls

The previous chapter showed how a program is *organized* — the program, bundle, namespace, and class scaffolding. This chapter is about what goes *inside* a method: the ordinary, everyday act of writing code. How you declare a variable, how you assign to it, which operators the language gives you, how expressions and statements differ, and — the thing you will do more than any other — how you call a method. None of this is exotic; it is the vocabulary you use on every line. But Polaron has a few firm rules here (assignment is not an expression, member access always goes through `this.`, every block is braced) that are worth stating plainly before you meet them scattered through later chapters.

Everything in this chapter lives inside a method body, so keep the whole-program skeleton from Chapter 2 in mind: what follows are the statements you would write inside `main`, or inside any method you declare.

3.1 Statements and blocks

A method body is a sequence of **statements**, each terminated by a semicolon, executed top to bottom. Polaron is strict about layout in two ways that never relax:

- **Every block is braced.** A `{ }` block groups statements; there is no brace-less form. `if (x) return;` written without braces is a *syntax error*, not a shortcut. An `if`, a loop, a method body — all take a full `{ }`.
- **One statement per line, each ending in `;`.** Declarations, assignments, calls, and `return` all end with a semicolon. Blank lines and indentation are yours to arrange for readability; they carry no meaning.

The statement kinds you will use constantly are: a **variable declaration** (`int n = 5;`), an **assignment** (`n = 6;`), an **expression statement** — usually a method call whose result you ignore (`this.log();`), a **return**, and the **control-flow** statements (`if`, `while`, `for`, `foreach`, `switch`, `match`) covered in Chapter 7. A bare block `{ ... }` is itself a statement and opens a new scope.

```
import System.IO.Console;
program Statements;

public bundle Main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                int a = 2;           // declaration
                int b = 3;           // declaration
                int sum = a + b;      // declaration initialised from an expression
                System.IO.Console.println($"sum = {sum}"); // expression statement (call)
                return;              // return statement
            }
        }
    }
}
```

3.2 Variables

A variable binds a name to a value of a known type. You declare one by writing the type, the name, and — almost always — an initializer:

```
int side = 4;
boolean ready = true;
char grade = 'A';
```

Two rules shape every declaration:

- **Immutable by default.** A variable you never reassign needs no annotation. If you intend to reassign it, you must mark it `mutable`. This makes the riskier choice — mutable state — the one that is visible in the source.
- **`var` infers the type of a local.** Inside a method body you may write `var` and let the compiler deduce the type from the initializer. Inference is a *local-only* convenience: fields, parameters, and return types always name their type explicitly.

```
int fixed = 10;           // immutable: reassigning it is a compile error
mutable int count = 0;    // mutable: expected to change
count = count + 1;        // fine

var total = fixed + count; // inferred as int
```

An immutable variable must be initialized where it is declared, since there is no later chance to give it a value. A `mutable` one may be assigned afterwards as often as you like.

3.3 Assignment

Assignment stores a new value in an existing `mutable` variable (or field, through `this.`). Two properties of assignment in Polaron catch newcomers, and both are deliberate:

Assignment is a statement, not an expression. It has no value, so it cannot appear inside a condition or be chained. `if (x = 5) { ... }` is an error (the compiler will not let a stray `=` masquerade as `==`), and `a = b = c;` does not compile — write the two assignments on their own lines. This closes off a classic family of bugs at the cost of one convenience.

Assignment copies. For a class value, `b = a;` makes `b` a deep, independent copy of `a`; the two do not share state afterwards. This is covered in full in Chapter 5; it is mentioned here only so the meaning of `=` is never a surprise. To make two names refer to the *same* object you use a pointer or reference, also in Chapter 5.

Compound assignment combines an operator with assignment as a shorthand: `a += b` is `a = a + b`. Polaron offers the full set — arithmetic and bitwise alike:

```
mutable int a = 20;
a += 5;      // a = a + 5  → 25
a -= 3;      // 22
a *= 2;      // 44
a /= 4;      // 11
a %= 3;      // 2
a <<= 4;     // 32  (bitwise shift-assign)
a &= 12;     // 0   (bitwise and-assign; also |= ^= >>=)
```

And **increment / decrement**, `++` and `--`, adjust a variable by one:

```
mutable int i = 0;
i++;        // 1
i--;        // 0
```

3.4 Operators

Polaron's operators are the familiar C-family set, with defined behavior on every edge case (overflow traps, division by zero traps — see Chapter 8). Grouped by role:

Group	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Comparison	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code>≤</code> <code>≥</code>
Logical	<code>&&</code> <code>&</code> <code> </code> <code>!</code>
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>
Increment / decrement	<code>++</code> <code>--</code>
Conditional (ternary)	<code>cond ? a : b</code>
Assignment	<code>=</code> and the compound forms above

The **logical** operators `&&` and `||` **short-circuit**: `a && b` never evaluates `b` if `a` is already `false`, and `a || b` never evaluates `b` if `a` is already `true`. This lets you guard an expensive or unsafe operand with a cheap test on its left.

The **ternary** operator is an *expression* — it produces a value — so it is the idiomatic way to choose between two values without an `if` statement:

```
int larger = a > b ? a : b;
```

Operators follow the usual precedence (multiplication before addition, comparison before `&&`, and so on). When in doubt, parenthesize; the parentheses cost nothing and document intent:

```
boolean inRange = (x ≥ 0) && (x < limit);
int packed = (hi << 8) | lo;
```

3.5 Calling methods

Calling a method is the workhorse of the language, and there are exactly four call shapes you will use.

On another object. Given a variable holding an object, name it, a dot, the method, and the arguments in parentheses. Arguments are ordinary expressions:

```
Account acc = new Account(100) on stack;  
acc.deposit(50);           // call deposit(50) on acc  
int bal = acc.balance();   // a call that returns a value
```

On yourself, through `this`. Inside a method, calling another method of the *same* object still goes through `this`. — there is no implicit receiver. `this.area()` calls this object's `area`; a bare `area()` would be an unknown name.

```
public method describe() returns void {  
    System.IO.Console.WriteLine($"area = {this.area()}");    // one method calling another  
    return;  
}
```

On a class (static). A `static` method belongs to the class, not an instance, so you call it through the class name: `Counter.inc()`, `Math.max(a, b)`. The standard-library console you have been using is exactly this — `System.IO.Console.WriteLine(...)` is a static call on the `Console` type.

Chained. When a call returns an object, you can call a method on that result directly, building a chain left to right. Each call runs on the value the previous one returned:

```
String s = builder.append("a").append("b").toString();
```

In every shape, **arguments are passed by copy** for value types — a method receives its own copy of a class argument and cannot alter the caller's object by mutating the parameter. When a method genuinely needs to modify the caller's object, the caller passes a pointer or reference (Chapter 5). A method's result comes back through `return`, and you either use it (assign it, feed it to another call, test it) or discard it by writing the call as its own statement.

A worked example ties the shapes together — one class whose method calls its own helper, and a `Main` that constructs it and calls across to it:

```

import System.IO.Console;
program Calls;

public bundle Main {
    public namespace app {

        public class Greeter {
            private int excitement;

            public constructor Greeter(int excitement) {
                this.excitement = excitement;
            }

            // A method that calls another method on the same object, through this.
            public method greet() returns void {
                System.IO.Console.println($"hello (level {this.level()})");
                return;
            }

            public method level() returns int {
                return this.excitement * 2;
            }
        }

        public class Main {
            public static method main(string[] args) returns void {
                Greeter g = new Greeter(3) on stack;
                g.greet(); // call across to another object → "hello (
                System.IO.Console.println($"max = {Main.max(4, 9)}"); // static call on this
                return;
            }

            // A static helper, called as Main.max(...)
            public static method max(int a, int b) returns int {
                return a > b ? a : b;
            }
        }
    }
}

```

3.6 Talking to the console

The most common thing a first program does is print, so it is worth pinning down the console calls you will reach for constantly. They all live on `System.IO.Console` and require `import System.IO.Console;` at the top of the file — nothing from the standard library is available without an import.

- `println(x)` writes its argument followed by a newline.
- `print(x)` writes without a trailing newline.
- `printf(format, args ...)` takes a C-style format string — `%d` for an integer, `%c` for a character, with `\n` and `\t` escapes — and substitutes the arguments in order.

A string on its own is **text, not a format**: `println("50% done")` prints a percent sign. The format reading applies only when there are arguments to substitute, which is what `printf` is for.

For building output from several values, **string interpolation** is usually clearer than `printf`. Inside a `"..."` string, each `{expr}` is evaluated and its value spliced into the text:

```
import System.IO.Console;
program Printing;

public bundle Main {
  public namespace app {
    public class Main {
      public static method main(string[] args) returns void {
        int x = 7;
        int y = 5;
        System.IO.Console.printf("x=%d y=%d\n", x, y);          // x=7 y=5
        System.IO.Console.println($"sum={x + y} max={x > y ? x : y}"); // sum=12 max=7
        System.IO.Console.print("no newline here");
        return;
      }
    }
  }
}
```

With variables, operators, expressions, statements, and the four call shapes in hand, you can write the *body* of any method. The next chapter steps back up to the values those statements move around — the primitive and composite types, and the rules that govern them.

4. Values & the Type System

Every value a Polaron program manipulates has a type, and that type is known at compile time. There is no dynamic typing, no "any", and no silent coercion that quietly reinterprets your data behind your back. The compiler knows the exact shape and size of every value before the program ever runs, and it uses that knowledge to reject whole classes of mistakes — mixing a 32-bit integer with a 64-bit one without saying so, letting a `null` slip into a value that promised never to be null, indexing past the end of an array — while the program is still on your screen rather than in your users' hands.

This chapter is a tour of the values you can build. We start with the primitive types that map directly onto machine registers, work through literals and the rules that govern how numbers convert into one another, and then climb up into the composite types — records, structs, unions, enums, catalogs — that let you give a name and a shape to your own data. Along the way we cover the two string types, the `nullable` opt-in, arrays, local type inference with `var`, generics and their variance, and finally casting and Polaron's promise that a cast never produces undefined behavior.

A recurring theme is worth stating up front, because it colors every example that follows: **assignment in Polaron is a deep copy, not a shared reference.** Writing `b = a` gives `b` its own independent copy of everything `a` holds. When you genuinely want two names to point at the same object, you say so explicitly with a pointer (`T*`) or a reference (`T&`). Keep this in mind — it is one of the language's load-bearing decisions, and it applies uniformly to primitives, structs, records, and full classes alike.

4.1 The primitive types

Primitives are the atoms: fixed-size values that live in registers or on the stack and carry no hidden machinery. Polaron draws a deliberate line between the *normal* names you use in everyday application code and the *bit-counted* names reserved for freestanding, close-to-the-metal work.

Integers

In **normal mode** — which is to say, ordinary applications, the mode you are in unless you explicitly declare a program `freestanding` — the integer types wear names that describe their role rather than their exact bit width. The idea is that most of the time you should not have to think about whether an integer is 16 or 32 bits; you just want "an integer".

Type	Bits	Signed?	Range (approx.)
byte	8	signed	−128 ... 127
short	16	signed	−32 768 ... 32 767
int	32	signed	−2.1 billion ... 2.1 billion
long	64	signed	-9.2×10^{18} ... 9.2×10^{18}
ubyte	8	unsigned	0 ... 255
ushort	16	unsigned	0 ... 65 535
uint	32	unsigned	0 ... 4.29 billion
ulong	64	unsigned	0 ... 1.8×10^{19}

`int` is the workhorse — 32-bit and signed — and is what you reach for by default. The unsigned family (`ubyte` ... `ulong`) is genuinely unsigned: it wraps and compares as an unsigned quantity, not as a signed one wearing a disguise.

```
int    count    = 42;
long   fileSize = 9000000000L;
ubyte  channel   = 200;      // fits in 0..255
ushort port      = 8080;
```

There is also `address`, a raw pointer-sized integer (64-bit on the current target). It exists for low-level and freestanding code that needs to treat a memory address as a number; you will rarely see it in application code.

`address` sits in the integer family so that arithmetic on it works, but it does **not** convert to or from the other integers on its own. Making an address out of a number is how a program reads memory nobody gave it, so the conversion is written down:

```
mutable long n = 4096;
address a = n;           // error: an address is not a 64-bit integer
address a = cast<address>(n); // ... unless you say so
```

Freestanding code is exempt, and not as a concession: on bare metal, making an address out of an integer is the work — the memory-mapped register at `0xB8000` is a number until you say otherwise, and no allocator is going to hand you the address instead.

The freestanding-only bit-counted names

You may have seen names like `int32` or `uint8` in other systems languages, and Polaron does have them — but **only in freestanding mode**. The bit-counted family

```
int8   int16  int32  int64
uint8  uint16 uint32 uint64
float32 float64
```

exists because when you are writing a kernel, a device driver, or a wire-format parser, the exact number of bits is part of the contract and hiding it would be a lie. In normal mode, however, using any of these names is a **compile-time error** — the compiler will point you at the normal-mode equivalent:

```
int32 x = 5;    // ERROR in normal mode:
                // "type 'int32' exists only in freestanding mode; use 'int'"
```

The mapping is exact: `byte` = `int8`, `short` = `int16`, `int` = `int32`, `long` = `int64`, and likewise for the unsigned family and for `float` = `float32`, `double` = `float64`. The two naming schemes describe the same machine types; they differ only in which mode is allowed to spell them. This chapter uses the normal names throughout.

Floating point

Normal mode offers four floating-point widths:

Type	Bits	Notes
<code>smallfloat</code>	16	half precision
<code>float</code>	32	single precision — the common choice
<code>double</code>	64	double precision
<code>quadruple</code>	128	quad precision

Note the crucial detail that trips up newcomers from other languages: in Polaron, **`float` is 32 bits and `double` is 64 bits**. They are distinct types, and one does not silently become the other. The bit-counted aliases `float32` and `float64` are, again, freestanding-only.

```
float ratio = 0.5f;    // 32-bit
double pi   = 3.14159; // 64-bit
```

`boolean`, `char`, and `void`

Three more primitives round out the set:

- **`boolean`** holds `true` or `false`. Conditions accept boolean expressions, and the language also permits truthy/falsy testing in conditions.
- **`char`** is a single character. It may be written with single or double quotes — `'a'` and `"a"` both denote a `char` — though the samples and this guide favor single quotes to keep `char` visually distinct from strings.
- **`void`** is the absence of a value; it appears only as the return type of a method that produces no result.

```
boolean done = false;
char grade = 'A';

public method greet() returns void {
    System.IO.Console.println("hi");
    return;
}
```

4.2 Literals and literal suffixes

Integer literals may be written in several bases, and an underscore may be used freely as a digit separator for readability — it has no effect on the value.

```
int decimal = 1_000_000;
int hex     = 0xFF;      // 255
int binary  = 0b1010;    // 10
long big    = 100L;      // the L suffix makes it a long
```

Floating-point literals default to `double`; the `f` suffix makes one a `float` instead:

```
double e = 2.71828; // double by default
float pi = 3.14f;   // 32-bit because of the f suffix
```


The `m` suffix: `Decimal`

The suffix `m` (or `M`) produces a value of the primitive `Decimal` type — a 128-bit fixed-point number carrying eighteen fractional digits. `Decimal` exists for money and any other domain where the rounding drift of binary floating point is unacceptable. Because it is fixed-point, `0.1m + 0.2m` is exactly `0.3m`, not the notorious `0.30000000000000004` you would get from `double`.

```
Decimal price = 19.99m;
Decimal tax   = 1.50m;
Decimal total = price + tax;    // exact: 21.49m
```

Size suffixes and user-defined literal suffixes

Polaris has no dedicated syntax for "kilobytes" or "milliseconds". Instead it offers a single general mechanism — the `comptime literal` function — from which such suffixes are built in the standard library and can be built by you. A `comptime literal` function takes exactly one argument (the literal it follows) and is evaluated entirely at compile time, so the suffix costs nothing at runtime.

The standard library's `System.Memory.Units` namespace provides six size suffixes that all return a `ByteSize`:

```
import System.Memory.Units.kilobytes;

region cache = itself.allocate(64 kilobytes);    // expands to kilobytes(64)
```

The expression `64 kilobytes` is rewritten by the compiler into the call `kilobytes(64)`, which — being `comptime` — folds to a constant `ByteSize` before the program runs. The available suffixes are `bytes`, `kilobytes`, `megabytes`, `gigabytes`, `terabytes`, and `exabytes`. Because the result is a distinct `ByteSize` struct rather than a bare `int`, an API that wants a size can demand exactly `ByteSize` and thereby refuse a raw, unit-less integer — you cannot accidentally hand `1024` to something expecting `1024 bytes`.

Defining your own is one declaration. A `comptime literal` is a member like any other — it takes exactly one argument, is evaluated at compile time, and its *name* becomes the suffix:

```
public class Units {
    public comptime literal kib(int x) returns long { return cast<long>(x) * 1024L; }
    public comptime literal kib(double x) returns long { return cast<long>(x * 1024.0); }
}

long a = Units.kib(64);           // qualified: 65536, folded before the program runs
long b = Units.kib(1.5);          // overloaded by the literal's type: 1536
```

It may be called two ways: **through its owner**, as above, or — once the owner is imported — as the suffix sugar `64 kib`. That import is what makes the sugar available, which is why `System.Memory.Units.kilobytes` is imported by name before `64 kilobytes` can be written. Suffixes may be **overloaded by the literal's type**, so `1.5 kib` and `2 kib` can take different paths.

Return a distinct type (a `struct` or a `newtype`, §4.12b) rather than a bare number and the unit becomes part of the type — which is how `ByteSize` refuses a unit-less integer.

One caution: a suffix name is matched wherever a literal is followed by an identifier, so avoid names that read as ordinary words. `step` is reserved for exactly this reason — it would be ambiguous against the range syntax.

Numeric coercion — there is (almost) none

This is the rule that most distinguishes Polaron's arithmetic from C's: **there is no implicit numeric promotion**. Converting between two integer widths, or between integer and floating point, requires an explicit `cast<T>` (covered in §4.12). The compiler will not quietly widen a `short` to an `int` for you, nor truncate a `long` into an `int`.

```
long total = 5000000000L;
int narrow = cast<int>(total); // explicit narrowing required
double d = cast<double>(narrow);
```

The one convenience the language does grant is for **compile-time integer literals**: a literal that provably fits its target coerces without a cast, because the value is known and the fit is provable.

```
byte b = 5; // OK: 5 fits in a byte
short s = 300; // OK: 300 fits in a short
byte x = 999; // ERROR: 999 does not fit in a byte
```

Nothing about this applies to *variables*; only literals whose value the compiler can see get this treatment.

Overflow: a defined wrap by default

By default, integer arithmetic that overflows **wraps** with two's-complement semantics — the zero-overhead choice. Crucially, this wrap is *defined behavior*, not the undefined behavior C leaves it as, so an overflow can never be steered into memory corruption; it just produces the wrapped value. When you want overflow to be an **error** instead, wrap the expression in `checked( ... )`, and signed `+`, `-`, and `*` trap deterministically on overflow. For per-operation control, every integer type carries explicit methods:

```
int fast = a + b; // defined two's-complement wrap, no overhead (the default)
int safe = checked(a * b); // signed overflow here traps deterministically
int a = x.wrappingAdd(y); // explicit wrap
int b = x.saturatingAdd(y); // clamps to the type's MIN/MAX
int c = x.uncheckedAdd(y); // wrap, no check
```

The full family covers add/sub/mul (and div where meaningful) under each policy. (Unsigned arithmetic, and all arithmetic in freestanding mode, always wraps.)

4.3 `String` versus `string`

Polaron has two string types, and the capital letter carries real meaning.

- **`String`** (capital S) is the **immutable** string class. Once built, its contents never change; any "modification" produces a fresh `String`. Because it can never change underneath you, a `String` is safe to share freely — no copy is forced on assignment or on passing it to a method.
- **`string`** (lowercase) is the **mutable** string type, meant for building up or editing text in place.

Reach for `String` by default; reach for `string` when you are assembling text incrementally and mutation is the point.

```
String greeting = "hello";    // immutable
string builder = "";          // mutable, to be appended to
```

String interpolation with `$" ... "`

Prefixing a string literal with `$` turns `{ }` into an interpolation hole: the expression inside is evaluated and its textual form spliced into the result. Without the `$`, braces are just ordinary characters.

```
String name = "Ada";
int age = 36;
String msg = $"Hello {name}, you are {age} years old";
String calc = $"Total: {price * quantity}";
String fmt = $"Pi is {pi:0.00}";    // :format spec after a colon
```

A hole may contain any expression, not merely a variable, and an optional format specifier follows a colon (`{pi:0.00}`). Interpolation is a first-class part of the `String` machinery and reads far more clearly than manual concatenation.

4.4 nullable types

By default **no type in Polaron can hold `null`**. A variable of type `Dog` always refers to an actual `Dog`; the possibility of "no dog" simply does not exist for it. If you need that possibility, you opt in with the `nullable` modifier:

```
Dog rex = null;    // ERROR: Dog is non-nullable
nullable Dog rex = null;    // OK: nullable opts the type into null
```

The enforcement rule is deliberately simple and lives entirely **at assignment boundaries**. A `null`, or a value that is itself `nullable`, may not flow into a non-nullable target — whether that target is a variable, a field, a parameter, or a return value. You decide, at each declaration, whether `null` is a legal value there; the compiler holds you to it.

```
nullable Dog rex = maybeDog();
Dog d = rex;    // ERROR: nullable cannot flow into non-nullable
nullable Dog r2 = rex;    // OK: nullable → nullable
```

What Polaron deliberately does **not** have is a flow-sensitive narrowing analysis. There is no "if you checked it for null on line 3, the compiler treats it as non-null on line 4", no force-unwrap operator, and no borrow-checker-style tracking. That machinery is exactly the complexity the language chose to avoid.

So what happens if you dereference a `nullable` that turns out to be `null` at runtime? You get a **deterministic trap** — a clean, defined program halt with a message — never undefined behavior and never a silent read through address zero. This is consistent with Polaron's broader no-UB principle: dereferencing a nullable is *allowed* (the compiler inserts a null check on the access), and if the value is null the program stops predictably instead of corrupting memory. A non-nullable value, by construction, can

never be null, so dereferencing it costs nothing extra. Comparing against `null` with `==` or `≠` is always permitted.

```
nullable Dog d = maybeDog();
int a = d.bark();           // allowed; traps deterministically if d is null
if (d ≠ null) { }          // comparison is always fine
```

4.5 Arrays

Arrays in Polaron are **dynamic and heap-allocated**. You create one with `new T[n]()`, which allocates space for `n` elements on the heap and zero-initializes them. Elements are read and written with the familiar `a[i]` syntax, the current length is read back with `a.length()`, and the array is released with `delete`.

```
int[] nums = new int[10]();           // ten zeroed ints on the heap
for (mutable int i = 0; i < nums.length(); i++) {
    nums[i] = i * i;
}
int first = nums[0];
delete nums;
```

Under the hood an array is a block laid out as a 64-bit length header followed by the elements, with the array value being a pointer to that block; `length()` simply reads the header. Every index is **bounds-checked**: an access outside `0 ... length-1` traps deterministically rather than wandering into adjacent memory. A single unsigned comparison catches both negative indices and indices past the end, so the check is cheap and there is no undefined behavior to exploit.

Arrays whose extent is in the type

`int[]` is an array whose length is **not stated**. `int[16]` is an array whose length is:

```
mutable int[16] key;                 // sixteen zeroed ints, right here
key[7] = 42;
int n = key.length();                // 16, a constant -- no header is read
```

The brackets say *array* in both. What is between them says whether the length is part of the type, and everything else follows from that one difference:

	<code>int[]</code>	<code>int[16]</code>
created with	<code>new int[n]()</code>	its own declaration
where it lives	the heap	where it is declared
representation	pointer to <code>[length][elements...]</code>	the elements, nothing else
<code>length()</code>	reads the header	a compile-time constant
released with	<code>delete</code>	nothing — it dies with its owner
copied	the pointer is copied	the elements are copied

A **stated extent** is an extent the value does not have to carry, so it needs no header; it is not reached through a pointer, so there is nothing to allocate; and it lives where it is declared, which is the rule this language already has for where things live (§5.3).

It works as a **field**, which is what lets a `layout` (§6.14) finally arrange an array, and as a **parameter**, carried by value with the aggregate that holds it:

```
public struct Key {
    public mutable int[4] bytes;
    public mutable int tag;
}
```

Extents nest, and read as C, Java and C# read them — the first group is the outer one:

```
mutable int[3][4] grid;           // three of int[4]
grid.length()                     // 3
grid[0].length()                  // 4
grid[2][1] = 5;
```

Two rules worth stating outright. A declared fixed array is **zeroed**, exactly as `new int[16]()` is — it cannot enter the *uninitialized* state the language otherwise defends, because an array is written an element at a time and no definite-assignment proof can follow that. And there is **no** `length(n)`: an array whose extent is in its type cannot be resized without becoming a different type, and the compiler says so rather than pretending.

4.6 `var`: type inference for locals

Inside a method body you may write `var` and let the compiler infer the type from the initializer. This is a convenience for locals **only** — fields, parameters, and return types always require an explicit type, because those form the published surface of a class and should never leave a reader guessing.

```
public method process() returns void {
    var dogs = new ArrayList<Dog>() on heap;    // inferred ArrayList<Dog>
    var n     = 42;                             // inferred int
}

public class Foo {
    var x = 5;                                   // ERROR: a field needs an explicit type
    public method bar(var p) { }                // ERROR: a parameter needs an explicit type
}
```

Two related rules are worth remembering here. First, **immutability is the default**: `var n = 42;` gives you an immutable `n`, and reassigning it is an error unless you wrote `mutable var n = 42;`. Second, **shadowing is forbidden** — you may not declare a variable that hides another of the same name in an enclosing scope.

4.7 record

A `record` is an immutable data carrier defined by its primary constructor parameters, which *are* its fields. Records are ideal for the small, value-like bundles of data — points, coordinates, small results — that you pass around but never mutate.

```
public record Point(int x, int y) {  
    public method sum() returns int { return this.x + this.y; }  
}  
  
Point p = new Point(3, 4);  
Point q = new Point(3, 4);  
boolean same = p.equals(q);    // true – equality is by value
```

Records come with strong guarantees baked in. They are always immutable (`mutable` is rejected outright). The compiler auto-generates `equals`, `hashCode`, and `toString` from the parameters, so two records with equal fields compare equal without you writing a line. They are implicitly `final` and cannot be extended, though they *may* implement interfaces and *may* carry methods and constants. What they may not do is add fields beyond the primary-constructor parameters, and they cannot hold a `persistent`.

4.8 struct (and bit fields)

A `struct` is a **mutable value object** of a fixed, compile-time-known size. Like a record it is copied on assignment and cannot participate in inheritance, but unlike a record it is mutable and does not auto-generate `equals` / `hashCode`. Structs are the right tool for small, mutable aggregates whose layout you care about — a vector, a color, a packet header.

```
public struct Vec3 {  
    public mutable float x;  
    public mutable float y;  
    public mutable float z;  
  
    public constructor Vec3(float x, float y, float z) {  
        this.x = x;  
        this.y = y;  
        this.z = z;  
    }  
}
```

Because a struct is a value, two struct variables never alias: assigning one to another copies every field, exactly like the deep-copy rule for classes. A struct may implement interfaces.

Bit fields

A struct field may declare a bit width with `field : N`, constraining it to `N` bits. This is essential for hardware registers and wire formats where individual fields occupy sub-byte spans.

```
public struct Flags {
    public mutable int a : 4;    // 0..15
    public mutable int b : 4;
    public mutable int c : 12;   // 0..4095
}
```

In the current implementation the constraint is enforced as **value masking**: a value that exceeds `N` bits is truncated to `N` bits on store (so writing `20` into a 4-bit field stores `4`). Physically packing the fields into a tighter struct layout is a planned refinement; the observable semantics — the masking — are in place today.

4.9 `union`

A `union` is C-style: all of its fields occupy the **same** storage, so writing one field and reading another reinterprets the same bits. This is a deliberately low-level tool, most at home in manual-memory and systems code where you need to view one block of memory through more than one lens.

```
public union Value {
    int    asInt;
    float  asFloat;
}

Value v = new Value();
v.asFloat = 1.5;
int bits = v.asInt;    // the bit pattern of 1.5f, reinterpreted as int
```

Because every field aliases the same memory, it is your responsibility to know which interpretation is currently valid — the compiler does not track it for you.

4.10 `enum` and `catalog`

Simple enums

The simplest enum is a named set of constants. Each constant is an ordinal value, and you refer to one by `EnumName.CONSTANT`.

```
public enum Color {
    RED,
    GREEN,
    BLUE
}

Color c = Color.GREEN;
```

Java-style enums with fields and methods

An enum can also be a full-fledged type: each constant carries constructor arguments, and the enum declares fields, a constructor, and methods just like a class. This is the form to reach for when each

constant needs associated data.

```
public enum Planet {
    EARTH(10, 2),
    MARS(30, 3);

    private final int mass;
    private final int radius;

    public constructor Planet(int mass, int radius) {
        this.mass = mass;
        this.radius = radius;
    }

    public method density() returns int {
        return this.mass / this.radius;
    }
}

int d = Planet.EARTH.density(); // 5
```

Every enum, simple or Java-style, comes with auto-generated helpers: `EnumType.values()` returns all constants in declaration order, `EnumType.count()` returns how many there are, `EnumType.random()` picks one, and `EnumType.parse(s)` returns an `Option<EnumType>` by name. On a `VALUE`, `v.name()` returns the declared identifier back as a `String` — the same name data `parse` reads in, read out the other way — and an enum that declares its own `name` method keeps it. A Java-style value also converts to its declaration ordinal with `cast<int>(v)` (never to its singleton's pointer bits).

Catalogs

A `catalog` is, in essence, an *interface for enums*. Where an ordinary interface constrains only the methods a type must provide, a catalog constrains both the shape (the methods) **and** specific values the implementing enum must contain. An enum satisfies a catalog by extending it and supplying the required values in a `byCatalog` block, implementing any required methods as normal enum methods.


```

public catalog TipoMotor {
    combustao,
    h2,
    eletrico

    method classify() returns TipoMotor;
}

public enum Motor extends TipoMotor {
    v8, v12, doisPistoes

    byCatalog {
        combustao,
        h2,
        eletrico
    }

    public method classify() returns TipoMotor { return combustao; }
}

```

Catalogs may provide default implementations, may extend other catalogs, and an enum may satisfy several catalogs at once — giving enums a form of multiple "interface" conformance that carries required values along with required methods.

A **catalog-typed value** is a first-class one: it may be stored, returned, and passed as an argument to a method, a class constructor, or an enum constant's constructor, and calling a catalog method on it dispatches to the constant's own implementation whichever enum that constant came from.

```

public class Scale {
    public static method weigh(Heavy h) returns int {
        return h.weight();           // dispatches to Ingot's or Stone's, by what was passed
    }
}

```

Catalogs and Java-style enums compose

A catalog-implementing enum may be Java-style: each constant — the enum's own and the `byCatalog` ones alike — takes constructor arguments, so every constant carries its data instead of a table on the side. The catalog's required methods are then implemented as ordinary enum methods reading the constant's own fields.

```

public catalog Scored {
    bronze,
    gold

    method score() returns int;
}

public enum Medal extends Scored {
    paper(1)

    byCatalog {
        bronze(10),
        gold(95)
    }
    ;
    private int points;

    public constructor Medal(int points) {
        this.points = points;
    }

    public method score() returns int {
        return this.points;
    }
}

Scored s = Medal.gold;
int g = s.score();    // 95 -- dispatch picks Medal, and `this` is the gold singleton

```

A value typed as the catalog still carries the (enum, ordinal) tag, so several enums — ordinal and Java-style mixed freely — can implement one catalog and a call through the catalog-typed value reaches the right implementation. Ordinals run through `byCatalog` in declaration order exactly as for an ordinal enum, and `values()`, `count()`, `random()` and `parse()` behave the same.

4.11 Generics

Generic types and methods let you write code once and use it across many element types. Polaron implements generics by **monomorphization**: for each concrete set of type arguments you actually use, the compiler stamps out a specialized copy — so `ArrayList<int>` and `ArrayList<Dog>` become genuinely distinct, fully-typed code with no boxing and no runtime type erasure.

```

public class Box<T> {
    private T content;
    public constructor Box(T c) { this.content = c; }
    public method get() returns T { return this.content; }
}

public method swap<T>(T a, T b) returns (T, T) {
    return (b, a);
}

Box<int> b = new Box<int>(42) on heap;
int v = b.get();

```

Const generics: a hole where a number goes

A type parameter is a hole where a **type** goes. `fixed int R` is a hole where a **number** goes, and it binds at stamping like everything else in the brackets:

```

public struct Grid<fixed T, fixed int R, fixed int C> {
    private mutable T[R * C] cells; // the extent is part of the type
    public readonly method rows() returns int { return R; }
    public method set(int r, int c, T v) returns void { this.cells[r * C + c] = v; }

    // The dimensions UNIFY: 'C' is this grid's column count, so 'other's row count must match
    public readonly method timesShape<fixed int N>(Grid<T, C, N>& other) returns int { ... }
}

Grid<int, 2, 3> g; // two shapes are two types

```

Four things follow, and each is one a region cannot give you. The extent is in the type, so the value carries no length header, needs no allocation, and **embeds** as a field of another value type. The dimensions are constants in the body, so a loop over `R * C` has a bound the optimizer can see. And they discriminate in a signature: a `4×3` times a `7×9` does not compile, where a matrix carrying its dimensions as runtime fields accepts anything and fails while somebody is using it.

`fixed` is per parameter and does not spread. A **bare** value parameter (`<int a>`) is reserved for the runtime-bound extent — a dimension supplied when the object is built, outside the type's identity — and is refused today rather than quietly given the stamped meaning.

Constraints

A type parameter may be constrained with `extends` or `implements`, so that generic code can rely on the capabilities of its type argument:

```

public class Cache<T extends Serializable> { }
public class Sorted<T implements Comparable<T>> { }
public method clamp<T extends Numeric>(T value, T min, T max) returns T { }

```

Variance

Polaron uses **declaration-site variance** in the style of C#. On a type parameter, `out` marks it covariant (the type produces `T` values), `in` marks it contravariant (the type consumes `T` values), and leaving the

annotation off makes the parameter **invariant** — the default and the safest choice.

```
public interface Producer<out T> {           // covariant: produces T
    method produce() returns T;
}

public interface Consumer<in T> {           // contravariant: consumes T
    method consume(T item) returns void;
}

public interface Box<T> { }                 // invariant (default)
```

Covariance lets a `Producer<Cat>` stand in where a `Producer<Animal>` is expected; contravariance lets a `Consumer<Animal>` stand in where a `Consumer<Cat>` is expected. Invariant parameters relate only to themselves.

sealed ... permits

A class may declare a closed set of subclasses with `sealed ... permits`. Only the listed classes may extend it, which turns the type into a finite, known set of possibilities — the compiler will reject any other class that tries to extend it.

```
public sealed class Shape permits Circle, Square, Triangle { }
```

Sealing interacts directly with pattern matching: when you `match` on a value of a `sealed` type, exhaustiveness becomes **mandatory** and no `default` arm is required — you must cover every permitted variant, and if you miss one the compiler tells you. (For a non-sealed type a `default` arm is required instead, since the compiler cannot know the possibilities are complete.)

4.12 Casting

All conversions between distinct types are explicit, written `cast<T>(expr)`. There is no hidden coercion; if you want a `long` from an `int`, you ask for it.

```
int    x = cast<int>(someLong);
double d = cast<double>(x);
Dog    dog = cast<Dog>(animal); // downcast, checked at runtime
```

The companion tests `is` and `as` let you probe and convert reference types: `animal is Dog` is a boolean test, `animal as Dog` performs the conversion, and `animal as? Dog` yields a nullable `Dog` that is `null` when the object is not a `Dog`.

Casting never invokes undefined behavior

Casting in Polaron is held to the same no-UB standard as the rest of the language. A cast can fail, but it fails cleanly and predictably — it never produces a poison value or a corrupt object. Three cases are worth spelling out:

- **Class downcasts are checked.** `cast<Dog>(animal)` verifies at runtime that the object really is a `Dog` (using the vtable) and throws a `ClassCastException` if it is not, rather than reinterpreting unrelated memory. Upcasts, identity casts, and plain reference reinterprets pass through unchecked because they cannot go wrong.
- **Float-to-integer conversions saturate.** Converting a floating-point value whose magnitude is too large for the target integer clamps it to the integer's minimum or maximum, and `NaN` becomes `0`. This uses hardware-supported saturating conversion, so it costs nothing at runtime while avoiding the poison result a naive truncation would give.

```
polaron double huge = 1.0e30; int n = cast<int>(huge); // saturates to int's MAX, not
garbage
```

- **Integer-to-integer conversions** widen (sign- or zero-extending as appropriate) or narrow by truncation — a defined, wrapping narrowing, never undefined.

Between raw integers and pointers, a `cast<T*>(addr)` or `cast<address>(ptr)` reinterprets the value, a facility meant for low-level and freestanding code. Finally, a class may define its own **conversion operators**, letting `cast<T>` invoke user-written conversion logic:

```
public class Celsius {
    private double temp;
    public operator explicit cast<Fahrenheit>() returns Fahrenheit {
        return new Fahrenheit(this.temp * 9.0 / 5.0 + 32.0);
    }
}

Fahrenheit f = cast<Fahrenheit>(myCelsius);
```

4.12b Naming a type: `typealias` and `newtype`

Two declarations give an existing type a new name, and they differ in exactly one way: whether the new name is the **same** type or a **different** one.

`typealias` is transparent. The alias and its target are fully interchangeable — it is a shorthand, and nothing more:

```
public typealias Celsius = int;
public typealias IntFn = function<int, int>;

Celsius t = 20;
int sum = t + 5;           // fine: Celsius IS int
```

Reach for it when a type expression is long enough to obscure the code that uses it — a `HashMap<String, ArrayList<Dog*>>`, a nested generic — and when you genuinely do not want a new type.

`newtype` is opaque. It creates a distinct nominal type over the same representation, and the two do not mix without a cast:

```
public newtype OrderId = long;
public newtype UserId = long;           // and distinct from OrderId, too

OrderId o = cast<OrderId>(cast<long>(1000));
long raw = cast<long>(o) + 7;           // cast out, compute, cast back
OrderId next = cast<OrderId>(raw);
```

That is the whole point: an `OrderId` and a `UserId` are both 64-bit integers and passing one where the other belongs is a compile error rather than a support ticket. The cost is the cast at each boundary, which is also the benefit — every place the raw number is treated as a number is visible.

Which to use: if mixing the two would be a bug, use `newtype`. If mixing them is fine and you only wanted a shorter name, use `typealias`.

4.13 Putting it together

The type system's job is to let you say precisely what you mean and to catch you the moment you contradict yourself. A `nums` you declared `int[]` cannot suddenly hold a `Dog`; a `nullable Dog` cannot leak into a `Dog`; a `long` cannot become an `int` without your explicit consent; an index cannot run off the end of an array without a clean trap. And because assignment copies, two names never accidentally share state — sharing is something you request out loud, with `T*` or `T&`.

The composite types layer meaning on top of the primitives: `record` for immutable value bundles, `struct` for mutable fixed-layout aggregates, `union` for reinterpreting memory, `enum` and `catalog` for closed sets of named values, and generics for writing one algorithm that works across many types with zero runtime overhead. Each is a way of pinning down the shape of your data so the compiler can be your first and most tireless reviewer.

5. Memory & Ownership

This is the chapter where Polaron stops looking like a friendly object-oriented language and starts showing its real personality. Polaron is a systems language: it compiles straight to native code, it has **no garbage collector**, and it never stops the world to sweep your heap behind your back. Every byte your program uses has a birth and a death that you can point to in the source. That is the whole bargain — you trade the convenience of a GC for **determinism**: you always know *when* memory is reclaimed, *where* an object lives, and *who* is responsible for it.

The surprising part is that this bargain does not feel dangerous in day-to-day code. Polaron achieves that with three ideas working together, and this chapter is the story of how they fit:

1. **Value semantics as the default.** Assignment copies. A variable *is* its object, not a handle to one somewhere else. Two variables never accidentally alias the same thing, so a mutation over here can never spookily change a value over there.
2. **RAII and scoped cleanup.** Objects clean themselves up when their scope ends — automatically, deterministically, and even when an exception is unwinding through the frame. You rarely write cleanup code by hand.
3. **Compile-time ownership when you need it.** When a resource genuinely must have a single owner (a file handle, a socket, a mutex guard), you say so in the type, and the compiler proves at *compile time* that the rule holds — with zero runtime cost.

And when the safe defaults get in the way — when you want a bump allocator, a memory-mapped hardware register, or state that outlives the object that created it — Polaron hands you the sharp tools: pointers, regions, and persistents. The design principle throughout is *no ceremony in the common case, a cannon when you need one*.

Let's build the model from the ground up.

5.1 Value semantics: assignment is a deep copy

In Polaron, **assignment copies the whole value**, and for a class instance that means a **deep copy**. This is the single most important sentence in the chapter, so let's sit with it. When you write `b = a` and both are class-typed, `b` becomes a brand-new, fully independent object with the same contents as `a`. Mutating `b` afterward cannot touch `a`, because there is no shared storage between them.

```

import System.IO.Console;
program ValueCopy;

public bundle main {
    public namespace app {
        public class Point {
            public mutable int x;
            public constructor Point(int x) { this.x = x; }
        }

        public class Main {
            public static method main(string[] args) returns void {
                Point a = new Point(1) on stack;
                Point b = a;          // deep copy -- b is a separate object
                b.x = 99;             // mutates only b
                System.IO.Console.println($"a={a.x} b={b.x}");    // a=1 b=99
                return;
            }
        }
    }
}

```

The copy is *deep*, not shallow. If a class owns an array or a nested value object, that storage is duplicated too, so the two objects share nothing at all:

```

public class Bag {
    public mutable int[] items;
    public constructor Bag() {
        this.items = new int[3]();
        this.items[0] = 10;
    }
    public method set(int i, int v) returns void { this.items[i] = v; }
    public method get(int i) returns int { return this.items[i]; }
}

// inside main:
Bag a = new Bag();
Bag b = a;          // the backing array is duplicated, not aliased
b.set(0, 999);      // mutate only b's array
System.IO.Console.println($"a[0]={a.get(0)} b[0]={b.get(0)}");    // a[0]=10 b[0]=999

```

Under the hood the compiler copies the object's bytes, then walks each field that *owns* its storage — arrays and nested value sub-objects — and duplicates those recursively. Primitive fields copy inline; the two `Bag` s end up with two completely separate `items` arrays.

Why default to copying? Because aliasing is the source of an enormous class of bugs, and a value default eliminates them by construction. In a reference-by-default language (Java, Python, C#), `b = a` makes `b` point at *the same* object, and now a write through either name is visible through both — often at a distance, often surprising. Polaron flips the default: sharing is the thing you must ask for explicitly, so when you *see* sharing in the source you know it is intentional. It also composes cleanly with manual memory: if every variable owns its own object, "who frees this?" almost always has the obvious answer — the scope that declared it.

Parameter passing follows the same rule: by value. A class-typed parameter receives an independent deep copy, exactly like assignment. Mutating the parameter inside the method does **not** affect the caller's object.

```
public static method tryToChange(Point p) returns void {
    p.x = 500;                // mutates the local copy only
}
// caller:
Point a = new Point(1);
Main.tryToChange(a);
System.IO.Console.WriteLine($"{a.x}"); // still 1
```

This is safe and predictable, but copying a large object on every call is wasteful, and sometimes you genuinely *want* the callee to see the caller's object. That is what pointers and references are for — the subject of the next section.

One consequence worth internalizing: abstract classes and interfaces are always handled by reference, because the concrete subclass behind them has a size the compiler cannot know statically. `String`, being immutable, is shared freely — copying an immutable value is pointless, and it can never surprise you.

How deep the copy goes, and how to go deeper

The copy duplicates the object's own storage, including its value-typed fields. A **pointer** field is copied *as a pointer* — so the copy points at the same thing the original did, and everything reachable through it is shared:

```
Node head = new Node(1) on stack;
head.next = &tail;

Node copy = head;                // `copy` is a new node ...
cast<Node*>(copy.next).v = 77;    // ... but `next` still points at the SAME tail
// head.next.v is now 77 as well
```

That is the right default — following every pointer would mean a copy could not be bounded, and `external` back-references would turn one copy into a copy of the whole program — but it is the place where "assignment is a deep copy" stops being the whole story. It matters most in exactly the structures built out of pointers: linked lists, trees, and graphs.

When you want a genuinely independent graph, that is what `cascade clone` is for (§10.5). It walks what the object *owns*, clones each node once, repoints the clone's pointers at the clones, and shares nothing:

```
cascade clone head into fresh;    // fresh's tail is a copy, not the same tail
```

It keeps a visited set, so a **cyclic** graph clones exactly once per node and comes out with the same shape rather than looping forever. `external` pointers are treated as associations and left pointing where they pointed, which is what stops a clone of one node from dragging in the world.

Fields the copy should not carry: `transient`

Some fields are not part of what an object *is* — a memoised result, a scratch buffer, a cached index. A `transient` field is declared as derived rather than canonical, and it is left out of both of the operations that ask "what is this object's value?": the deep copy, and serialization.

```
public class Counter {
    private mutable int value;
    private transient mutable int cachedDouble;    // derived from value; not canonical
}
```

On assignment the copy gets `value` and a **default** `cachedDouble`, so it starts clean and rebuilds the cache on demand while the source keeps its own. That is usually what you want from a cache, and getting it wrong the other way is a real bug: a copied-over cache that no longer matches the field it was derived from.

`transient` and `persistent` contradict each other — one says "not part of the value", the other "survive the object" — and writing both is a compile error.

5.2 Sharing on purpose: pointers (`T*`) and references (`T&`)

When you want two names to see and mutate **the same** instance, you opt out of value semantics with a pointer (`T*`) or a reference (`T&`). Both share the underlying object; they differ in ergonomics and intent.

A **pointer** `T*` holds the address of an object. You take an address with the `&` operator, and — this is a small but important convenience — you access members through a pointer with the ordinary dot (`.`), not a separate arrow operator:

```
Point a = new Point(1) on stack;
Point b = a;           // deep copy -- independent
Point* p = &a;         // p shares a's instance
p.x = 7;               // writes through to a
System.IO.Console.WriteLine($"a={a.x} b={b.x}");    // a=7 b=99
```

Here `b` was copied and stayed at `99`, while writing through `p` changed `a` to `7`, because `p` is not a copy — it is a second name for the very same object.

A **reference** `T&` is a bound alias: it is tied to one object when it is created and stands in for that object from then on. Use a reference when "this is just another name for that object" is the whole story; use a pointer when you need an address you can pass around, store, rebind, or compare with `null`.

```
Point a = new Point(1);
Point* p = &a;    // an address: rebindable, can be null, supports & and pointer ops
Point& r = a;     // an alias: bound to a, use it exactly like a
```

For passing large objects into a method without paying for the deep copy, declare the parameter as a reference (or pointer). Now the method sees the caller's object and can mutate it:

```
public static method bump(Point& p) returns void {
    p.x = p.x + 1;    // mutates the caller's object
}
```

Pointer arithmetic (`p++`) is permitted on any pointer, but the compiler warns on class pointers, because advancing a pointer into the middle of an object rarely means anything and can corrupt memory. The safe, idiomatic use of `T*` is *sharing*, not walking.

Pointers to pointers: `T**`, `T***`, and the `*` operator

A pointer is itself a value with an address, so you can take a pointer *to* a pointer, to any depth: `T**` is a pointer-to-pointer, `T***` a triple pointer, and so on. Each `&` adds a level and the prefix `*` operator peels one off — `*pp` on a `T**` yields the `T*`, and `**pp` reaches the underlying `T`. This is the classic tool for an *out-parameter* (a method that rebinds the caller's pointer) and for pointer-based data structures:

```
mutable int x = 5;
int* p = &x;      // p → x
int** pp = &p;    // pp → p
int*** ppp = &pp; // and so on, to any depth

int v = ***ppp;   // 5 -- read through three levels
**pp = 99;        // write through two levels: x is now 99

// An out-parameter rebinds the caller's pointer through a T**:
public static method rebind(int** slot, int* target) returns void {
    *slot = target; // *slot as an lvalue: store through the pointer
}
```

The one restriction: a pointer to a *generic* type may be only one level deep (`Box<int>*` is fine, `Box<int>**` is not), because a generic's internal name can itself end in `*` and a second one would be ambiguous. Every non-generic type — primitives, classes, structs — supports pointers to any depth. `T&` references, by contrast, are always a single level: a reference is a bound alias, not an address you build towers of.

The mental model to carry forward: **value by default, share by exception**. A plain variable is an object; a `T*` / `T&` is a way to reach into someone else's object. When you read Polaron and see a `*` or `&`, that is the code announcing "sharing happens here."

5.3 Where objects live: stack vs heap

Every allocation answers a question: *where does this live, and when does it die?* Polaron makes you name the answer with `new ... on stack` or `new ... on heap` — but only when the default is not what you want.

- **on stack** places the object in the current frame. It is reclaimed automatically when the enclosing scope ends. No `delete`, no bookkeeping — the object simply ceases to exist at the closing brace (running its destructor on the way out; see §5.5).
- **on heap** allocates from the heap and hands you ownership of a long-lived object. Heap objects are **not** reclaimed automatically — you must `delete` them.

```
Dog rex = new Dog("Rex", 5) on stack;      // dies at end of scope
Dog* big = new Dog("Big", 10) on heap;     // lives until you delete it
delete big;                               // reclaim the heap object
delete big from heap;                     // the explicit, unambiguous form
```

The placement is optional — no ceremony in the common case. When you omit it, the compiler picks the sensible default: **objects go on the stack** (RAII, freed automatically), and **arrays and dynamic collections go on the heap** (they are naturally dynamic). You write the placement explicitly only to force something other than the default, or when the type itself demands the heap (a `T*` you intend to return, an object that must outlive its scope).

```
Dog rex = new Dog("Rex", 5);              // no ceremony → stack (RAII)
Dog* big = new Dog("Big", 10) on heap;    // the "cannon": manual heap, needs delete
int[] scores = new int[16]();            // arrays default to the heap
```

The design intent is a smooth gradient: reach for a stack object and forget about it in the easy 90% of code; drop down to explicit `on heap` + `delete` exactly when a value has to outlive the scope that made it. Both live in the same language, one keystroke apart.

5.4 No undefined behavior: dangling, double-free, and the trap

Manual memory has a reputation for foot-guns: use-after-free, double-free, dangling pointers. In C and C++ these are *undefined behavior* — the program might crash, might silently corrupt data, might appear to work until it doesn't. Polaron takes a firm position here, the **no-UB principle**: a memory mistake must **fail deterministically**, never silently corrupt.

Consider a double-`delete`:

```
Dog* d = new Dog("Rex", 5) on heap;
delete d;
delete d;    // deterministic panic, not silent corruption
```

The second `delete` does not quietly free a recycled block and march on. Before running the destructor, the compiler emits a liveness check (`__polaron_check_live`) against the allocator: if the block has already been freed, the program panics immediately with a clear message. A live pointer, or a pointer the allocator doesn't own, passes through untouched. The same guard covers `delete` of an already-released object. You still bear the responsibility of freeing heap objects once, but freeing *twice* turns a lurking corruption bug into a loud, reproducible crash at the exact offending line.

Other memory hazards are handled in the same spirit elsewhere in the language: dereferencing a `nullable` that happens to be null traps deterministically rather than segfaulting; out-of-bounds array indexing traps at the access; integer overflow can be made to trap under `checked( ... )`. The unifying rule is that Polaron would rather stop your program with a precise diagnostic than let it wander into corrupted state.

The best defense, though, is to not manage memory by hand at all where you don't have to — which is exactly what the next three sections are about.

The compile-time half: the region binder

Trapping is what happens when a mistake reaches the running program. Some mistakes never need to get that far, and the **region binder** is the analysis that stops them at the compiler. It is **on by default**.

An object allocated in a method's own frame — `new X()`, or `new X() on stack` — lives exactly as long as the call. A pointer or reference to it that *leaves* the call is dangling before it is ever used once:

```
public static method make() returns Point* {
    Point* p = new Point(7);
    return p;                                // error[Polaron-1721]: the frame is gone by the time
}                                           // the caller reads it
```

The same rule covers storing such an object into something that outlives the call — a field, or a parameter a callee stores into its own receiver — and it covers a closure handed to a `Thread` that captures plain mutable state instead of an `atomic<T>`, `Mutex<T>` or `Channel<T>`.

The analysis has a chapter of its own: 5b. The region binder. What follows here is the shape of it. Chapter 5b is the whole thing — the four regions and the order between them, how it compares to a garbage collector and to a borrow checker, where each value's region comes from, the six ways to answer a refusal, and every diagnostic it emits. Read it once before writing a program that shares objects between long-lived owners.

A **heap** object is not caught, and that distinction is the whole of it: `on heap` means the pointer dies at return and the object does not, so handing one out is a factory and storing one in a field is the ownership transfer every linked structure is built from.

```
Node* n = new Node(v) on heap;
this.head = n;                // fine: the object outlives the call
```

If a program genuinely traffics in pointers to dead frames, `--no-region-binder` turns the analysis off for that program. There is deliberately no per-line version: a line that looks ordinary is exactly how this mistake survives review, so the way to opt out is one flag, visible in the build. `polc --check` takes the same flag and the same default, so the editor and the compiler always agree about what is an error.

5.5 RAI: destructors and automatic cleanup

RAI — *Resource Acquisition Is Initialization* — is the backbone of Polaron's memory determinism. The idea: tie a resource's lifetime to an object's lifetime, so that cleanup is automatic and happens at a well-defined moment. In Polaron that moment is **scope exit**, and the mechanism is the **destructor**.

A destructor is written `~ClassName` and runs when the object's lifetime ends:

```
public class Board {
    private mutable char[] cells;
    public constructor Board() { this.cells = new char[9](); }
    public destructor ~Board() returns void {
        delete this.cells;      // release the array this object owns
    }
}
```

For a **stack object**, the destructor runs automatically at the end of the block that declared it. You do not call it; you cannot forget it:

```
public method play() returns void {
    Board board = new Board();      // on the stack by default
    board.place(0, 'X');
    return;                          // ~Board() runs here, freeing cells
}
```

Destructors run in **reverse order of construction** (last built, first destroyed), which is what you want: a later object may depend on an earlier one, so it must be torn down first. When one class extends another, the derived destructor runs first, then the base — inner-to-outer, always deterministic.

Crucially, **cleanup also runs on exception unwind**. If an exception is thrown and propagates out through a frame, every live stack object in that frame still has its destructor run as the stack unwinds. There is no path — normal `return` or exceptional throw — where a stack object silently skips its cleanup. This is what makes RAII trustworthy rather than merely convenient: the guarantee holds precisely in the moments (errors, early exits) when hand-written cleanup is most likely to be forgotten.

Heap objects are the exception to automatic cleanup — their destructor runs when *you* `delete` them, because their whole point is to outlive the current scope. RAII plus `delete` is the two-speed system again: stack objects clean themselves; heap objects clean up on command.

5.6 `defer` and `using`: cleanup that can't be skipped

Sometimes the resource you need to release is not neatly wrapped in a stack object — it's a heap allocation, a lock, an open handle — and you want to guarantee its cleanup runs no matter how the block exits. Polaron gives you two scoped tools for that, and both share RAII's iron guarantee: **they run on normal exit and on exception unwind alike**.

`defer` schedules a block to run when the enclosing scope ends:

```
public method processFile(String path) returns void {
    File f = new File(path) on heap;
    defer { delete f; }           // runs at scope exit, even if read() throws
    f.read();
}
```

The deferred block runs whether the method returns normally or an exception tears through it. You put the cleanup right next to the acquisition, so you can never lose track of it across a long method with many exit points.

Multiple `defer`s run in **LIFO order** — last deferred, first run — mirroring destructor order, so nested resources unwind in the correct sequence:

```
defer { System.IO.Console.WriteLine("deferred 1"); }
defer { System.IO.Console.WriteLine("deferred 2"); }
System.IO.Console.WriteLine("body");
// prints: body, then "deferred 2", then "deferred 1"
```

`using` is sugar for a resource with a well-defined scope. It declares the resource, runs the block, and disposes the resource at the closing brace — running its destructor and, for a heap resource, freeing it:

```
using (Resource r = new Resource(3) on heap) {
    System.IO.Console.WriteLine("inside using");
} // r is disposed here: ~Resource() runs, then the heap block is freed
```

If you explicitly `delete r` inside the block, `using` notices and does not double-dispose it. Like everything else in this section, `using` disposal fires on the exception path too, so a throw inside the block still releases the resource before unwinding onward.

The three mechanisms — destructors, `defer`, `using` — are one idea seen from three angles: *cleanup is attached to a scope, ordered LIFO, and guaranteed on every exit*. Together they mean that in idiomatic Polaron you write very little explicit cleanup, and the cleanup you do write cannot be skipped.

5.7 Regions: arena allocation

Individual `new / delete` is precise but has a cost, and sometimes it's the wrong shape for the problem. When you're about to allocate *thousands* of small objects that all die at the same time — a parse tree, a frame of a simulation, every entity in a level — you don't want thousands of separate frees. You want to allocate them all in one arena and throw the arena away in a single stroke. That's a **region**.

`region` is a primitive type in Polaron, on the same footing as `int` or `boolean`. A region owns a slab of memory and hands out pieces of it with a trivial *bump* — advance a pointer, that's the whole allocation. You create one with the `itself` self-reference and an amount expressed in real units:

```
import System.Memory.Units.kilobytes;

region pen = itself.allocate(1 kilobytes);
Dog* a = new Dog(5) in region pen; // bump-allocated inside pen
System.IO.Console.WriteLine($"a = {a.getAge()}");
// pen is freed automatically at end of scope (region RAII)
```

`itself` is a pronoun that, inside a declaration's initializer, refers to the variable being declared — so `itself.allocate( ... )` reads naturally without repeating the name. The `1 kilobytes` is not a bare integer: the `kilobytes` suffix (a compile-time `literal` function from `System.Memory.Units`) produces a `ByteSize`, a distinct type, so you can't accidentally hand a region a raw `int` meant for something else.

Objects placed with `new T() in region r` do **not** get individually freed. The whole region is released at once — either automatically when its scope ends (region RAII, the same discipline as stack objects), or explicitly:

```
release region pen;    // frees the whole arena at once; destructors of its objects run
```

When a region is released, every object still living in it has its destructor run, then the entire slab goes back in one operation. That is the performance rationale in a sentence: **you pay for allocation once per region, not once per object**. The bump allocation itself is nearly free; the runtime even caches a released slab per thread so a hot `allocate ... release` loop reuses the block instead of round-tripping a multi-megabyte request through the OS every iteration.

Regions can also enforce **what may live in them**, via `accepts` and `rejects` — chained onto the allocation and validated by the type system:

```
region pen = itself.allocate(1 kilobytes).accepts({Dog});
Dog* d = new Dog(42) in region pen;    // OK: Dog is accepted
// putting a Cat in `pen` is a compile error (and a runtime exception if only known dynamically)

region io = itself.allocate(8 kilobytes).rejects({String});
```

A region can be a **class field**, so an object owns its own arena for the objects it manages:

```
public class Pool {
    public region arena;
    public constructor Pool() { this.arena = itself.allocate(1 kilobytes); }
    public method makeDog(int n) returns Dog* {
        return new Dog(n) in region this.arena;    // allocate into the field's region
    }
}
```

Finally, a region can sit over a **fixed physical address** with `itself.at(addr, size)`. This is the basis for memory-mapped I/O in freestanding code — a VGA text buffer, a device register — where objects must live at an address the hardware dictates. Releasing such a region frees only its small header; it never touches the fixed memory itself.

```
import System.Memory.Units.bytes;

region hw = itself.at(0xB8000, 4000 bytes).accepts({VGACChar});
// objects placed in `hw` land at the hardware address; ideal for MMIO
```

Regions are Polaron's answer to "GC-like ergonomics without a GC": batch the lifetime, and a whole graph of objects becomes as cheap to reclaim as a single free.

5.7.1 Region flavors

The plain `region` above is a **bump** allocator: allocation just advances a cursor, and nothing is reclaimed piecemeal — the whole slab goes back at once. That is the right shape for "everything dies together," but not every arena has that shape. Polaron lets you pick the allocator's *flavor* by writing a soft keyword **before** `region`:


```

region a = itself.allocate(64 kilobytes);           // bump (the default)
pool region b = itself.allocate(64 kilobytes);      // free-list
stack region c = itself.allocate(64 kilobytes);     // mark / rollback (LIFO)
fixedslot region d = itself.allocate(64 kilobytes).accepts({Dog}); // one fixed-size type
ring region e = itself.allocate(96 bytes).accepts({Log}); // circular, auto-evicting

```

- **bump** (default) — advance a pointer; the cheapest possible allocation. Reclaimed only when the whole region is released. Best when the objects share one lifetime.
- **pool** — a free list. `delete x from region r` runs `x`'s destructor and returns its slot to the free list, so a pool sustains an endless churn of allocate/delete without the region growing. (See "Reclaiming one object" below.)
- **stack** — LIFO reclamation with explicit marks: allocate, then rewind the cursor to an earlier point to free everything allocated since, all at once.
- **fixedslot** — a pool specialized to a *single* type, so every slot is the same size and both allocation and free are O(1) index math. It **requires** `.accepts({T})` naming that one type; without it the compiler reports an error.
- **ring** — a fixed circle of slots. When it fills, the *oldest* live object is evicted (its destructor runs) to make room for the new one — a bounded "most recent N" buffer. You never `delete` from a ring; it evicts on its own.

A region carries exactly one flavor — writing two (`pool stack region ...`) is an error, and a flavor keyword only qualifies a `region` (`pool int x` is an error).

growable. Any flavor except `ring` can be made `growable`, which chains a fresh block when the current one fills instead of failing the allocation (a ring is inherently bounded, so `growable ring` is rejected):

```

growable pool region g = itself.allocate(256 bytes); // grows in 256-byte blocks as needed

```

Flavors work on **region fields**, too — the pattern behind Forge's terminal pool, where a long-lived object churns short-lived ones through its own free list:

```

public class Kennel {
    private pool region den;
    public constructor Kennel() { this.den = itself.allocate(4 kilobytes); }
    public method adopt(int n) returns void {
        Dog* d = new Dog(n) in region this.den;
        // ... use d ...
        delete d from region this.den; // reclaims the slot for the next dog
    }
}

```

Reclaiming one object

- **delete x from region r** — on a **pool** or **fixedslot** region, runs `x`'s destructor and returns its slot to the free list for reuse. On a bump region it runs the destructor and marks the slot dead, but the space is not reclaimed (bump has no free list). On a ring it is an error — rings evict automatically.
- **extract x from region r** — moves `x` *out* of the region onto the heap (a deep relocation) and returns the new heap pointer, which you must bind:

```
polaron pool region kennel = itself.allocate(2 kilobytes); Dog* d = new Dog(7) in region
kennel; Dog* out = extract d from region kennel; // relocate to the heap; the kennel slot is
reclaimed // `d` is now moved-from; use `out`. Delete `out` yourself later.
```

Extract is how you let a single object *survive* an arena that is about to go away. The compiler rejects an extract whose object still holds a pointer into the same region (that pointer would dangle after the move), and rejects a bare `extract` whose result isn't bound.

Marks and rollback (stack regions)

A `stack` region reclaims in LIFO order. Take a `checkpoint` with `mark`, allocate scratch, then `rollback` to that checkpoint to free everything since — destructors run in reverse:

```
stack region tmp = itself.allocate(1 megabytes);
checkpoint m = mark of region tmp; // remember the cursor
Node* scratch = new Node(1) in region tmp;
// ... build a throwaway working set ...
rollback region tmp to m; // reclaim everything allocated since m
```

`checkpoint` is a built-in cursor type. `mark` requires a stack region, and a checkpoint may only be rolled back on the region it came from — mixing them is a compile error. Marks nest, so a hot loop can `mark` / `build` / `rollback` each iteration and reuse the same memory every time.

5.7.2 region class : a type with one place

Everything above places objects one allocation at a time. A `region class` turns that around: every instance of the type comes from **one** region owned by the type, and that region holds nothing else.

```
public region class Node {
    public mutable int value;
    public constructor Node(int v) { this.value = v; }
}

Node* a = new Node(10); // no `on heap`, no `in region`: there is only one place it can go
Node* b = new Node(20);
```

The allocation is fast for the same reason any region is, but speed is not what the declaration buys — regions already beat `malloc` by roughly 12× in the benchmark suite without it. What only **totality** buys is the three things that need "*there is nowhere else*":

- an O(1) answer to `unimport`'s question "*is any instance of this type still alive?*",
- a linear walk over every instance of a type, with no registry to maintain,
- and, eventually, a `Node*` that is a 32-bit offset rather than a 64-bit pointer.

Totality is also enforced downward: everything reachable beneath a region class must itself live in that region. The region binder places `new Node( ... )` in the type's own region automatically (§5b.5), which is what lets a tree link its own children without a word about lifetimes.

5.8 Ownership disciplines: `movable`, `unique`, `partitionable`

Value semantics is a fine default, but some things must **not** be copied. A file handle, a socket, a mutex guard, a GPU buffer — duplicating one of these is either meaningless or actively dangerous (two owners closing the same file). For these, Polaron lets you declare an **ownership discipline** on the class itself, and then proves the rules at *compile time*. The discipline is part of the type's contract: anyone using your class learns the rules by reading its declaration, not its documentation.

There are three disciplines, declared as a prefix before `class`:

Default (no prefix) — copy semantics. Everything in §5.1. Assignment copies; multiple independent instances coexist. This is what you've used so far.

`movable` — transfer, don't copy. A `movable` class forbids the silent deep copy. Ownership moves from one variable to another with the explicit `move` keyword, and the source variable is *invalidated* by the move — the compiler tracks it and rejects any later use:

```
public movable class Connection {
    public mutable int id;
    public constructor Connection(int id) { this.id = id; }
}

Connection c1 = new Connection(1) on heap;
Connection c2 = move c1;    // ownership transfers to c2; c1 is now "moved"
// c1.id;                  // COMPILE ERROR: 'c1' used after it was moved
```

A plain `Connection c2 = c1;` (no `move`) is a compile error for a movable type — the whole point is that transfer must be visible in the source. That holds at **every place a value can land**, not only at a variable: a field, a call argument and a return are the three ways a value outlives the method it was written in, so they are exactly where it matters most.

```
this.held = c;           // error[Polaron-0404]: field 'held' needs an explicit 'move' (move c)
this.keep(c);            // error[Polaron-0404]: the parameter of 'keep' needs an explicit move
return c;                // error[Polaron-0404]: this return needs an explicit 'move'
```

A moved-from variable is not garbage you must tiptoe around; it is simply *empty*, and you can bring it back to life by reassigning it:

```
mutable Connection c = new Connection(1) on heap;
for (int i in 0..10) {
    consume(move c);      // c is moved out ...
    c = new Connection(i) on heap; // ... and reassigned: valid again
}
```

`unique` — one live owner, ever. A `unique` class guarantees that at any instant only one variable in the whole program references the object. Every assignment is an implicit move (the `move` keyword is optional but recommended for clarity), and passing a `unique` value by value to two places, or storing it in a container that would duplicate it, is rejected at compile time.

```
public unique class FileHandle {
    public constructor FileHandle(string path) { /* open */ }
}

FileHandle f1 = new FileHandle("data.txt") on heap;
FileHandle f2 = f1;    // implicit move; f1 is now invalid (== move f1)
```

partitionable — **opt in to moving fields out one at a time.** By default you cannot move a single field out of an object, because that would leave the parent in a half-built state. A **partitionable** class permits it for its **movable** / **unique** fields; the compiler then tracks each field's state independently:

```
public partitionable class Connection {
    public movable Socket socket;
    public mutable Config config;
}

Connection c1 = new Connection() on heap;
Socket s = move c1.socket;    // OK because Connection is partitionable
// c1.socket is now moved-out and inaccessible until reassigned
// c1.config is still fully valid
c1.socket = new Socket() on heap;    // reactivate the field
```

unique and **partitionable** are contradictory — partitioning hands out independent references to pieces of an object, which breaks the single-owner guarantee — so the compiler rejects that combination outright.

The flow analysis behind all this is the key to why it's free. The compiler assigns every **movable** / **unique** variable one of three states — *valid*, *moved*, or *uninitialized* — and tracks how that state flows through assignments, moves, branches, and loops. Using a *moved* variable is a compile error with the exact line of the move in the message. And because the compiler knows which variables are *valid* at each scope exit, it runs destructors **only** on those — a moved-from variable's destructor does not run, so ownership transfer never causes a double-free. All of this happens during compilation and produces no runtime checks, no reference counts, no hidden flags: **ownership safety is zero-cost**. Contrast C++, where `std::move` is a cast that doesn't actually move anything and leaves the source "valid but unspecified" — a fertile source of silent bugs. Polaron makes the state real and checks it.

move is a small family of operations, not just variable-to-variable transfer. It also moves objects **between regions** without logically reallocating them:

```
Car* c2 = move c1 from region staging to region production carrying persistents;
// c1's object now lives in production; staging can be released and c2 survives
```

And **cascade move** transfers an object *and the entire graph it owns* from one region to another — promote a whole subtree from a scratch arena to a permanent one in one statement:

```
cascade move tree from region old to region fresh;    // tree + everything it owns
release region old;                                   // old is freed; tree lives in fresh
```

move also appears **in a signature**, on a parameter or a return type, to say that the method takes or gives ownership — for any type, not only a **movable** one:

```
public method consume(move Connection c) returns void { ... }
public method create() returns move Connection { return move c; }

consume(move c1);    // the call must say `move` too; c1 is finished afterwards
```

The keyword appearing in both places is the point rather than an oversight. Reading the *call* tells you ownership changed hands, without going to look at the declaration — so a `move` parameter passed plainly is rejected, and a method declared `returns move T` must `return move ...`.

In freestanding (kernel) mode the ownership keywords `move`, `movable`, `unique`, `partitionable`, and `into` all remain available — precisely because they are compile-time only and need no managed runtime, which makes them ideal for low-level code that wants ownership safety at zero cost.

5.9 Persistents and `reattach`: state that outlives its object

The last piece of Polaron's memory model turns the usual object lifetime on its head. A **persistent** is a class member whose *lifetime is decoupled from the instance that declares it*. It survives its parent object's destructor, and it can **reattach** to a new object that occupies the same conceptual slot. Think of it as durable state that a class carries across the birth and death of its instances — a cache, a counter, a skeleton that shouldn't be rebuilt every time.

A persistent's identity is bound to a **triple**:

```
(lexical scope, variable name, region)
```

Two variables that differ in *any* of the three coordinates get *independent* persistents; two that match on all three share the same persistent. When a variable with a matching triple is created again after the previous one was deleted, its persistent fields reattach automatically — the new object sees the values that were live before.

```
public class Car {
    public eternal persistent mutable int chassi = 0;
    public constructor Car() {}
}

// Each call builds a fresh Car, bumps chassi, and deletes the Car.
// Because chassi is persistent, it lives outside the object and counts up
// across calls: 1, 2, 3 -- even though the Car is deleted every time.
public static method bump() returns int {
    Car c = new Car() on heap;
    c.chassi = c.chassi + 1;
    int r = c.chassi;
    delete c;           // the Car dies; chassi persists
    return r;
}
```

The persistent even remains accessible *after* the parent is deleted, through the qualified path — it genuinely outlived the object. Polaron's persistents are **in-process**: they live for the duration of the program run, keyed by that triple, and reattach in memory within the same run.

Persistents work in freestanding mode, which the chapter overview once said they did not. A persistent bound to a named variable costs *nothing*: the compiler gives it a private global, so a bare-metal image gains no runtime call and no allocation. That makes them a natural fit for exactly the state a kernel is made of — a counter, a cache, a table that has to survive the object that owns it being torn down and rebuilt.

The one form that needs support is the indexed one (`table[i] = new T()` , §18.5), which is keyed by slot at runtime. Freestanding gets a **fixed** registry rather than the hosted growing one: no allocator is involved, blocks come from one static arena so an address stays stable for the whole run, and exhausting either the slot table or the arena is a panic that names the limit. Raise `POLARON_FS_PERSIST_SLOTS` or `POLARON_FS_PERSIST_ARENA` if a program needs more. A bound you can see is worth more in a kernel than a table that quietly grows.

Because a persistent that nobody ever releases is a memory leak by construction, Polaron enforces a **release obligation at compile time**. A non- `eternal` persistent must have a `release persistent` somewhere in the program, or compilation fails with an error telling you to either release it, mark it `eternal` , or annotate that release is delegated elsewhere:

```
public class Cache {
    public persistent mutable int slot = 0;    // non-eternal: must be released
    public constructor Cache() {}
    public method use() returns void { this.slot = this.slot + 1; }
}

// somewhere in the program:
Cache c = new Cache() on heap;
c.use();
release persistent c.slot;    // discards the accumulated state
delete c;
```

Releasing a persistent **discards what it accumulated**: a later reattach starts from zero, exactly as the first attach of the run did. The block's storage itself is not freed — a stable address across reattach is the whole point of a named persistent — and the runtime reclaims it at shutdown.

The escape hatch is `eternal` : a persistent that is *meant* to live for the whole run (a global config cache, a session-long counter) is marked `eternal persistent` , and then no release is required — the runtime frees it at program shutdown, running its destructor. `eternal` is actually a general lifetime modifier — it applies to regions, threads, channels, and static singletons too, always with the same meaning: *lives for the whole program, no explicit cleanup required*.

Persistents interact with everything else in this chapter. They can be disambiguated by region with `of region` , they follow (or don't) an object across a `move` via the `carrying` / `leaving` / `releasing` qualifiers, and they can be placed in a named region at declaration. They are the most advanced corner of Polaron's memory model — reach for them when you have state whose lifetime honestly doesn't match any single object, and let the default value semantics handle everything else.

Putting it together

Step back and the whole design reads as one coherent stance on memory:

- **Default to values.** Assignment deep-copies; every variable owns its object; nothing aliases by accident. Bugs of shared mutation simply don't arise.
- **Share deliberately.** `T*` and `T&` opt in to sharing, and their presence in the source is the signal that sharing is intended.
- **Let scopes clean up.** Stack objects, `defer`, and `using` reclaim resources automatically and deterministically — on normal exit *and* on exception unwind.
- **Batch what dies together.** Regions turn a graph of allocations into a single arena and a single free, for GC-like ergonomics at a bump allocator's speed.
- **Prove ownership when it matters.** `movable`, `unique`, and `partitionable` push single-owner rules into the type system, enforced by compile-time flow analysis at zero runtime cost.
- **Fail loud, never silent.** Double-free, use-after-free, null deref, and out-of-bounds trap deterministically. There is no undefined behavior to exploit you.
- **Keep durable state where it belongs.** Persistents give state a lifetime independent of any one object, with a compile-time guarantee that it is eventually released.

No garbage collector, no hidden pauses, no mystery about when memory is reclaimed — and yet, in the code you write most of the time, almost no manual bookkeeping. That balance — determinism without drudgery — is the heart of Polaron.

5b. The region binder

The previous chapter is about *what you can write*: `new`, `delete`, destructors, regions, pointers, `move`. This one is about the analysis that reads all of it and decides whether the result can dangle. It runs on every compile, it is **on by default**, and it is the reason Polaron can offer manual memory management without the usual bargain attached.

Its name is both words. What it binds is **regions**: it works out, for every value in the program, which region that value lives in — and then it enforces exactly one rule about references between them.

The rule, complete: a reference may only point at a region that **outlives** its own.

Everything in this chapter is either how a value's region is worked out, what "outlives" means between two of them, or what to write when the answer is "*nothing in this program says*".

5b.1 The idea in one page

A region is a lifetime with a name. Not an inferred interval, and not a runtime object: a place whose end is a point the program already states — the closing brace of a call, a `release region`, the destruction of the object that holds the field.

Polaron's claim is that **the program already contains this information**, and that it does not need to be annotated a second time. Where an object lives is written at its `new`. When a region ends is written at its scope. Who owns a field is written in the destructor that frees it. The region binder's whole job is to read those three things and to check them against each other:

```
public static method make() returns Point* {
    Point* p = new Point(7);    // this call's frame
    return p;                  // ...and the caller reads it after the frame is gone
}
```

```
error[Polaron-1721]: region-binder: returning 'p', which names an object living in this
method's own frame; the frame is gone by the time the caller reads it. Allocate it with
'on heap' so it outlives the call, or return it by value so the caller gets a copy
```

Nothing about that program was annotated. `new Point(7)` said "here", `return` said "there", and "here" does not outlive "there".

The second half of the idea is the one that costs something, and it is worth stating plainly at the top:

A value the analysis cannot place is refused.

Not warned about — refused. A checker that answers "*allowed*" where it means "*I could not tell*" finds bugs but cannot state a guarantee, because every guarantee has a shape it does not recognise, and that shape is where the bug will be. Refusing the unplaceable is what turns the region binder from a bug-finder into a promise.

5b.2 The four regions, and the order between them

Every value lives in exactly one of four kinds of region. They are **totally ordered**, longest-lived first:

Root $\supseteq$ Object $\diamond o$ $\supseteq$ Region R $\supseteq$ Activation $\diamond m$

Region	Written in the source as	Ends when
Root	a static field, a string literal, a fixed address, <code>on heap</code> before anyone takes it	never (or: not while the program runs)
Object $\diamond o$	a field of some object; anything that object owns	that object is destroyed
Region R	<code>new T() in region r</code>	<code>release region r</code> , or its scope closes
Activation $\diamond m$	<code>new T() / new T() on stack</code> — the frame of the call it is written in	the call returns

In a diagnostic these are named the way you think of them, not the way the model writes them: *static storage*, *this object*, *what 'p' belongs to*, *region pen*, *this call*.

Three details of the order do most of the work:

A heap allocation nobody has taken yet is Root. `new T() on heap` produces something that outlives everything, because no one has claimed it. It therefore fits *anywhere* — which is what makes `this.head = new Node(v) on heap`; the ordinary way a class takes ownership rather than an error. What happens to it afterwards is the receiving object's business, and that is read from the receiving object's destructor (§5b.4).

Two different objects are incomparable. Not "unknown, so allowed" — *incomparable*, and that is a final answer rather than a gap:

```
public class Session {
    private mutable Cache* cache;      // whose? and does it outlive a Session?
}
```

Nothing anywhere in the program says whether a `Cache` outlives a `Session` or the reverse, so a pointer from one to the other cannot be shown valid for as long as it is reachable. The program has to say. §5b.6 is the list of ways to say it, and there are six.

Two explicit regions are ordered by birth. Regions are released last-in-first-out — at scope exit, in reverse declaration order — and a region declared inside a block is necessarily born after the region enclosing it. So *born earlier* is exactly *dies later*, and **one number per region** orders an arbitrarily deep nest without modelling the nest at all:

```
region durable = itself.allocate(64 kilobytes);
{
    region scratch = itself.allocate(4 kilobytes);
    Node* tmp = new Node(1) in region scratch;
    Node* keep = new Node(2) in region durable;
    tmp.next = keep;      // fine: durable outlives scratch
    keep.next = tmp;      // error: scratch is released first
}
```

Both directions matter. A rule that refused both would not be an order.

5b.3 Against the alternatives

Three strategies exist for the same problem — *"is this reference still valid?"* — and they differ in **when they answer** and **what the programmer has to write down**.

The garbage collector answers at run time, by not answering

A tracing GC makes the question unaskable: nothing is freed while anything can reach it. That is a real guarantee and it costs nothing to write — no annotation, no discipline, no shape of program ruled out. What it costs is paid elsewhere: a runtime that traces, pauses that arrive when the collector decides rather than when the program does, memory held past its last use, and object graphs that must be laid out the collector's way rather than the cache's.

And it answers only the memory question. *When is this file closed? when is this GPU buffer released? when is this lock dropped?* — none of those are memory, none are handled by tracing, and every managed language grows a second mechanism for them (`using` , `try-with-resources` , `defer`) precisely because the first one does not reach.

Polaron's destructors run at a point the program states, and that is the same mechanism for memory, files, sockets and locks. The region binder is what makes those points safe.

The borrow checker answers at compile time, by tracking aliasing

Rust's borrow checker proves more than the region binder does. It tracks **exclusivity** — one mutable borrow *xor* many shared ones — which yields not only temporal safety but data-race freedom and iterator invalidation as consequences of the same rule.

The price is that lifetimes become part of the *type*. When elision cannot infer them they are written into signatures (`fn longest<'a>(x: &'a str, y: &'a str) → &'a str`), they propagate outward through every caller, and a data structure whose shape is not a tree — a graph, a back-pointer, an observer list — is expressed through `Rc<RefCell<T>>` , arena indices, or `unsafe` . That is not a flaw; it is the cost of proving a stronger statement.

The region binder answers at compile time, by tracking placement

Polaron proves the weaker of the two statements — **temporal** safety, not exclusivity — and buys something specific with the difference: **there are no lifetime annotations in the language**. Not elided ones; none. Look at any Polaron signature in this manual and there is nowhere a lifetime could be written.

That is possible because of the model's first sentence: **regions are inferred from structure the language already has, and the region tree IS the composition tree**. A class contains its fields, so its fields live in its region. A region contains what is allocated in it. A frame contains its locals. Composition is a tree the programmer already drew, and containment is already an order — so the ordering comes for free, out of code that was going to be written anyway.

The one thing that could not be read off the structure is which of two pointers is *ownership*. Polaron reads that off the **destructor** (§5b.4), which the author already had to write, and which says it unambiguously.

Side by side

	Garbage collector	Borrow checker	Region binder
Answers	at run time	at compile time	at compile time
Proves	reachable $\Rightarrow$ alive	temporal safety + exclusivity	temporal safety
You write	nothing	lifetimes in signatures, 'a , sometimes Rc / RefCell	destructors, on heap / in region , occasionally weak
Runtime cost	tracing, pauses, headroom	none	none
Non-memory resources	a separate mechanism	same mechanism (Drop)	same mechanism (destructors)
Graphs & back- pointers	free	Rc<RefCell> / indices / unsafe	weak , or one region for both
Data races	not addressed	prevented by the same rule	a separate rule (ch. 9)
Escape hatch	unsafe blocks in some	unsafe blocks	--no-region-binder , whole program only

Read the last row carefully, because it is a real difference in kind. Rust and C# let you turn the rule off **in one place**. Polaron does not, and the reason is in §5b.8.

What this means for how you write

If you have written Rust, the adjustment is that **you never annotate a lifetime and you often write a destructor instead**. Where Rust asks "how long does this borrow last, relative to that one?", Polaron asks "who frees this?" — and the answer goes in a destructor rather than a signature.

If you have written C++ or C, the adjustment is that **the compiler now insists you answer the question you were already supposed to have answered**. The refusals mostly land on the same lines where a code review would ask "wait, who owns that?".

If you have written Java, C# or Go, the adjustment is real: object graphs must have a direction. The next two sections are the practical part.

5b.4 The destructor is where ownership is declared

A `T*` field can be two completely different things and the type says nothing about which:

```
private mutable ArrayList<Row*>* rows;      // does this class free these rows, or borrow them
```

The destructor says:

```

public class Table {
    private mutable ArrayList<Row*>* rows;
    public destructor ~Table() returns void {
        for (var r in rows.toArray()) { delete r; } // "a Table owns its rows"
        delete rows;                               // "...and the list itself"
    }
}

public class ResultSet {
    private mutable ArrayList<Row*>* rows;
    public destructor ~ResultSet() returns void {
        delete rows;                               // the list only: a VIEW of somebody's
    }
}

```

Two classes, the same field, opposite meanings — and the region binder now treats them oppositely. `table.rows.add(row)` is a **handover** and needs no ordering. `result.rows.add(row)` is a **borrow** and must be proven safe.

Four consequences worth knowing:

Owning a container is not owning what is in it. `~ResultSet` frees a field called `rows`, and so does `~Table`. A check that asked only "does it free `rows`?" could not tell a table from a view — which is the exact bug the whole analysis exists to catch. So two answers come out of a destructor: which **fields** it frees, and whose **contents** it frees. A value landing *in* a field asks the first; a value landing *among* a field's elements asks the second.

Every spelling counts. `delete this.f`, `delete this.f.get(i)`, a loop that deletes the elements, and a `this.clear()` whose body does the deleting — all four say the same thing about the field.

Adding a destructor can turn a program that compiled into one that does not, and adding one is usually the *fix* rather than the problem. It is the sentence that says who frees what, and the refusal was there precisely because nobody had said it.

Ownership crosses calls. The compiler computes, for every method, which of its parameters get stored into the receiver and into which field — an *escape summary* — and carries it through the `.polh` so it survives a bundle boundary. That is why this reports at the call:

```

Row scratch = new Row(1);           // this call's frame
result.keep(scratch);               // error: 'keep' keeps that argument

```

...and why the complaint lands where both objects are named, instead of inside `keep`, on a line that could not be written any other way.

5b.5 Where a value's region comes from

The full inference. Read it as a lookup table for "why did it say that?".

The expression	Its region
<code>new T()</code> , <code>new T()</code> on stack	this call

The expression	Its region
<code>new T()</code> on heap	Root — nobody owns it yet, so it fits anywhere
<code>new T()</code> in region <code>r</code>	region <code>r</code>
<code>new T()</code> where <code>T</code> is a region class	the type's own region — it needs no <code>on heap</code> and no <code>in region</code> , because there is only one place its instances can go
<code>this</code>	this object
a parameter, or a local holding a heap object	that object (the object <i>is</i> the name)
a local declared <code>region r</code>	region <code>r</code>
a field of a class that owns it (<code>recv.f</code>)	the receiver's region
a field the class borrow s	the receiver's region — as a <i>lower bound</i> , see below
<code>a[i]</code>	the array's region
<code>&x</code> , <code>*x</code>	whatever <code>x</code> 's region is — taking an address changes what you hold, not how long the thing lasts
<code>c ? a : b</code>	the shorter-lived of the two arms
<code>null</code>	Root — it points at nothing, so it fits everywhere
a static field, a class name	Root
<code>cast<T*>(0x09000000)</code>	Root — a fixed address was there before the program and will be after
<code>r.at(i)</code> where <code>at</code> returns a field <code>r</code> owns	the receiver's region
a method that returns fresh storage	Root — whoever takes it becomes the owner
any other call	unplaceable — and therefore refused

Two entries deserve their own paragraph.

A borrowed field read as a source is a lower bound, and that is sound. When you read `this.head` out of a class that does not own it, the analysis answers "the holder's region" even though the true owner is elsewhere. That is legitimate because of induction: every store *into* that field was itself checked by this same rule, so whatever is in there already outlives the holder. A source position may be understated — it can refuse a program that was fine, never accept one that was not. Answering "unknown" instead would refuse `node.next = this.head`, which is the second line of every linked list ever written.

A fresh object's region is lowered by what its constructor keeps. `new Parser(tokens)` on heap reads those tokens for as long as it exists, so it is no longer-lived than they are — whatever the allocation says. The `new` itself is never the error, because nothing has gone wrong there yet; the bound travels with the object and the diagnostic lands where it is stored, returned or kept.

5b.6 How to obey it

When a store or a return is refused, there are six answers. They are in the order you should consider them, and the first two cover most cases.

1. Store a copy

Assignment in Polaron is a deep copy, so a value that is copied in has no lifetime relationship to anything. If the data is small — an id, a date, three floats, a record — this is the right answer and it removes the question rather than answering it.

```
private mutable Config settings;      // a copy: no ordering to prove
```

A `record` is the shape built for this: copied on assignment, complete the moment it is built.

2. Own it, and say so in a destructor

If the holder *should* free the thing, write the destructor that frees it. The reference becomes ownership and there is nothing left to prove.

```
public class Panel {  
    private mutable Button* ok;  
    public destructor ~Panel() returns void { delete ok; }    // a Panel owns its button  
}
```

3. Put both in one region

Two objects in the same region are born and released together, so there is no order to establish — the question disappears entirely. This is the answer for graphs, parse trees, and anything where "these all die at once" is true.

```
region ast = itself.allocate(64 kilobytes);  
Node* root = new Node("+") in region ast;  
root.left = new Node("1") in region ast;      // same region: nothing to order
```

`region class` takes this further: every instance of the type comes from one region owned by the type, so `new Node(v)` inside it needs no placement at all.

4. `weak` for a view that must outlive nothing

A `weak T*` is **emptied when its target dies**, so it cannot outlive what it points at — and the binder therefore asks for no ordering on a store into one.

```
private mutable weak nullable Grid* ground;    // the world's; nothing orders the two
```

A weak slot is two pointers wide (the reference, plus the registration that lets the target's `delete` find and clear it). Use it for back-pointers, observers and caches — the shapes that genuinely have no lifetime relationship.

Do not reach for it to silence a diagnostic. If the holder should free the thing, the answer is #2. And note the rule is all-or-nothing: if a value lands in two fields and only one is `weak`, the ordinary one still holds a raw pointer after the target dies, and that is the reference the rule is about.

5. `move` / `unique` for a single owner

`unique` or `movable` on a type says there is exactly one owner, and makes every handover a `move` the compiler follows. Where that is the truth, say it once on the declaration and every call site reads as what it

is.

6. Give it a longer life at the allocation

The remaining case is that the object was simply allocated in the wrong place. `on heap` gives it a lifetime longer than the call and makes the caller responsible for `delete`; a region gives it the region's lifetime.

```
public static method make() returns Point* {  
    return new Point(7) on heap;    // the caller deletes it  
}
```

Decide an object's lifetime **where it is allocated**, not where it is used — the allocation is the one place that has to know.

5b.7 The refusals, one at a time

Five diagnostics come out of this analysis. `polaron explain <code>` prints the full write-up for any of them.

Polaron-1721 — this hands out a pointer to storage that is about to disappear

The frame-escape case. Something living in this call's frame is returned, or stored somewhere that outlives the call.

```
Node n = new Node(1);  
holder.kept = n;    // 1721: the frame is freed first
```

Fix: #6 (allocate it where it should live), or #1 (store a copy).

Polaron-1722 — nothing in the program says which of these two dies first

The incomparable case. Both values live as long as *some* object, and nothing anywhere states which object outlives the other.

```
region-binder: nothing orders this object against what 'cache' belongs to, so storing  
that reference in field 'cache' cannot be proven safe: neither is known to outlive the  
other. Store a copy, or have one of them own the value outright.
```

This is not a gap in the analysis; it is a gap in the program. **Fix:** #2 (own it), #1 (copy),

3 (one region), or #4 (weak) — in that order of preference.

Polaron-1723 — past this point there is no proof to be had

A pointer is being handed to an `extern` function. There is no Polaron body to read, so nothing in this program can say whether the foreign side keeps it.

Fix: declare the parameter as `address`. That is a raw machine word, which is exactly what a pointer crossing into C is, and the refusal goes away because the *claim* goes away — the declaration now states where this program's proof stops.

Polaron-1724 — what this holds was freed by an earlier call

A value built out of another object's contents, read after those contents were freed. This is use-after-free spread over three statements, which is why it survives review: building the view is correct, emptying the source is correct, reading the view is correct, and only their **order** is wrong.

```
watcher.watch(subject); // watcher now holds a reference into subject
delete subject;         // ...and that is what it was pointing at
watcher.read();         // 1724: use of 'watcher' after 'subject' was emptied
```

What establishes a borrow. Six shapes, and they were closed one at a time, each by writing the wrong program and finding that the analysis said nothing:

shape	example
a call that keeps an argument	<code>watcher.watch(subject)</code>
a method that returns a borrow of a parameter	<code>var view = Query.over(table);</code>
a method that returns a borrow of its receiver	<code>byte[] raw = owner.record();</code>
...including through one call , or out of a field whose contents the class owns	<code>return this.items.get(i);</code>
a field path read directly	<code>Leaf* deep = tree.branch.leaf;</code>
an alias , of an object or of a borrow	<code>Box* q = p;</code>

The receiver half is the one that matters most in practice, because an accessor is the commonest method a class has. It was also the last to be written: for a long time the analysis knew that `view.of(table)` kept a borrow of its ARGUMENT and did not know that `owner.record()` handed back a borrow of the OWNER. A compound-file reader kept a directory record past the file that owned it exactly that way — read the record, delete the file, lay out a new one from what was read — and the program ran for months before a value type of a different size passed through the same allocator and turned it into a crash two frames away.

What breaks a borrow. `delete` — including `delete this`, which frees the one name every other line in the method reads. Any method that frees the **contents** of one of its own fields: a `clear()` that deletes each element invalidates every borrow taken out of that object, which is `DELETE FROM people` arriving while an earlier `SELECT` is still holding the rows. Any method that frees **the field itself** and puts a fresh value there — `reset()` leaves the object perfectly healthy and everybody holding a reference out of it holding the old one. And `release region`, which is the same sentence with a different keyword.

Freeing a borrow is the same fact, used the other way round. `b` is `o`'s, `o`'s destructor frees it, so `delete b` frees it twice — a double free written as ordinary cleanup, with nothing above it for the author to notice and the second free happening in a destructor, possibly in another file. Refused with the same code. It applies to what a **call handed back**, and deliberately not to the other two things the table above

records: an **alias** is a second name for what this frame owns (deleting through either name is right; deleting through both is the plain use-after-free rule), and a **field path** read out of an object is half of grow-and-replace — `oldK = this.keys`, `this.keys = new K[ ... ]`, `delete oldK` — where the object handed ownership over in between. Both are in the standard library, fourteen times between them, and both are correct.

A lambda is a body that runs somewhere else. It captures by reference, so what it holds is what the enclosing frame holds — and it is analysed where it is *written*, with the state that holds there, where everything is always still alive. Only the call site knows what has been freed since, so the body is analysed **at the call**, with the state that holds at the call. The call above a `delete` stays silent and the one below it does not, which is the whole difference between this and a rule about lambdas capturing pointers.

Two owners for one object is the one shape reported as advice rather than refused (0B53): `this.buf = other.buf`, where both classes free that field in their destructors, makes two owners of one object, and whichever is destroyed second frees what the first already freed. It stays advice only because the cure — `this.buf = move other.buf` — is refused today for a non-partitionable class, and a compiler that refuses a shape has to leave a way to write it.

A borrow taken on any path is taken, and so is a `delete`. Obligations survive control flow the way `moved`, `deleted` and `freed` do — taken in one arm of an `if`, or inside a loop body, they are live afterwards. Both joins used to lose them: a branch kept a borrow only where both arms agreed, so one `if` walked out of the rule, and a loop restored the state from before its body, so one `for` walked out of the plain use-after-free rule as well. A loop may run zero times, so a PROOF made inside it does not survive; an obligation is the other way round, because the body can run.

...and it is taken again on the next iteration. A loop body used to be analysed once, with the state that holds at the TOP — so a body that frees at the bottom and reads at the top was checked in the only order where it is innocent. The body is now replayed with the state the previous iteration left, which is what a second iteration *is*: no new rule, and every obligation the analysis tracks crosses the back edge, including the ones added after this was written. A path that frees and then LEAVES (`if (done) { delete this; return; }`) never reaches the end of an iteration, so it carries nothing back and stays silent — which is why this is a replay of the flow rather than a search for a shape.

A copy is not a borrow. `String name()` reading a field still gives the caller a copy — assignment copies in this language, and `T*`, `T&` and `T[]` are how you ask to share instead. The rule reads the shape of what is handed back, so an accessor returning a value is not in it.

Fix: read before the mutation, or hold copies (`record`) so emptying the source leaves the result intact. `Slice<T>` is the opposite choice said out loud — a window that does not own what it looks at, fine as long as it is read before the owner moves on.

Polaron-0803 and friends — regions used incorrectly

The region-specific rules: releasing a parent before the sub-region carved out of it, using a value after `release region`, a `region` field with no release in the destructor, `mark / rollback` on the wrong flavor, `extract` of an object whose field is still in the region.

use of 'tmp' after 'release region scratch': it lives in that region, so its storage is gone. Read it before the release, or copy what is needed out of the region first

5b.8 Reading a refusal you disagree with

The first move is to read *which two regions* the sentence names — it always names both, and the fix follows from the pair. "this call outlives what 'p' belongs to" is a different problem from "nothing orders this object against what 'cache' belongs to".

When the complaint is that a call *keeps* an argument, the question turns on **which field** it landed in and whether that field is owned. That is printable:

```
POLARON_TRACE_KEEPS=1 polaron build  
keeps: ResultSet.keep#0 → [rows]
```

Now the argument is concrete: `keep` puts it in `rows`, and `~ResultSet` does not free the elements of `rows`, so it is a borrow. Either the destructor is wrong or the call is.

Two shapes pass that look like they should not, and knowing them saves time:

- **Storing what the caller handed you.** `Parser(tokens) { this.tokens = tokens; }` is not checked inside the constructor — the answer is knowable only where both objects are named, which is at the `new`. The obligation is not dropped, it *moves* to every call site.
- **A second pointer to what this object already owns.** `this.first = v; this.last = v;` is the head-and-tail of every linked structure; the first store made this object the owner and the second names the same node again.

Finally, the escape hatch. `--no-region-binder` turns the analysis off **for a whole program**, and that is the only choice on offer:

```
polc program.pol --no-region-binder
```

There is deliberately no per-line version, and the reason is not severity — it is that the analysis divides a program into regions, and declaring that some part of it belongs to none makes every proof about the *rest* unavailable, because those proofs are relative to a partition that no longer covers the program. A partial guarantee here is not a weaker guarantee; it is not one. So the choice offered is not *more safety or less*, but **"cannot write this bug" against "can write it anywhere"** — and it is visible in the build rather than buried on a line that looks ordinary.

5b.9 A worked example: the result set that outlived its rows

This is the program that motivated the analysis, reduced to its bones. A table owns rows; a query returns a result set holding some of them; a later statement deletes rows from the table.

```

public class Table {
    private mutable ArrayList<Row*>* rows;
    public destructor ~Table() returns void {
        for (var r in rows.toArray()) { delete r; }
        delete rows;
    }
    public method select(int col) returns ResultSet* { ... }
}

public class ResultSet {
    private mutable ArrayList<Row*>* rows;           // rows the TABLE owns
    public method keep(Row* r) returns void { rows.add(r); }
    public destructor ~ResultSet() returns void { delete rows; }
}

```

Every line is individually correct. Run it and the second statement prints the first statement's data, or garbage, depending on what the allocator did in between — the classic shape, and one that survives review because there is no wrong line to point at.

The region binder points at `keep`:

```

error[Polaron-1722]: region-binder: 'keep' keeps that argument, and nothing orders what
'result' belongs to against what 'table' belongs to: neither is known to outlive the
other, so this reference cannot be proven safe. Store a copy, or have one of them own
the value outright.

```

Three fixes, and the choice is a design decision the program had been avoiding:

1. **The result set owns its rows** — copy each row in, free them in `~ResultSet`. Correct when results outlive statements.
2. **The result set is explicitly a window** — hold a `Slice<Row*>`, and read it before the table changes. Correct when results are consumed immediately, and now says so.
3. **Both live in one region** — a query region released when the statement ends. Correct when the whole engine is phase-structured.

The analysis does not choose. It refuses to let the question go unanswered.

5b.9b Regions are part of the analysis, not a thing you remember to use

A region is the answer to a shape the compiler can see, so the compiler is the one that should mention it. Three ways it takes part:

It names the flavour. A refusal that says "region `r`" hands the reader a name and keeps the fact. Which kind of region it is decides what to do about the value: a `bump` region gives everything back at once when it is released, a `pool` hands a slot back the moment an element is freed, a `ring` overwrites its oldest, and a `fixedslot` refuses rather than growing. So the diagnostics say *the bump region 'r'*, and the advice differs accordingly — a pool or a ring can leave a reference stale with nothing released at all, and "release it later" is no answer there.

It asks for one. Polaron-0B52 fires on a method that allocates and frees three or more things by hand:

```
warning[Polaron-0B52]: this method allocates and frees 4 things by hand, which is a bump
region written out longhand
```

One `new ... on heap` with its `delete` in the same block is a stack object that took the long way, and Polaron-0B1A says so. Three of them is a different fact: the method is keeping a manual ledger, every entry has to stay in step with its partner, and every exit anybody adds later has to remember all of them. A region does not make that safer — it makes the safety unconditional. The flavour in the message is chosen from what the method does: pairs inside a loop want storage that comes back each turn, which is a `pool`; pairs freed once at the end want the block released at once, which is a `bump`.

It refuses the misuse. A value from a region outliving its release, a region-allocated value stored where the region does not reach, `mark / rollback` on a flavour that has no such thing, `extract` of an object whose field is still inside — all errors, not advice. A region is a lifetime with a name, and the whole point of naming it is that the name can be checked.

5b.9c `surveyed` : the one escape, and why it is not an `unsafe`

Everything above rests on the analysis covering the whole program. That is also why the only escape this language had — `--no-region-binder`, which turns it off everywhere — was never really an escape: a partition that stops covering part of the program makes every proof about the rest unavailable.

`surveyed` is the escape that keeps the cover.

```
extern cdecl static method csDecrypt(byte* buf, int len) returns int;

public surveyed method decrypt(byte[] into, int count) returns boolean {
    return Cslibu.csDecrypt(&into[0], count) == 0;
}
```

Inside, exactly one refusal is suspended: Polaron-1723, the one about a value the analysis cannot *place*. That is the refusal a foreign call earns — nothing in this program can say whether a C function keeps the pointer it was handed — and it is the only one that has to go. The other three stay: if the analysis can still prove that a line is wrong, proving it is right whether or not the method is marked. A `surveyed` method that frees an object and then reads a borrow of it is refused like any other.

Outside, the boundary goes to the worst case. The word carries no content, so the compiler cannot know what the method lends; the only sound reading of "*I cannot tell you what this does*" is that it does everything it could. Every reference parameter is treated as kept, and a reference result as a borrow of everything in reach:

```
error[Polaron-1721]: 'decrypt' keeps that argument, and what 'file' belongs to outlives
this call, so what it keeps is freed first
```

That inversion is the whole design:

The freedom is local. The suspicion is exported.

An `unsafe` block frees its own body and tells the callers nothing. `surveyed` frees its own body and makes every caller stricter — so marking a method has a price, paid by the code that uses it, and nobody reaches for the word to make a diagnostic go away. It also explains the name: `unsafe` says *this may be rotten*; `surveyed` says *this ground was walked and written down*.

It is inherited. An override may only be `surveyed` if the method it overrides is. The binder already unions an override's summary into its base — a call through a base reference is checked against every override that could run — so a surveyed override does make the base conservative, soundly. What it does not do is let a reader see it: `Sink.take` would look like an ordinary method and be conservative because a subclass in another file said so. A conservative summary nobody can see is the thing the word exists to prevent.

Not on a destructor. §5b.4 says ownership is declared in the destructor, and the analysis reads what a class owns out of it. A surveyed destructor would erase the language's own source of truth about who frees what — not a risk to be weighed, a sentence with no meaning.

Not on an `extern` either. It has no body, so there is nothing to not-derive, and marking it would make every call site conservative with no line of Polaron to point at. Write the wrapper: the foreign boundary gets a name, a file and a line.

`--no-region-binder` stays. Whoever wants no guarantee at all keeps the whole-program switch. What changes is that it is no longer the only door.

5b.10 What it does not do

Stated plainly, because a guarantee is only as good as its boundary.

- **It does not prove exclusivity.** Two live references to the same mutable object are fine here. That is the borrow checker's stronger claim, and it is not made. The one place it would have been needed and is not — changing a collection while walking it — is answered directly instead: the collection is named in the loop header, and a method that writes a field of its own receiver is one the walk cannot survive.
- **It does not prevent data races.** Concurrency safety is a separate rule with its own mechanisms — `atomic<T>`, `Mutex<T>`, `Channel<T>`, and a check on what a thread closure captures (ch. 9). It arrives by decision, not as a side effect.
- **It does not track values through arbitrary calls.** A call result is placed when the callee is an accessor returning something the receiver holds, when it hands back a borrow of a parameter, when it reads an element out of a field whose contents the class owns, or when it is known to return fresh storage. Anything else is unplaceable, and unplaceable is refused. The fix is usually to name the value at its source.
- **It does not free anything.** It is entirely a compile-time analysis; `delete`, destructors and `release` do the freeing, and the runtime's own liveness check (§5.4) catches what gets past.

What it cost to turn on is worth recording, since "refuse the unplaceable" sounds expensive: the number of sites needing a change went 35, then 23, then 18, then 10, then 0 — across the 20,000-line standard library, 799 sample programs and a 27-file SQL engine. Every step that removed sites taught the analysis to place another real shape; none of them relaxed a rule. The number was never a count of dangerous code. It was a count of the analysis's own silence.

See also: §5.4 (the runtime half — deterministic traps), §5.7 (regions in full), §5.8 (movable / unique / partitionable), §17.2 and §17.3 (destructors and weak at the seams), ch. 9 (concurrency), §11 (address and the FFI boundary).

6. Object-Oriented Programming

Polaron is an object-oriented language in the strict sense: object orientation is not a style you may adopt but the only way to organize code. There are no free functions, no top-level variables, and no bare statements floating outside a type. Every piece of behavior you write is a method that belongs to a class, and every piece of state is a field of some object or a `static` member of some class. This chapter explains how classes are built, how they relate to one another through inheritance and interfaces, how method calls are resolved at runtime, and how the finishing touches — properties, operator overloading, and enums — round out the model.

If you come from Java or C#, most of this will feel familiar, but Polaron makes several deliberate choices that a newcomer must internalize early: assignment copies by default (so class variables behave like values, not references), member access always goes through `this.`, mutability is opt-in rather than the default, and overriding an inherited member is never accidental — you must say `override`.

6.1 Everything Lives in a Class

A Polaron program is a nested hierarchy: a `program` declaration names the compilation target, a `bundle` groups related code, a `namespace` carves out a naming scope, and classes live inside namespaces. Execution begins in a `static` method named `main` that takes a `string[]` and returns `void`. Because there is nowhere else to put code, even the smallest program is a class:

```
import System.IO.Console;
program Hello;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                System.IO.Console.println("hello, world");
                return;
            }
        }
    }
}
```

The `import` line brings the `Console` facilities into scope; the standard library always requires an explicit import. Notice that `main` is `static`: it runs without an instance, because at program start no objects exist yet. From `main` you construct the first objects and the program grows outward from there.

6.2 Fields, Constructors, and Destructors

A class bundles together the data it owns (its **fields**) with the operations that act on that data (its **methods**). Fields declare the shape of every instance. Each field carries an explicit visibility keyword — `public`, `private`, `protected`, or `internal` — because Polaron never infers access silently. A field is immutable unless you mark it `mutable`; this is the reverse of most languages and it means that a field you never reassign needs no annotation, while one you do reassign announces that fact.

Fields can be given a value in two places: inline at the point of declaration, or inside a **constructor**. A constructor is introduced by the `constructor` keyword followed by the class name and a parameter list; it runs once, when an instance is created, and its job is to leave every field in a valid state. Inside any method or constructor, access to a member is always written through `this.` — there is no implicit `this`, so `this.balance` is a field access and `balance` alone would be an unknown name.

```
public class Account {
    private mutable int balance;
    private int id = 1000;          // inline initializer

    public constructor Account(int opening) {
        this.balance = opening;    // reassignable: declared mutable
    }

    public method deposit(int amount) returns void {
        this.balance = this.balance + amount;
        return;
    }

    public method getBalance() returns int {
        return this.balance;
    }
}
```

A class has **exactly one** constructor, for the same reason it has one method per name: Polaron has no overloading. Where another language would offer several, give the alternatives distinct names as static factory methods, or take one constructor with the widest parameter list.

The mirror image of a constructor is a **destructor**, written `destructor ~Name() returns void`. It runs when an object's lifetime ends and is where you release whatever the object owns — heap memory, file handles, and the like. Polaron has no garbage collector; memory is managed manually, and the destructor is the hook that makes manual management ergonomic. For an object that lives on the stack, the destructor fires automatically when the enclosing scope exits (this is RAII, the same discipline C++ uses); for an object on the heap you trigger it with `delete`. The following program prints `ctor`, then `hello`, then `dtor`, in that order, because the stack object `g` is destroyed as `main` returns:


```

public class Greeter {
    public constructor Greeter() {
        System.IO.Console.printf("ctor\n");
    }
    public method hello() returns void {
        System.IO.Console.printf("hello\n");
        return;
    }
    public destructor ~Greeter() returns void {
        System.IO.Console.printf("dctor\n");
    }
}

public class Main {
    public static method main(string[] args) returns void {
        Greeter g = new Greeter() on stack;
        g.hello();
        return;                // ~Greeter() runs here
    }
}

```

An object is created with `new`, optionally followed by a placement clause: `on stack` puts it in the current frame, and `on heap` allocates it (and requires a matching `delete`). The placement is optional and defaults sensibly — a plain object defaults to the stack — but spelling it out is good style while you are learning where each object lives.

6.3 Methods and `this`

Instance methods are declared with the `method` keyword, a unique name, a parameter list, and a mandatory `returns` clause naming the result type (`void` if there is none). The `return` statement is required in a non-void method and optional in a void one.

Polaron forbids method overloading. Within a class, a method name is unique — you cannot declare two methods called `add` that differ only in their parameters. This keeps dispatch unambiguous and error messages precise; when you need variants, give them different names (`addInt`, `addAll`) rather than relying on the compiler to pick by signature. Because names are unique, every call site names exactly one method.

Every block requires braces and there is no shorthand: `if (x) return;` written without braces is a syntax error. This uniformity means a method body always reads as a sequence of brace-delimited statements.

```

public class Rectangle {
    private int width;
    private int height;

    public constructor Rectangle(int w, int h) {
        this.width = w;
        this.height = h;
    }

    public method area() returns int {
        return this.width * this.height;
    }

    public method describe() returns void {
        System.IO.Console.WriteLine($"rectangle {this.width}x{this.height} = {this.area()}");
        return;
    }
}

```

Note how `describe` calls `this.area()`: one method invoking another on the same object still goes through `this.`. The `" ... "` form is string interpolation, and `{ ... }` splices an expression's value into the output.

6.4 Static Members

Some state and behavior belong to a class as a whole rather than to any single instance. Marking a field or method `static` lifts it to the class level. A static field is shared by every instance and exists for the whole run of the program; a static method runs without a receiver. Both are addressed by class name — `Counter.count`, `Counter.inc()` — never through an instance. Static state that changes over time must still be declared `mutable`, exactly like an instance field.

```

public class Counter {
    private static mutable int count;

    public static method inc() returns void {
        Counter.count = Counter.count + 1;
    }
    public static method get() returns int {
        return Counter.count;
    }
}

public class Main {
    public static method main(string[] args) returns void {
        Counter.inc();
        Counter.inc();
        Counter.inc();
        System.IO.Console.WriteLine($"count = {Counter.get()}");    // count = 3
        return;
    }
}

```

6.5 Value Semantics: Assignment Copies

This is the single most important rule to absorb about Polaron objects, and the one most likely to surprise a Java programmer: **assignment is a deep copy**. When you write `Account b = a;`, `b` becomes an independent duplicate of `a`, recursively copying every field. Mutating `b` afterwards leaves `a` untouched. Passing a class value as a method argument follows the same rule — the parameter is a fresh deep copy, so a method cannot change the caller's object merely by mutating its parameter.

```
Account a = new Account(100) on stack;
Account b = a;           // deep copy: b is a separate account
b.deposit(50);
// a.getBalance() is still 100; b.getBalance() is 150
```

When you genuinely want two names to refer to the *same* object — the sharing semantics that references give you in Java — you ask for it explicitly with a pointer (`T*`) or a reference (`T&`). These are the tools for sharing and for avoiding the cost of copying a large object.

There are principled exceptions to the copy rule, and they all share one justification: copying would be either meaningless or unsafe. Interface-typed and abstract-class-typed values are always references, because such a value is necessarily some concrete subclass whose true size is not known at the declaration point — copying it by the base's own (often smaller) layout would truncate the object. `String` is immutable, so sharing its buffer is observationally identical to copying it. And a Java-style enum constant is a shared singleton whose identity must be preserved. For everything else — plain classes, structs, records — assignment means copy.

6.6 Inheritance with `extends`

A class may extend exactly one base class using `extends`. Polaron has single inheritance; a class inherits all of its base's fields and methods and may add its own. The `protected` visibility exists precisely for this relationship: a `protected` member is visible to the declaring class and its subclasses but not to unrelated code.

When a subclass is constructed, the base class's constructor runs first so that inherited fields are initialized before the subclass touches them. If the base has a no-argument constructor, this happens implicitly — you write nothing. When the base constructor needs arguments, you forward them explicitly with a `super( ... )` call as the first action of the subclass constructor:

```

public class Animal {
    protected int legs;
    public constructor Animal(int legs) {
        this.legs = legs;
    }
    public method legCount() returns int {
        return this.legs;
    }
}

public class Dog extends Animal {
    private int barks;
    public constructor Dog(int barks) {
        super(4);           // forward 4 to Animal's constructor
        this.barks = barks;
    }
    public method describe() returns void {
        System.IO.Console.println($"legs = {this.legCount()} barks = {this.barks}");
    }
}

```

Here `Dog` inherits both the `legs` field and the `legCount()` method; `super(4)` tells `Animal` how to initialize its part of the object, and the subclass then initializes its own `barks`. If `Animal` had a parameterless constructor, `Dog` could omit `super` entirely and the base constructor would still run.

6.7 Interfaces

An interface describes a capability as a set of method signatures without committing to an implementation. A class declares that it provides that capability with `implements`, and unlike class inheritance a class may implement several interfaces at once. Interface methods are abstract by default — just a signature terminated by a semicolon — and the implementing class supplies the body, marking it `override` because it is fulfilling an inherited obligation.

```

public interface Drawable {
    method describe() returns void;
}

public class Sprite implements Drawable {
    public override method describe() returns void {
        System.IO.Console.println("a sprite");
        return;
    }
}

```

An interface method may also carry a body, in which case it is a **default method**. Classes that implement the interface inherit the default automatically and may override it if they want different behavior. A default method can call the interface's other (still-abstract) methods on `this`, and those calls dispatch virtually to whatever the concrete class provides — so a default written once adapts to every implementer:

```

public interface Greeter {
    method name() returns int;
    method greet() returns int {
        return this.name() + 100;    // default, built from the abstract name()
    }
}

public class Alice implements Greeter {
    public override method name() returns int { return 1; }
    // inherits the default greet() → returns 101
}

public class Bob implements Greeter {
    public override method name() returns int { return 2; }
    public override method greet() returns int { return 999; }    // overrides the default
}

```

Because an interface can never be instantiated on its own — any value of interface type is really an instance of some concrete class — interface-typed values are always references, never copied. The same is true of abstract classes, discussed next.

6.7b `partial` classes and `deprecated` members

`partial` lets one class be declared in several parts, which may live in different files. The compiler folds them into one class before anything else looks at it, so a part can use a field a different part declared:

```

public partial class Dog {
    private mutable int barks;
    public constructor Dog() { this.barks = 0; }
    public method bark() returns void { this.barks = this.barks + 1; }
}

// Elsewhere – another part of the same class, possibly another file.
public partial class Dog {
    public method count() returns int { return this.barks; }
}

```

Its use is separating generated code from hand-written code, and splitting a large type along a seam that means something. It is not a licence to scatter a class across a project: every part must carry `partial`, and the fold happens before analysis, so nothing about the resulting class differs from one written in a single piece.

`deprecated` marks a declaration as on its way out. Every call site gets a compiler warning and the code still runs:

```

public deprecated static method oldWay(int x) returns int { return x * 2; }

```

That is the whole feature, and its value is the migration it makes possible: the replacement lands, the old name keeps working, and every caller is told — by the compiler, at the exact line — without anything breaking on the day of the change.

6.8 Abstract, Final, and Sealed

The `abstract` modifier expresses "incomplete on purpose." An abstract class cannot be instantiated; it exists to be extended. An abstract method has no body — only a signature — and forces every concrete subclass to supply one. A class with any abstract method must itself be abstract. This lets you write a base type that defines a contract (and possibly some shared, concrete helpers) while leaving the essential behavior to be filled in below.

```
public abstract class Shape {
    public abstract method area() returns int;    // no body; subclasses must provide it
}
```

At the opposite end, `final` closes a door. A `final` method cannot be overridden by any subclass, and a `final` class cannot be extended at all. Use it when a piece of behavior must not be redefined, or when a class is a leaf by design.

`sealed` sits between open and closed: `sealed class Shape permits Circle, Square` allows exactly the listed classes to extend `Shape` and no others. A sealed hierarchy is a closed, known set of subtypes, which is what lets the compiler check a `match` over it for exhaustiveness — if you have handled every permitted case, no `default` arm is required.

```
public sealed abstract class Shape permits Circle, Square {
    public abstract method area() returns int;
}
public class Circle extends Shape {
    private int r;
    public constructor Circle(int r) { this.r = r; }
    public override method area() returns int { return this.r * this.r * 3; }
}
public class Square extends Shape {
    private int side;
    public constructor Square(int side) { this.side = side; }
    public override method area() returns int { return this.side * this.side; }
}
```

6.9 Virtual Dispatch and Polymorphism

Polymorphism is the payoff of the whole model: a variable typed as a base class or an interface can hold any subtype, and a method call on it runs the *most-derived* override for the object's real runtime type. Polaron implements this the classic way, with virtual method tables (vtables). A class that participates in a hierarchy carries a hidden vtable pointer as its first field; each concrete class has one vtable, laid out so that every distinct virtual method name occupies a stable slot. A call through a base-typed variable loads the function pointer from that slot and calls it indirectly, so the code that runs is chosen by the object, not by the declared type of the variable.

Overriding is explicit and checked: you write `override` on the subclass method, and the compiler verifies that a matching method really exists to override. Forgetting `override` when you meant to override, or writing it when nothing matches, is an error rather than a silent new method.

The compiler devirtualizes when it safely can. If a receiver's concrete class is known to have no subclasses at this point in the program, the call is compiled as a direct call with no vtable indirection — you get polymorphism's flexibility where you need it and a plain call where you don't.

```
public class Animal {
    public method speak() returns void {
        System.IO.Console.println(" ... generic animal");
        return;
    }
}
public class Dog extends Animal {
    public override method speak() returns void {
        System.IO.Console.println("woof");
        return;
    }
}
public class Cat extends Animal {
    public override method speak() returns void {
        System.IO.Console.println("meow");
        return;
    }
}

public class Main {
    public static method main(string[] args) returns void {
        Animal a = new Dog();    // static type Animal, runtime type Dog
        a.speak();                // → woof
        Animal b = new Cat();
        b.speak();                // → meow
        return;
    }
}
```

The canonical shape example ties inheritance, abstraction, interfaces, and virtual dispatch together. `Shape` is an abstract class that implements the `Drawable` interface; `Square` and `Circle` each override `area()` and `describe()`. A variable of static type `Shape` calls the override belonging to the object it actually holds:

```

public interface Drawable {
    method describe() returns void;
}
public abstract class Shape implements Drawable {
    public abstract method area() returns int;
}
public class Square extends Shape {
    private int side;
    public constructor Square(int s) { this.side = s; }
    public override method area() returns int { return this.side * this.side; }
    public override method describe() returns void {
        System.IO.Console.WriteLine($"Square area: {this.area()}");
        return;
    }
}
public class Circle extends Shape {
    private int radius;
    public constructor Circle(int r) { this.radius = r; }
    public override method area() returns int { return 3 * this.radius * this.radius; }
    public override method describe() returns void {
        System.IO.Console.WriteLine($"Circle area: {this.area()}");
        return;
    }
}

public class Main {
    public static method main(string[] args) returns void {
        Shape s1 = new Square(4);    // prints Square area: 16
        Shape s2 = new Circle(2);    // prints Circle area: 12
        s1.describe();
        s2.describe();
        return;
    }
}

```

6.10 Properties

A property is a field-like member whose reads and writes are mediated by code. To the outside it looks like a field — `r.area`, `a.balance = 50` — but under the hood a getter and/or setter runs. Properties use the soft keywords `get`, `set`, and `init` inside a brace block on a typed member.

The simplest form is an **auto-property**, where you write only `get;` and `set;` and the compiler synthesizes the backing storage. A property with `get;` `set;` is a readable, writable field; one with `get;` `init;` is **init-only** — it may be assigned inside the constructor but is immutable thereafter. A **computed** property provides a getter body and no setter, deriving its value on each read rather than storing it:


```

public class Rect {
    public int w { get; set; }           // auto-property: read/write
    public int h { get; init; }         // init-only: set in the ctor, then immutable
    public constructor Rect(int w, int h) {
        this.w = w;
        this.h = h;
    }
    public int area { get { return this.w * this.h; } } // computed, get-only
}

// r.area is recomputed on each read; r.w = 10 goes through the setter

```

You can also write full custom bodies for both accessors, backing them with a private field and running logic on the way in or out. Inside a setter, the incoming value is available as `value`. This is how you validate, clamp, or transform:

```

public class Temp {
    private mutable int celsius;
    public constructor Temp() { this.celsius = 0; }
    public int fahrenheit {
        get { return this.celsius * 9 / 5 + 32; }
        set { this.celsius = (value - 32) * 5 / 9; }
    }
}

// t.fahrenheit = 212 stores celsius = 100; reading t.fahrenheit yields 212

```

Properties are first-class members of the dispatch model. When a subclass overrides a property, **both** the getter and the setter dispatch virtually, exactly as overridden methods do: reading or writing the property through a base-typed variable runs the most-derived accessor for the object's real type. A property is not a second-class citizen that quietly bypasses polymorphism — it is a method pair wearing a field's clothes.

Bidirectional properties

The getter/setter pair above converts *one way each*. When a property is really two representations of the same datum — Celsius/Fahrenheit, radians/degrees, bytes/kilobytes — a **bidirectional** property expresses both conversions over a backing field, as a pair of `X to Y:` rules. Reading computes the property from the field; assigning computes the field from the property:

```

public class Temperature {
    private mutable double celsius;
    public constructor Temperature(double c) { this.celsius = c; }

    public bidirectional double fahrenheit {
        celsius to fahrenheit: celsius * 9.0 / 5.0 + 32.0;
        fahrenheit to celsius: (fahrenheit - 32.0) * 5.0 / 9.0;
    }
}

Temperature* t = new Temperature(100.0) on heap;
double f = t.fahrenheit; // reads celsius, converts → 212.0
t.fahrenheit = 32.0;     // converts and stores → celsius = 0.0

```

Each rule names the source and destination and gives the expression; the backing field and the property name are both in scope inside the rules, so the two directions read as the plain formulas they are.

6.11 Operator Overloading

A class can give meaning to the built-in operators when applied to its instances. You declare an operator with the `operator` keyword followed by the operator symbol, a parameter list, and a `returns` type. The most common case is a binary operator such as `+`: `a + b` where `a`'s class defines `operator +` compiles to a call of that operator with `b` as the argument.

```
public class Vec3 {
    public mutable int x;
    public mutable int y;
    public mutable int z;
    public constructor Vec3(int x, int y, int z) {
        this.x = x; this.y = y; this.z = z;
    }
    public operator + (Vec3 other) returns Vec3 {
        return new Vec3(this.x + other.x, this.y + other.y, this.z + other.z) on heap;
    }
}
// Vec3 c = a + b;    →    a.operator+(b)
```

Beyond binary arithmetic and comparison operators, Polaron also lets you overload:

- **Unary increment/decrement**, `operator ++ ()` and `operator -- ()`, taking no parameters. `c++` reassigns `c` to the operator's result.
- **Indexing for reads**, `operator [] (int i)` returns `T`, so `t[i]` calls it.
- **Indexing for writes**, `operator []= (int i, T v)` returns `void`, so `t[i] = v` routes the assigned value through it.

```
public class Pair {
    private mutable int a;
    private mutable int b;
    public constructor Pair() { this.a = 0; this.b = 0; }
    public operator [] (int i) returns int { return i == 0 ? this.a : this.b; }
    public operator []= (int i, int v) returns void {
        if (i == 0) { this.a = v; } else { this.b = v; }
    }
}
// p[0] = 10;    →    operator[]= ;    p[0]    →    operator[]
```

6.12 Enums

An enum names a fixed set of constants. In its simplest form an enum is a list of names, and each name is a distinct ordinal value starting at zero. You refer to a constant as `EnumName.CONSTANT`, compare enum values with `==` and `≠`, and use them in interpolation, where a simple enum prints as its ordinal.

```
public enum Color {
    RED,          // 0
    GREEN,        // 1
    BLUE          // 2
}
// Color.GREEN compares and prints as 1
```

Every enum automatically gains a small set of built-in members: `EnumName.count()` returns how many constants there are, `EnumName.values()` returns an array of all constants in declaration order (handy to iterate with `foreach`), `EnumName.random()` returns an arbitrary constant, and `EnumName.parse(name)` returns an `Option` of the constant matching a name.

```
public enum Color { RED, GREEN, BLUE }

public class Main {
    public static method main(string[] args) returns void {
        System.IO.Console.println($"count = {Color.count()}"); // 3
        mutable int sum = 0;
        for (int c in Color.values()) {
            sum = sum + c; // 0 + 1 + 2
        }
        System.IO.Console.println($"sum = {sum}"); // 3
        return;
    }
}
```

Enums can be far richer than a list of ordinals. In the **Java-style** form, each constant carries constructor arguments, and the enum declares fields, a constructor, and methods just like a class. Every constant is a single shared singleton instance, constructed with its arguments — because the constants are shared by identity, they are reference values and are never copied. This lets an enum bundle data and behavior with each named value:

```

public enum Planet {
    EARTH(10, 2),
    MARS(30, 3);

    private final int mass;
    private final int radius;

    public constructor Planet(int mass, int radius) {
        this.mass = mass;
        this.radius = radius;
    }

    public method density() returns int {
        return this.mass / this.radius;
    }
}

public class Main {
    public static method main(string[] args) returns void {
        System.IO.Console.println($"earth = {Planet.EARTH.density()} mars = {Planet.MARS.density()}");
        // earth = 5, mars = 10
        return;
    }
}

```

6.13 Field Layout: `affinity`

By default a class's fields are laid out in declaration order. For data-oriented code — where a tight loop touches a few fields of every object, every frame — you often want the *hot* fields packed together so a pass pulls in fewer cache lines, and the *cold* ones (debug ids, bookkeeping) pushed to the end. `affinity` blocks express that intent without hand-splitting the class into parallel arrays:

```

public class Particle {
    public affinity cold {
        mutable int id;
        mutable int spawnFrame;
    }
    public affinity hot {
        mutable float x;
        mutable float y;
        mutable float vx;
        mutable float vy;
    }
    // The emitted object is { vtable, x, y, vx, vy, id, spawnFrame } -- hot first, regardless
    // of the order the blocks appear in.
}

```

Only the *layout* changes; the fields, their access, and the class's semantics are exactly as if they were declared plainly. Affinity is inherited safely: a subclass's object still begins with exactly the base's layout, and only the subclass's own fields are grouped.

The same question about CODE is a method modifier: `cold` says a path is rarely taken, so its body moves off the hot line and is not inlined into one. The inliner costs a method by its size, which is a poor proxy when the bulk of one runs once in a thousand calls — `ArrayList.ensureCapacity` is the case that asked for it. There is no `hot` beside it, on purpose: `hot` is what the optimizer already assumes, so the word would spend a token and add no fact.

```
private cold method grow() returns void { ... }
```

With classes, inheritance, interfaces, virtual dispatch, properties, operators, enums, and

6.14 Layouts: an interface for memory

An interface says what a type **does**. A layout says how a type **arranges itself**. Those are separate questions, so a layout is not a fourth species beside `struct`, `record` and `union` — it crosses them:

```
public layout ThreeToALine {
    onArrange {
        itself.fitWithin(20 bytes);
        itself.refuse("a beast must stay three to a cache line");
    }
}

public struct Beast implements ThreeToALine {
    public mutable float x;
    public mutable float y;
    public mutable int    herd;
    public mutable short stamina;
    public mutable short age;
}
```

`implements` needed no new grammar for this. A struct cannot `extends` — it has no vtable to inherit through — but implementing was never inheriting, which is why the door was already open.

Implementing a layout authorizes the compiler to order the fields, and that is the point of the feature rather than a side effect. A check that only refuses is a guard against a problem that could have been solved; here it can be solved, because the compiler knows every size and alignment and Polaron exposes no offsets. Fields are ordered widest-first, so the padding a declaration order pays for simply is not spent. The build refuses only what could not be made to fit, and says so:

```
error: `Tick` is arranged by `Packed`, which fits it within 12 bytes -- it measures 16.
The fields were already ordered widest-first to make it fit, so reordering them by
hand will not help: something has to be narrower, or leave
(a tick must stay five to a cache line)
```

What a layout is not

A layout **establishes no contract** in the `requires / ensures` sense. A contract survives into the binary and can fail while the program runs; a layout is consumed entirely by the compiler, and a layout that cannot be satisfied means there is no program at all.

Nothing inside `onArrange` names a library type — not even the byte unit, which the compiler reads directly. So layouts hold in freestanding by construction, rather than by a rule forbidding the library. Inside the hook, `itself` is the arrangement being decided.

A layout applies to value aggregates because only they own a layout at the point of use: where a `Beast` is used, twenty bytes are there. Where a class is used, a pointer is — and the vtable slot at the front of the instance belongs to the compiler, not to the author.

Modifiers. A layout may be `public / private / internal`, `final` (it cannot be refined), or `sealed ... permits` (only the listed types may implement it). `abstract` and `comptime` are refused with a reason: a layout is already both. So are `partial`, the ownership modifiers, and `heap`.

For conditions that are *not* about a type's size — a table length, a protocol constant, arithmetic that has to come out a certain way — use `demand` instead.

6.15 Transformers: a transformation relation between classes

A **transformer** is a construct that establishes a **transformation relation between two classes**. It answers a question none of the other declarations do:

`class` says what a thing **is**. `interface` says what it **must be able to do**. `layout` says how it **arranges itself**. **transformer** says what a type gains — and how it relates to the other types that gain the same.

A transformer is a noun. It has a name, you can point at it, and it is **never instantiated**: it produces nothing you can hold. What it produces is what the types that apply it gain.

What lives inside one

Two things: **variables**, and **transformable methods**.

A transformable method is an abstract method that may nevertheless carry a **default implementation inside the transformer itself**, and that implementation can still be swapped for one belonging to a class. That combination has no equivalent among ordinary methods, and it is why the word is different: in Polaron these are **procedures**.

A METHOD's signature is fixed where it is declared.
A PROCEDURE's signature is completed at the type that applies it.

Inside a transformer, `itself` is *the type that will apply this* — a type that does not exist yet. A procedure returning `itself` returns a different type in every type that applies it. That is what "abstract, but with an implementation" means here: the body exists, but it is not code for any type yet. It is not floating behaviour — **its subject exists, it just does not know its name**.

The two operations: application and call

A transformer is **not implemented and not extended. It is applied**.

```
public class Dog applies TDescriber { }
```

Applying a transformer does not mean applying all of its procedures — only the ones the class chooses. Supplying a class's own body for a procedure is *also* called applying — applying that procedure:

```
public class Dog applies TDescriber {  
    public procedure describe() returns string { return "a dog"; }  
}
```

When a procedure is applied, its implementation changes — whether or not the transformer defined one.

An applied procedure overrides the transformer's own body, and there may be exactly **one application per procedure per class**: a class applying `TDescriber` may apply `describe` once, and only once.

Once applied, the procedure **belongs to the class**, and **its visibility travels with it**.

The visibility written on a procedure is **not** about who may reach it inside the transformer — nothing can reach into a transformer at all. It is the visibility the member **enters the applying class with**.

Procedures are therefore **private by default**: applying a transformer is equipment, not a promise made to the outside world, and the only way to get at a procedure at all is to apply the transformer that holds it.

A transformer cannot be instantiated, so there is no receiver to write to the left of a dot. That is what the second operation is for — **call**:

```
public class Cat applies TDescriber {  
    public procedure describe() returns string {  
        return "[" + call TDescriber.describe() + "]";    // the transformer's own body  
    }  
}
```

`call T.p()` reaches the **transformer's** procedure directly, without going through the application. So a class can use both: the applied procedure that now belongs to it, *and* the transformer's original, which its own application would otherwise have covered over. Calling is **reserved to the class that applied the transformer** — a transformer's procedure cannot be called from outside it.

The relation

When two or more classes apply the same transformer **and the same procedure**, those classes establish a transformation between themselves. It comes in three shapes:

shape	meaning	declared with
unidirectional, single	$A \rightarrow B$	nothing — it emerges from the pair
unidirectional, multiple	$A \rightarrow B, C, D$	collective
bidirectional	$A \leftrightarrow B$	mutual

Nothing declares "A converts to B". Two types apply the same transformer and implement its procedures, and **the transformation emerges from the pair** — the direction too: one side implemented is one way, both sides is both. What does *not* emerge is the **obligation** to write the other side, and that is the whole of what those two words do.

`collective` is the multiple case, and it brings its own economy: the conversions you write are edges, and the compiler completes the graph along them. Three classes in a cycle are three procedures and six conversions. The rules that keep it honest are in the specification, §32.12.1 — no path is an error, and two equally short paths is an error too, because a composed conversion may never pick one.

This is a **safe way to turn objects into other objects**, or something of one object into something of another: a route for an `A` to become a `B` **without necessarily converting vtables**.

`entrusts` : consenting to be built from outside

`applies` says "*I gain this equipment*". `entrusts` is a different and stronger promise, made by the type on the receiving end of a conversion: it hands the transformer the right to **construct** this type, filling its storage field by field — private ones included.

```
public transformer TConverter {
    public procedure into<each Other>() returns Other;
}

public class Metres applies TConverter {
    private mutable int whole;
    public procedure into<Feet f>() returns Feet {           // `f` is bound, and this fills it
        f.whole = itself.whole * 3;
        f.fraction = 0;
        return f;
    }
}

public class Feet entrusts TConverter {                     // "you may assemble me"
    private mutable int whole;
    private mutable int fraction;
}
```

Only the type itself can agree to that, because only it knows what its invariants are — which is why `entrusts` sits on `Feet` and not on `Metres`. What comes back is an ordinary object; nothing about it records that it was built from outside.

The consent buys an obligation, and the compiler enforces it: **the assembly must be complete**. Leaving `f.fraction` unassigned above is an error, checked with exactly the dataflow a constructor already passes — same map, same join at a branch. It matters more than an uninitialised local would: a local reads stack garbage, which is usually absurd and fails loudly, while an unassigned field reads whatever was last in that storage, so the wrong value is plausible, stable, and everything downstream works perfectly with it.

The cost

Transformers are resolved at compile time, so they create no vtables. A transformer is expanded by the same machinery that expands a generic, running over another list — no vtable, no allocation, no indirection. What you pay is exactly the code you would have written by hand, which is what makes the feature usable in a kernel without thinking twice.

Naming

`T` + stem + `er` — `TDescriber`, never `Describe`. A transformer is **the coupling between two types and neither of them**: an electrical transformer has two windings and is not either winding, so it is named

for the agent of the relation. The leading `T` says at a glance which of the three list-like clauses on a class line you are reading — `extends` is identity, `implements` is obligation, `applies` is equipment. The compiler warns and *generates* the suggested name rather than only demanding one.

field-layout hints in hand, you have the whole object model Polaron offers. The recurring themes — explicit over implicit (visibility, `this.`, `override`), value semantics with opt-in sharing, and immutability by default — are worth keeping in mind as you read the chapters that follow, because the rest of the language is built on top of exactly these objects.

7. Control Flow

Control flow is where a program stops being a list of declarations and starts making decisions. Polaron gives you the full imperative toolkit you would expect — conditionals, loops, multi-way branching, and pattern matching — plus a small, deliberately unusual set of low-level jump constructs (the "chaos tetrad") for the rare occasions when ordinary structured flow gets in your way. This chapter walks through all of it, from the everyday `if` to the kernel-grade `abstainfrom`, and it shows each construct with a short program you can actually compile and run.

Two rules cut across everything in this chapter, so it is worth stating them once and loudly before we begin.

Every block is braced. There is no such thing as a one-line `if` in Polaron. A conditional, a loop, a `switch` case, a `match` arm — every one of them delimits its body with `{ }`. This is not stylistic pedantry: the classic "dangling else" and the "goto fail" family of bugs both come from bodies whose extent is implicit. Polaron removes the ambiguity at the grammar level.

```
if (x == null) { return; }    // OK
if (x == null) return;       // compile error: block requires braces
```

Assignment is not an expression. You cannot write `if (x = 5)` and accidentally assign where you meant to compare — the compiler rejects it. A condition must be a genuine boolean-valued (or truthy) expression, never an assignment. The whole category of `=`-versus-`==` slips is designed out of the language.

With those two invariants in place, the rest of the chapter is mostly familiar territory told carefully.

7.1 Conditionals: `if`, `else if`, `else`

The `if` statement runs its block when the condition holds. An optional `else` supplies the alternative, and chaining `else if` gives you a ladder of mutually exclusive cases. Because braces are mandatory, the shape of the code always matches its meaning — there is never a hidden statement dangling off the end of a branch.

```

import System.IO.Console;
program Grade;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                int score = 82;
                if (score ≥ 90) {
                    System.IO.Console.println("A");
                } else if (score ≥ 80) {
                    System.IO.Console.println("B");           // this branch
                } else if (score ≥ 70) {
                    System.IO.Console.println("C");
                } else {
                    System.IO.Console.println("F");
                }
                return;
            }
        }
    }
}

```

Conditions are evaluated for truthiness. A boolean expression is the obvious case, but a reference is also usable directly: an object reference is "truthy" when it is non-null, so `if (dog)` reads as `if (dog ≠ null)`. This lets a null check double as a presence check without extra ceremony.

```

if (dog) {           // equivalent to: if (dog ≠ null)
    dog.bark();
}

```

The boolean operators `&&` and `||` short-circuit, exactly as you would hope. In `a() && b()`, `b()` is never evaluated when `a()` is already false; in `a() || b()`, `b()` is skipped when `a()` is already true. That is not merely an optimization — it is a guarantee you can lean on to guard a potentially unsafe second operand behind a cheap first one.

7.2 `while` and `do - while`

A `while` loop tests its condition *before* each iteration, so a loop whose condition is false at the outset runs its body zero times. A `do - while` loop tests *after* each iteration, which means the body always runs at least once. Reach for `do - while` when the work must happen before you can even decide whether to repeat it — reading a value, then looping while that value is unsatisfactory, is the canonical shape.

```

import System.IO.Console;
program DoWhile;

public bundle main {
  public namespace app {
    public class Main {
      public static method main(string[] args) returns void {
        mutable int n = 0;
        mutable int count = 0;
        do {
          count = count + 1;
          n = n + 2;
        } while (n < 10);
        System.IO.Console.println($"count = {count}");    // n: 2,4,6,8,10 → 5 iterations

        // The body runs at least once even when the condition is false up front.
        mutable int runs = 0;
        do {
          runs = runs + 1;
        } while (runs < 0);
        System.IO.Console.println($"runs = {runs}");      // 1
        return;
      }
    }
  }
}

```

Note the `mutable` on every variable the loop reassigns. Polaron is immutable-by-default: a plain `int n = 0;` is a constant binding, and attempting to reassign it is a compile error. Any counter, accumulator, or flag that a loop updates must be declared `mutable`.

7.3 The classic `for`

The three-clause `for` loop packages initialization, a per-iteration condition, and a step into one header. It is the right tool when the loop is fundamentally driven by an index or a hand-managed cursor.

```

import System.IO.Console;
program ForClassic;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                mutable int total = 0;
                for (mutable int i = 0; i < 10; i++) {
                    total = total + i;
                }
                System.IO.Console.println($"sum 0..9 = {total}");    // 45
                return;
            }
        }
    }
}

```

The loop variable `i` is scoped to the loop: it exists only inside the header and body, and it is gone the moment the loop ends. The initializer declares it `mutable` because the step clause reassigns it every pass.

7.4 `foreach` : iterating ranges and collections

The `for (T item in source)` form — "foreach" — iterates without an explicit index. It comes in two flavors that share one syntax: iterating a **range** of integers, and iterating any **collection** that models `Iterable<T>` (see the collections chapter). In both, you may spell the element type explicitly or write `var` and let it be inferred.

Both spellings exist and mean exactly the same thing. `foreach` is a keyword of its own, and it parses to the same statement as the `for ... in` form:

```

for (var x in xs) { total = total + x; }
foreach (var x in xs) { total = total + x; }           // identical
foreach (index i, int v in xs) { dot = dot + i * v; }

```

Polaron keeps both so that a C# or Java programmer and a C++ programmer each find the one they expect. Pick one per codebase and stay with it; the samples in this manual use `for ... in`.

7.4.1 Ranges

A range literal produces a sequence of integers. `a..b` is *half-open* — it includes `a` and excludes `b` — which is the convention that makes lengths and offsets line up cleanly. `a..=b` is *closed*: it includes both endpoints. Adding `step k` changes the stride from the default of 1.

```

import System.IO.Console;
program ForRange;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                mutable int sum = 0;
                for (int i in 0..5) {           // 0,1,2,3,4 → 10
                    sum = sum + i;
                }
                System.IO.Console.printf("excl=%d\n", sum);

                mutable int inc = 0;
                for (int i in 1..=5) {         // 1,2,3,4,5 → 15 (closed)
                    inc = inc + i;
                }
                System.IO.Console.printf("incl=%d\n", inc);

                mutable int stepped = 0;
                for (int i in 0..10 step 2) {   // 0,2,4,6,8 → 20
                    stepped = stepped + i;
                }
                System.IO.Console.printf("step=%d\n", stepped);
                return;
            }
        }
    }
}

```

A range is not just loop sugar; it is a first-class `Range<T>` value you can name, store, and pass around, then iterate later. Naming the type requires importing it, like any standard-library type:

```

import System.Collections.Range;

Range<int> r = 0..100 step 5;
for (int i in r) { /* ... */ }

```

7.4.2 Collections

The same `for ( ... in ... )` iterates arrays and any `Iterable<T>`. Here the element type is inferred with `var`:

```

import System.IO.Console;
program Foreach;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                int[] nums = new int[5]();
                for (mutable int i = 0; i < 5; i++) {
                    nums[i] = i * i;
                }
                mutable int sum = 0;
                for (var x in nums) {           // inferred element type
                    sum = sum + x;
                }
                System.IO.Console.println($"sum of squares = {sum}");    // 0+1+4+9+16 = 30
                delete nums;
                return;
            }
        }
    }
}

```

7.4.3 Iterating with an index

When you need the position alongside each element, add an `index` binding. The form `for (index i, T v in source)` gives you the 0-based position `i` and the element `v` on every pass, over both arrays and ranges. It is cleaner than a separate hand-maintained counter and it cannot drift out of sync with the iteration.

```

import System.IO.Console;
program ForeachIndex;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                int[] xs = new int[4]();
                xs[0] = 10; xs[1] = 20; xs[2] = 30; xs[3] = 40;
                mutable int dot = 0;
                for (index i, int v in xs) {    // sum of i*v = 0 + 20 + 60 + 120 = 200
                    dot = dot + i * v;
                }
                System.IO.Console.printf("dot=%d\n", dot);
                delete xs;
                return;
            }
        }
    }
}

```

7.4.4 Generators: producing a sequence with `yield`

The other side of `foreach` is *producing* the sequence. A method that returns `Iterator<T>` and whose body contains `yield` is a **generator**: instead of building a whole collection and returning it, its body runs *lazily* — one element per `yield`, suspending right there and resuming on the consumer's next request.

```
import System.Collections.Iterator;

public static method evens(int limit) returns Iterator<int> {
    mutable int n = 0;
    while (n ≤ limit) {
        yield n;           // hand back n, suspend here
        n = n + 2;         // resumes here on the next request
    }
}
```

You consume a generator like any iterable — with `foreach`:

```
mutable int sum = 0;
for (int e in Sequences.evens(8)) {
    sum = sum + e;          // 0 + 2 + 4 + 6 + 8
}
```

Because generation is lazy, a generator can be **infinite** — the consumer decides when to stop, and `break` simply leaves the body suspended forever:

```
public static method primes() returns Iterator<int> {
    mutable int n = 2;
    while (true) {          // never ends on its own
        if (Sequences.isPrime(n)) {
            yield n;
        }
        n = n + 1;
    }
}

for (int p in Sequences.primes()) {
    if (p > 30) {
        break;              // the loop stops; the generator is abandoned
    }
    // ... use p ...
}
```

An **instance** generator captures `this`, so its body can read the object's fields, and an early `return;` ends the sequence:


```

public method ticks() returns Iterator<int> {
    mutable int t = this.from;
    while (t > 0) {
        yield t;
        t = t - 1;
    }
    yield 0;
    return; // ends the sequence
}

```

Under the hood the compiler lowers the body to a heap state machine (the same coroutine lowering `async / await` uses) wrapped in a synthesized class that implements `Iterator<T>`. So a generator is an ordinary iterator: you can hold it in an `Iterator<T>` variable, drive it by hand with `hasNext()` / `next()`, and `delete` it when done.

```

Iterator<int> it = Sequences.evens(4);
if (it.hasNext()) {
    int first = it.next();
}
delete it;

```

`yield` has a second, unrelated use — the value of a block arm in a `match` expression (see §7.7.2). The compiler tells them apart by context: `yield` inside a method that returns `Iterator<T>` is a generator; `yield` inside a `match` arm is that arm's value.

7.5 `break` and `continue`, plain and labelled

Inside any loop, `break` abandons the loop entirely and `continue` skips to the next iteration. In their bare form they act on the innermost enclosing loop.

Nested loops raise a familiar problem: from inside the inner loop, how do you break out of the *outer* one? Polaron answers with **loop labels**. Prefix a loop with `name:` and then `break name;` or `continue name;` targets that specific loop rather than the nearest one. This removes the usual workaround of a `found` flag threaded through both loops, and it keeps the intent explicit at the jump site.

```

import System.IO.Console;
program Labeled;

public bundle main {
  public namespace app {
    public class Main {
      public static method main(string[] args) returns void {
        // break to a labelled outer loop
        mutable int found = 0;
        outer: for (mutable int i = 0; i < 5; i++) {
          for (mutable int j = 0; j < 5; j++) {
            if (i * j == 6) {
              found = i * 10 + j;
              break outer;
            }
          }
        }
        System.IO.Console.println($"found = {found}");    // i=2, j=3 → 23

        // continue to a labelled outer loop: abandon the current row, move to the next
        mutable int sum = 0;
        rows: for (mutable int r = 0; r < 3; r++) {
          for (mutable int c = 0; c < 3; c++) {
            if (c == 1) { continue rows; }
            sum = sum + 1;
          }
        }
        System.IO.Console.println($"sum = {sum}");        // 1 per row x 3 = 3
        return;
      }
    }
  }
}

```

`break outer;` unwinds both loops at once; `continue rows;` abandons the rest of the current row and jumps straight to the next value of `r`. These loop labels are a convenience of the looping constructs and are distinct from the standalone `label` statements used by the jump constructs in section 7.8 — do not confuse the two.

7.6 `switch`: multi-way branching with fall-through

`switch` dispatches on a value against a list of `case` constants. Polaron's `switch` has two firm rules that set it apart from the C tradition it otherwise resembles.

First, **every case body is braced** — `case 1 { ... }`, not `case 1: .` There are no colon-introduced fall-through regions; a case is a block, like everything else in the language.

Second, a **default is mandatory**. The compiler rejects a `switch` that does not handle the residual case, so there is no silent "and if none of these, do nothing" hiding in your control flow. If doing nothing is genuinely what you want, you write an empty `default { }` and say so.

Fall-through is still available — it is often exactly what you want for grouping cases — but it is now *opt-in per case*. A case that ends without `break` falls through into the next one; a case that ends with `break` does not. Because the boundary of each case is a brace rather than the next `case` label, the fall-through is a deliberate choice you make by omitting `break`, not an accident of forgetting a colon.

```
import System.IO.Console;
program Switch;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                for (mutable int x = 0; x < 4; x++) {
                    switch (x) {
                        case 0 {
                            System.IO.Console.printf("zero ");
                            // no break: falls through into case 1
                        }
                        case 1 {
                            System.IO.Console.printf("one ");
                            break;
                        }
                        case 2 {
                            System.IO.Console.printf("two ");
                            break;
                        }
                        default {
                            System.IO.Console.printf("other ");
                        }
                    }
                }
                System.IO.Console.println(""); // "zero one one two other "
                return;
            }
        }
    }
}
```

`switch` is not limited to integers. It matches enum constants by their name and — a notable extension — it matches `String` values *by content*, not by pointer identity. Two strings built separately at runtime that hold the same characters take the same case.

```
switch (dir) {
    case Dir.NORTH { code = 0; break; }
    case Dir.EAST  { code = 1; break; }
    default        { code = 9; }
}

switch (command) {
    case "hi" { greet(); break; }
    case "hello" { greet(); break; } // matched by value, even for a runtime-built string
    default { unknown(); }
}
```

When you are branching on the *dynamic type* of an object rather than on a scalar value, `switch` is the wrong tool — that is what `match` is for, and it is where we go next.

7.7 `match`: exhaustive pattern matching

`match` (specification §16) is `switch`'s type-aware sibling. Instead of comparing a scalar against constants, it dispatches on the *runtime type* of an object across a class hierarchy, binds the matched object's fields to fresh locals in one step ("destructuring"), and — crucially — can be checked for **exhaustiveness** at compile time. It exists in two forms: a *statement* form that runs a block per case, and an *expression* form that produces a value.

7.7.1 The statement form and positional destructuring

Each `case` names a concrete subtype and, in parentheses, a list of bindings positionally matched to that type's fields in declaration order. When an object matches a case, its fields are pulled out into those named locals, ready to use in the arm's body. No manual downcast, no field-by-field access — the dispatch and the extraction happen together.

```
import System.IO.Console;
program MatchDispatch;

public bundle main {
  public namespace app {
    public sealed abstract class Shape permits Circle, Rect {
      public abstract method tag() returns int;
    }
    public class Circle extends Shape {
      public int r;
      public constructor Circle(int r) { this.r = r; }
      public override method tag() returns int { return 1; }
    }
    public class Rect extends Shape {
      public int w;
      public int h;
      public constructor Rect(int w, int h) { this.w = w; this.h = h; }
      public override method tag() returns int { return 2; }
    }

    public class Main {
      public static method main(string[] args) returns void {
        Shape* s = new Rect(3, 4) on heap;
        match (s) {
          case Circle(int r)      { System.IO.Console.println($"circle {r}"); }
          case Rect(int w, int h) { System.IO.Console.println($"rect area {w * h}"; }
        }
        delete s;
        return;
      }
    }
  }
}
```

Here `s` has static type `Shape*` but points to a `Rect`. The `match` inspects the dynamic type, selects the `Rect` arm, and binds `w = 3` and `h = 4` positionally from the instance's fields. Under the hood this is the same virtual-dispatch machinery that powers method calls — matching on type is not a chain of hand-written casts.

7.7.2 The expression form: `→` and `yield`

A `match` can also *be* a value. In expression form, each arm uses `→` to supply its result, and the whole `match` evaluates to the result of the selected arm. This turns a five-line assign-in-each-branch statement into a single expression.

```
double area = match (shape) {
  case Circle(double r)      → PI * r * r;
  case Square(double s)      → s * s;
  case Triangle(double b, double h) → b * h / 2.0;
};
```

When an arm needs more than a single expression, write it as a block and hand back the value with `yield`:

```
int a = match (s) {
  case Circle(int r) → {
    int p = r * r;      // multi-statement block arm
    yield p * 3;        // Circle(3): 9 * 3 = 27
  };
  case Square(int side) → side * side;
};
```

Every arm — whether `→ expr` or `→ { ... yield ... ; }` — must produce a value of the same type, and that type is the type of the whole `match` expression.

7.7.3 Exhaustiveness, `sealed`, and `default`

The payoff of `match` is that the compiler can *prove* you handled every case. This hinges on whether the matched type is `sealed`.

A `sealed` class fixes its set of direct subtypes with a `permits` clause: `sealed abstract class Shape permits Circle, Square` declares that `Circle` and `Square` are the *only* shapes there will ever be. Given that closed set, the compiler can check a `match` for completeness. If you cover every permitted subtype, **no default is needed** — and if you add a new subtype to the `permits` list later, every non-exhaustive `match` in the program becomes a compile error until you handle it. That is the feature working for you: the type system turns "did I update all the matches?" from a manual audit into a guaranteed one.

```

import System.IO.Console;
program Sealed;

public bundle main {
  public namespace app {
    public sealed abstract class Shape permits Circle, Square {
      public abstract method area() returns int;
    }
    public class Circle extends Shape {
      private int r;
      public constructor Circle(int r) { this.r = r; }
      public override method area() returns int { return this.r * this.r * 3; }
    }
    public class Square extends Shape {
      private int side;
      public constructor Square(int side) { this.side = side; }
      public override method area() returns int { return this.side * this.side; }
    }
  }

  public class Main {
    public static method main(string[] args) returns void {
      Shape* s = new Circle(3) on heap;
      match (s) {
        // no default: sealed and exhaustive
        case Circle(int r) { System.IO.Console.println($"circle r = {r}"); }
        case Square(int side) { System.IO.Console.println($"square side = {side}"); }
      }
      delete s;
      return;
    }
  }
}

```

When the matched type is **not** sealed, the compiler cannot know the full set of subtypes, so a `default` arm is **required** to cover whatever you did not name. The rule is symmetric and easy to remember: *sealed and complete needs no default; anything else needs a default*. You can, of course, use a `default` on a sealed type too, when you deliberately want to collapse several permitted subtypes into one fallback:

```

int d = match (u) {
  case Circle(int r) → r;
  default           → 7;      // Square (and any other) falls here
};

```

7.8 The chaos tetrad: `goto`, `comefrom`, `abstainfrom`, `reinstate`

Beyond structured control flow, Polaron exposes four low-level jump and toggle constructs, collectively the "chaos tetrad" (specification §7.9–§7.11). They are powerful, they are niche, and they are honest about being both. You will not use them in ordinary application code — structured loops and `match` cover that ground better. Their home is systems programming: kernels, drivers, bootloaders, embedded power

management, and the occasional retry or state-machine pattern where an ordinary loop reads worse than a jump.

Three properties define the tetrad and keep it disciplined.

They are intra-method only. Every one of the four references `label` s declared *in the same method*.

There is no jumping into another method, and no jumping across a class boundary — `goto otherMethod.label` is a compile error, and so is the equivalent for the other three. Cross-method jumping was considered and deliberately removed: action-at-a-distance between methods is precisely the kind of chaos that makes these constructs infamous elsewhere. By confining them to a single method body, Polaron keeps the unit of flow-control reasoning small and local. (The old "jump to file line *N*" forms — `goto line N`, `comefrom line N` — were removed for the same reason and because they are fragile under edits; use a named `label` .)

They are available in freestanding mode. This is worth stating plainly because it is easy to assume otherwise: all four members of the tetrad work in `freestanding` programs (specification §36). None of them depends on the managed runtime. `goto` and `comefrom` lower to ordinary compile-time branches; `abstainfrom` / `reinstate` lower to a single global integer — a reference counter — manipulated with atomic load/add/store. There is no allocator, no syscall, and no runtime library involved, so everything compiles straight to bare-metal instructions. In fact freestanding is where `abstainfrom` shines brightest (power management, feature flags, conditional debug instrumentation in a kernel), so its availability there is by design, not by accident. The managed features that freestanding *does* drop are a different list entirely — exceptions, `async/await`, `unimport/reimport`, reflection, `lazy`, `using`, `within`, `persistents`, and the managed `Console` — and the tetrad is not among them.

A `label` itself is a bare statement marker, written `label name;`. It names a point in the method that the tetrad constructs can target. Only labels you write explicitly are valid targets; labels the compiler, runtime, or standard library generate are off-limits.

7.8.1 `goto` and `label`

`goto name;` transfers control directly to `label name;` in the same method. The most common use is a forward jump that skips a stretch of code:

```

import System.IO.Console;
program GotoForward;

public bundle main {
  public namespace app {
    public class Main {
      public static method check(int x) returns void {
        if (x < 0) { goto negative; }
        System.IO.Console.println("non-negative");
        return;
        label negative;
        System.IO.Console.println("negative");
      }
      public static method main(string[] args) returns void {
        Main.check(5);          // non-negative
        Main.check(0 - 3);      // negative
        return;
      }
    }
  }
}

```

There is a second, deliberately unchecked form of `goto` for low-level and FFI work: a jump to a raw *address* or an `extern` function. `goto myFunction;` treats an external function as a jump target; `goto 0x1000;` jumps to a literal address. This is a one-way transfer — control does not come back, and whatever follows the `goto` is unreachable — so the programmer is fully responsible for the target's validity. It exists for bootloader and kernel-entry scenarios, where handing control to code loaded at a known address is exactly the job. This address form is a freestanding/low-level tool; the label form above is the one you use in ordinary methods.

7.8.2 `comefrom`: inverse interception

`comefrom` is `goto` turned inside out. Where `goto` announces the jump at the *source* ("go there"), `comefrom` announces the interception at the *destination* ("when execution reaches label L, come here instead"). When control reaches the referenced label, flow is redirected to the point where the `comefrom` sits. It reads as a declarative statement about a label elsewhere in the method, which is what makes it interesting — and what makes it demand tooling support to read comfortably.

The order of the two lines is the whole construct. The `comefrom` is the *landing* and the `label` is where control is *taken*, so which of them you write first decides whether you have a skip or a loop:

- **`label above, comefrom below`** — control reaches the label, is sent forward to the landing, and carries on. What sits between the two never runs: that is the forward skip.
- **`comefrom above, label below`** — control passes the landing, runs on, reaches the label and is sent *back*. That is the backward retry, and it needs a way out between the two or it never returns. Local variables keep their values across the jump — there is no implicit reset — which is exactly what a retry wants.

Get the order the wrong way round and the code does the opposite of what it reads like; the compiler warns about the case it can see (`Polaron-0B55`).


```

import System.IO.Console;
program Comefrom;

public bundle Main {
    public namespace App {
        public class Main {
            // FORWARD: the label steals, so what sits between it and the `comefrom` never runs
            public static method forward() returns void {
                label skip;
                System.IO.Console.println("X");           // between the two: skipped
                comefrom skip;
                System.IO.Console.println("forward ok");
                return;
            }

            // BACKWARD: the `comefrom` is the landing and comes first, so the label below serves
            // control back to it. The `if` is the way out.
            public static method backward() returns void {
                mutable int n = 0;
                comefrom again;
                n = n + 1;
                System.IO.Console.println($"n={n}");
                if (n ≥ 3) {
                    System.IO.Console.println("done");
                    return;
                }
                label again;
                return;
            }

            public static method main(string[] args) returns void {
                Main.forward();           // forward ok
                Main.backward();          // n=1 n=2 n=3 done
                return;
            }
        }
    }
}

```

The discipline around `comefrom` is real. At most one `comefrom` may target any given label (two would make the order ambiguous), it fires *after* the labelled statement runs, and the compiler emits a warning when static analysis spots an obvious infinite cycle. It does not, and cannot, undo I/O or external effects — a `comefrom` that jumps back over a heap allocation leaves that memory to you (wrap the risky region in a `region` or use `defer` for automatic cleanup). Polaron is upfront about the limits.

7.8.3 `abstainfrom` and `reinstate` : toggling code at runtime

The last pair lets a method switch a block of its own code on and off at runtime without threading boolean flags through the logic. `abstainfrom name;` disables the code from `label name;` up to the next label (or the end of the method); `reinstate name;` turns it back on. While a label is abstained, execution skips its block entirely.

Because the disabled region runs from the label to the end of the method, the `abstainfrom / reinstate` calls must sit *before* the label they control — otherwise they would land inside the very block they are trying to toggle and could never re-enable it. The canonical shape is to read some state at the top of the method and toggle accordingly, then let the guarded block follow:

```
import System.IO.Console;
program GotoAbstainfrom;

public bundle main {
    public namespace app {
        public class Main {
            public static method run(int mode) returns void {
                if (mode == 1) { abstainfrom body; } // disable `body` from now on
                if (mode == 2) { reinstate body; }    // re-enable it
                label body;
                System.IO.Console.printf("body\n"); // skipped while abstained
            }
            public static method main(string[] args) returns void {
                Main.run(0); // body
                Main.run(1); // abstains body → silent
                Main.run(0); // still abstained → silent
                Main.run(2); // reinstates body → body
                return;
            }
        }
    }
}
```

The on/off state is a **reference count**, not a single flag. Two `abstainfrom body;` calls stack to a count of 2, and the block only wakes up when a matching number of `reinstate body;` calls bring the count back to 0. This lets several independent reasons to disable a block coexist without stepping on each other — no one caller has to know whether another has already abstained. Reinstating an already-active label is a harmless no-op (a warning, not an error), and the counter is per-execution: every program run starts with everything reinstated.

The counter's increments and decrements are atomic and sequentially consistent, so `abstainfrom / reinstate` behave predictably under concurrency: a toggle affects only executions that *reach* the label afterward, while any thread already inside the block runs to completion. And because that atomic counter is the entire runtime cost — one global integer, no allocator, no scheduler — the whole mechanism is freestanding-safe, which is why it is a natural fit for kernel power-management and driver feature-flag code.

For safety, `abstainfrom` refuses a long list of targets outright: compiler-generated labels, import/unimport machinery, reimport validation, type-system checks, bounds checks, constructors and destructors, region and persistent lifecycles, contract checks, and more. You cannot use it to switch off the language's own correctness and safety mechanisms.

Here is the same pattern in a `freestanding` bundle, emitting through a bare `extern` function — proof that the tetrad needs nothing from the managed runtime:

```

program FreestandingTetrad freestanding;

public bundle main freestanding {
  public namespace boot {
    public class Sys {
      public extern cdecl static method putchar(int c) returns int;
    }
    public class Main {
      public static method tick(int mode) returns int {
        if (mode == 1) { abstainfrom body; } // disable body
        if (mode == 2) { reinstate body; }   // re-enable body
        label body;
        Sys.putchar(66); // 'B'
        return 0;
      }
      public static method main(string[] args) returns int {
        Main.tick(0); // B
        Main.tick(1); // abstains → silent
        Main.tick(0); // silent
        Main.tick(2); // reinstates → B
        Sys.putchar(10);
        return 0;
      }
    }
  }
}

```

7.9 Choosing the right construct

With so many ways to branch and loop, a short guide helps.

- Use **if / else if / else** for a handful of boolean conditions.
- Use **switch** to branch a single scalar, enum, or string value against known constants — and remember the mandatory **default** and the opt-in fall-through.
- Use **match** whenever you are branching on an object's *dynamic type*, especially over a **sealed** hierarchy where you want the compiler to guarantee you covered every case, or when you want branching to *produce a value* via the **→ / yield** expression form.
- Use **while / do - while** for condition-driven repetition, **classic for** for index-driven loops, and **foreach** for ranges and collections — reaching for the **index** binding when you need positions.
- Use **loop labels** with **break / continue** to control an outer loop from within a nested one.
- Reach for the **chaos tetrad** only in low-level or systems code, where a jump genuinely reads better than structured flow — and lean on the fact that it stays entirely within one method and works in freestanding mode.

The everyday constructs keep your code obvious; the tetrad is the escape hatch for the day you need to talk to the machine on its own terms.

8. Errors, Results & Contracts

Every real program has to reckon with the moment things go wrong: a file that isn't there, a number the user typed that can't be parsed, a division whose denominator turned out to be zero. Polaron does not pretend those moments away, and it does not leave them to chance either. The language gives you three complementary tools for dealing with failure, and a fourth guarantee that sits underneath all of them.

The three tools are *exceptions* (`try` / `catch` / `finally` with `throw`), *result types* (`Result<T, E>` and `Option<T>` , matched exhaustively or threaded with the `try?` operator), and *contracts* (`requires` , `ensures` , `invariant`). The guarantee underneath is the no-undefined-behaviour principle: when an operation cannot produce a meaningful answer — an integer that overflows, an index past the end of an array, a dereference through a null reference — the result is always *deterministic*, never an exploitable crash into the weeds.

This chapter explains what each tool does, how the compiler implements it, and, just as importantly, *when to reach for which one*. The short version: use exceptions for genuinely exceptional, cross-cutting failures; use `Result` / `Option` for expected, recoverable outcomes on the value path; use contracts to state the rules a method assumes and promises; and lean on the no-UB guarantee to know that even the failures you *didn't* plan for stop cleanly.

8.1 Exceptions: `try`, `catch`, `finally`, `throw`

An exception is a way to abandon the current computation and hand control to a handler further up the call stack. You raise one with `throw` , you guard a region of code with `try` , and you handle raised exceptions with `catch` . A `finally` block runs no matter how the guarded region is left.

Here is the whole shape in one runnable program. `run(true)` throws and is caught; `run(false)` completes normally. In both cases the `finally` block runs.

```

import System.IO.Console;
program ExceptionsIntro;

public bundle main {
    public namespace app {
        public interface Throwable { }

        public class MyError implements Throwable {
            public constructor MyError() { }
        }

        public class Main {
            public static method run(boolean fail) returns void {
                try {
                    if (fail) { throw new MyError() on heap; }
                    System.IO.Console.println("body ok");
                } catch (MyError e) {
                    System.IO.Console.println("caught");
                } finally {
                    System.IO.Console.println("finally");
                }
                return;
            }

            public static method main(string[] args) returns void {
                Main.run(true);    // prints: caught / finally
                Main.run(false);   // prints: body ok / finally
                return;
            }
        }
    }
}

```

What you can throw

A thrown value must be a **heap-allocated object that participates in a class hierarchy** — that is, a class that either `extends` another class or `implements` an interface, so that it carries a vtable. Two rules follow, and the compiler enforces both:

- **You cannot throw a bare primitive.** `throw 5;` is a compile error, not a runtime surprise. An exception has to be an object the handler can identify by type.
- **You cannot catch a standalone (non-polymorphic) class.** A class with no base and no interface has no vtable, so a handler would have no type information to match against; the compiler rejects `catch (Plain p)` for such a `Plain`.

The object must also live on the heap, because it has to outlive the stack frame that raised it:

```

throw new MyError() on heap;    // ok

MyError e = new MyError() on stack;
throw e;                        // compile error: a thrown object must be heap-allocated

```

The spec's canonical example roots exceptions in a base `Exception` class (`public class IOException extends Exception { }`); implementing a small `Throwable` interface, as above, works exactly the same

way. What matters is only that the thrown type is polymorphic.

Type-matched catch

Catch clauses are tried top to bottom, and a clause matches when the thrown object's runtime type **is** the caught type *or a subtype of it*. Polaron does the matching itself — it compares the thrown object's vtable pointer against the set of vtables for the caught class and all its subclasses — rather than relying on C++ RTTI. So an ancestor clause catches a descendant, exactly as you'd expect:

```
public class AppError implements Throwable { public constructor AppError() { } }
public class ParseError extends AppError { public constructor ParseError() { } }

try {
    throw new ParseError() on heap;
} catch (AppError e) {           // matches: ParseError is-a AppError
    System.IO.Console.println("handled as AppError");
}
```

Put the most specific clauses first; a broad `catch (AppError e)` placed before a `catch (ParseError e)` would shadow the narrower one.

The `throws` clause

A method may declare the exception types it is expected to raise with a `throws` clause. `throws` is a real keyword, not an annotation:

```
public method readFile(String path) throws(IOException, NetworkException) returns string {
    if (!this.exists(path)) {
        throw new IOException() on heap;
    }
    // ...
}
```

Polaron's exceptions are *unchecked*: the `throws` clause documents intent and drives a compiler **warning**, it does not force callers to handle anything. If a `throw` reaches a point where it is neither caught by an enclosing `try` nor covered by the method's own `throws` clause, the compiler warns that the exception "is neither caught nor declared in the method's 'throws' clause" — a nudge to either handle it or declare it, never a hard error that stops the build.

`finally` always runs

The defining property of `finally` is that it runs on *every* way out of the guarded region, not just the tidy ones. That includes:

- normal fall-through off the end of the `try` body,
- a caught exception (the matching `catch` runs, then `finally`),
- an *uncaught* exception that propagates to an outer handler,
- and an early `return`, `break`, or `continue` that jumps out of the `try`.

The uncaught case is the subtle one. Here an inner `try` has only a `finally`; the throw is caught by the *outer* `try`, but the inner `finally` still runs on the way out:

```
try {
    try {
        System.IO.Console.println("throwing");
        throw new MyError() on heap;
    } finally {
        System.IO.Console.println("inner finally");    // runs, though uncaught here
    }
} catch (MyError ex) {
    System.IO.Console.println("outer caught");
}
// prints: throwing / inner finally / outer caught
```

And the early-exit case — a `return` out of a `try`, or a `break` that leaves a `try` nested in a loop — runs `finally` before control actually departs:

```
public static method withReturn() returns int {
    try {
        return 1;
    } finally {
        System.IO.Console.println("finally-return");    // runs before the return leaves
    }
}
```

The unwinding model, and why cleanup is reliable

Polaron lowers exceptions to the platform's **native landing-pad unwinding**, not to `setjmp / longjmp`. On Windows the compiler emits Microsoft's structured-exception tables and routes through `__CxxFrameHandler3`; on Linux and other ELF targets it emits the Itanium/DWARF model, throwing through `__cxa_throw` with the `__gxx_personality_v0` personality. In both cases every Polaron exception travels as a single canonical carrier — the object pointer — and the handler does the real type-matching in ordinary code.

The practical payoff is a **zero-cost happy path**: when nothing is thrown, the `try` region is just straight-line code with no per-statement bookkeeping and no overhead. You pay only when an exception is actually raised and the runtime walks the stack to find a handler.

That stack walk is where reliability matters, and this is a point Polaron takes seriously. As the stack unwinds through each frame, *all* scope cleanup runs, in the correct order:

- `finally` blocks,
- `defer` statements (LIFO, see Chapter on scoped resources),
- `using` bindings (their `delete` fires),
- region releases,
- and `synchronized / mutex` unlocks.

None of these are skipped just because the exit path is an exception rather than a normal return. A resource acquired before a throw is released as the throw passes back through its scope:

```
public method process(String path) returns void {
    File f = new File(path) on heap;
    defer { delete f; }           // runs even if f.read() throws
    f.read();
}
```

This closes a whole category of leaks-on-error that plague languages where cleanup and unwinding are bolted on separately.

8.2 `Result<T, E>` and `Option<T>`

Exceptions are the right tool when a failure is genuinely exceptional and you want it to punch through several call frames at once. But a great many "failures" are ordinary, expected outcomes: parsing might not succeed, a lookup might find nothing. For those, Polaron gives you two sum types in the prelude that make the failure part of the value itself, so the type system forces you to deal with it.

Both are declared `mustuse`, so a statement that drops one is warned about (Polaron-0B19): a `Result` whose answer is thrown away is an error nobody handled, which is the one failure the type exists to make impossible to have by accident. Any type of your own may carry the same word, and so may a single method. Where dropping the answer is what you mean, say it at the line — `discard theCall();`.

`Result<T, E>` holds either a success value of type `T` (an `Ok`) or an error value of type `E` (an `Err`).

`Option<T>` holds either a present value (a `Some`) or nothing (a `None`). Both are sealed types — the set of cases is fixed — which is what lets `match` check exhaustiveness. They live under `System.Errors`:

```
import System.Errors.Result;
import System.Errors.Option;
```

Constructing them

The explicit form is an ordinary heap allocation:

```
Result<int, int>* ok  = new Ok<int, int>(7) on heap;
Result<int, int>* err = new Err<int, int>(404) on heap;
Option<int>* some  = new Some<int>(9) on heap;
Option<int>* none  = new None<int>() on heap;
```

More commonly you use the construction *sugar* — `Ok(x)`, `Err(x)`, `Some(x)`, `None()` — and let the compiler infer the generic arguments from the expected type, which is either the method's declared return type or the type of the variable you are assigning to:

```
public static method parse(int n) returns Result<int, int>* {
    if (n < 0) { return Err(404); } // E inferred as int from the return type
    return Ok(n);                  // T inferred as int
}

Result<int, int>* a = Ok(7);           // both inferred from the declared type
Option<int>* o = Some(9);
Option<int>* p = None();
```


Consuming them with `match`

Because `Result` and `Option` are sealed, a `match` over one must cover every case — and, having covered them, needs no `default` arm. Each `case` binds the payload positionally:

```
public static method grade(Result<int, int>* r) returns int {
    match (r) {
        case Ok(int v) { return v; }
        case Err(int e) { return 0 - e; }
    }
    return 0;
}
```

The same reads naturally for `Option`:

```
match (o) {
    case Some(int v) { System.IO.Console.printf("some %d\n", v); }
    case None()      { System.IO.Console.printf("none\n"); }
}
```

If you forget a case, the compiler tells you at build time rather than letting a missing branch slip through — that is the whole point of making failure a value with a closed set of shapes.

Propagating with `try?`

Threading a `Result` up through several layers by hand — `match`, pull out the value on success, return the error on failure, repeat — is tedious and easy to get wrong. The `try?` operator does exactly that in one token. Applied to a `Result` or `Option`, it evaluates to the inner value when the operand is `Ok / Some`, and otherwise **immediately returns** the `Err / None` from the enclosing method:

```
public static method parse(int n) returns Result<int, int>* {
    if (n < 0) { return Err(0 - n); }
    return Ok(n);
}

public static method doubleIt(int n) returns Result<int, int>* {
    int v = try? Main.parse(n); // Ok → v; Err → return that Err from doubleIt
    return Ok(v + v);
}
```

`doubleIt(5)` runs `parse(5) = Ok(5)`, binds `v = 5`, and returns `Ok(10)`. `doubleIt(-3)` runs `parse(-3) = Err(3)`, and `try?` short-circuits: `doubleIt` returns that `Err` without ever reaching the next line.

Two constraints keep `try?` honest, and the compiler checks both:

- the operand must actually be a `Result` or `Option` — `try?` on anything else is a compile error ("try? requires a Result or Option operand"); and
- the enclosing method must itself return a `Result` or `Option`, since that's where the propagated error goes. Using `try?` inside a method that returns `void` is a clean compile error, not a runtime crash.

When to prefer Results over exceptions

Reach for `Result / Option` when failure is an *expected, recoverable* part of the job — parsing input, looking something up, opening a file that might be absent. The failure stays on the value path, every caller can see from the signature that it must be handled, and there is no stack-unwinding machinery involved.

Reach for exceptions when a failure is genuinely *exceptional* and should unwind across many frames to a distant handler — the kind of error most call sites have no useful way to react to locally.

There is also a hard rule: **freestanding mode has no exceptions**. When a program or bundle is declared `freestanding` (for kernels and bare-metal targets, where there is no unwinder and no C++ personality routine), the compiler rejects `try`, `catch`, and `throw` outright. In that environment `Result` and `Option` are your *only* structured error-handling tools — which is one more reason to build the habit of using them for anything a caller can reasonably recover from.

8.3 Contracts: `requires`, `ensures`, `invariant`

Contracts let a method state, in executable form, the assumptions it makes and the promises it keeps. They are optional — you attach them where a rule is worth enforcing — and they read as part of the method's signature, before the body.

- `requires` states a *precondition*: something that must be true on entry.
- `ensures` states a *postcondition*: something the method guarantees on exit.
- `invariant` states a *class invariant*: something that must hold for every instance across its observable lifetime.

A contract should call a method only if that method is `readonly` — declared to write nothing, and checked through the whole call graph. The reason is the one a reader would guess: a check that could change state would make the checking part of the program's behaviour, so the same program would mean different things depending on whether contracts were compiled in.

Here is an `Account` that spells out all three:

```

public class Account {
    private mutable int balance;

    invariant this.balance ≥ 0;

    public constructor Account(int start)
        requires start ≥ 0
        ensures this.balance == start
    {
        this.balance = start;
    }

    public method deposit(int amount) returns void
        requires amount > 0
        ensures this.balance == old(this.balance) + amount
    {
        this.balance = this.balance + amount;
    }

    public method withdraw(int amount) returns void
        requires amount > 0
        requires amount ≤ this.balance
        ensures this.balance == old(this.balance) - amount
    {
        this.balance = this.balance - amount;
    }
}

```

Each clause is an ordinary boolean expression. You may stack several `requires` (or several `ensures`); they all have to hold.

When each check runs

The compiler weaves the checks into the generated code:

- `requires` clauses are evaluated **on entry**, before the body runs.
- `ensures` clauses are evaluated **on exit**, after the body has done its work.
- `invariant` clauses are also evaluated on method exit, so every public operation leaves the object in a valid state.

When a clause holds, execution proceeds. When one fails, the program stops with a non-zero status. A violation is a bug in the program, so it stops deterministically rather than raising a catchable exception — you are meant to fix the caller or the method, not to `catch` a broken contract.

What a violation says

A contract exists to name a disagreement, so one that fails says **which** one, and on `stderr` rather than in the middle of the program's own output:

```

contract violated: requires
→ src/gfx/GlWorld3D.pol:345:32 in GlWorld3D.upload
| requires wrote == counted
| left = 17561340, right = 23966928

```

The clause is quoted as you wrote it, read back out of the source. The last line appears whenever the clause is a comparison whose two sides can be read a second time **without consequences** — identifiers, fields, literals, casts of those. Never a call: a diagnostic that changes the state it is diagnosing is worse than none, so anything that could log, mutate or allocate simply goes unquoted.

The no-UB guards — an index off the end, a division by zero, a full region — report in the same shape, with their own two numbers (`index = 9`, `length = 6`).

A guard that fires ends the run with an **exit code**, which is the only part of the report a script outside the program can read:

Code	What fired
70	an index off the end of an array
71	integer division by zero, including the <code>INT_MIN / -1</code> corner
72	<code>checked(...)</code> left the range of its type

All of it holds in freestanding too, including the values, which a kernel formats without `stdio`. There it arrives through `__polaron_panic`, which a kernel may override to route reports to its own console; overriding it keeps the whole message, because everything is composed before that call rather than printed around it.

`old( ... )` in postconditions

A postcondition often needs to compare the final state to the state *at entry* — "the balance went up by exactly `amount`." The `old(e)` form gives you the value `e` had when the method was entered. The compiler captures each `old( ... )` expression on entry into a hidden slot and reads it back when the `ensures` clause runs:

```
ensures this.balance == old(this.balance) + amount
```

`old( ... )` is only meaningful inside an `ensures` clause; using it elsewhere is a compile error.

`result` in postconditions

Every example above is a `void` mutator promising something about `this`. The other half of what a postcondition is for is the *returned value* — "what I hand you satisfies this" — and inside an `ensures` clause the name `result` is that value:

```
public method takePage() returns int
    ensures result % 4096 == 0
{
    ...
}
```

This is the guarantee that meets a caller's `requires`. A frame allocator that promises page-aligned addresses and a mapper that requires them state the two halves of one contract, and neither has to trust the other.

`result` is in scope in `ensures` only. A `requires` clause runs on entry, before there is a result, so naming it there is an undeclared-name error like any other. It is not a reserved word: a local actually called `result`

shadows the binding and means itself. With several `return` statements, each one's postconditions see that return's own value.

Invariants are inherited

A class invariant binds not only the class that declares it but every subclass. If `Counter` declares `invariant this.count ≥ 0;`, then a `Stepper` extends `Counter` is held to that same invariant on exit of its own methods too:

```
public class Counter {
    protected mutable int count;
    invariant this.count ≥ 0;
    public constructor Counter() { this.count = 0; }
    public method get() returns int { return this.count; }
}

public class Stepper extends Counter {
    public constructor Stepper() { this.count = 0; }
    public method up(int n) returns void { this.count = this.count + n; }
    public method down(int n) returns void {
        this.count = this.count - n;    // checked against the inherited invariant on exit
    }
}
```

If `down` ever drove `count` below zero, the inherited invariant would fire on the way out of `down`, even though the rule was written on `Counter`.

demand : the compile-time cousin

Contracts are checked as the program runs. Sometimes the thing you want to state is knowable at *build time* — a buffer size, a table length, a bit of arithmetic that must come out a certain way. `demand <condition> otherwise "why";` settles a constant condition during compilation and emits no runtime code at all:

```
demand 2 + 2 == 4 otherwise "math is broken";
demand 16 * 1024 < 65536 otherwise "the buffer does not fit";
```

If the condition is false the build stops with your reason. If it is true the demand disappears entirely — there is nothing left to run. Think of it as a `requires` clause on the program's own assumptions, paid once while building rather than on every call.

It is a **statement**, like `return` or `throw`, so it lives in a method body and carries no parentheses of its own. Its condition must be knowable then: one that does not fold is refused outright, never quietly deferred to runtime.

For the **size of a type**, use a `layout` rather than a demand. A byte budget is a property of the type, not a condition about the program — and a layout lets the compiler order the fields to meet it, which is a guarantee rather than a check. See §10 for how `demand` shares its evaluator with `comptime` and `fixed`.

8.4 The no-UB principle and failure

Underneath all three tools sits a promise that shapes how Polaron handles the failures you *didn't* write a handler for: **there is no exploitable undefined behaviour**. Where C would quietly wander into the undefined — signed overflow, a stray index, a null dereference — Polaron produces a *deterministic* outcome. Either the operation is given a defined answer, or the program stops cleanly with a message via a runtime panic. It never becomes a silent memory-corruption bug an attacker can steer.

These deterministic panics are distinct from exceptions: they are not thrown, not caught, and not part of the `try / catch` machinery. They terminate the program. They are the language refusing to lie about a computation it cannot honestly perform.

Integer overflow

Integer arithmetic has, by default, a **defined two's-complement wrap**. That's the zero-overhead choice, and because the result is defined it is not undefined behaviour — an overflowing `int` wraps predictably rather than poisoning the program. When you want overflow to be an *error* instead, wrap the expression in `checked( ... )`, and signed `+`, `-`, and `*` will trap deterministically on overflow:

```
int fast = a + b;           // defined wrap, no overhead
int safe = checked(a * b + c); // signed overflow here panics deterministically
```

For per-operation control there are explicit methods: `wrappingAdd` / `wrappingSub` / `wrappingMul` (and `unchecked ...` aliases) for guaranteed wrap, and `saturatingAdd` / `saturatingSub` / `saturatingMul` to clamp to the type's minimum or maximum instead of wrapping:

```
int clamped = x.saturatingAdd(y); // pins to the type's max/min instead of wrapping
int wrapped = x.wrappingAdd(y);   // explicit wrap
```

(Unsigned arithmetic and freestanding mode always wrap.)

Division by zero

Integer division by zero — and the `INT_MIN / -1` overflow corner — do not invoke undefined behaviour. They panic deterministically with a clear message:

```
int q = a / b; // if b == 0, the program panics: "integer division by zero or overflow"
```

Array bounds

Every array index is bounds-checked. An out-of-range access panics rather than reading or writing past the block:

```
int[] xs = new int[3]();
int bad = xs[7]; // panics: "array index out of bounds"
```

The check is cheap and, where the optimizer can prove an index is in range (a loop counter over the array's own length, say), it is elided entirely — so the guarantee costs nothing in the common, provably-safe case.

Null dereference

Dereferencing a **nullable** reference that happens to be null does not fault into address zero. The compiler emits a null check on the accesses that actually dereference, so a null receiver traps deterministically with a message instead of segfaulting:

```
nullable Dog maybe = this.find(name);
maybe.bark();    // if maybe is null, this traps deterministically, not a wild segfault
```

Non-nullable references pay nothing for this — the check is emitted only where the type says the value *could* be null. And taking an address (`&x`) does not dereference, so it is exempt.

Float-to-int conversion

Converting a floating-point value to an integer is *saturating* and therefore always defined: a value too large for the target type clamps to that type's extreme rather than producing a garbage bit pattern.

8.5 Choosing the right tool

To close, a rule of thumb for a chapter that has offered several overlapping tools:

- **Result<T, E> / Option<T>** — for expected, recoverable failures that belong on the value path, where you want every caller to see and handle the outcome. Thread them with `try?`. In **freestanding** code, these are the only structured error tools available.
- **Exceptions (try / catch / throw)** — for genuinely exceptional failures that should unwind across many frames to a distant handler. Trust `finally`, `defer`, `using`, region releases, and mutex unlocks to run during that unwind.
- **Contracts (requires / ensures / invariant)** — for stating and enforcing the rules a method assumes and promises; a violation is a bug and stops the program. Use `demand` for the assumptions you can settle at compile time.
- **The no-UB guarantee** — you don't reach for this; it is always on. Overflow, division by zero, out-of-bounds access, and null dereference are all deterministic, so even the failures you never anticipated stop cleanly instead of becoming security holes.

9. Concurrency

Most of what a program does with more than one thing at a time, Polaron expresses through ordinary library types rather than dedicated syntax. `Thread`, `Mutex<T>`, `atomic<T>`, and `Channel<T>` are all classes in the standard library, imported and used like any other class. Only `async` and `await` remain keywords, and they earn that status for a concrete reason: they change how the compiler generates code. An `async` method is not compiled as a straight-line function; it is rewritten into a state machine that can pause in the middle and continue later. That rewrite cannot be a library feature, so the two words that trigger it are part of the language.

The result is two facilities that solve two different problems. **OS threads** give you genuine parallelism: separate execution contexts scheduled by the operating system, each running a piece of code at the same time as the others. **Async/await** gives you cooperative concurrency over a fixed pool of worker threads: many logical tasks that suspend and resume without each one consuming a whole OS thread while it waits. The two systems are conceptually distinct, they can coexist in the same program, and this chapter treats them in turn before covering the primitives that let concurrent code share state without corrupting it.

Everything here depends on the runtime and the operating system. **Concurrency is not available in freestanding mode** (see §9.9): the compiler rejects `async` / `await` outright there, and the thread, channel, and lock types have nothing to lower to without an OS underneath.

9.1 Two models, side by side

Before the details, hold the distinction clearly. A `Thread` is a real operating-system thread. When you `start()` one, the OS scheduler may run its code on a different core in true parallel with your other threads. You pay for that with the cost of an OS thread, and you are responsible for synchronizing any state the threads share.

An `async` method, by contrast, does not get its own OS thread. Calling it hands a small unit of work to a shared pool of worker threads managed by the language runtime. While that work waits for something (typically another `async` task), it is *suspended*: it steps off the worker entirely, freeing that worker to run other tasks, and it is resumed later when what it was waiting for is ready. This is how you can have thousands of outstanding `async` operations backed by a handful of OS threads.

You reach for threads when you want to run CPU-bound work on multiple cores, or when you need a long-lived background worker. You reach for `async` when you have many operations that spend most of their time waiting and you do not want to burn an OS thread on each one.

9.2 Threads

`Thread` lives at `System.Concurrency.Thread`. A thread runs an `Action` — a command that takes no arguments and answers nothing — which you hand to its constructor. Calling `start()` spawns the OS thread and begins running it; calling `join()` blocks the calling thread until that thread has finished.


```

import System.IO.Console;
import System.Concurrency.Thread;
import System.Commands.Action;
program ThreadSpawn;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                mutable int n = 21;
                Action* work = command () carries (int seed = n) into pack returns void {
                    System.IO.Console.printf("worker n=%d\n", pack.seed * 2);
                };
                Thread t = new Thread(work) on heap;
                t.start(); // the command now runs on its own OS thread
                t.join(); // wait for it to finish before continuing
                System.IO.Console.printf("main done\n");
                return;
            }
        }
    }
}

```

The command **carries** `n`, and baggage is copied when the command is built — so the worker thread receives its own copy and does not race the main thread over the original variable. That is not a default to remember: it is the only mode there is, and the list says out loud what crossed. The compiler reads the same list to decide whether the crossing is safe at all (§9.5). Because `main` calls `join()` before printing its last line, the output is deterministic — the worker runs to completion first, so the program prints `worker n=42` and then `main done`. Without the `join()`, the two prints could interleave in either order, which is exactly the kind of nondeterminism threads introduce.

Under the hood `start()` calls into the runtime, which uses `CreateThread` on Windows and `pthread_create` on POSIX; `join()` maps to `WaitForSingleObject` / `pthread_join`. You allocate the `Thread` on heap here because it must outlive the statement that starts it.

9.3 Async, await, and `Task<T>`

An `async` method is written like any other method, with the `async` modifier in front, but it behaves very differently at the call site. Instead of running to completion and returning a `T`, it returns immediately with a `Task<T>` — a handle to a computation that will eventually produce a `T`. The body is scheduled to run on the worker pool. You get the value out later by writing `await` on the task.

```

import System.IO.Console;
import System.Concurrency.Task;
program AsyncBasic;

public bundle main {
    public namespace app {
        public class Main {
            public static async method sumA() returns int {
                mutable int s = 0;
                for (mutable int i = 1; i ≤ 100; i++) { s = s + i; }
                return s;
            }
            public static async method sumB() returns int {
                mutable int s = 0;
                for (mutable int i = 1; i ≤ 200; i++) { s = s + i; }
                return s;
            }
            public static method main(string[] args) returns void {
                Task<int> a = Main.sumA();    // scheduled on the pool; returns at once
                Task<int> b = Main.sumB();    // also scheduled; both may run concurrently
                int ra = await a;             // block here until a finishes
                int rb = await b;
                System.IO.Console.printf("a=%d b=%d\n", ra, rb);
                return;
            }
        }
    }
}

```

The two calls to `sumA()` and `sumB()` do not compute anything at the call site; each returns a `Task<int>` right away, and the actual summing happens on pool workers. Only when `main` reaches `await a` does it wait for a result. Notice that `main` is an ordinary (non-async) method: awaiting a task from ordinary code is legal and simply *blocks* the current thread until the task completes. The distinction between blocking and suspending is the heart of the model, and the next sections make it precise.

`Task<T>` is `System.Concurrency.Task`; it wraps the runtime task handle. You almost never touch its internals — you create one by calling an async method and consume it with `await`.

9.3.1 How an async method becomes a state machine

When the compiler sees `async`, it does not emit a normal function. It rewrites the method into a **state machine** whose local variables no longer live on the call stack but in a heap-allocated *state object*. Each `await` in the body marks a point where the method may pause. The compiler splits the body at every such point into a *suspend* half and a *resume* half, and it builds a small dispatch at the top of the resume function — an entry switch — that jumps to whichever resume point the method last stopped at.

This rewrite is why locals move to the heap. A value defined before an `await` and used after it must survive the pause, and a stack frame does not survive a function returning. By storing locals in the state object, the method can return control to the worker pool at an `await`, come back later, and find all its variables exactly where it left them.

The worker pool itself is a fixed set of OS threads created once, sized to the machine (the number of logical processors, clamped to a sane minimum and maximum). They pull ready-to-run work — pairs of

(*resume function, state object*) — off a shared queue and run it. A worker never sleeps waiting on an individual task; when a task suspends, the worker just picks up the next queued item.

9.3.2 What `await` actually does

Inside an async method, `await someTask` does three things. It saves the awaited task's handle into the state object, it advances the state counter to the resume point that follows, and it asks the runtime whether the awaited task is already finished.

If the task is **not** yet done, `await` registers the current method's resume function as a *continuation* on that task and then *returns* — the worker is released to do other work, and no OS thread is parked. When the awaited task later completes, the runtime schedules the registered continuation back onto the pool, a worker picks it up, the entry switch jumps straight to the resume point, and execution continues as if it had never stopped.

If the task **is** already done, `await` takes a fast path: there is nothing to wait for, so it does not suspend at all and simply reads the result and continues in place.

```
import System.IO.Console;
import System.Concurrency.Task;
program AsyncChain;

public bundle main {
    public namespace app {
        public class Main {
            public static async method val(int n) returns int { return n; }
            public static async method pipeline() returns int {
                int a = await Main.val(10);    // suspend; resume with a = 10
                int b = await Main.val(20);    // suspend; resume with b = 20
                int c = await Main.val(30);    // suspend; resume with c = 30
                return a + b + c;              // 60
            }
            public static method main(string[] args) returns void {
                Task<int> t = Main.pipeline();
                System.IO.Console.printf("sum=%d\n", await t);    // 60
                return;
            }
        }
    }
}
```

`pipeline()` suspends three times, once at each `await`, and each time its locals `a`, `b`, and `c` persist in the state object across the pause. The final `return a + b + c` *completes* the task with the value `60`, which is what `main`'s `await t` ultimately reads.

9.3.3 Awaiting inside loops and conditionals

Because the async lowering is a genuine coroutine transform and not a shallow trick, `await` is allowed anywhere in the method body, including inside loops and `if` blocks. The control flow you write is the control flow you get; the compiler splits each `await`'s block into a suspend/resume pair and wires the entry switch so that resuming jumps back into the loop at the right place.

```

import System.IO.Console;
import System.Concurrency.Task;
program AsyncLoop;

public bundle main {
    public namespace app {
        public class Main {
            public static async method val(int n) returns int { return n; }
            public static async method sumLoop() returns int {
                mutable int total = 0;
                for (mutable int i = 1; i ≤ 5; i++) {
                    int x = await Main.val(i);    // suspend/resume once per iteration
                    total = total + x;
                }
                return total;                    // 15
            }
            public static method main(string[] args) returns void {
                Task<int> t = Main.sumLoop();
                System.IO.Console.printf("loopsum=%d\n", await t);    // 15
                return;
            }
        }
    }
}

```

Here `total` and the loop counter `i` both live in the state object, so they keep their values across the suspension that happens on every iteration. The method resumes back into the loop body each time until the loop finishes normally and the task completes with `15`.

There is one subtlety worth naming. When several `await` s appear in a *single expression*, a value computed before one `await` must still be alive after the next one. The compiler handles this by *spilling* such intermediate temporaries into scratch storage inside the state object and reloading them in the resume block, so an expression like `await f() + await g() + await h()` evaluates correctly even though each `await` splits the expression in two.

```

public static async method combine() returns int {
    // the result of the first await survives the second and third awaits:
    int s = await Main.val(10) + await Main.val(20) + await Main.val(30);    // 60
    // a plain value computed before an await survives it too:
    int t = Main.twice(5) + await Main.val(7);                                // 17
    return s + t;                                                            // 77
}

```

9.3.4 Results and exceptions both propagate

An async task can finish in one of two ways: it can produce a value, or it can throw. Both outcomes travel back to the awaiter. If the body reaches a `return`, the task completes with that value and the awaiter's `await` yields it. If the body throws an exception that escapes it, the task completes in a *failed* state carrying the exception, and when the awaiter reaches its `await`, the exception is re-thrown there. In other words, `await` propagates a failure into the awaiting context exactly as if the awaited work had run inline.

```

import System.IO.Console;
import System.Concurrency.Task;
program AsyncTry;

public bundle main {
    public namespace app {
        public interface Throwable { }
        public class MyError implements Throwable {
            public mutable int code;
            public constructor MyError() { this.code = 1; }
        }
        public class Main {
            public static async method val(int n) returns int { return n; }
            public static async method work(boolean fail) returns int {
                int base = await Main.val(10);          // stays live across the whole try
                mutable int out = 0;
                try {
                    int a = await Main.val(20);
                    if (fail) { throw new MyError() on heap; }
                    out = base + a;                      // 30 when not failing
                } catch (MyError e) {
                    out = base + await Main.val(5);      // 15; note: await inside catch
                } finally {
                    out = out + 100;                     // always runs
                }
                return out;
            }
            public static method main(string[] args) returns void {
                Task<int> ok = Main.work(false);
                System.IO.Console.printf("ok=%d\n", await ok);    // 130
                Task<int> bad = Main.work(true);
                System.IO.Console.printf("bad=%d\n", await bad);   // 115
                return;
            }
        }
    }
}

```

This example is a good stress test of the model: an `await` sits inside a `try`, another sits inside a `catch`, a `throw` is caught locally, and a `finally` runs on both paths. The value `base` is computed before the `try` and remains valid through all of it because it lives in the state object. Exceptions compose with `async` just as they do in ordinary code.

9.3.5 Awaiting outside an async method

`await` is not restricted to `async` bodies. When you write it in an ordinary method — as `main` does in every example above — it cannot suspend a state machine, because there is none. Instead it simply *blocks* the calling thread until the task completes and then reads the result. This is the natural way to drive `async` work from a synchronous entry point: kick off the tasks, then block on their results at the top level.

`await` also works on a **channel receive**, which is a special, degenerate case. A `Channel`'s `receive()` already blocks until a value is available, so `await ch.receive()` is a pass-through: the `await` adds no suspension of its own, it just reads the value the blocking receive produced. This lets `async`-flavored code interoperate with channels without special ceremony.

```
// producer thread fills the channel; the consumer awaits each receive
mutable int sum = 0;
for (mutable int i = 0; i < 5; i++) {
    sum = sum + await ch.receive();    // await on a channel receive == the blocking receive
}
```

9.4 `Mutex<T>` and `synchronized`

When two threads need to touch the same data, unsynchronized access is a bug. Polaron's answer is `Mutex<T>` (from `System.Concurrency.Mutex`), a lock that *owns* the value it protects rather than sitting beside it. You cannot reach the guarded value except through the lock, which makes it structurally impossible to read or write it without holding the mutex.

The only way in is the `synchronized` statement:

```
synchronized (m) using T& c {
    // c is a reference to the protected value; the lock is held for this whole block
}
```

`synchronized (m)` acquires the lock; `using T& c` binds `c` as a **reference** to the value inside the mutex, so reads and writes of `c` hit the protected storage directly; and the closing brace releases the lock. Crucially, the release is registered as a scope cleanup, which means it runs on **both** normal exit and exception unwinding — if the body throws, the lock is still released as the stack unwinds, so a failure inside a critical section cannot leave the mutex locked forever and deadlock the next thread that wants it.

```

import System.IO.Console;
import System.Concurrency.Thread;
import System.Concurrency.Mutex;
program MutexSynchronized;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                Mutex<int> counter = new Mutex<int>(0) on heap;
                Action* work = command () carries (Mutex<int> cell = counter) into pack
                returns void {
                    for (mutable int i = 0; i < 100000; i++) {
                        synchronized (pack.cell) using int& c {
                            c = c + 1;    // read-modify-write, protected by the lock
                        }
                    }
                };
                Thread t1 = new Thread(work) on heap;
                Thread t2 = new Thread(work) on heap;
                t1.start();
                t2.start();
                t1.join();
                t2.join();
                synchronized (counter) using int& c {
                    System.IO.Console.printf("total=%d\n", c);    // exactly 200000
                }
                return;
            }
        }
    }
}

```

Two threads each increment the shared counter one hundred thousand times. The `c = c + 1` inside the `synchronized` block is a read-modify-write, which is not atomic on its own; without the lock the two threads would routinely read the same value, both add one, and both write it back, losing an update. With the lock, the total is exactly `200000`. Note that both threads share the *same* mutex: the command carries `counter`, and `Mutex` is a heap object referred to through its handle, so both threads lock the one counter — which is why the data-race rule lets this baggage across at all.

The mutex generalizes to any type. A `Mutex<ArrayList<Dog>>` guards a whole list, and `synchronized (m) using ArrayList<Dog>& list { list.add(rex); }` mutates it safely.

There is one rule the compiler enforces: **you may not await while holding a mutex**. An `await` inside a `synchronized` block is a compile-time error. The reason is the classic deadlock: suspending an async task while it holds a lock can let the very work that must run to release it get stuck behind the held lock. Forbidding the combination outright removes an entire family of hangs.

9.5 `atomic<T>`

For a single integer counter or flag, a full mutex is heavier than necessary. `atomic<T>` (from `System.Concurrency.atomic`) is a **lock-free** cell whose operations compile directly to hardware atomic

instructions — no lock is taken, no thread ever blocks, and there are no lost updates. It is the lock-free counterpart to the mutex example above.

```
import System.IO.Console;
import System.Concurrency.Thread;
import System.Concurrency.atomic;
program AtomicCounter;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                atomic<int> counter = new atomic<int>(0) on heap;
                Action* work = command () carries (atomic<int> cell = counter) into pack
                returns void {
                    for (mutable int i = 0; i < 100000; i++) {
                        pack.cell.increment();    // one indivisible atomic add
                    }
                };
                Thread t1 = new Thread(work) on heap;
                Thread t2 = new Thread(work) on heap;
                t1.start();
                t2.start();
                t1.join();
                t2.join();
                System.IO.Console.printf("total=%d\n", counter.get());    // exactly 200000
                return;
            }
        }
    }
}
```

The atomic cell offers both a method form and an operator form. The methods are:

- `get()` — an atomic load of the current value.
- `set(v)` — an atomic store.
- `add(n)` — atomically add `n` and return the **new** value.
- `increment()` — add one (the special case of `add`).
- `compareAndSet(expected, desired)` — if the current value equals `expected`, store `desired` and return `true`; otherwise leave it unchanged and return `false`. This is the primitive on which lock-free algorithms are built.

The operators read more naturally for simple counters and lower to the same atomic instructions:

```
atomic<int> c = new atomic<int>(5) on heap;
c++;           // 6   (atomic add of 1)
c += 4;        // 10  (atomic add)
c--;           // 9   (atomic sub of 1)
c -= 2;        // 7   (atomic sub)
c = 100;       // atomic store of 100
int v = c.get();
```


Every one of these is sequentially consistent, the strongest and easiest-to-reason-about memory ordering, so you never have to think about instruction reordering when using them.

9.6 Channel<T>

Channels are the message-passing side of Polaron concurrency: instead of sharing memory and guarding it with a lock, threads communicate by handing values to each other. `Channel<T>`

(`System.Concurrency.Channel`) is a **bounded blocking queue**. You give it a capacity at construction. `send(v)` blocks while the channel is full, and `receive()` blocks while it is empty. This back-pressure is the point: a fast producer cannot outrun a slow consumer without limit, because it stalls once the buffer fills.

```
import System.IO.Console;
import System.Concurrency.Thread;
import System.Concurrency.Channel;
program ChannelDemo;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                Channel<int> ch = new Channel<int>(4) on heap;    // capacity 4
                Action* producer = command () carries (Channel<int> out = ch) into pack
                returns void {
                    for (mutable int i = 1; i ≤ 5; i++) {
                        pack.out.send(i);    // blocks once the buffer is full
                    }
                };
                Thread t = new Thread(producer) on heap;
                t.start();
                mutable int sum = 0;
                for (mutable int i = 0; i < 5; i++) {
                    sum = sum + ch.receive();    // blocks while empty
                }
                t.join();
                System.IO.Console.printf("sum=%d\n", sum);    // 15
                return;
            }
        }
    }
}
```

The producer sends `1..5` into a channel that holds only four, so it blocks on the fifth send until the consumer has taken one out. The consumer receives five values and sums them to `15`. Because every value crosses the channel one at a time under the channel's own lock, no external synchronization is needed: the channel is the synchronization. Values ride through as 64-bit slots, which covers integers and object references alike.

9.7 Channel.select

Sometimes you need to wait on *several* channels at once and act on whichever becomes ready first. Earlier drafts of the language had a `select` keyword for this; the language settled instead on a fluent builder, `Channel.select()`, which reuses ordinary static methods and commands and needs no new syntax. You chain a `.receive(channel, handler)` arm for each channel you want to watch, optionally a `.timeout(milliseconds, handler)` arm, and finish with `.run()`.

```
import System.IO.Console;
import System.Concurrency.Thread;
import System.Concurrency.Channel;
program ChannelSelect;

public bundle main {
  public namespace app {
    public class Main {
      public static method main(string[] args) returns void {
        Channel<int> a = new Channel<int>(4) on heap;
        Channel<int> b = new Channel<int>(4) on heap;
        Action* prod = command () carries (Channel<int> out = b) into pack returns void {
          pack.out.send(42);
        };
        Thread t = new Thread(prod) on heap;
        t.start();
        t.join(); // b now holds a value; a is still empty
        Channel.select()
          .receive(a, command (int x) returns void {
            System.IO.Console.printf("a=%d\n", x);
          })
          .receive(b, command (int y) returns void {
            System.IO.Console.printf("b=%d\n", y); // this arm fires: prints b=42
          })
          .run();
        return;
      }
    }
  }
}
```

The chain is entirely static — the compiler can see every arm at compile time — so it lowers the builder into a poll loop rather than a runtime data structure. Each pass tries a *non-blocking* receive on every listed channel; the first channel that has a value wins, and its handler runs with that value. Channel `b` holds `42` and `a` is empty, so the `b` arm fires and prints `b=42`.

A `.timeout` arm bounds the wait. When no channel becomes ready within the given number of milliseconds, the timeout handler runs instead, so `select` never hangs forever:

```

Channel<int> a = new Channel<int>(4) on heap;    // stays empty
Channel.select()
    .receive(a, command (int x) returns void {
        System.IO.Console.printf("a=%d\n", x);
    })
    .timeout(20, command () returns void {
        System.IO.Console.println("timeout");    // no value within ~20 ms: this runs
    })
    .run();

```

9.8 A note on data races

A data race — two threads touching the same location at the same time, at least one of them writing, with no synchronization between them — is undefined behavior in most systems languages and a source of some of the hardest bugs there are. Polaron does not make races impossible, but its concurrency toolkit is designed so that the *straightforward* way to share state is also the *safe* way.

Three ideas do the work. First, value semantics: assignment in Polaron is a deep copy, and a command's baggage is copied when the command is built, so handing work to another thread tends to hand it a private copy rather than a shared, aliased one. Two threads that each own their own copy cannot race over it — and where the baggage is a POINTER, the declaration says so, which is exactly what the compiler reads to refuse the crossing (`region-binder: this thread's work carries ...`) unless the type is `shareable`, an `atomic<T>`, a `Mutex<T>`, a `Channel<T>`, or was handed over with `move`. Second, when threads genuinely must share mutable state, `Mutex<T>` makes the shared value *unreachable* except while the lock is held, so you cannot forget to synchronize — the type system routes you through the lock — and the lock is released even when the critical section throws. Third, `atomic<T>` gives you correct shared counters and flags with no lock at all, and `Channel<T>` lets threads cooperate by passing values instead of sharing memory, which sidesteps the whole problem: if the only thing that crosses between threads is a message through a channel, there is nothing to race over.

The async side adds one guardrail of its own: the compiler forbids holding a mutex across an `await`, closing off the most common way async code deadlocks itself.

9.9 Concurrency is not available in freestanding mode

Freestanding programs (§36 of the specification) run without an operating system and without the Polaron runtime, so none of this chapter applies there. The compiler enforces the boundary directly: declaring an `async` method or writing an `await` in freestanding mode is a compile-time error. The thread, channel, mutex, and atomic-cell types likewise depend on OS threading primitives and the runtime's worker pool, none of which exist in a freestanding target. If you are writing a kernel or a bare-metal program, concurrency has to be built from whatever primitives your target hardware offers, outside the facilities described here.

10. Compile-Time, Reflection & Universal Prefixes

Most of what you have read so far describes what a Polaron program *does* when it runs. This chapter is about the two moments that bracket runtime: the work the compiler performs *before* the program exists, and the questions the program can ask *about itself* while it runs. Between those two poles sits a small, deliberately chosen vocabulary of **universal prefixes** — six keywords that attach to almost any declaration or operation and bend its lifetime, its evaluation time, or its optimization rules.

The through-line of the chapter is *when code executes*. `comptime` moves work to compile time so it costs nothing at runtime. `reflection` does the opposite, deferring type decisions to runtime. `lazy` slides an initializer from declaration time to first-use time. `eternal` stretches a lifetime to the end of the program; `cascade` widens an operation from one object to a whole graph; `volatile` forbids the optimizer from moving or erasing an access; and `final` freezes something so it can never change again. Once you see each feature as a decision about *timing*, they stop looking like a grab bag and start looking like one coherent idea.

A practical note runs through everything below: some of these features are pure compile-time or zero-runtime machinery and work everywhere, including **freestanding** mode (§11); others lean on the managed runtime and are therefore rejected in freestanding. The chapter flags which is which at each step, and §10.6 collects the answers in one place.

10.1 Compile-time evaluation

Polaron has a real compile-time evaluator. It is not a preprocessor and it is not string substitution: it interprets a genuine subset of the language — arithmetic, comparisons, locals, loops, recursion, and calls to other compile-time functions — and folds the result straight into the binary. Anything it computes has **zero runtime cost**, because by the time the program runs the answer is already a constant baked into the machine code.

Named constants with `fixed`

The simplest compile-time value is a named constant. In Polaron a constant is introduced with the `fixed` keyword (the specification's prose calls this a `const`; the compiler spells the keyword `fixed`). A `fixed` value must be initialized by a constant expression, and the analyzer evaluates that expression at compile time.

```
public class Cfg {
    public static fixed int MAX = 100;
    public static fixed int DOUBLED = Cfg.MAX * 2;    // 200, folded at compile time
}
```

A constant may refer to a sibling constant through its owning type (`Cfg.MAX`), and the evaluator resolves the whole chain before any code is generated. Because the result is a true constant, it can be used

anywhere the language expects a compile-time value — the condition of a `demand`, the count in a `new T[N]()`, another `fixed`.

This paragraph used to end *"including as the size of a stack array"*. There are no stack arrays: every array in Polaron is heap-allocated and dynamic (§4.5), and nothing in the compiler or the samples has ever accepted one. The sentence described a feature that does not exist, which is the one kind of error a manual cannot be trusted to catch about itself — it reads exactly like the true sentences around it.

What a type answers about itself

A type can be asked things directly, and every answer is folded into the binary as a constant:

```
int.sizeof()           // 4
Dog.typeName()         // "Dog"
Pace.length()          // how many members a closed family has -- enums only
Point.isValue()        // true: copied where it stands, not reached through a pointer
Dog.owns()             // true: destroying one does work
```

Also available: `align()`, `isMovable()` and `isUnique()`.

This is not reflection. Reflection (§10.4) is a runtime service: it costs, and it is unavailable in freestanding mode. These are facts the compiler is already holding, so asking costs nothing and works everywhere.

The point is generic code. Inside `Box<T>` the parameter *is* the concrete type by the time the question is answered, so `T` can be asked directly and no constraint has to promise anything:

```
public class Box<T> {
    public method holds() returns String { return T.typeName(); }
    public method frees() returns boolean { return T.owns(); }
}
```

Because each instantiation is checked on its own, a `T` that cannot answer is an error **at that instantiation**, naming the type that could not — `Pace.length()` is a count, and asking a class or a primitive for one is refused rather than answered with a zero.

`owns()` is the one to reach for when writing a container. A container of **values** whose element type has a destructor double-frees: each copy destroys what the original holds. Until this existed there was no way to ask in advance (§17.1).

`comptime` locals

Inside a method you can force a local to be computed at compile time by prefixing it with `comptime`. The initializer runs during compilation, and the variable becomes a constant you can use in constant contexts — for example, as an array length:

```
comptime int x = 2 + 3 * 4;    // evaluated to 14 during compilation
int[] buf = new int[x]();     // a comptime value is a valid array size
```

comptime methods

A method marked `comptime` runs at compile time whenever it is called with constant arguments. This is the mechanism behind precomputed lookup tables, perfect hashes, and build-time configuration. The evaluator handles recursion, iteration, and local mutation — it is a small interpreter, not a peephole folder:

```
public class Math {
  public static comptime method fib(int n) returns int {
    if (n < 2) { return n; }
    return Math.fib(n - 1) + Math.fib(n - 2);    // recursion at compile time
  }
  public static comptime method factorial(int n) returns int {
    mutable int acc = 1;
    for (mutable int i = 2; i ≤ n; i++) {        // a loop, at compile time
      acc = acc * i;
    }
    return acc;
  }

  public static fixed int FIB10 = Math.fib(10);    // folds to 55
  public static fixed int FACT5 = Math.factorial(5); // folds to 120
}
```

The evaluator is numeric over both integers and doubles, so a `comptime` method may mix an integer loop counter with a floating-point accumulator and return a `double` constant:

```
public static comptime method scale(int n, double factor) returns double {
  mutable double acc = 0.0;
  for (mutable int i = 0; i < n; i++) { acc = acc + factor; }
  return acc;
}
public static fixed double TOTAL = Math.scale(3, 1.5);    // folds to 4.5
```

`comptime` can also be written as a trailing keyword on the method signature, matching the spec's §28.3 form; the prefix form shown above is the one used throughout the samples and is preferred for readability.

comptime if

A `comptime if` chooses its branch at compile time from a constant condition, and — crucially — the branch that is not taken produces *no code at all*. This is how you compile out debug instrumentation or select a code path per build without a runtime test:

```

public class Cfg {
    public static fixed boolean DEBUG = false;
    public static fixed int LEVEL = 2;
}

comptime if (Cfg.DEBUG) {
    System.IO.Console.printf("debug on\n");
} else {
    System.IO.Console.printf("debug off\n");    // only this branch is emitted
}

comptime if (Cfg.LEVEL ≥ 2) {
    System.IO.Console.printf("level high\n");    // and only this one
}

```

Compile-time string DSLs

Because the evaluator understands string operations too, a `comptime` method can take a `comptime string` parameter, parse it during compilation, and fold the result to a constant — a compile-time DSL with no runtime parser. The classic use is validating a query, a regex, or a format string before the program ever runs. Here a `comptime` method counts the columns in a CSV header:

```

public static comptime method colCount(comptime string s) returns int {
    mutable int n = 1;
    mutable int i = 0;
    while (i < s.length()) {
        if (s.charAt(i) == ',') { n = n + 1; }
        i = i + 1;
    }
    return n;
}

public static fixed int COLS = Sql.colCount("id,name,email,age");    // folds to 4

```

The argument to a `comptime` parameter must itself be a compile-time constant. Passing a runtime value is a compile error, which keeps the "no runtime parser" guarantee honest:

```

mutable string r = "hello";
int n = Sql.len(r);    // error: the argument to a comptime parameter must be constant

```

demand

`demand <condition> otherwise "why";` settles a condition while the program is being built. It is a **statement**, not a call — it stands beside `return`, `delete`, `throw` and `release`, takes no receiver, returns nothing and emits nothing. It shares the one compile-time evaluator with `comptime` and `fixed`, so its condition may reference named constants *and* call `comptime` methods:

```

demand 2 + 2 == 4 otherwise "math is broken";
demand 16 * 1024 < 65536 otherwise "the buffer does not fit";
demand Math.fib(10) == 55 otherwise "fib(10) must be 55";    // calls a comptime method
demand Fruit.count() == 3 otherwise "three fruit are drawn on the labels";    // an enum's size

```

An **enum's** `count()` folds here because an enum is a closed set written out in the source: its size is settled the moment the declaration is parsed. That is what lets a table numbered across several enums hold its own arithmetic together — give each family a slice of one index space and the offset of a family is the size of everything before it, so

```
demand 3 == Fruit.count() otherwise "crates are numbered after the fruit";
```

stops the build when somebody adds a fourth fruit, instead of silently renumbering every crate. A *class* with a static `count()` of its own is not an enum and stays an ordinary runtime call.

If the condition holds, nothing at all reaches the executable. If it fails, compilation stops:

```
error[Polaron-0806]: demand not met: fib(10) must be 55
```

The `otherwise` text is the **reason**, not a restatement of the condition. The condition already says `== 55`; the message is there to say why 55 matters, and it is what the failure prints.

A demand belongs in a method body, like any other statement. Its condition must be knowable at build time — a non-constant condition is refused rather than deferred to runtime, because a check that silently moves to runtime is a guarantee you have lost without being told.

Use it for the assumptions your code depends on: table lengths, protocol constants, arithmetic that has to come out a certain way. For **the size of a type**, reach for a `layout` instead — a byte budget is a property of the type rather than a condition about the program, and a layout lets the compiler arrange the fields to meet it instead of merely refusing when they do not.

Guardrails

The compile-time evaluator is bounded. A recursion or step budget protects the compiler from programs that would otherwise loop forever at build time — an unbounded `comptime` recursion is rejected with a clear error rather than hanging the compiler:

```
public static comptime method spin(int n) returns int {  
    return M.spin(n) + 1;    // unbounded: rejected by the step budget, does not hang the build  
}
```

Compile-time evaluation, `fixed`, `comptime`, `comptime if`, and `demand` are all pure compile-time machinery. They emit no runtime support code, so they are fully available in **freestanding** mode.

10.2 Reflection

Reflection is the runtime counterpart to compile-time evaluation: instead of the compiler asking questions about your program, the *program* asks questions about its own types while it runs. Polaron exposes this through the `reflect` namespace, and using any of it requires an explicit import:

```
import reflect;
```


Forgetting the import is an error (`reflection` requires `'import reflect;'`). Reflection is also the one feature in this chapter that is **explicitly rejected in freestanding mode** — it depends on runtime type information, the real `String`, and `stdlib` collections such as `ArrayList`, none of which exist in the bare-metal subset. Attempting it there fails with `reflection is not available in freestanding mode`.

Type tokens

The entry point is `reflect.typeOf<T>()`, which returns a `Type` token describing a class. From a `Type` you can read the class name as a real `String`:

```
Type t = reflect.typeOf<Dog>();
System.IO.Console.println(t.name());    // Dog
String n = t.name();
System.IO.Console.printf("len=%d\n", n.length());    // 3
```

Enumerating members

A `Type` exposes the class's declared methods and fields. You can enumerate them by name through the `count/index` pair, or take them as collections:

```
Type t = reflect.typeOf<Dog>();
System.IO.Console.printf("methods=%d fields=%d\n", t.methodCount(), t.fieldCount());
for (mutable int i = 0; i < t.methodCount(); i++) {
    System.IO.Console.println(t.methodName(i));
}
ArrayList<Method> ms = t.methods();    // the collection form
ArrayList<Field> fs = t.fields();
```

Reading and writing fields

A `Field` token reads and writes a field on a live object. Values cross the reflection boundary boxed as `Object` — a reference passes through directly, and a primitive is boxed, so you cast it back on the way out:

```
Point p = new Point() on heap;
Field f = reflect.typeOf<Point>().fields().get(0);
int before = cast<int>(f.get(p));    // read, unboxing the Object
f.set(p, 99);                        // write it back
int after = cast<int>(f.get(p));    // 99
```

Invoking methods and constructing objects

A `Method` token dispatches a call by name (scoped to no-argument methods), and `Type.instantiate()` allocates an object and runs its no-argument constructor, returning it as `Object`. Together they let you drive an object known only at runtime:

```
Type t = reflect.typeOf<Dog>();
Dog d = cast<Dog>(t.instantiate());    // dynamic construction via the no-arg ctor
Method m = t.method("bark");
System.IO.Console.println(m.name());    // bark
m.invoke(d);                          // Woof!
t.method("sit").invoke(d);             // chained lookup + invoke
```

Reflecting over annotations

Both `Type` and `Method` report the annotations applied to them, as an `ArrayList<Annotation>`; each `Annotation` exposes its `name()`. This is what makes user annotations useful — a framework can discover them at runtime and act on them:

```
Type t = reflect.typeOf<Widget>();
ArrayList<Annotation> anns = t.annotations();
System.IO.Console.printf("count=%d\n", anns.size());
for (Annotation a in anns) {
    System.IO.Console.println(a.name());    // Marker, Audited
}
```

A `Method` mirrors this with its own `annotations()`, so you can, for instance, find every method tagged `[Audited]` and wrap it.

10.3 Annotations

Annotations attach declarative metadata to a class, method, or field without changing what that declaration *does*. Polaron keeps a sharp line between language concepts and user metadata: built-in modifiers such as `override`, `final`, `static`, and `volatile` are **keywords**, while everything else that decorates a declaration is an annotation.

The two spellings

An annotation may be written either way, and the two are **exactly** equivalent — same meaning, same parsed result:

<code>[Test]</code>	<code>@Test</code>
<code>[Ignore(reason: "flaky")]</code>	<code>@Ignore(reason: "flaky")</code>
<code>[MaxLength(value: 100)]</code>	<code>@MaxLength(value: 100)</code>

This holds for both the annotations the standard library ships and the ones you declare, because there is no difference between them: a built-in is simply an annotation the stdlib declares. Pick one spelling and stay with it; this documentation uses `[Name]` throughout.

Arguments are always **named** (`field: value`), never positional, in both spellings.

A compiler **attribute** is a different thing and keeps its own spelling: double brackets, no `@` form — `[[no_bounds_check]]` (§11). Attributes instruct the compiler; annotations carry metadata.

Declaring an annotation

You declare an annotation with the `annotation` keyword. Its body lists fields, each of which is either required or carries a `default`:

```
public annotation MaxLength {
    int value;
    String errorMessage default "too long";    // optional, has a default
}
```

Applying an annotation

Applications sit on the line above the thing they annotate and pass fields by name. They can decorate a class, a method, or a field:

```
[MaxLength(value: 100, errorMessage: "name too long")]
public class UserName {
    [MaxLength(value: 50)]
    private mutable int length;

    [Marker]
    public method length() returns int { return this.length; }
}
```

The compiler validates every application: the annotation must exist, each named argument must be a real field of that annotation, and every required field must be supplied. Once validated, the annotations ride along on the declaration and become visible to reflection (§10.2).

[CompileTimeProcessor]

[CompileTimeProcessor] is a built-in annotation you apply *to another annotation* to mark it for compile-time processing — the hook by which an annotation can drive code generation during the build rather than being read back at runtime:

```
[CompileTimeProcessor]
public annotation Marker { }
```

An annotation carrying [CompileTimeProcessor] is recognized by the compiler as a build-time concern; a plain annotation is ordinary runtime metadata discovered through reflection.

10.4 Lifecycle hooks

Constructors and destructors describe the life of an *instance*. Lifecycle hooks describe the life of the *class itself* — moments that have nothing to do with any single object. They are written as bare named blocks in the class body, with no visibility and no parameters:

```
public class Server {
    onClassLoad          { System.IO.Console.printf("Server loaded\n"); }
    onFirstInstance      { System.IO.Console.println("pool setup"); }
    onLastInstanceDestroyed { System.IO.Console.println("pool teardown"); }
    onClassUnload        { System.IO.Console.println("cache unloaded"); }
}
```

Each hook fires at a precise moment:

- **onClassLoad** runs once, at program start, *before* `main` — even if the class is never instantiated. It is the place for class-level setup that must exist before anyone uses the type.
- **onFirstInstance** runs when the count of live instances first leaves zero ($0 \rightarrow 1$). It is backed by a per-class counter maintained in the constructor, and is ideal for lazily setting up state that is shared across all instances.

- **onLastInstanceDestroyed** runs when the last instance is destroyed ($1 \rightarrow 0$). The counter is decremented in the destructor; this is the matching teardown for shared state.
- **onClassUnload** runs when the class is `unimport`ed (`$ unimport`), right before its code is torn out of memory. Because `unimport` belongs to the managed runtime, this hook only fires in managed programs.

The counter-based hooks make a common pattern trivial: a connection pool sets up on the first live object and tears down when the last one dies, without any global flag written by hand.

`unimport` and `reimport` : hot unloading a class

Polaron's managed runtime can unload a class's *code* at runtime and load it back. This is the basis for hot-reload and plugin systems.

`unimport X` does two things: it marks the class dead, and it physically overwrites its machine code in memory with trap instructions (`int3`, byte `0xCC`) — the code is really gone, not just flagged. Any later use of the type — a `new`, or a method call on a surviving instance — is caught by the alive guard and throws `UnimportedTypeException`, so control never branches into the destroyed code. **`onClassUnload`** (above) fires just before the code is torn out.

```
import System.Runtime.UnimportedTypeException;

unimport Dog;
try {
    a.bark(); // a survives, but Dog's code is gone
} catch (UnimportedTypeException e) {
    System.IO.Console.WriteLine("Dog is unloaded");
}
```

`reimport X` brings the code back by reloading the class's original bytes from the program's own file on disk, after which the type works normally again.

For safe hot-reload you usually want to *verify* that the reloaded code is the version you expect. The **`expecting`** challenge–response does this: `unimport X expecting using <inputs> { ... }` runs a block in the **old** code to produce a validation value, and the matching `reimport X expecting <value> using <inputs> { ... }` on `Failure` { ... } runs the block in the **new** code and compares its result, bit for bit, with the saved value. On a match the program continues; on a mismatch the `onFailure` block runs instead.

```
int salt = 7;
var proof = unimport Plugin expecting using salt {
    return Plugin.fingerprint(salt);
};

reimport Plugin expecting proof using salt {
    return Plugin.fingerprint(salt); // new code must reproduce `proof`
} onFailure {
    System.IO.Console.WriteLine("rejected: the reloaded code is not the expected version");
};
```

Because `unimport/reimport` depend on the managed runtime, they are unavailable in freestanding mode (see §10.6).

10.5 The universal prefixes

Six keywords are promoted to **universal prefixes**: `cascade`, `eternal`, `lazy`, `comptime`, `volatile`, and `final`. Each has one consistent meaning that it carries into every context where it is legal, so they compose into precise intentions without a proliferation of one-off keywords. A prefix modifies *the thing it precedes* — a declaration, or, for `cascade` / `lazy` / `comptime`, an operation — and the sections below take them one at a time.

`cascade` — propagate an operation across an owned graph

`cascade` runs an operation not just on its target but recursively across everything that target **owns**. The rules are what make it safe: it follows composition (owned value fields and owned pointers) but skips associations — a pointer marked `external` is treated as a back-reference and is not followed — it detects cycles with a runtime visited-set so each node is touched exactly once, and it respects visibility.

The most common form is `cascade delete`, which frees an object and everything reachable through ownership. A plain `delete` would run only the outermost destructor and leak the rest:

```
Outer o = new Outer() on heap;
cascade delete o;    // runs ~Outer, then ~Mid, then ~Leaf – the whole owned graph
```

This works through *inherited* ownership too: if a value field is declared in a base class, cascade-deleting a derived object still frees it. Ownership propagates the same way through owned pointers, and the visited-set makes even a cyclic owned graph free exactly once:

```
Node* a = new Node() on heap;    // a → b → c → a, an owned cycle
// ... link them ...
b.owner = a;                    // an `external` pointer: an association, skipped
cascade delete a;                // ~Node fires once per node, never twice
```

`cascade` generalizes well beyond deletion. `cascade clone X into Y` deep-copies X's entire owned graph into a fresh graph stored in Y, sharing nothing — mutating the clone leaves the original untouched. `cascade move t from region old to region fresh` relocates an object and its owned graph between regions (§ regions), so the graph survives even after the source region is released. And any plain operation can be cascaded across the graph — `cascade validate(x)` checks each node's invariant, and `cascade Console.println(x)` describes every node:

```
cascade clone a into c;          // c is an independent deep copy of a's graph
cascade move t from region old to region fresh; // t and its Leaf now live in `fresh`
cascade validate(a);             // check the invariant of every node
cascade Console.println(a);      // describe every node, once each
```

Optional parameters bound the walk — `cascade(depth: 3) delete tree`, `cascade(except: {World}) delete player`, `cascade(types: {Item, Pet}) delete inventory`. `cascade` is a runtime graph walk with no reliance on the managed services, so it works in **freestanding** — with the single exception of `cascade unimport`, which needs the managed runtime and is rejected there along with plain `unimport`.

eternal — a lifetime that spans the whole program

eternal declares that a resource lives for the entire run of the program. The compiler stops requiring explicit cleanup, and — importantly — an eternal object's destructor is **not** run at scope end:

```
public class Res {
    public destructor ~Res() returns void { System.IO.Console.printf("freed\n"); }
    public method use() returns int { return 1; }
}

eternal Res r = new Res() on stack;    // ~Res will NOT run when the scope ends
System.IO.Console.printf("using=%d\n", r.use());
// no "freed" is ever printed
```

This is the right tool for genuinely program-long state — a global cache, a logger, a config table — where scope-based destruction would be wrong. **eternal** is a lifetime decision the compiler enforces by *omitting* cleanup code, so it adds no runtime machinery and is available in **freestanding**.

lazy — defer initialization until first access

lazy moves an initializer from declaration time to the first time the value is actually read, and runs it at most once. A **lazy** binding that is never touched is never initialized at all — which is exactly the point when the initializer is expensive:

```
lazy Thing a = new Thing(1) on heap;    // deferred
lazy Thing b = new Thing(2) on heap;    // never accessed → never constructed
int x = a.get();    // first access triggers construction of `a` now
int y = a.get();    // already built → not repeated
```

The same applies to a class **field**: a **lazy** field is built on first access rather than during construction, so building a **Holder** does not yet build its **Thing**:

```
public class Holder {
    private lazy Thing* thing = new Thing() on heap;    // not built at construction
    public method use() returns int { return this.thing.val(); }    // first call builds it
}
```

lazy extends to other resources with the same "on first reference" meaning — **lazy region**, **lazy import**, and **lazy thread** all defer their heavy step until the resource is first used. The specification defines lazy initialization as **thread-safe**, guarded by an implicit mutex so two threads racing on the first access still build exactly once. That thread-safe deferred initialization is part of the managed runtime, so **lazy** — unlike the other prefixes in this section — is **not** part of the freestanding subset.

comptime — evaluate at compile time (as a prefix)

Used as a prefix, **comptime** is the same feature described in §10.1: it forces the thing it precedes to be evaluated during compilation. As a local declaration it produces a compile-time constant (**comptime** `int x = 2 + 3 * 4;`); as a method modifier it makes the method run at build time; and as a statement prefix it drives **comptime if**. Because it is pure compile-time machinery it is fully available in **freestanding**.

volatile — never reorder, never elide

volatile tells the optimizer that an access has meaning beyond its value: it must not be reordered, duplicated, folded, or removed. Every read performs a real load and every write performs a real store. This does not change the result of a single-threaded program, but it is essential for memory-mapped I/O and hardware registers, whose values change outside the program's control:

```
public class Device {
    private volatile mutable int reg;
    public method poke(int v) returns void { this.reg = v; } // every store really happens
    public method peek() returns int { return this.reg; } // every load really fetches
}

Device d = new Device() on heap;
d.poke(7);
d.poke(42); // the first write is NOT folded away, even though it looks dead
```

volatile composes onto more than fields. A **volatile method** is emitted **noinline** / **optnone**, so it is always executed and never optimized away; and a **volatile region** marks every access to objects placed in it as volatile — the natural way to describe an MMIO window:

```
public class Sensor {
    public volatile method read() returns int { return 42; } // never inlined or elided
}

volatile region mmio = itself.allocate(4 kilobytes);
Reg* r = new Reg() in region mmio;
r.ctrl = 7; // a volatile store into mapped memory
```

volatile is an instruction to the code generator, not a runtime service, so it is available in **freestanding** — where, being the language of MMIO, it is most at home.

final — cannot be changed, overridden, or extended

final freezes something. On a field or a local it means explicitly immutable — the value is set once and can never be reassigned (this is the deliberate opposite of **mutable**):

```
public class C {
    private final int x;
    public constructor C() { this.x = 5; } // set once, in the constructor
}

final int y = c.get(); // an explicitly immutable local
```

On a class, **final** forbids inheritance; on a method, it forbids overriding. Used correctly these compile like any other declaration; misused, they are hard errors — extending a **final class** or overriding a **final method** is rejected:

```
public final class Color {
    public final method value() returns int { return this.rgb; }
}

// public class Sub extends Color { ... } // error: cannot extend a final class
```

`final` also applies to `import` — `final import Dog` declares an import permanent and blocks any later `unimport`. `final` is a compile-time constraint with no runtime footprint, so it is available in **freestanding**.

Composition, canonical order, and invalid combinations

Prefixes combine freely when their meanings are compatible, which is where the "compositional vocabulary" idea pays off. `eternal lazy region` is a region that lives forever but only allocates its backing block when the first object enters it; `eternal comptime int` is a constant computed at build time and kept for the whole program; `volatile comptime int` reads a hardware base address at compile time yet forbids caching it at runtime.

To keep declarations readable, the compiler enforces a **canonical order** of modifiers:

```
[annotations]
[visibility] [eternal] [lazy] [final] [comptime] [volatile] [cascade] [static] [mutable] ...
```

Writing them out of order (for example `public static eternal lazy int x`) is rejected with a suggestion to reorder. For operations rather than declarations, only `cascade`, `lazy`, and `comptime` apply, in that order.

Contradictory combinations are compile errors, because they ask for two incompatible things at once:

- `mutable final` — mutable means reassignable, `final` means it cannot be reassigned.
- `comptime volatile` — computed before runtime, yet allowed to change at runtime.
- `persistent transient` — survives the destructor, yet is not serialized.

The compiler reports each with a message that names the conflict and tells you to keep only one.

10.6 Availability in freestanding mode

Freestanding mode (§11) drops the managed runtime, so the features in this chapter split cleanly by whether they need it. The table below is the summary; each rule is enforced by the analyzer.

Feature	Freestanding?	Why
<code>fixed constants</code> , <code>comptime (local / method / if)</code> , <code>demand</code> , <code>layout</code>	Yes	Pure compile-time; emit no runtime code.
<code>cascade (delete / clone / move / validate / ...)</code>	Yes	A self-contained runtime graph walk.
<code>cascade unimport</code>	No	Unimport is a managed-runtime service.
<code>eternal</code>	Yes	A lifetime decision the compiler enforces by omitting cleanup.
<code>volatile (field / local / method / region)</code>	Yes	A code-generation instruction; the language of MMIO.
<code>final (field / local / class / method /</code>	Yes	A compile-time constraint with no runtime footprint.

Feature	Freestanding?	Why
import)		
lazy (local / field / region / import / thread)	No	Thread-safe deferred init belongs to the managed runtime.
Reflection (reflect.typeOf , Type , Field , Method , Annotation)	No	Explicitly rejected; needs RTTI, String , and stdlib collections.
Lifecycle hooks	Partial	onClassLoad / onFirstInstance / onLastInstanceDestroyed are ordinary; onClassUnload fires only where unimport exists (managed).

The unifying idea is worth repeating: everything that happens *before* runtime, or that costs *nothing* at runtime, survives into freestanding; everything that leans on managed services — reflection, thread-safe laziness, unimport — does not. Keep that test in mind and you can predict the availability of any prefix or feature without consulting the table.

11. Systems Programming: Builtins, FFI & Freestanding

Most of what you have read so far is about *the language*: classes, generics, regions, ownership, pattern matching. This chapter is about the layer beneath the language — the place where Polaron stops being an abstraction over "objects and methods" and becomes an abstraction over "bytes and addresses." It covers three closely related topics: the **compiler builtins** that the rest of the standard library is built on, the **foreign function interface (FFI)** that lets Polaron talk to C and to the operating system, and **freestanding mode**, in which Polaron runs with no operating system underneath it at all.

The unifying idea is *no hidden machinery*. A high-level language earns the right to hide the machine behind convenient syntax; a systems language earns the right to *show* it when you ask. Polaron tries to do both: the ordinary object model stays clean, and when you need to reach through it — to a hardware register, a C library, a fixed physical address — the primitives in this chapter are waiting.

11.1 Compiler builtins vs. the standard library

It helps to draw a line first. Polaron has two very different kinds of "built-in" thing:

- **The standard library** is ordinary Polaron code — classes like `ArrayList<T>`, `HashMap<K, V>`, `Strings`, `Files`, `Regex`. It is written in the language, compiled like your code, and you reach it with `import`. You could, in principle, have written it yourself.
- **Compiler builtins** are types and operations the compiler *knows about intrinsically*. They are not classes you could write in Polaron, because they lower directly to machine-level constructs: a `String`'s `{length, data}` layout, a `vec4`'s SSE register, a `Decimal`'s 128-bit mantissa, a raw `Memory.write<int>`. They are the bedrock the standard library stands on.

This distinction matters for the rest of the chapter, and especially for freestanding mode — where the builtins are always available, and the library is available in the part that does not need an operating system, which turns out to be most of it. Collections, `Buffer`, `Math` and the value types all work bare metal; what does not is the handful of things that call the host — see [what is removed](#) for the list and the reasoning.

Let us walk through the builtins first.

11.2 Strings: `String` and `string`

Polaron gives you two string builtins with deliberately different personalities.

`String` is the **immutable** one. Once constructed, its bytes never change; every operation that "modifies" it actually returns a fresh `String`. This makes it safe to share and cheap to reason about. It is the type most APIs speak.

```

import System.IO.Console;
program StringDemo;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                String a = "Hello";
                String b = "World";
                String hw = a.concat(", ").concat(b).concat("!"); // a new String each step
                System.IO.Console.println(hw); // Hello, World!
                System.IO.Console.printf("len=%d empty=%d\n", hw.length(), hw.isEmpty());
                System.IO.Console.printf("c0=%c c7=%c\n", a.charAt(0), hw.charAt(7));
                System.IO.Console.println(hw.substring(7, 12)); // World
                System.IO.Console.printf("eq=%d\n", a.equals("Hello")); // 1
                return;
            }
        }
    }
}

```

The core `String` methods the compiler recognizes are `length()`, `isEmpty()`, `charAt(int)`, `equals(String)`, `concat(String)`, and `substring(int, int)`. The allocating ones (`concat`, `substring`) build a new heap `String`; the query ones do not allocate. Everything richer — `split`, `join`, `replace`, formatting — lives in the `Strings` helper class in the standard library, which is just Polaron code layered on top of these primitives.

`string` (lowercase) is the **mutable** counterpart: a growable buffer you build up in place.

```

string m = "hi";
m.append("!");
m.append(" there"); // "hi! there", grown in place
System.IO.Console.println(m);

```

Both string types obey Polaron's value semantics: assigning a string **copies** it. Mutating the copy leaves the original untouched — no aliasing surprises.

```

string a = "x";
string b = a; // a deep copy
b.append("y");
// a is still "x"; b is "xy"

```

Under the hood a string is a `{length, data}` pair: an `i64` length followed by a pointer to the character bytes. You rarely see that layout — but when you hand a `String` to a C function (§11.7), it is the raw `data` pointer that crosses the boundary, and knowing the layout explains why.

11.3 `$" ... "` string interpolation

Interpolation is the ergonomic way to build a `String` from a mix of literal text and expressions. Inside a `$" ... "` literal, anything in `{ ... }` is evaluated and stitched into the result. The value produced is a real

`String` object — you can store it, take its `length()`, pass it on.

```
int n = 7;
String who = "world";
String s = $"n={n}, hello {who}!";           // "n=7, hello world!"
System.IO.Console.WriteLine(s);
System.IO.Console.Printf("len=%d\n", s.Length());
String t = $"{n} doubled is {n + n}";        // nested expressions are fine
```

The compiler infers how to render each hole from the expression's type — integers, characters, other strings, and so on — so you never write format specifiers inside `" ... "`. Interpolation is one of the few conveniences that survives into freestanding mode, precisely because it can be lowered without a dynamic `String` allocation when the result is only consumed, not stored.

11.4 `Decimal`: exact fixed-point arithmetic

Binary floating point (`float`, `double`) is fast and wrong for money: `0.1 + 0.2` is famously not `0.3`. Polaron's answer is the `Decimal` builtin — an exact base-10 fixed-point type backed by a 128-bit mantissa scaled by `1018`, giving eighteen fraction digits with no binary rounding error. Decimal literals carry an `m` suffix, which is what tells the compiler "this is a `Decimal`, not a `double`."

```
import System.IO.Console;
program Money;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                Decimal price = 19.99m;
                Decimal qty   = 3m;
                Decimal total = price * qty;           // exactly 59.97, not 59.9699999...
                System.IO.Console.WriteLine($"total={total}");
                Decimal tax = total * 0.08m;
                System.IO.Console.WriteLine($"tax={tax}");
                Decimal half = cast<Decimal>(5) / cast<Decimal>(2); // exactly 2.5
                System.IO.Console.WriteLine($"half={half}");
                return;
            }
        }
    }
}
```

`Decimal` arithmetic (`+` `-` `*` `/`) is exact, it compares and orders like any number, it converts to and from the ordinary numeric types with `cast<...>`, and its textual form (via `toString` or interpolation) trims trailing fraction zeros so `59.97m` prints as `59.97`, not `59.970000...`. Reach for `Decimal` whenever a rounding error would be a bug — currency, tax, billing, ledgers.

11.5 Low-level memory: `address`, raw pointers, and the `Memory` API

This is where Polaron opens the floor and lets you touch memory directly. Three builtins work together.

`address` is a primitive integer holding a raw machine address with no type attached — the Polaron equivalent of `uintptr_t`. You can do arithmetic on it and cast it to and from typed pointers.

It is sixty-four bits on every target, including the 32-bit ones, and that is a decision rather than an accident. The obvious alternative — let `address` be as wide as the machine — makes the same source describe two different types depending on where it is compiled, and a program that moves between targets then drifts in ways nothing diagnoses. So the width is fixed and the narrow forms are what a program reaches for when it means a specific one:

type	bits
<code>address</code>	64, everywhere
<code>half address</code>	32 — a machine pointer on i686 and the other 32-bit targets
<code>short address</code>	16 — an I/O port, a real-mode segment
<code>byte address</code>	8

On a 32-bit target, an FFI declaration that means "a pointer the callee will read" is written `half address`. Writing `address` there is not refused — it is a 64-bit value, which the C ABI on those machines passes in two words.

Int↔pointer casts convert freely between an `address` (or any pointer-sized integer) and a typed pointer `T*`, using `cast<T*>(addr)` one way and `cast<address>(ptr)` the other. This is the reinterpretation escape hatch: "treat these bytes as an array of `int`."

Raw pointer indexing `p[i]` on a pointer obtained this way is an *unchecked* access — a plain pointer-plus-offset (a GEP), with no array header and no bounds check. It is exactly `*(p + i)` in C terms. That is the point: over hardware you do not get a length word, so you must not assume one.

The `Memory` API is the typed front door to a raw block:

- `Memory.alloc(bytes)` returns an `address` to a fresh raw block.
- `Memory.free(address)` releases it.
- `Memory.read<T>(address)` reads a `T` from an address; `Memory.write<T>(address, value)` writes one.
- `Memory.getMemory(x)` returns the raw `address` backing a value.

```

import System.IO.Console;
import System.Memory;
program LowLevelMemory;

public bundle main {
    public namespace app {
        public class Main {
            public static method main(string[] args) returns void {
                address mem = Memory.alloc(16);           // raw block → an address
                int* p = cast<int*>(mem);                   // reinterpret the address as int*
                p[0] = 100;                                  // unchecked raw write
                p[1] = 200;
                System.IO.Console.printf("sum=%d\n", p[0] + p[1]);           // 300

                Memory.write<int>(mem, 999);
                System.IO.Console.printf("read=%d\n", Memory.read<int>(mem)); // 999

                address back = cast<address>(p);           // pointer → address round-trip
                System.IO.Console.printf("delta=%d\n", cast<int>(back - mem)); // 0
                Memory.free(mem);
                return;
            }
        }
    }
}

```

Everything here is a direct machine operation — a load, a store, an add. Nothing calls into a garbage collector or a bounds-checked collection. That is what makes this API usable in freestanding mode, where `Memory.alloc` maps onto whatever allocator *your kernel* provides rather than libc's `malloc`.

11.6 SIMD vectors: `vec2` / `vec3` / `vec4` and `mat4`

Graphics and physics math is dominated by small fixed-size vectors of floats. Polaron makes them first-class builtins that lower straight to LLVM `<N x float>` — that is, to SSE/AVX registers — so a component-wise add is one instruction, not a loop.

`vec2`, `vec3`, and `vec4` are float vectors of two, three, and four lanes. You construct them by calling the type, do element-wise `+` `-` `*` `/` (a scalar operand *broadcasts* to all lanes), read lanes with `.x` / `.y` / `.z` / `.w` or with `[i]`, and pass or return them by value.

```

import System.IO.Console;
program Simd;

public bundle main {
    public namespace app {
        public class Main {
            public static method dot3(vec3 a, vec3 b) returns float {
                vec3 p = a * b;           // element-wise multiply
                return p.x + p.y + p.z;
            }
            public static method main(string[] args) returns void {
                vec4 a = vec4(1.0, 2.0, 3.0, 4.0);
                vec4 b = vec4(10.0, 20.0, 30.0, 40.0);
                vec4 c = a + b;           // element-wise
                vec4 d = a * 2.0;         // scalar broadcast → (2,4,6,8)
                System.IO.Console.printf("cx=%d dz=%d\n", cast<int>(c.x), cast<int>(d.z));

                vec3 u = vec3(1.0, 2.0, 2.0);
                System.IO.Console.printf("len=%d\n", cast<int>(u.length()));    // 3
                return;
            }
        }
    }
}

```

Vectors also carry the geometry methods you expect: `a.dot(b)`, `a.length()`, `a.normalize()`, and (on `vec3`) `a.cross(b)`.

```

vec3 a = vec3(1.0, 2.0, 2.0);
vec3 b = vec3(3.0, 0.0, 0.0);
float d = a.dot(b);           // 3
float l = a.length();         // 3
vec3 n = a.normalize();       // (1/3, 2/3, 2/3)
vec3 x = a.cross(b);          // (0, 6, -6)

```

For transforms there is `mat4`, a 4×4 float matrix laid out row-major as `<16 x float>`. It offers `mat4.identity()`, matrix×matrix `multiply`, and matrix×vector `transform`.

```

mat4 id = mat4.identity();
vec4 p = vec4(1.0, 2.0, 3.0, 1.0);
vec4 tp = id.transform(p);    // identity * p = p
mat4 t = mat4(1.0, 0.0, 0.0, 5.0,
              0.0, 1.0, 0.0, 6.0,
              0.0, 0.0, 1.0, 7.0,
              0.0, 0.0, 0.0, 1.0);
vec4 q = t.transform(vec4(0.0, 0.0, 0.0, 1.0)); // origin → (5, 6, 7)

```

These types were added to make real 3D code — the kind that used to require a C++ math library — expressible directly in Polaron without giving up performance.

11.7 `System.IO.Console` : the managed console

`System.IO.Console` is the built-in console. It is a *managed* builtin: it assumes an operating system with standard streams underneath it, which is why it is one of the things freestanding mode takes away (§11.9). In hosted programs it is your everyday I/O, and it must be imported.

```
import System.IO.Console;
// ...
System.IO.Console.printf("x=%d y=%c\n", 42, 'A'); // format string + args
System.IO.Console.print("no newline");
System.IO.Console.println("with newline");
String line = System.IO.Console.read();           // read a line, returns a String
```

`printf` takes a format string (a string literal or a `" ... "` interpolation) followed by arguments; `print` and `println` take a string literal, an interpolation, or a `String` value; `read()` returns a whole line as a `String` (parse it yourself for other types, e.g. `toInt`). Note the deliberate design tension: the console is the most convenient builtin *and* the first one a kernel loses, because printing to a screen without an OS means writing bytes to a hardware buffer yourself — which is exactly what §11.9's VGA example does.

11.8 Foreign Function Interface (FFI)

No systems language is an island. Sooner or later you must call C: `libc`, a graphics driver, a database client, or — in a kernel — a hand-written assembly routine. Polaron's FFI (spec §26) keeps that inside the object model. A foreign function is declared as an `extern static method` of a class, with no body; it resolves to a symbol with C linkage.

```
import System.IO.Console;
program FfiExtern;

public bundle main {
  public namespace app {
    public class LibC {
      public extern cdecl static method abs(int x) returns int;
      public extern cdecl static method labs(long x) returns long;
    }
    public class Main {
      public static method main(string[] args) returns void {
        int a = LibC.abs(-42); // 42
        long b = LibC.labs(cast<long>(-1000000)); // 1000000
        System.IO.Console.printf("abs=%d\n", a);
        return;
      }
    }
  }
}
```

The shape of the declaration is `public extern <language> static method name(params) returns T;`, and the word after `extern` names the foreign **language**, not a calling convention:

Word	Means
<code>cdecl</code>	C.
<code>cdecl</code>	C++ — C++ ABI rules for aggregates and the implicit <code>this</code> . There is no mangler: the symbol is taken literally, so a mangled name goes in <code>symbol(" ... ")</code> .
<code>rustdecl</code> , <code>zigdecl</code>	Reserved, and lowered to the C ABI today. Neither language has a stable native ABI — what is callable from outside Rust is <code>#[no_mangle] extern "C"</code> , and Zig's <code>export</code> is the C ABI — but the words declare intent, and a mismatch warning can be written against them.
unknown <world>	A raw ABI with no language behind it: <code>pe</code> , <code>elf</code> , <code>macho</code> , or <code>win64</code> , <code>sysv</code> , <code>aapcs</code> .
<code>syscall(<n>)</code>	Not a language and not a symbol: an instruction with a number. <code>extern syscall(1) method write(...)</code> .

The language is the axis because the language is what decides name mangling, how aggregates travel, exceptions and ownership. A calling convention is an ABI detail below that — but two of them are still spelled in this position, meaning *C, called this way*:

Word	Means
<code>stdcall</code>	C with the 32-bit x86 <code>stdcall</code> convention: the callee cleans the stack and the symbol is decorated <code>@N</code> . Every Win32 entry point is called this way on i686.
<code>fastcall</code>	C with the <code>fastcall</code> convention: the first two integer arguments ride in ECX and EDX.

Both are legal to write on any target and change the instructions only where there are instructions to change: on x86-64 all three conventions collapsed onto the one platform ABI, so a `stdcall` declaration there lowers exactly as `cdecl` does. The `@N` decoration follows from the convention, so nothing spells a decorated symbol by hand.

The enclosing class is pure organization: it groups related externs and gives them a namespace. You call through it (`LibC.abs( ... )`), but the actual C symbol is the method's **simple name** (`abs`), so it links against the real `abs` in `libc`.

A pointer that crosses is an `address`. Past an `extern` there is no Polaron left to reason about: nothing can say whether the callee keeps a pointer it was handed, so nothing can prove the storage outlives it, and the region binder refuses a `T*` parameter on a foreign declaration. Declare it `address` and hand across `cast<address>(p)` — which is how a program says the lifetime is outside the language, and is the truth for a buffer the operating system fills during the call.

Marshalling — how Polaron values become C values at the boundary — is where the interesting work happens, and Polaron handles the common cases automatically:

- **String / string** → `char*`. A string argument is passed as its NUL-terminated `data` pointer, not as the `{length, data}` object. The C side sees plain characters.

```
polaron public class LibC { public extern cdecl static method puts(String s) returns int; }
// ... LibC.puts("hello from polaron"); // C receives a char*
```

- **Small structs by value.** A value-type `struct` small enough to travel in a register (1/2/4/8 bytes on Win64) is passed and returned by value: the compiler loads its bytes as the ABI integer for the call, and stores a returned register back into a fresh struct. A `Point { int x; int y; }` is eight bytes and matches a C `struct { int x, y; }`.

```
polaron public struct Point { public mutable int x; public mutable int y; public constructor
Point(int x, int y) { this.x = x; this.y = y; } } public class PointApi { public extern cdecl
static method polaron_point_sum(Point p) returns int; public extern cdecl static method
polaron_point_scale(Point p, int k) returns Point; }
```

- **Capture-free lambdas → C callbacks.** A lambda with no captured environment is lowered to a raw C function pointer, so a C function can call straight back into it. Lambdas that *do* capture cannot cross the boundary (there is nowhere to put the environment).

```
polaron public class Cb { public extern cdecl static method polaron_apply_cb(function<int,
int> f, int x) returns int; } // ... int r = Cb.polaron_apply_cb(lambda(int n) returns int {
return n * n; }, 7); // 49
```

- **Variadics.** A trailing `...` in the declaration marks a variadic C function, so you can bind things like `printf` directly.

```
polaron public class LibC { public extern cdecl static method printf(String format, ...)
returns int; } // ... LibC.printf("x=%d\n", 42);
```

Linking system libraries. Declaring an extern says *what* to call; you still have to tell the linker *where* the symbol lives. The class that groups the externs is the thing that knows, so the class says it:

```
public class Wgl library OpenGL {
    public extern cdecl static method wglCreateContext(long hdc) returns long;
    // ...
}
```

`OpenGL` there is a **logical name**: an identifier, ours, the same on every platform, and therefore checkable against the declaration that uses it. The **file** it means is theirs — `opengl32.lib` on Windows, `libGL.so.1` on Linux — unspellable as an identifier and different per platform, so it lives in the manifest of the project that declares the class:

```
[libraries]
OpenGL = { windows = "opengl32", linux = "GL" }
Zlib    = "z"                                     # one name, every platform
```

An unmapped name resolves to itself, so a library whose file is spelled like its logical name needs no entry at all; a name mapped to the empty string needs no link flag; and `C` — the C runtime, which every platform links already — is answered that way by default.

A dependency's mapping is inherited. A program that plugs Polaron-OpenGL writes nothing about `opengl32`, `gdi32`, `user32`, `kernel32` or `gdiplus`: the classes are in the library, the mapping is in the library's manifest, and the link reads both. A program's own `[libraries]` entries win over a dependency's, so it can still override one that was mapped badly.

The older `[build] native_libs = "opengl32, user32"` still works and simply appends to the link line. It is blunter: it says "link this" and never "this is where `class Wgl` comes from", so a class could name a library the build did not link, and nothing noticed until the linker did.

11.9 Freestanding mode

Freestanding mode (spec §36) is Polaron with the operating system removed. It is the mode for kernels, bootloaders, firmware, hypervisors — bare metal, where there is no `malloc`, no thread scheduler, no filesystem, and no exception handler until *you* write them.

The central principle: no hidden runtime calls

Everything in freestanding mode rests on a single guarantee:

No freestanding feature emits a hidden call to runtime code. Everything that executes at runtime corresponds directly to something the programmer wrote. Memory layout is predictable. There is no implicit allocation. There are no background threads.

This is the modern equivalent of what C and C++ offer with `-ffreestanding`, but with a much larger language surface kept intact. A feature is allowed in freestanding mode if, and only if, it can be lowered to code with no dependency on a managed runtime.

What is removed, and why

The removed set is exactly the features that *cannot* keep that guarantee. Each removal has a concrete technical reason:

- **Exceptions** — `try / catch / finally / throw / throws`. Stack unwinding needs runtime unwind tables, dynamic lookup, and compiler-generated cleanup paths. *Alternative:* `Result<T, E>` and `Option<T>` remain, and become the single error-handling mechanism.
- **Persistents** — `persistent`, `reattach`, `release persistent`. These depend on a global registry keyed by the `(scope, name, region)` triple, an implicit mutex, and runtime lookup. *Alternative:* ordinary `static` fields cover "a value that lives for the whole program."
- **Async** — `async / await`. These require a language-managed worker-thread scheduler. But the kernel is what *implements* schedulers; it cannot depend on one. *Alternative:* raw threads the kernel itself provides.
- **Unimport / hot reload** — `unimport`, `reimport`. These need a live-type registry and a mini-linker loading and unloading code segments at runtime. In a kernel, code is static after boot.
- **Lazy initialization**. Lazy fields need runtime tracking and an implicit mutex on first read.
- `using`. The resource-token form depends on managed teardown.
- **defer with a timeout**. Timeout-based defer needs a runtime timer. *Static defer* — the plain "run this at scope exit" form, with no unwinding — is **kept**.
- **The runtime teardown lifecycle hooks** — `onClassUnload`, `onLastInstanceDestroyed`. These fire during dynamic destruction, which needs runtime tracking. The *static-init* hooks `onClassLoad` and `onFirstInstance` are **kept** and are how you do boot-time initialization.
- `within`. Its scoping depends on the managed runtime.
- **Reflection (`reflect.*`)**. Reflection needs enormous runtime metadata — type tables, method names as strings, parameters, annotations. A kernel has no room for it and no need to inspect types dynamically.
- **The parts of the standard library that call an OPERATING SYSTEM**. This entry used to say "the managed standard library generally", and list `ArrayList` and `HashMap` alongside `Console` and

`Thread`. That was wrong, and wrong in a way that cost real work: it sent people to reimplement collections by hand under a prohibition the compiler never enforced.

The real test is not "*does this allocate*" — allocation has an answer bare metal, namely the program's own `heap` class. It is "*does this reach for a host operating system*", which has no answer at all. So the list is short, and the compiler now refuses exactly it, **at the import** rather than at the first use:

`Console` · `File / Paths / Directory` · `Net / Socket / TcpClient / TcpListener` ·
`Process / Env / Subproc / Conpty` · `Time` · `Thread / Task / Channel / Mutex / Semaphore / Latch` ·
`String / StringBuilder` · `Test`

Everything else works. `ArrayList<T>`, `HashMap<K, V>` and the rest of the collections compile in a freestanding program and link against your `heap` class; `System.Memory.Buffer` — a library class with a contract on every access and a destructor — works, and so do `Math`, `Raw`, `Allocator`, the unit literals, `Result / Option`, and every value type.

This is the same split C++ makes: a freestanding implementation drops `<iostream>`, `<fstream>` and `<thread>` and keeps the rest. Polaron is in that position, not in C's.

String is on the list for now, and only for now. It lowers to a family of eleven `__polaron_str_*` symbols (copy, free, eq, hash, index, trim, upper, lower, repeat, ...) that no bare-metal runtime provides yet. They need nothing a kernel does not have — the program's heap and `memcpy` — so this is code nobody has written rather than a restriction. Until then, use a byte literal `b" ... "` for fixed text and `byte* / byte[]` with the raw Memory API for text you build.

What is kept — and this is the point

Freestanding Polaron is not a toy subset. What survives is almost the entire *value* of the language:

- **Full OOP** — `class`, inheritance, `virtual / abstract / sealed`, `interface`, generics with constraints and variance, `struct` with bit fields, `record`, `union`, `enum`, `catalog`.
- **The chaos tetrad** — `goto`, `comefrom`, `abstainfrom`, `reinstate`. None needs the managed runtime: `goto / comefrom` are compile-time branches, and `abstainfrom / reinstate` are a single atomic global counter. `abstainfrom` is in fact a key power-management primitive for kernels and embedded code (disable a code path, re-enable it later).
- **Ownership** — `move`, `movable`, `unique`, `partitionable`, `into`. These are *compile-time only* disciplines with zero runtime cost, so they are essential for kernel devs who want ownership safety for free. (The contextual qualifiers `carrying / leaving / releasing` are the exception — they are removed, because they depend on persistents.)
- **Regions** — `region`, `release region`, `itself`, `new ... in region, including at address`. Type-safe manual memory with predictable layout; see the MMIO use below.
- `comptime` functions, `fixed` constants, `demand`, `layout`, and compile-time string DSLs.
- **The bit-counted numeric types** — `int8 ... int64`, `uint8 ... uint64`, `float32 / float64`, and `address`. These are actually **freestanding-only**: in normal mode Polaron uses the named types (`byte / short / int / long`, `smallfloat / float / double`, etc.) and the compiler *rejects* the bit-counted names. Freestanding is where exact widths matter — a hardware register is 32 bits, full stop — so that is where the bit-counted spellings live.
- Static `defer`, contracts (checked at compile time where possible), pattern matching, string interpolation without dynamic allocation, `volatile` fields, FFI with explicit calling conventions, and

immutability-by-default.

A couple of runtime *rules* also tighten up: arrays keep bounds checking on by default (opt out per hot path), and integer overflow simply wraps (no automatic checking) — with `checked( ... )` available where you want it back.

Declaring freestanding and how it is enforced

You can pass `--freestanding` on the command line, but the robust way is to declare it in the source, so the requirement travels with the code. A `program` or a `bundle` can be marked `freestanding`, and the compiler then rejects any non-freestanding feature *even without the flag* — you cannot accidentally reach for the managed runtime.

```
program FreestandingKernel freestanding;

public bundle main freestanding {
    public namespace boot {
        public class Sys {
            public extern cdecl static method putchar(int c) returns int;
        }
        public class Main {
            public static method main(string[] args) returns int {
                address buf = Memory.alloc(8);
                int* msg = cast<int*>(buf);
                msg[0] = 79;    // 'O'
                msg[1] = 75;    // 'K'
                msg[2] = 10;    // '\n'
                for (mutable int i = 0; i < 3; i++) {
                    Sys.putchar(msg[i]);           // I/O via FFI, not the managed Console
                }
                Memory.free(buf);
                return 0;
            }
        }
    }
}
```

Try to use the managed console, `async`, exceptions, `unimport`, or reflection inside a freestanding unit and the compiler stops you with a message that names the feature and the spec section. For example, `System.IO.Console.println( ... )` becomes *"Console (managed stdlib) is not available in freestanding mode; use FFI for I/O."*

And because the tetrad is kept, the kernel power-management idiom works unchanged:

```
public static method tick(int mode) returns int {
    if (mode == 1) { abstainfrom body; }    // disable `body` from now on
    if (mode == 2) { reinstate body; }      // re-enable it
    label body;
    Sys.putchar(66);                        // 'B'
    return 0;
}
```

Inline assembly: `asm` and `naked`

At the very bottom there are instructions no language construct stands for — `cli`, `lgdt`, a `syscall` entry. An `asm` **block** emits verbatim assembly, and it **names the architecture it is written for**:

```
asm("x86_64") {  
    nop  
}
```

The body is captured raw — the Polaron lexer does not tokenise it — and the block is emitted as side-effecting inline assembly, so nothing reorders it away.

The architecture name is checked against the one being built for. That is worth more than it looks: an unchecked intent tag lets an x86 block travel all the way down to clang's assembler on an ARM build, where it fails as `<inline asm>:1:3: error: too few operands for instruction` — a message about a file that does not exist, with no path back to the Polaron line that wrote it. Porting consists largely of finding exactly these, so the compiler reports it itself, at the right line.

A `naked` method takes the same idea up one level: no prologue, no epilogue — no frame set up, no registers saved, no return sequence. The body is the whole function, which is why it is almost always a single `asm` block.

```
public naked static method _start() returns void {  
    asm("x86_64") {  
        mov $0, %rbp  
        call kmain  
    }  
}
```

Reach for it where the standard frame would already be wrong before the first instruction runs: an entry point the hardware jumps to with no valid stack, a trampoline that runs on the caller's frame, a hand-written ABI shim. Everywhere else it is the wrong tool — an ordinary method costs a handful of instructions and keeps the debugger, the unwinder and the optimiser working.

`heap class`

Bare metal there is no allocator until the program declares one. A `heap class` is that declaration: a class marked `heap` whose two static methods *are* the program's allocator.

```
public heap class Heap {  
    private static mutable address next;  
    private static mutable boolean armed;  
  
    public static method allocate(address size) returns address {  
        if (!Heap.armed) { Heap.next = 1048576; Heap.armed = true; }  
        address at = Heap.next;  
        Heap.next = Heap.next + size + 15;  
        return at;  
    }  
    public static method release(address p) returns void { return; }  
}
```

Declaring one is what makes the rest of the language work with no operating system under it. The compiler generates its allocator bridge from this class rather than from libc, so `new ... on heap`, every collection, `System.Memory.Buffer` and even `String / string` compile and link in a freestanding program — the `__polaron_str_*` helpers are written purely in terms of `allocate`, `release` and a memory copy, and codegen emits them against *your* heap. A kernel that declares one needs not a single libc symbol.

The example above is a bump allocator that never reclaims, which is the honest starting point for a kernel before it has a page allocator. Anything with the same two signatures works: a free list, a buddy allocator, a slab. The language's only requirement is that the program says where memory comes from, once, in a place the compiler can find.

`region at address` for memory-mapped I/O

The freestanding jewel is combining regions with fixed addresses. `itself.at(address, size)` builds a region whose objects live at a *fixed* physical address — exactly what you need for a memory-mapped hardware register or a framebuffer. `new ... in region` bump-allocates starting there, with full type safety over the layout; `release region` frees only the region's small header, never the fixed memory itself.

```
region hw = itself.at(mem, 256 bytes);    // `mem` here is a fixed hardware address
Cell* c = new Cell() in region hw;       // placed at that address, type-checked
c.v = 77;
release region hw;                       // frees the header only
```

A memory-mapped type should be a `struct`, not a `class`: a struct is a value type with a predictable layout and no vtable, so its fields sit at exactly the hardware offsets. A class would carry an `Object` vtable pointer at offset 0 and shift everything. This is the classic VGA text buffer, straight out of the spec's minimal kernel:

```
public region vga_text = itself.at(0xB8000, 4000 bytes).accepts({VGACChar});

public struct VGACChar {
    public mutable uint8 character;
    public mutable uint8 attribute;
}
```

Cross-compiling with `--target`

Finally, you tell the compiler *which machine* to emit for with `--target=<triple>`. For a hosted program the default is the native host (e.g. `x86_64-windows`). For a real bare-metal target you pass a triple whose OS field is `none`:

```
polaron build --freestanding --target=x86_64-unknown-none --output=kernel.elf
```

A bare-metal triple changes the entry point. A hosted program (even a freestanding one) still gets an ordinary `main` that the C runtime calls. But when the triple contains `none`, there is no C runtime to build an `argc / argv` array — so the compiler emits an entry named `kmain` that your assembly boot stub calls directly, with nothing that needs libc. The result is a bare-metal object (for `x86_64-unknown-none`) with predictable layout and no dynamic dependencies, ready to be linked into a kernel image.

11.9b `interrupt`: a method the program does not call

Every other method in this manual is *called*. An `interrupt` is **entered** — by something outside the program, at a moment the program did not choose, on top of whatever was running. That is a different thing, so it is a different declaration form rather than a method with an annotation on it.

```
public class Keyboard {
    private volatile mutable int scancodes;
    public constructor Keyboard() { this.scancodes = 0; }

    public interrupt(Trap t) returns void {           // no name: one device, one handler
        this.scancodes = this.scancodes + 1;
    }
    public method count() returns int { return this.scancodes; }
}
```

It is **nameless** because a class needs only one, and it belongs to an *object* — which is the whole reason it is a declaration form and not a free function: the handler exists to reach the device state, and `static` would take that away.

Getting its address is how you install it, and the two are one act:

```
address entry = kbd.interrupt;           // the entry point, bound to this object
Idt.install(33, entry);                  // freestanding: an IDT vector
signal(SIGINT, entry);                   // hosted: the C runtime's own signal()
```

A vector holds an address and nothing else, so there is no way to obtain the address without saying whose handler it is.

The rules, each of them checked:

Rule	Why
It cannot be called	Calling one is <i>simulating</i> an interrupt, which is a different thing wearing the same name
It returns <code>void</code>	It is entered, not called, so a returned value has nowhere to go
It cannot be <code>static</code>	The object is the point
One per class	Two handlers means two objects — which the driver model already says
Its body cannot block or allocate	The lock it would take may be the one the interrupted code is holding, and the handler cannot wait for a holder that cannot run
The frame parameter is read-only	It arrives as a private copy; writing it changes nothing, and pretending otherwise would be worse than refusing

The blocking rule is about what the body **reaches**, not what it says. A handler that looks innocent and calls one method that, two hops down, allocates is refused at the handler.

State shared with an interrupt must say so. The handler and the code it preempts both touch it, and the language already spells that two ways for two real cases: `volatile` for a plain field, or an `atomic<T>` where the update must be indivisible. No third marker was invented.

It works hosted, too, and the meaning is what makes it portable — a POSIX signal is exactly "entered by something outside the program at a moment it did not choose". Only the lowering differs:

	entry point	installed with
freestanding	x86_intrcc , the CPU frame passed byval	Idt.install(vector, obj.interrupt)
hosted	void(i32) , ordinary C ABI	signal(SIGINT, obj.interrupt)

The **mode** decides the lowering, not the target triple — the triple answers an ABI question, and `freestanding` answers a language one. That also shapes the parameter: bare metal hands over the frame the CPU pushed, so it is a class; a hosted world hands over a *code* (a signal number), so it is an integer. **The parameterless form is the intersection** — a handler that does not care where it came from compiles for both worlds unchanged.

11.9c `reentrant` : the property `interrupt` was one case of

An interrupt handler carries a list of prohibitions — it must not allocate, must not free, must not take a lock, must not reach shared mutable state — and the compiler checks every one over the call graph. Look at what that list *is*, and it is not four rules. It is four consequences of one sentence:

`reentrant` — this may be entered again while an earlier entry is still running.

Two things follow, and they are exactly the two the handler rule already checked:

	why
it reaches no storage another activation could be inside	the allocator has global mutable state, and you may have interrupted it mid-update
it touches no shared mutable state	the earlier activation may be half-way through writing the same thing

So `reentrant` is a **member modifier** anything can carry, and `interrupt` is implicitly one:

```
public class Ring {
    private volatile mutable int head;
    private int[] slots;                // fixed at construction; only the CONTENTS move

    public reentrant method push(int v) returns void {
        this.slots[this.head & 7] = v;
        this.head = this.head + 1;
        return;
    }
}
```

Naming the reason rather than a mechanism buys three things a "does not allocate" marker would not:

1. `interrupt` **stops having rules of its own**. One property, one checker, and the bespoke list collapses into an implication of it.
2. **It catches the lock**. A `reentrant` method that takes a `Mutex` **deadlocks against itself** — a real kernel bug class no allocation rule can see. `atomic<T>` stays legal, because a single atomic instruction has no half-done state to interrupt. `Mutex<T>` does not.

3. **A method that is not a handler gains a way to say so.** A scheduler entry, a page-fault path, a destructor running during teardown — each must be equally careful, and none could declare it.

It is viral, and the diagnostic names the path

The obligation travels along the call graph, including into the destructor that runs at the end of a scope — that is a call, and calls are checked. The message says not merely that a method allocates, but *which* reentrant method reaches it and *how*:

```
error: a `reentrant` method must not free memory (reached via refill)
```

That distinction is the whole value. "*Ring.refill frees memory*" is a fact about Ring, and a fact about Ring is not a bug. "*pump is reentrant and reaches it, via refill*" is the bug — and it is the sentence that says which of the two to change.

What makes the rule livable

Everything in Polaron is immutable by default, so a reentrant method reading a base address, a port number or a buffer fixed at construction is untouched by the shared-state half. What it catches is exactly the state a handler and a main loop both *write*: a ring buffer's head, a tick counter, a ready flag. For those it asks for a word the language already has — `volatile` when hardware is on the other end, `atomic<T>` for a counter or a flag — and invents no marker of its own.

`reentrant` is a member modifier and not a universal prefix: `reentrant field` means nothing.

11.10 Putting it together

The three subjects of this chapter are really one subject seen from three distances. Builtins are the primitives; FFI is how those primitives reach outward to existing native code; freestanding mode is what remains when you take the operating system away — and it is *almost everything*, because Polaron was designed so that its abstractions (OOP, regions, ownership, comptime) cost nothing at runtime and therefore survive the trip to bare metal. You can write a hardware driver as a set of classes, guard its memory with type-safe regions at fixed addresses, prove its ownership at compile time, and never once depend on a runtime that will not be there. That combination — full OOP plus type-safe regions plus predictable layout, with no hidden machinery — is what Polaron offers that the older systems languages do not.

12. Keyword Reference

This chapter catalogs every reserved word Polaron recognizes. Each entry gives what the keyword does, its **status**, and — where it clarifies — a short example. Use it as a lookup companion to the preceding chapters, which explain each feature in depth.

Source of truth. What is *reserved today* comes from the compiler itself: `src/lexer/lexer.cpp` + `src/lexer/token.h` (hard keywords) and `src/parser` + `src/semantic/analyzer.cpp` (soft/contextual keywords and semantically-resolved type names). Where the written spec and the implementation diverge, this chapter documents what the compiler accepts today and flags the difference.

How to read each status

Status	Meaning
hard	Reserved in the lexer; can never be an identifier.
soft (contextual)	Tokenized as an identifier; becomes a keyword only in a specific parser context, and is a normal identifier elsewhere.
type (semantic)	Not a lexer keyword; a type name recognized by semantic analysis.
freestanding-only	Recognized, but usable only in freestanding mode.
removed in freestanding	A hard keyword in full mode; forbidden in freestanding mode.
reserved (spec) – not yet implemented	Appears in the spec/catalog, but the current compiler does not recognize it. Documented for completeness; do not rely on it.
migrated to the stdlib	Was a keyword in an older design; today it is a library type/method, not a reserved word.

Spec-vs-implementation notes flagged in this chapter (the spec wins on design; the implementation decides what compiles today): - A named compile-time constant uses `fixed T NAME = expr;`, not `const`. `const` appears in the spec/catalog but is **not reserved** in the lexer. - `switch / case` uses `case V { ... }` (braces), per spec 7.3 — not the legacy `case V:` colon form. - `operator` uses `operator + ( ... )`, not the legacy `operator method +( ... )`. - `cdecl / stdcall / fastcall / byCatalog / expecting / onFailure` are called "contextual" in spec 39, but the lexer reserves them as **hard keywords**. After `extern`, `cdecl` and its siblings name a foreign language; `stdcall` and `fastcall` name C plus a 32-bit x86 convention. - `yield` has two uses: the value of a `match`-expression arm (spec 16.2), and producing the next element of a generator method that returns `Iterator<T>` (spec 22.6).

Index

1. Organization & structure
2. Type members
3. Visibility & modifiers

4. Inheritance & polymorphism
 5. Types & type operations
 6. Memory, ownership & resources
 7. Universal prefixes
 8. Control flow
 9. Ranges & iteration
 10. Exceptions & contracts
 11. Concurrency
 12. FFI / interop
 13. Compile-time, modules & imports
 14. Managed runtime (persistents, unimport, hooks, chaos tetrad)
 15. Primitive types & literals
 16. Soft / contextual keywords
 17. Reserved in the spec but not implemented
 18. Identifiers reserved by the stdlib
 19. Freestanding mode — summary
 20. Counts
-

1. Organization & structure

program

hard. The outermost organizational unit; names the program. One declaration per root file.

```
program HelloWorld;
```

bundle

hard. An independent compilation unit within a program; contains namespaces. May be declared `freestanding`.

```
public bundle main { /* namespaces ... */ }  
public bundle kernel freestanding { /* ... */ }
```

namespace

hard. Logical organization inside a bundle; contains classes, interfaces, enums, and so on. Governs cross-namespace visibility (access requires `import`).

```
public namespace game.entities { public class Player { } }
```

class

hard. Declares a class — the fundamental unit of OOP in Polaron.

```
public class Dog { private string name; }
```

interface

hard. Declares an interface (a contract). May carry default methods.

```
public interface Drawable { method draw() returns void; }
```

struct

hard. A value-type composite; supports bit fields (`field : N`).

```
public struct PacketHeader {  
    public mutable ubyte version : 4;  
    public mutable ubyte kind : 4;  
}
```

record

hard. An immutable DTO-style type; generates a constructor and equality from its positional parameters.

```
public record Point(int x, int y);
```

union

hard. A C-style union — the fields share the same storage (an alternative interpretation of the same bytes).

enum

hard. An enumeration. Simple form (ordinal constants) and Java-style form (fields, a constructor, methods).

catalog

hard. An interface for enums: requires the implementing enum to supply both methods **and** values.

byCatalog

hard (spec 39 lists it as contextual; the lexer reserves it). Inside an enum body, drives generation from a catalog.

2. Type members

method

hard. Declares a method. The keyword is mandatory. There is no overloading — one unique name per method.

constructor

hard. The special creation method; same name as the class.

destructor

hard. The special destruction method; syntax `~ClassName()` . Runs via RAI at the end of a stack object's scope, or on `delete` .

operator

hard. Declares an operator overload. Spec syntax: `operator <op> ( ... ) (no method)`.

procedure

hard. A transformer's member: the conversion itself. `static procedure from<Other>(Other value) returns itself` names the SOURCE; `procedure into<Fahrenheit f>() returns Fahrenheit` names the target and binds it, so the body fills that storage in. A procedure is not a method — it belongs to the relation between two types rather than to either one — and the bound target is storage with no constructor run over it yet, which is why the body IS its construction and why every field of it must be assigned before the body ends.

command

hard. The third kind of member, beside `method` (behaviour of the instance) and `procedure` (behaviour of the relation): **portable behaviour** — something a class hands out for somebody else to run later.

```
public command aboveFloor(int x) carries (int floor) into pack returns boolean {
    return x > pack.floor;
}
```

Naming it without calling it BUILDS one: `Gate.aboveFloor(10)` is a value carrying `floor = 10`, and the value is called by naming it — `test(11)`. At namespace level the same word declares the **role** an API takes, which is how a method can accept a command without naming any particular one:

```
public command IntTest(int x) returns boolean;           // the role
public static method count(int[] xs, IntTest* test) returns int; // the API
```

A command satisfies a role when the signatures agree — neither side names the other. There is also an inline form, written where the value is wanted, with the carried values on the line: `command (int x) carries (int floor = 10) into pack returns boolean { return x > pack.floor; }`.

readonly

hard. On a method: it writes nothing — no field of its own, no field of anything it was handed, no static, no output. Declared, and then checked through the whole call graph, so a body that writes nothing itself but calls something that does is refused too.

```
public readonly method area() returns int { return this.w * this.h; }
```

It is what makes a method callable from a `requires` or an `ensures`: a contract that could change state would make the check part of the program's behaviour. It also lets the optimizer drop a repeated call and hoist one out of a loop. A cache is a write, however invisible from outside — the spelling for that is `lazy`.

cold

hard. On a method: this path is rarely taken. The body moves off the hot line and stops being inlined into one — the inliner costs a method by its SIZE, which is a poor proxy when the bulk of it runs once in a thousand calls.

```
private cold method grow() returns void { ... }
```

There is no `hot` : `hot` is what the optimizer already assumes, so the word would spend a token and add no fact. The same word also tags an `affinity cold { }` field group, which is about where fields sit rather than where code sits.

`mustuse`

hard. On a type or a method: the answer is the point, so a statement that drops it is warned about (`Polaron-0B19`). `Result` and `Option` carry it — a `Result` whose answer is thrown away is an error nobody handled — and any type may.

```
public mustuse class Ticket { ... }           // once, for every method that returns one
public static mustuse method reading() returns int { ... } // or just this one
```

The valve is `discard` (soft): `discard theCall();` says the dropping was deliberate, at the line, where a reader sees a decision rather than a missing diagnostic.

`carries`

hard. What a command holds: `carries (int minAge) into pack`. The list is the command's state, copied when one is built, and reached inside the body through the name `into` gives it (`pack.minAge`) — so a read of carried state never looks like a read of anything else. Capture is impossible by grammar: a name from the surrounding scope that is not on this list is not in scope inside the body.

`returns`

hard. Introduces a method's / constructor's return type.

`return`

hard. Returns from a method, with or without a value.

3. Visibility & modifiers

`public`

hard. Accessible from anywhere.

`private`

hard. Accessible only within the declaring class.

`protected`

hard. Accessible by the class and its subclasses.

`internal`

hard. Accessible only within the same bundle.

`static`

hard. Belongs to the class, not an instance. Called via `ClassName.member` — **always**, including from inside that very class, and the compiler refuses a bare `member( ... )`. An instance method may be written bare because `this` is its subject and the language supplies it; a static one has none, and an action with no

subject is the shape this language refuses. Where two classes share a short name, the namespace is what separates them: `Text.Reader.parse(1)`.

abstract

hard. A class that cannot be instantiated directly; a body-less method that a concrete subclass must implement.

final

hard. Also a [universal prefix](#). A method that cannot be overridden, or a class that cannot be extended.

override

hard. Mandatory when overriding an inherited method (from a class or interface).

surveyed

hard. On a method: the region binder does not derive this method's lifetime summary from its body — a person did, and answers for it. Inside, the one refusal about a value the analysis cannot *place* is suspended; every other refusal stays, so anything it can still prove wrong is still refused. Outside, the boundary goes to the worst case: every reference parameter is treated as kept and a reference result as a borrow of everything in reach.

The freedom is local and the suspicion is exported, which is the opposite of what an [unsafe](#) does — marking a method costs its callers, not nobody. It is inherited: an override may only be [surveyed](#) if the method it overrides is. Not on a destructor, which is where ownership is *declared* and read from. See §5b.

mutable

hard. Allows reassignment/mutation. Everything is immutable by default; use [mutable](#) only where a value is actually reassigned.

nullable

hard. Marks a type as possibly [null](#). Declaration-only: the check happens at assignment (no flow narrowing); dereferencing a null value traps deterministically.

sealed

hard. Restricts subclasses to those listed in [permits](#); enables exhaustive [match](#) with no [default](#).

permits

hard. Lists the allowed subclasses of a [sealed](#) class.

partial

hard. A class declared in several parts (across files); the compiler merges them into one type.

deprecated

hard. Marks a declaration as deprecated; using it raises a compiler warning.

4. Inheritance & polymorphism

extends

hard. Class inheritance (or an enum extending a catalog).

implements

hard. Interface (or catalog) implementation.

this

hard. A reference to the current instance. **this.** is **mandatory** to access one's own members.

super

hard. A reference to the superclass: `super(args)` in the constructor and `super.method()` to call the base implementation.

5. Types & type operations

transformer

hard. Declares a **transformer**: a named relation between types, holding the `procedure s` that convert between them and any rule that comes with the conversion. `public mutual transformer TConverter` declares one both ends may implement. A transformer is not a value and has no instances; it is the thing two types agree about. By convention its name starts with `T` and reads as an agent noun — `TConverter`, `TDescriber` — and the compiler warns when it does not.

applies

hard. `class Celsius applies TDescriber` — the type takes the transformer's behaviour. What the transformer brings is written once and every type that applies it has it. Also used in a condition: `when itself applies TName`.

entrusts

hard. `class Fahrenheit entrusts TConverter` — stronger than `applies`, and a different promise: it hands the transformer the right to CONSTRUCT this type, filling its storage field by field, private ones included. Only the type itself can agree to that, because only it knows what its invariants are. What comes back is an ordinary object; nothing about it says it was built from outside.

call

hard. `call TName.procedureName(args)` — reaches the TRANSFORMER's own body rather than this type's override of it. The word exists because there is no receiver to write to the left of the dot: a transformer is not a value, so `TName.p()` would be a static call on a type that is not one. It means *"my type replaced this, and I want the original anyway"*.

var

hard. Type inference — permitted **only** for local variables.

is

hard. A dynamic type check; returns a boolean.

as

hard. A reference cast/downcast between compatible types (checked at runtime).

cast

hard. An explicit cast with the type in `< >`, including numeric conversions (saturating, no UB).

```
uint crc = cast<uint>(4294967295);
```

null

hard. The absence-of-value literal; valid only for `nullable` types.

typealias

hard. A type alias (no new identity).

newtype

hard. A wrapper with its own type identity (distinct from the underlying type).

6. Memory, ownership & resources

new

hard. Allocates an instance; the placement is optional (`on stack` / `on heap` / `on static` / `in region`), with sensible defaults (objects → stack, arrays → heap).

delete

hard. Frees memory allocated with `new`, running the destructor first.

on

hard. Specifies the allocation site: `on stack`, `on heap`, or `on static`.

`on static` is storage that is **part of the image**: an array whose length is known when it is compiled, laid out where every Polaron array is laid out, allocated by nobody. It is legal on a static field's initialiser, which is the only place where "before the program runs" names a moment.

```
private static mutable byte[] room = new byte[1024]() on static;
```

It exists for the layer that has nothing to allocate *with* — the failure reporter a fired guard calls, an allocator's own free lists, a region's bootstrap pool. Those were C++ for exactly this reason: a file-scope array costs nothing and asks nobody. The trade is said out loud rather than hidden — **one buffer for the whole program**, so two users at once share it.

`on stack` on an ARRAY is not honoured and never was: the clause parsed, nothing read it, and the array went to the heap whatever was written. That is `Polaron-0B54` now instead of silence.

in

hard. Two uses: the region target of a `new (in region X)` and iteration in `for (x in coll)`.

region

hard. A native type: a named arena of memory with type-acceptance rules. Allocated via `itself.allocate( ... )`.

```
region pen = itself.allocate(64 kilobytes);
Dog* a = new Dog(5) in region pen;    // freed by RAII at scope end
```

of

hard. Region disambiguation when declaring a pointer/variable.

accepts

hard. Lists the types a region accepts (`.accepts({ ... })`).

rejects

hard. Lists the types a region rejects.

itself

hard. The self-reference pronoun in a declaration's initializer — refers to the entity being declared (e.g. a region allocating its own backing memory).

release

hard. Frees a persistent or a region (`release region r` ;). `release region` is **available in freestanding** (regions are freestanding-safe); the persistent form is not, since persistents do not exist there.

move

hard. Transfers ownership between variables, regions, or disciplines; invalidates the source.

movable

hard. A class discipline: requires an explicit `move` to transfer ownership; a plain assignment is an error.

unique

hard. A class discipline: at most one live reference at a time; assignment is an implicit move, and copying is forbidden.

reentrant

hard. A member modifier: **this may be entered again while an earlier entry is still running.**

Two things follow from that one sentence, and both are checked: the method reaches **no storage another activation could be inside** (the allocator has global mutable state you may have interrupted mid-update), and it touches **no shared mutable state**.

`interrupt` is one case of it and is implicitly `reentrant` — the handler's bespoke list of prohibitions is now one property with one checker. Naming the *reason* rather than a mechanism is what makes it wider than a "does not allocate" marker: **it also catches the lock**, because a `reentrant` method that takes a `Mutex` deadlocks against itself. `atomic<T>` stays legal, since a single atomic instruction has no half-done state to interrupt.

Viral. Everything the method calls carries the obligation, including the destructor that runs at the end of a scope — and the diagnostic names the path that reached the violation, because "*Ring allocates*" is a fact about Ring while "*this reentrant method reaches it, via refill*" is the bug.

Not a universal prefix: `reentrant field` means nothing.

shareable

hard. A type safe to reach from **several threads at once**, so the region binder may hand it across a thread boundary. A modifier and not a marker interface: it names no methods and dispatches nothing, and under `dynamic` an `implements` clause would buy an eight-byte header and an indirection for a property that generates no calls.

It is checked, not trusted, and the rule is decidable from the declaration alone — which is why it needs no whole-program knowledge and travels in a `.polh`:

legal when every mutable field is `atomic<T>`, or is itself `shareable` — or when the type is entirely immutable.

A bare permission would be a one-word hole in the no-UB principle: written over a type with a plain mutable `int`, it is exactly the race the compiler otherwise refuses. This is the house pattern — **you declare the intent and the compiler confirms it** — beside `override` declaring and the hierarchy checking, and `layout` stating and the arrangement refusing.

`dynamic`

hard. *Decided at run time.* On a class it means the instance **carries its type**: a dispatch pointer as field zero, eight bytes on every instance, and membership of the `Object` root — so it may be held as an `Object`, asked `is`, and dispatched through.

Without it, **a class is its fields**. `class Pair { int a; int b; }` is eight bytes, not sixteen. C++ hangs the same cost on a per-method `virtual` and lets the header appear as a side effect the declaration never mentions; here one word says both *which bodies are chosen at run time* and *which types are known at run time*, and a class that does neither pays for neither.

Four kinds carry it without the word, because for them it is not a choice: a class that `extends` something (the base already decided), one that `implements` an interface (interface dispatch needs a table to dispatch from), an `abstract` class (it exists to be inherited from), and a `region class` or `heap class` (their instances are walked, and a run of objects that do not say what they are cannot be).

It is never inferred. Using a class as an `Object*` without the word is a compile error naming the word and the eight bytes, rather than the class being marked `dynamic` because the use was seen — inference needs whole-program knowledge, breaks across a bundle boundary, and moves the cost away from the declaration, which is the one property the design rests on.

`weak`

hard. A field that points at an object without owning it and without keeping it alive: `private weak nullable Cell* watcher;` . When the object it refers to is deleted, the field reads as null rather than as a pointer into freed memory — which is what makes an observer, a parent link or a cache safe to hold. A weak slot is two pointers wide, not one (the reference plus the registration that lets the delete find it), and it must be `nullable`, because becoming null is the whole point.

`partitionable`

hard. A class modifier allowing individual fields to be moved out (opt-in via `into`). Contradictory with `unique`.

`persistent`

hard. A field that outlives its parent object's destructor; reattaches automatically by identity. **Removed in freestanding.**

transient

hard. A non-serializable field (excluded from snapshots/serialization).

eternal

hard. Also a [universal prefix](#). A resource that lives for the whole program run (no cleanup).

defer

hard. Defers a block to the end of the current scope (LIFO order); also runs during exception unwind.

```
defer { file.close(); }
```

using

hard. The bound variable in [synchronized](#), and scoped resource disposal. **Removed in freestanding.**

external

hard. A field modifier marking a **non-owned** association — [cascade](#) does not follow it.

7. Universal prefixes

cascade

hard. Propagates an operation recursively across the target's owned dependencies/fields (delete, clone, move, and so on).

eternal

hard. Program-lifetime. See §6.

lazy

hard. Defers execution/initialization until first access (implicitly thread-safe). **Removed in freestanding.**

comptime

hard. Runs at compile time; zero runtime overhead. See §13.

volatile

hard. Not optimizable by the compiler; reads/writes always happen for real (MMIO).

final

hard. Not modifiable/overridable/removable. See §3.

8. Control flow

if / else

hard. Conditional with mandatory braces; [else if](#) chains.

`while`

hard. A top-tested loop.

`do`

hard. A do-while loop: runs at least once, tests at the end.

`for`

hard. The classic three-clause `for` and the for-in over ranges / collections (`for (int x in items) { ... }`). See `in` , `index` , `step` .

`foreach`

hard. Iterates a range or collection: `foreach (int x in items) { ... }` . A C#-style spelling of the for-in loop; `for (x in items)` and `foreach (x in items)` are both accepted. See `in` , `index` , `step` .

`switch`

hard. A switch with fall-through. **Braces are mandatory on each `case`** (spec 7.3).

`case`

hard. A `switch` or `match` clause. In `match` it can destructure positionally.

`default`

hard. The default clause of `switch` / `match` .

`break`

hard. Exits the current loop/switch (supports a label).

`continue`

hard. Advances to the loop's next iteration (supports a label).

`match`

hard. Exhaustive pattern matching on dynamic type (statement form, and an expression form with `→`).

`yield`

hard. Two uses: the value of a block arm in a `match` expression (spec 16.2); and, inside a method that returns `Iterator<T>` , producing the next element of a generator (spec 22.6) — the compiler lowers such a method to a state machine.

`goto`

hard. A jump to a `label` (intra-method; also goto-address in freestanding). Part of the "chaos tetrad".

Available in freestanding.

`label`

hard. Marks a statement as the target of `goto` / `comefrom` / `abstainfrom` (`label name` ;).

`comefrom`

hard. The inverse interception of `goto` : declared at the destination, so reaching the label diverts control. Intra-method scope. **Available in freestanding** (a compile-time branch).

`abstainfrom`

hard. Disables a label and the block it introduces (reference-counted; `reinststate` re-enables it). Intra-method scope. **Available in freestanding** (a global atomic counter, no runtime).

`reinststate`

hard. Re-enables a label previously disabled with `abstainfrom`. Intra-method scope. **Available in freestanding.**

9. Ranges & iteration

`step`

hard. A custom stride in a range (`a..b step n`).

`index`

hard. Exposes the index in a for-in loop.

10. Exceptions & contracts

`try`

hard. A protected block. **Removed in freestanding.**

`catch`

hard. Catches an exception. **Removed in freestanding.**

`finally`

hard. A block always executed (with or without an exception). **Removed in freestanding.**

`throw`

hard. Throws an exception. **Removed in freestanding.**

`throws`

hard. Declares the exceptions a method may throw. **Removed in freestanding.**

`requires`

hard. A precondition (contract), validated on method entry.

`ensures`

hard. A postcondition, validated on exit. Accepts `old( ... )` for the prior value (a soft keyword).

`invariant`

hard. A class invariant, checked before/after every public method.

`demand`

hard. A statement that settles a condition while the program is built: `demand <cond> otherwise "why";` . The condition must be knowable then, and it must hold. Emits nothing.

otherwise

hard. Introduces the reason on a **demand**.

layout

hard. An interface for memory, implemented by a value aggregate: it says how the type arranges itself. Its **onArrange** hook runs at build time and leaves nothing behind; implementing one lets the compiler order the type's fields. See §6.14.

12. Concurrency

async

hard. Marks an asynchronous method; runs on a worker pool and may use **await**. **Removed in freestanding.**

await

hard. Suspends the async method until the awaited task completes. **Removed in freestanding.**

synchronized

hard. A critical section with an implicit mutex; the locked value is bound via **using**.

```
synchronized (counter) using int& c { c = c + 1; }
```

12. FFI / interop

extern

hard. Declares an external function (FFI); may specify a calling convention and library.

cdecl

hard (spec 39 calls it contextual; the lexer reserves it). Says the foreign language of an **extern** is C. Its siblings — **cppdecl**, **rustdecl**, **zigdecl**, **unknown <world>** and **syscall(<n>)** — are matched as identifiers rather than reserved, so no program loses a name it was already using.

stdcall

hard. In **extern**, C with the 32-bit x86 stdcall convention: the callee cleans the stack and the symbol is decorated **@N** — how Win32 is called on i686. Legal on any target; on x86-64, where the three conventions are one ABI, it lowers exactly as **cdecl**.

fastcall

hard. As **stdcall**, with the fastcall convention: the first two integer arguments in ECX and EDX.

unknown

hard. **extern unknown <world> method ...** — a raw ABI with no language behind it, for a binary whose origin has to be stated rather than guessed. The world is required: a binary FORMAT (**pe**, **elf**, **macho**,

resolved per target) or a raw ABI outright (`win64` , `sysv` , `aapcs`). `unknown c` means "this target's own C ABI", which is the portable answer and the one a C-facing export wants.

naked

hard. A method the compiler emits with no prologue and no epilogue — no frame set up, no registers saved, no return sequence. The body is the whole function, which is why it is almost always `asm` . For an entry point the hardware or another ABI jumps to, where the standard frame would be wrong before the first instruction ran. Freestanding (spec 36).

interrupt

hard. `public interrupt(Trap t) returns void { ... }` — a hardware interrupt handler. Freestanding, it lowers to `x86_intrcc` : LLVM writes the save of every register the body clobbers, the `cld` , the pop of a pushed error code, and the `iretq` . Hosted, the same declaration installs through `signal()` . One per class: an interrupt vector has room for an address and nothing else, so the bound receiver waits in a single slot.

freestanding

hard. Marks a `program` / `bundle` as freestanding (bare-metal): forbids `async`, exceptions, `unimport`, reflection, and the managed `Console` .

13. Compile-time, modules & imports

comptime

hard. Runs at compile time (a method/value/ `if`). See also [prefixes](#).

literal

hard. Declares a function as a numeric-literal suffix. Must be `comptime` , with exactly one parameter.

fixed

hard. Declares a **named compile-time constant** (`fixed T NAME = expr;`), at class or namespace level — and, in a type-parameter list, marks a parameter that **binds at stamping**.

```
public struct Grid<fixed T, fixed int R, fixed int C> {
    private mutable T[R * C] cells;           // the extent is part of the type
    public method get(int r, int c) returns T { return this.cells[r * C + c]; }
}
Grid<int, 2, 3> g;                            // two shapes are two types
```

`fixed int R` is a hole where a **number** goes — the thing a generic could not express, and the reason a runtime-dimension matrix computes its index with a load, cannot be embedded as a field, and accepts any matrix at all in `multiply` . Marked `fixed` , `R` participates in the type, the class is monomorphized per value, and `R` is a literal in the body.

It is per parameter and does not spread: `<fixed T, int a>` marks the first only. A **bare** value parameter is reserved for the runtime-bound extent that arrives later, and is refused today (`Polaron-010B`) rather than quietly given the stamped meaning.

import

hard. Loads a symbol (class, namespace, bundle) into the program. The stdlib requires explicit imports.

annotation

hard. Declares a custom annotation (metadata for classes/methods/fields).

14. Managed runtime

Persistents, unimport, lifecycle hooks, and the chaos tetrad.

unimport

hard. Removes a symbol from memory at runtime (unloads its code). **Removed in freestanding.**

reimport

hard. Reloads, at runtime, a previously `unimport` ed symbol (reloads its code from the executable on disk). **Removed in freestanding.**

expecting

hard (spec 39 calls it contextual). An authenticity-validation block on import/unimport. **Removed in freestanding.**

onFailure

hard. The mandatory block fired when an `expecting` validation fails. **Removed in freestanding.**

methodref

hard. A receiver bound to one of its instance methods — a command carrying that receiver, and compiled as one, so dispatch through it stays virtual.

```
Animal cat = new Cat() on heap;  
IntMap* sp = methodref cat.speak;    // runs Cat.speak
```

lambda and function are gone. A command replaces both: the baggage is declared rather than captured, and a callable's type is a role somebody named (`Predicate<T>` , `Comparer<T>`). Neither word is reserved any more — they are ordinary identifiers again. See 14.1 and 14.3.

15. Primitive types & literals

The core primitive type names (also keywords). Normal mode uses these:

Type	Meaning
<code>byte</code>	signed 8-bit integer (equivalent to <code>int8</code>)
<code>short</code>	signed 16-bit integer
<code>int</code>	signed 32-bit integer
<code>long</code>	signed 64-bit integer

Type	Meaning
<code>float</code>	32-bit floating point
<code>double</code>	64-bit floating point
<code>boolean</code>	<code>true</code> / <code>false</code>
<code>char</code>	a character
<code>void</code>	absence of a return type
<code>string</code>	a mutable string
<code>String</code>	an immutable string (a class)

```
int n = 42;
double d = 3.14;
char c = 'X';
boolean ok = true;
mutable string buf = "hi";
```

Semantically-resolved type names (not lexer keywords)

`ubyte`, `ushort`, `uint`, `ulong` (unsigned 8/16/32/64), plus `smallfloat` (16-bit) and `quadruple` (128-bit), are **type names** recognized by semantic analysis rather than reserved words.

Bit-counted names (hard keywords, freestanding-only)

`int8`, `int16`, `int32`, `int64`, `uint8`, `uint16`, `uint32`, `uint64`, `float32`, `float64`. In normal mode, using them is an error (the compiler suggests `byte` / `short` / `int` / `long` / `ubyte` /.../ `float` / `double`).

```
// only inside a freestanding bundle:
uint8 version = 4;
float32 x = 1.0;
```

Boolean literals

- `true` (hard) — the boolean true literal.
- `false` (hard) — the boolean false literal.

Other literals (not keywords): integers in several bases, floats (`1.5`), `Decimal` (the `m` suffix, e.g. `1.50m`), chars (`'x'`), strings (`" ... "`), and interpolation (`"$x = {expr}"`).

16. Soft / contextual keywords

Tokenized as identifiers; they become keywords only in the context shown, and are ordinary identifiers elsewhere.

Word	Context	Use
<code>get</code>	property body	<code>public int age { get; }</code>
<code>set</code>	property body	<code>public int age { get; set; }</code>
<code>init</code>	property body	<code>public string key { get; init; }</code>

Word	Context	Use
from	import / move	import Dog from bundle pets; / move c from region a to region b
to	move	move c to region prod
into	move	move c1 into region prod
out	generic type-parameter position	interface Producer<out T> { } (covariance)
old	inside ensures(...)	ensures(this.n == old(this.n) + 1)
carrying	move qualifier	move c1 carrying persistents (default)
leaving	move qualifier	move c1 leaving persistents
releasing	move qualifier	move c1 releasing persistents
onClassLoad	class body (hook)	onClassLoad { init(); }
onFirstInstance	class body (hook)	onFirstInstance { setup(); }
onLastInstanceDestroyed	class body (hook)	onLastInstanceDestroyed { teardown(); } (removed in freestanding)
onClassUnload	class body (hook)	onClassUnload { cleanup(); } (removed in freestanding)
asm	asm("arch") { ... }	an inline assembly block; a normal identifier otherwise
methodptr	type position	methodptr<int, int> — the address of code: one machine word, no environment, plain C ABI. Class.method without a call is one (static only).
named	requires named	requires named — a parameter that must be passed by name
explicit	conversion operator	explicit operator Foo(...) — an explicit-only conversion (spec 6.6)
implicit	conversion operator	implicit operator Foo(...) — an implicit conversion (spec 6.6)
bidirectional	type declaration	a type with a two-way conversion pair (spec 32.6)
affinity / hot / cold	affinity { ... } block	field layout / cache-locality hints (spec 32.9)
within	defer within ...	defer within 5 seconds { ... } — a deferred action with a timeout (spec 32.10)
library	extern clause	extern ... library "name" — names the native library to link
bump / pool / stack / fixedslot / ring	region declaration	the region's allocator flavor (spec 17.11); default is bump
growable	region declaration	a region that chains a new block when the current one is full

Word	Context	Use
<code>mark</code> / <code>rollback</code>	region ops	<code>mark of region R</code> / <code>rollback region R to m</code> (stack regions)
<code>extract</code>	<code>extract X from region R</code>	relocate an object out of a region onto the heap

`carrying` / `leaving` / `releasing` are **removed in freestanding** (they depend on persistents). The contextual type names (`ubyte` / `ushort` / `uint` / `ulong` / `smallfloat` / `quadruple` / `address`) are covered in §15 and §19.

17. Reserved in the spec but not implemented

These words appear in the spec and/or the legacy catalog, but the current compiler does **not** recognize them (they are neither in the lexer nor handled by the parser/semantic layers). Documented for completeness — they do not work today.

Word	Documented intent	Situation
<code>const</code>	compile-time constant	Replaced by <code>fixed</code> in the implementation; <code>const</code> is not reserved.
<code>module</code>	future organizational unit	Reserved in the spec only; not implemented.
<code>package</code>	future package system	Reserved in the spec only; not implemented.
<code>delegate</code>	field prefix: satisfy the interfaces by forwarding to this field	Composition instead of inheritance; a method the class writes wins. Compile-time only, so freestanding too. For a callable VALUE use a <code>command</code> , a <code>methodref</code> , or <code>methodptr</code> at the C border.
<code>force</code>	an <code>unimport</code> modifier	Contextual in the spec; not recognized.
<code>timeout</code>	an <code>unimport</code> modifier	Not recognized as an <code>unimport</code> keyword (exists only as a <code>Channel.select</code> method).
<code>serializable</code>	marks something serializable	Documented; not recognized.
<code>version</code>	bundle versioning	Reserved in the spec only; not implemented.
<code>checked</code> / <code>saturating</code> / <code>wrapping</code> / <code>unchecked</code>	arithmetic modes	<code>checked(expr)</code> exists as a builtin , not a keyword; the others became <code>stdlib</code> methods.

Migrated to the `stdlib` (not keywords in any mode): `thread`, `channel`, `select`, `snapshot`, `restore`, `reverse`, `reversible`, `forward`, `backward`, `witness`, `assert`, `tests`, `saturating`, `wrapping`, `unchecked`, `allocate`, `at`, `kilobytes`, `megabytes`.

18. Identifiers reserved by the stdlib

Not technically keywords, but the stdlib reserves them; do not use them as identifiers.

- **System** — I/O and system: `System.IO.Console.println( ... )`, `System.exit(code)`, `System.Memory.Units` (size suffixes).
 - **Console** — the I/O shortcut: `Console.println(x)`, `Console.print(x)`, `Console.printf( ... )` (requires `import System.IO.Console`).
 - **Memory** — low-level operations (freestanding): `Memory.read<T>(addr)`, `Memory.write<T>(addr, v)`, `Memory.alloc / free / zero / copy`.
-

19. Freestanding mode — summary

Freestanding mode (spec 36) removes the keywords that depend on the managed runtime and unlocks the bit-counted type names.

Removed (hard keywords forbidden in freestanding):

```
async await catch finally lazy persistent
throw throws try unimport reimport using
expecting onFailure
onClassUnload onLastInstanceDestroyed
```

Plus the contextual persistent-move qualifiers `carrying`, `leaving`, `releasing`.

Kept and essential: the ownership disciplines (`move`, `movable`, `unique`, `partitionable`, `into`), `itself`, `literal`, `region` / `release region` / `accepts` / `rejects`, `extern` / `cdecl` / `stdcall` / `fastcall`, **the chaos tetrad** (`goto` / `label` / `comefrom` / `abstainfrom` / `reinstate` — compile-time branches plus a global atomic counter, no runtime), bit fields, and — **freestanding-only** — the bit-counted names (`int8` ... `float64`) and the `address` type (a raw memory address; `int`→`pointer` casts).

20. Counts

Design counts per the spec; the current implementation reserves a slightly different subset (see §17 for the divergences).

Category	Count (spec)
Core keywords	133
Contextual keywords	13
Primitive types (also keywords)	20
Freestanding mode	112–115 keywords

Accuracy note. This is the canonical keyword reference: it lists what the compiler **recognizes today** (lexer + parser + semantic), which is the authority on the language's keywords. It is kept in sync with

the compiler as the language evolves.

13. Diagnostics

Every error and warning the Polaron compiler emits carries a stable code — `Polaron-NNNN` — and, by default, a short explanation of *what* went wrong, *why*, *how to fix it*, and *how to avoid it next time*. The goal is that a diagnostic teaches rather than just scolds: you should rarely have to search the web to understand a Polaron error.

13.1 Anatomy of a diagnostic

A rich diagnostic (the default) looks like this:

```
error[Polaron-0101]: use of undeclared variable 'coutn'
→ game/Board.pol:42:17
  |
42 |         return coutn;
  |                ^^^^^ not declared in this scope

why:    A name must be declared before it is read -- as a local (var/mutable), a
        parameter, a field (via this.), or a namespace constant. This name matches
        nothing visible here, which is almost always a typo or a missing declaration.
fix:    If it is a typo, use the suggested name ('count'). Otherwise declare it before
        this point, e.g. `mutable int coutn = ...;`.
prevent: Let the editor autocomplete names. Forge's live check flags an undeclared name
        as you type, and Alt+Enter applies the suggested fix.
```

The parts:

- **error / warning** — the severity. Warnings do not stop the build.
- **[Polaron-NNNN]** — the stable diagnostic **code**. It never changes for a given kind of problem, so it is safe to search for, suppress-by-policy, or look up with `polaron explain`.
- **the title** — the specific problem, naming the actual identifiers/types involved.
- **the source span** — file, line, column, and a caret (`^^^`) under the exact tokens.
- **why / fix / prevent** — the canonical explanation for that code. This wording is written once, in the compiler's diagnostic catalog, so it is identical wherever the code appears: the terminal, `polaron explain`, and Forge's hover.

Concise mode

Rich output is the default. For dense build logs or CI, pass `-q` (quiet) to get the one-line form and drop the why/fix/prevent block:

```
game/Board.pol:42:17: error[Polaron-0101]: use of undeclared variable 'coutn'
```

13.2 `polaron explain`

To read the full why / fix / prevent for a code without triggering it, ask the driver:


```
polaron explain Polaron-0101      # the canonical explanation for one code (use the full Polar
polc --explain                    # list every code the compiler knows
```

`polaron explain` forwards to the compiler, which owns the catalog, so the text is always in sync with the version you have installed.

13.3 Code ranges

Codes are grouped by the compiler phase that raises them. The ranges are stable; individual codes are added over time.

Range	Phase / area
00xx	Lexing & syntax
01xx	Name & type resolution
02xx	Visibility
03xx	Type checking
04xx	Mutability & ownership
05xx	Flow & exhaustiveness
06xx	Declarations & inheritance
07xx	Literals, interpolation & bit widths
08xx	Feature-specific semantics (operators, reflection, regions, vectors, unimport, atomics, interrupts, transformers, tests)
09xx	Freestanding mode
0Axx	Not implemented yet
0Bxx	Warnings
17xx	Regions: flavors, operations, and the region binder

13.4 Every code

The complete list, with the one-line title each carries. The **why / fix / prevent** for any of them is in the compiler — `polaron explain Polaron-0405` prints it — because writing that text twice is how the two copies come to disagree.

Code	Title
Polaron-0001	unexpected syntax here
Polaron-0002	cannot read this text
Polaron-0101	not declared in this scope
Polaron-0102	no such field on this type
Polaron-0103	no such method on this type
Polaron-0104	unknown type

Code	Title
PoLaron-0105	no such name
PoLaron-0106	this name was brought in under a different path
PoLaron-0107	a static method is called through its class
PoLaron-0108	a command type says what will be called, not what it holds
PoLaron-0109	an inline command has no class to belong to
PoLaron-010A	an inline command carries its values with it
PoLaron-010B	a value type parameter needs <code>fixed</code> , and a bare one is reserved
PoLaron-0201	not accessible from here
PoLaron-0300	null where a value is required
PoLaron-0301	wrong type here
PoLaron-0302	wrong number of arguments
PoLaron-0303	return type does not match
PoLaron-0304	argument type does not match the parameter
PoLaron-0305	this cast is not allowed
PoLaron-0306	cannot index this
PoLaron-0307	operator applied to the wrong type
PoLaron-0307b	type argument does not satisfy the bound
PoLaron-0401	this is not mutable
PoLaron-0402	used after it was moved
PoLaron-0403	this cannot be moved
PoLaron-0404	cannot assign to this
PoLaron-0405	a movable value is transferred, never copied — say so
PoLaron-0406	<code>weak</code> describes a reference, and this is not one
PoLaron-0501	not all paths return a value
PoLaron-0502	match does not cover every case
PoLaron-0503	<code>try?</code> needs a Result/Option method
PoLaron-0504	<code>try?</code> propagates a failure this method cannot carry
PoLaron-0601	already declared
PoLaron-0602	duplicate field
PoLaron-0603	duplicate member
PoLaron-0604	inheritance cycle
PoLaron-0605	cannot extend this type
PoLaron-0606	cannot override a final method
PoLaron-0607	these modifiers contradict each other
PoLaron-0608	this static field's initializer has no value before the program starts

Code	Title
Polaron-0609	this program has nowhere to start
Polaron-0610	a class has one constructor, and this is the second
Polaron-0611	this name already belongs to a type the compiler provides
Polaron-0612	this field is left with no value by the constructor
Polaron-0701	bad literal suffix
Polaron-0702	cannot interpolate this value
Polaron-0703	printf needs a literal format string
Polaron-0704	this value does not fit the bits the field was given
Polaron-0705	a packed bit field has no address of its own
Polaron-0407	this type cannot be copied: it owns a <code>unique</code> value
Polaron-0408	<code>shareable</code> over a field nothing synchronises
Polaron-0409	a <code>reentrant</code> method reaches something it may have interrupted
Polaron-0410	an entity's field has no width decided here
Polaron-0411	a struct cannot hold a <code>weak</code> field
Polaron-0706	this is not an instruction the assembly checker knows
Polaron-0707	this operand cannot be put where the block says
Polaron-0801	malformed operator overload
Polaron-0802	reflection used incorrectly
Polaron-0803	region used incorrectly
Polaron-0804	vector/matrix operation is malformed
Polaron-0805	unimport/reimport used incorrectly
Polaron-0806	demand
Polaron-0807	this must be a compile-time constant
Polaron-0808	a persistent is never released
Polaron-0809	this is wider than the machine can do atomically
Polaron-0810	an interrupt is entered, not called, and it runs where almost nothing is safe
Polaron-0811	this transformer's contract is not met here
Polaron-0812	this annotation cannot go on this method
Polaron-0813	no arrangement satisfies this layout
Polaron-0814	a channel hands the object over, so <code>send</code> transfers it
Polaron-0815	<code>readonly</code> is a promise this body does not keep
Polaron-0901	not available in freestanding mode
Polaron-0A01	not implemented yet
Polaron-0B01	this name reads against the convention the rest of the language follows
Polaron-0B02	the standard library already has a type with this short name

Code	Title
PoLaron-0B03	this local hides a field of the same name
PoLaron-0B04	this exception leaves the method without being declared
PoLaron-0B05	a pointer to a class usually points at one object, not at an array
PoLaron-0B06	this changes a copy, and the caller will not see it
PoLaron-0B07	the calling convention and the symbol it binds do not agree
PoLaron-0B08	this still works and is not going to keep working
PoLaron-0B09	these persistents will be told apart by identity, not by their contents
PoLaron-0B0A	this test reads a fixture whose setup it does not run
PoLaron-0613	this does not match how the annotation was declared
PoLaron-0B0B	this builds a String by re-copying it on every iteration
PoLaron-0B0C	this [Allow] never suppressed anything
PoLaron-0B0D	nothing ever assigns to this, so <code>mutable</code> claims a freedom it does not use
PoLaron-0B0E	this <code>catch</code> neither handles the failure nor lets anyone know it happened
PoLaron-0B0F	this method is <code>async</code> and never awaits anything
PoLaron-0B10	this class is only static methods, which is a namespace with a class round it
PoLaron-0B11	this is public fields with no methods and no invariant, which is a record
PoLaron-0B12	these constants share a prefix, which is a set being kept by hand
PoLaron-0B13	this is an instance method with the receiver written out
PoLaron-0B14	this name spells a type the declaration already gives
PoLaron-0B15	this has a default over a set whose members the compiler already knows
PoLaron-0B16	these branches all compare the same thing, one after another
PoLaron-0B17	this number is written out several times in one method
PoLaron-0B18	this try raises and catches its own exception
PoLaron-0B19	this call's answer is the point, and the statement drops it
PoLaron-0B1A	this is allocated on the heap and deleted in the same block
PoLaron-0B1B	this cleanup is repeated before more than one return
PoLaron-0B1C	this throw is inside a loop
PoLaron-0B1D	every subtype of this is in this program, and the declaration does not say so
PoLaron-0B1E	this abstract class has exactly one subtype
PoLaron-0B1F	the destructor frees this field, and the declaration does not say it owns it
PoLaron-0B20	several methods here open by checking the same thing about this object
PoLaron-0B21	this returns a boolean and writes its real answer through a parameter
PoLaron-0B22	this public method opens by rejecting its argument
PoLaron-0B23	this compares a non-nullable value against null
PoLaron-0B24	several parameters of one primitive type, which any order will satisfy

Code	Title
PoLaron-0B25	these static arrays are one row read sideways
PoLaron-0B26	this converts to and from another type by hand
PoLaron-0B27	this loop allocates and frees on every iteration
PoLaron-0B28	this allocates a bigger array, copies into it, and frees the old one
PoLaron-0B29	several of one type are allocated here one at a time
PoLaron-0B2A	this condition is decided at compile time
PoLaron-0B2B	this region is set up and torn down for one object
PoLaron-0B2C	this copies the same value on every iteration
PoLaron-0B2D	this walks the whole collection, inside a loop
PoLaron-0B2E	this holds a lock across work that has nothing to do with what it protects
PoLaron-0B2F	these two waits are added together and did not have to be
PoLaron-0B30	nothing in this program ever moves one of these
PoLaron-0B31	this is public and nothing outside its bundle mentions it
PoLaron-0B32	nothing in this program ever holds one of these
PoLaron-0B33	nothing reads this field
PoLaron-0B34	this field is public and only its own class ever writes it
PoLaron-0B35	these two point at each other, and neither says which one owns the other
PoLaron-0B36	an enum beside a crowd of fields it silently chooses between
PoLaron-0B37	this lock guards a single integer
PoLaron-0B38	this is lazy and the constructor reads it
PoLaron-0B39	nothing in this program ever receives from a channel
PoLaron-0B3A	this deep-copies something nothing reads afterwards
PoLaron-0B3B	this list starts empty and is filled in a loop
PoLaron-0B3C	this keeps a raw address in a field
PoLaron-0B3D	this is written and written again before anything reads it
PoLaron-0B3E	this takes a pointer and only ever reads through it
PoLaron-0B3F	this tests the negation of the method's own <code>requires</code>
PoLaron-0B40	this loop indexes an array with a bound that is not its length
PoLaron-0B41	this region is released long after the last thing put into it
PoLaron-0B42	two regions are opened and released in the same block
PoLaron-0B43	this is declared eternal and released by hand
PoLaron-0B44	this dispatches through an interface on every element
PoLaron-0B45	the declared field order costs bytes that a layout would recover
PoLaron-0B46	this takes a large aggregate by value
PoLaron-0B47	the compiler can already work this value out

Code	Title
Polaron-0B48	several call sites check the same thing before calling
Polaron-0B49	this loop writes one array and reads another, with nothing separating them
Polaron-0B4A	element-wise arithmetic with a call in the middle of it
Polaron-0B4B	this is handed to something that keeps it, and never read again here
Polaron-0B4C	this override has the same body as the method it overrides
Polaron-0B4D	this number is being used as an address, and it is not typed as one
Polaron-0B4E	an address put into a number stops being an address
Polaron-0B4F	<code>on stack</code> cannot be honoured on a returned object
Polaron-0B50	this field is left out of the generated equality
Polaron-0B51	a layout is <code>arranges</code> , not <code>implements</code>
Polaron-0B52	this hand-allocates and hand-frees several things: a region written longhand
Polaron-0B53	two objects now own one object, and both destructors free it
Polaron-0B54	an array cannot honour <code>on stack</code> , and this one was accepted and ignored
Polaron-0B55	this come from loops back with no way out between its label and itself
Polaron-1710	a region has exactly one flavor
Polaron-1711	a fixedslot/ring region needs its single element type
Polaron-1712	growable does not apply here
Polaron-1713	mark/rollback need a stack region
Polaron-1714	this checkpoint belongs to another region
Polaron-1715	a ring region auto-evicts; individual delete is not allowed
Polaron-1717	use of a variable after it was extracted from its region
Polaron-1718	cannot extract an object whose field lives in the same region
Polaron-1719	a flavor modifier only qualifies a region
Polaron-1720	an <code>extract</code> result must be bound
Polaron-1721	this hands out a pointer to storage that is about to disappear
Polaron-1722	nothing in the program says which of these two dies first
Polaron-1723	past this point there is no proof to be had
Polaron-1724	what this holds was freed by an earlier call

13.5 In the editor

Forge surfaces the same codes end to end. Its live checker runs the compiler on the unsaved buffer, so diagnostics appear as you type; hovering a squiggle shows the code's why/fix/prevent (the same catalog text), **Alt+Enter** applies a suggested quick-fix where one exists, and **Ctrl+F1** runs `polaron explain` on the diagnostic under the caret.

14. Commands, method references & tuples

Methods are Polaron's primary unit of behavior, but they are not the only one. Sometimes a class needs to hand out behaviour for somebody else to run later — a comparison, a predicate, a handler. This chapter covers **commands**, which is how that is written; bound method references; bare C code addresses for FFI; multiple return values via tuples; and named arguments.

14.1 Commands

A **command** is the third kind of member, beside `method` and `procedure`. A method is behaviour of the instance; a procedure is behaviour of the relation between two types; a command is **portable behaviour** — behaviour written in one place and run in another.

```
public class Gate {
    public command aboveFloor(int x) carries (int floor) into pack returns boolean {
        return x > pack.floor;
    }
}
```

Naming it without calling it **builds** one. `Gate.aboveFloor(10)` is a value that carries `floor = 10`; the arguments are the `carries` list, not the parameter list, because the parameters are what the *caller* will supply later. The value is then called by naming it:

```
var over10 = Gate.aboveFloor(10);
boolean yes = over10(11);          // true
boolean no  = over10(3);           // false
```

What it carries, and why the list exists

`carries (T name, ...) into <pack>` is the command's state. It is copied when the command is built — ordinary assignment semantics, so a value copies and a `T*` carries the pointer in plain sight — and it lives under the name `into` gives it. Inside the body it is always reached through that name:

```
public command countClick(Event& e) carries (Counter* stats) into pack returns void {
    pack.stats.increment(e.kind());
}
```

Two things follow, and both are the point of writing it this way:

- **Capture is impossible by grammar.** A name from the surrounding scope that is not on the `carries` list is not in scope inside the body. A command cannot quietly reach out and hold something you did not hand it, so what one keeps alive is readable from its first line.
- **this is never captured implicitly.** If the instance has to travel, it goes on the list like anything else (`Kennel.countClick(this.stats)`). A command with no `this` in its baggage provably does not hold the instance — a fact the region binder reads off the list rather than inferring.

Omit the clause when there is nothing to carry; the command is then stateless.

```
public command isPositive(int x) returns boolean { return x > 0; }
```

The role: a command type

An API that takes a command has to say what it will call, and it must be able to say it without naming any particular command. That is a **command type** — the same word, at namespace level, with a signature and no body:

```
public command IntTest(int x) returns boolean; // the role

public static method count(int[] xs, IntTest* test) returns int {
    mutable int n = 0;
    for (mutable int i = 0; i < xs.length(); i = i + 1) {
        if (test(xs[i])) { n = n + 1; }
    }
    return n;
}
```

Conformance is **structural**: a command satisfies a role when the signatures agree. `Gate` never heard of `IntTest`, `count` never heard of `aboveFloor`, and `Gate.count(xs, Gate.aboveFloor(10))` works because the two signatures are the same shape. A command may satisfy several roles at once.

A command type is a role, not a form, so it never says what a command *holds*: the baggage differs between commands playing the same role, and writing it on the role would demand they all carry the same things.

The inline form

Where the command is used once, right where it is written, it can be written there:

```
int floor = 6;
int over = Gate.count(xs, command (int x) carries (int limit = floor) into pack returns boolean
    return x > pack.limit;
});
```

The carried values are given **on the line**, because the inline form is written at the use and there is no other place they could come from. (The declared form separates them — `carries (int floor)` at the command, `Gate.aboveFloor(10)` at the use — precisely because there the two are in different files, methods and minds.)

An inline command is lifted onto the class the expression is written in, so it is the same construct as the declared one and not a second thing: the same rule that refuses a loose method applies, which is why an inline command outside any class is an error rather than a free function in disguise.

14.2 The roles the standard library names

Most APIs do not want a command type of their own; they want the shape everybody means. Those live in `System.Commands`, and a command satisfies them structurally like any other role:

Role	Shape	Where it shows up
Action	<code>() → void</code>	a thread's work, a test body, a notification handler
Action1<T> / Action2<A, B>	<code>(T) → void, (A, B) → void</code>	forEach, an event subscription, a chunked parallel loop
Predicate<T>	<code>(T) → boolean</code>	filter, any, all, count, find
Mapper<T, R>	<code>(T) → R</code>	map, and a request handler
Comparer<T>	<code>(T, T) → int</code>	every sort and search in the library
Folder<R, T>	<code>(R, T) → R</code>	reduce, fold, a map's merge
Combiner<A, B, R> / Combiner3<A, B, C, R>	<code>(A, B) → R, (A, B, C) → R</code>	the general two- and three-input shapes

They overlap on purpose. A `(Dog*) → boolean` command is a `Predicate<Dog*>` and also a `Mapper<Dog*, boolean>`, and neither role has to know about the other — which is what lets each stay named for what it MEANS at the call site rather than for its arity.

```
import System.Commands.Comparer;

nums.sort(command (int a, int b) returns int { return a - b; });
```

14.3 What went away, and what to write instead

`lambda`, `function<...>` and the `[captures: ...]` clause are not part of the language. A lambda's capture list could be left out entirely, and then what the value held was decided by its body and stated nowhere; `function<Ret, Params...>` named a shape and no role, so an API taking one could not say what it wanted beyond the arrow.

Was	Is
<code>function<boolean, Dog*> p</code>	<code>Predicate<Dog*>* p</code> , or a command type you name
<code>lambda(int x) returns int { ... }</code>	<code>command (int x) returns int { ... }</code>
<code>lambda[captures: byvalue n](...)</code>	<code>command (...)</code> carries <code>(int n = n)</code> into pack ..., reading <code>pack.n</code>
<code>lambda[captures: move chunk](...)</code>	carries <code>(Chunk* mine = move chunk)</code> into pack
<code>lambda[captures: byref counter](...)</code>	nothing — baggage is copied; share an <code>atomic<T></code> or a <code>Mutex<T></code>

The last row is a deliberate loss. A command that shared a caller's local could not outlive the frame it was written in, which is the one thing a command is for.

14.5 Method references: `methodref`

`methodref obj.method` binds a receiver to a method — which is a command carrying one thing, and is compiled as exactly that. Dispatch stays virtual: if the receiver is statically a base type but dynamically a subclass, the override runs, because the call inside the binding is an ordinary call on an ordinary pointer.

```
Animal cat = new Cat() on heap;           // Cat overrides speak()
IntMap* sp = methodref cat.speak;         // any role of the right shape
int e = sp(4);                           // runs Cat.speak(4)
```

It binds an **instance** method only: a static one belongs to the class and has no receiver to bind, so its address is `Class.method` — a `methodptr`, below.

14.6 A code address: `methodptr`

A `methodptr<Ret, Params...>` is the address of code: one machine word, no environment, the plain C ABI, and a typed signature. It is what an entry point loaded at run time is (`wglGetProcAddress` for modern OpenGL, `GetProcAddress`), what a driver's dispatch slot holds, and what a registry stores against a bare pointer when the things it names belong to many different classes.

Its type parameters put the **return type first**, then the argument types, matching how a C signature reads:

```
public mutable methodptr<int, int> createShader;           // int create(int)
public mutable methodptr<void, int, int, long, long> shaderSource; // void source(int,int,lon
```

`Class.method` without a call is that method's address, which is where one comes from without a foreign symbol. Only a **static** method: an instance method's receiver is not named by that expression, and a pointer to one without it is a `this` nobody supplied.

```
methodptr<int, int> twice = Maths.twice;
twice(21);                      // 42 -- a plain indirect call, with nothing in fr
```

It completes the customs post, and the pair is the point: `address` is the address of *data*, blind to shape and to guarantees; `methodptr` is the address of *code*, blind to guarantees and sighted as to shape. Use a `command` for Polaron behaviour, which carries state and is an object; use `methodptr` where the value really is a bare address — a C callback, a driver's dispatch slot, an entry point resolved at run time.

The type was spelled `funcptr`. It is gone from the grammar: in Polaron there are methods, procedures and commands, and the word "function" does not survive at the border either.

14.7 Multiple return values: tuples

A method can return several values at once by declaring a **tuple** return type — a parenthesized list of types — and returning a parenthesized list of values:

```
public static method divmod(int a, int b) returns (int, int) {
    return (a / b, a % b);
}
```

The caller **destructures** the result into fresh locals:

```
(int q, int r) = MathX.divmod(17, 5);    // q = 3, r = 2
```

Tuple components may be named for documentation; the names don't change how the tuple is used:

```
public static method bounds() returns (int low, int high) {  
    return (3, 9);  
}  
(int lo, int hi) = MathX.bounds();
```

Tuples are also ordinary values — you can build one from a literal and destructure it:

```
(int x, int y) = (lo + 1, hi + 1);
```

14.8 Named arguments

At a call site, an argument may be passed **by name** with `name: value`, which documents intent and frees you from remembering positional order:

```
validate(value: 100, errorMessage: "name too long");
```

A parameter can be *required* to be passed by name by prefixing its declaration with `requires named` (spec 22.4), so callers can never pass it positionally — useful for boolean flags and other arguments whose meaning isn't obvious from position alone:

```
public static method spawn(requires named boolean detached) returns void { /* ... */ }  
// Main.spawn(true);           // error: 'detached' must be passed by name  
Main.spawn(detached: true);    // ok
```

15. The toolchain & projects

`polaron` is the one command you drive the language with: it builds and runs projects, checks and formats source, manages dependencies and environments, and scaffolds new projects. Under it sits `polc`, the compiler proper; you rarely call `polc` directly.

15.1 The `polaron` command

Command	What it does
<code>polaron run [file.pol] [--args ...]</code>	Build and run — the current project, or a single bare file. Arguments after <code>--</code> go to the program.
<code>polaron build</code>	Build the current project into <code>build-output/</code> .
<code>polaron check [--project <dir>] [--overlay <file>=<tmp>]</code>	Type-check and print diagnostics without emitting anything. <code>--overlay</code> checks a file as it reads in a temp buffer — the hook editors use for live diagnostics.
<code>polaron compile <file.pol></code>	Compile one file to an <code>.exe</code> , without running it.
<code>polaron test [--args ...]</code>	Build and run the project's <code>[Test]</code> methods. Arguments after <code>--</code> go to the runner: <code>--filter <text></code> narrows the run, <code>--list</code> names the tests without running them, <code>--timing</code> adds durations.
<code>polaron fmt [file.pol]</code>	Format the project's source (or one file) in place.
<code>polaron doc</code>	Render the public API to HTML from <code>///</code> doc comments.
<code>polaron explain <code></code>	Print the why / fix / prevent for a diagnostic code (see §13).
<code>polaron plug [<url\ name>[@version]] [-e]</code>	Download a dependency (or all of them if none is named).
<code>polaron unplug <name> [-e]</code>	Remove a dependency.
<code>polaron env new\ list\ remove [<name>]</code>	Manage shared environments (toolchain/dependency sets reused across projects).
<code>polaron new <name> / polaron init</code>	Scaffold a new project (in a new directory, or in the current one).
<code>polaron clean</code>	Remove <code>build-output/</code> .
<code>polaron studio</code>	Open the TUI project manager.
<code>polaron --version / polaron --help</code>	Print the version / usage.

Turning off the escape checks

The **region binder** — the analysis that stops a pointer to a dead frame from leaving the frame (§5.4) — is on by default. `polc --no-region-binder` turns it off for a whole program.

There is deliberately no per-line version. A line that hands out a pointer to storage that is about to disappear looks like ordinary code, which is exactly how the mistake survives review; so the way to opt out is one flag, visible in the build, rather than an annotation buried in the source. `polc --check` accepts the

same flag and has the same default, so the editor's live diagnostics and the build never disagree about what is an error.

15.2 Project layout

A project is a directory with an `polaron.toml` manifest and a `src/` tree. `polaron build` compiles every `.pol` under `src/` as one program and links the result into `build-output/`.

```
myapp/
├── polaron.toml
├── src/
│   └── main.pol
└── build-output/      (produced by `polaron build`)
```

15.3 The `polaron.toml` manifest

```
[polaron_project]

[program]
name = "MyApp"
version = "0.1.0"
language_version = "1.0"
entry = "src/main.pol"

[dependencies]
Polaron-OpenGL = "https://github.com/jvpts11/Polaron-OpenGL@v1.0.0"

[build]
output = "build-output/"
target = "x86_64-windows"
freestanding = false
subsystem = "windows"           # a GUI app: no console window on launch
```

Nothing here names `opengl32` or `gdi32`, though the program links both. The classes that declare those externs say which library they come from, the library's own manifest maps that name to a file per platform, and the link inherits it — see §11.8.

- **[program]** (or **[library]** for a reusable bundle) — `name`, `version`, `language_version`, and `entry` (the file holding `main`). A **[library]** builds an `.polb` bundle plus an `.polh` header instead of an executable.
- **[dependencies]** — each entry names another Polaron library and says where it is:
- a **plugged** library, fetched by `polaron plug` and installed under `libraries/`: `Polaron-OpenGL = { path = "libraries/Polaron-OpenGL", source = "https://...@v1.0.1" }`. `path` is where it lives now; `source` is the link it came from. Writing the bare link (`Name = "https://...@v1.0.1"`) is the same entry with the location left to its default, which is what a command line gives and what `plug` expands.
- a **path** dependency — a sibling project built from source with yours: `somelib = { path = "../somelib" }`. No `source`, because it was never fetched.

- **[build]** — `output` (the build directory), `target` (an LLVM triple, e.g. `x86_64-windows`), `freestanding` (bare-metal mode; see §11), `subsystem` (`windows` links a GUI app with no console window; the default is a console app), `native_libs` (the older blunt way to name import libraries for FFI), and `environment` (a named shared environment to build against).
- **[libraries]** — what each logical foreign-library name (`class Wgl library OpenGL`) means on each platform: `OpenGL = { windows = "opengl32", linux = "GL" }` . A dependency's entries apply to whoever links it, so a consumer does not repeat its dependency's link list.

15.4 Dependencies and environments

`polaron plug` fetches a dependency into `libraries/` inside your project, compiles it — every source file of it, not just its entry — and records it in `[dependencies]` as what it is, where it is, and where it came from. `polaron unplug` removes both the directory and the entry. The `libraries/` directory is build output, not source: `polaron new` puts it in `.gitignore`, and `polaron plug` with no arguments rebuilds it from the manifest and `polaron.lock`, which pins the exact resolved version of everything installed.

A **path dependency** points at a sibling project on disk instead, and is built from source as part of the build: the pattern for a library being developed alongside its consumer.

Environments (`polaron env`) are shared toolchain/dependency sets: create one with `polaron env new <name>`, list them with `polaron env list`, and point a project at one via the `environment` key in `[build]`, so several projects share one resolved set of dependencies instead of each re-fetching them.

15.5 Editor integration

The Forge IDE and the VS Code extension drive this same toolchain: `polaron check --overlay` powers live diagnostics on the unsaved buffer, `polaron build / run / test` back the build and run commands, `polaron fmt` formats on demand, and `polaron explain` backs the "explain this diagnostic" action.

16. Testing

A test in Polaron is an annotated static method that lives next to the code it tests. There is no `test` keyword and no `assert` keyword — it is annotations, static methods, and classes, the same pieces the rest of the language is made of. `polaron test` finds them, runs them, and exits non-zero if any failed.

This chapter is about writing tests that are worth having: not only "does `add(2, 2)` return 4", but "is the world this generator produced acceptable", "does this run give back all the memory it took", "does the whole program still do the right thing end to end".

16.1 A first test

Annotate a `public static` method with `[Test]`, and assert:

```
import System.Test.Test;
program MathTests;

public bundle Main {
  public namespace App {
    public class MathUtils {
      public static method add(int a, int b) returns int {
        return a + b;
      }

      [Test]
      public static method add_basic() returns void {
        Test.assertEqual(MathUtils.add(2, 2), 4);
        Test.assertEqual(MathUtils.add(-1, 1), 0);
        return;
      }
    }
  }
}
```

```
$ polaron test
PASS MathUtils.add_basic
tests: 1 passed, 0 failed, 0 skipped
```

A `[Test]` method may return `void`, as above — its verdict is "no assertion failed" — or `boolean`, in which case the test is its own verdict:

```
[Test]
public static method still_boolean() returns boolean {
  return MathUtils.add(20, 22) == 42;
}
```

Tests are discovered only in your own code: the standard library and your dependencies are not scanned, so a dependency's tests belong to that dependency's own project.

Annotations may equally be written `@Test` ; the two spellings are identical (§10.3). This documentation uses the bracket form throughout.

16.2 Saying what broke

An assertion that fails prints the numbers. Numbers alone are rarely enough:

```
expected 8..22, got 31
```

`Test.checking( ... )` names the criterion the following assertions are testing, and the failure becomes readable:

```
Test.checking("mountain share stays in band");
Test.assertBetween(mountainPercent, 8, 22);
```

```
[mountain share stays in band] expected 8..22, got 31
```

The label is cleared at the start of every test, so it costs nothing when you do not use it, and there is no bookkeeping to get wrong. Reach for it whenever a test checks more than one thing.

Two more reporting calls round it out:

- `Test.fail(String why)` records a failure directly, for a check no assertion covers.
- `Test.skip(String why)` gives up at runtime on an **unmet precondition** — no GPU, no network, a fixture that could not be built. The test is reported as SKIP, which is neither a pass nor a defect. A test that skips is honest; one that passes because it never ran is not.

16.3 The assertions

Group	Calls
Equality	<code>assertEqual(int, int)</code> , <code>assertNotEqual</code> , <code>assertEqualLong</code> , <code>assertEqualString</code> , <code>assertEqualChar</code> , <code>assertEqualBoolean</code> , <code>assertEqualDouble</code>
Predicates	<code>assertTrue</code> , <code>assertFalse</code>
Text	<code>assertContains</code> , <code>assertStartsWith</code> , <code>assertEndsWith</code>
Ranges	<code>assertBetween(int, int low, int high)</code> , <code>assertAtLeast</code> , <code>assertAtMost</code> , <code>assertBetweenDouble</code>
Tolerances	<code>assertWithin(double, double, double absolute)</code> , <code>assertNear(double, double, double relative)</code>
Arrays	<code>assertEqualIntArray</code> , <code>assertEqualLongArray</code> , <code>assertEqualStringArray</code> , <code>assertEqualDoubleArray(..., tolerance)</code> , <code>assertSorted(int[])</code>
Exceptions	<code>assertThrows<E>(function<void>)</code> , <code>assertDoesNotThrow(function<void>)</code>
Memory	<code>assertNoLeaks(function<void>)</code> , <code>liveBytes()</code> — see §16.9
Output	<code>captureOutput(function<void>)</code> returns <code>String</code> , <code>assertMatchesGolden(actual, path)</code> — see §16.9

Every array comparison reports the **first differing index**: with a thousand elements the index is the whole diagnosis, and "arrays differ" is not.

Two of these deserve a note.

Ranges, not exact values. Most measurements of a generated or simulated system are only ever "within an acceptable band". Writing that as `assertTrue(v ≥ 8 && v ≤ 22)` compiles fine and throws away everything the report needed: which bound, and by how much. `assertBetween` keeps the numbers.

assertNear is relative. `assertWithin(actual, expected, 0.5)` means "within half a unit"; `assertNear(actual, expected, 0.05)` means "within 5%". Use the relative form for quantities whose scale varies — a cell count that depends on world size, an elapsed time. It falls back to an absolute comparison when the expected value is zero, where a relative one has no meaning.

16.4 One test, many inputs

A test that should run over a list of inputs takes **one parameter** and names the static method that supplies the rows:

```
public static method seeds() returns int[] {
    mutable int[] rows = new int[3]();
    rows[0] = 2;
    rows[1] = 7;
    rows[2] = 11;
    return rows;
}

[Cases(source: "seeds")]
[Test]
public static method every_seed_lands_in_band(int seed) returns void {
    Test.checking("the seed stays inside the accepted band");
    Test.assertBetween(seed, 1, 15);
    return;
}
```

```
PASS Parser.every_seed_lands_in_band[0]
PASS Parser.every_seed_lands_in_band[1]
PASS Parser.every_seed_lands_in_band[2]
```

Each row is its own result, so one bad input fails alone and names itself. The source is called **once** per test, not once per row.

The source must be a `public static` method of the same class returning an array of exactly the parameter's type — `int[]` for an `int` parameter, `String[]` for a `String` one. For several values per row, declare a `record` and take that; the machinery is the same. A parameter with no `[Cases]`, or a source of the wrong type, is a compile error.

16.5 Repeating, expecting failure, tagging, budgeting

`[Repeat(times: N)]` runs the whole test — hooks included — N times and reports **one** verdict. For flakiness: a test that fails 3 times in 100 is a broken test, and the report names the runs that failed rather

than printing a hundred lines.

`[ExpectedToFail(reason: " ... ")]` inverts the verdict. Failing is the expected outcome, reported as `XFAIL`; **passing is a failure**, because the bug got fixed and the annotation is now a lie. Unlike `[Ignore]` the test still *runs*, so the day it starts working you hear about it:

```
[ExpectedToFail(reason: "nested quotes are not handled yet")]
[Test]
public static method nested_quotes() returns void {
    Test.fail("the parser drops the inner quote");
    return;
}
```

`[Tag(name: "slow")]` groups tests. Repeatable, and selected with `--tag` / `--exclude-tag`, so the slow half of a suite can sit out the inner loop and still run in CI.

`[MaxTime(ms: 500)]` turns a passing test that took longer into a failure. It is a **budget, not a timeout**: it is checked after the fact and does not abort a test that hangs. It catches the test that quietly got ten times slower.

16.6 Benchmarks

`[Benchmark]` marks a `public static` method taking no arguments that is *measured*, not judged:

```
[Benchmark(iterations: 1000, warmup: 100)]
public static method summing_speed() returns void {
    ...
}
```

An untimed warmup pass lets caches and branch predictors settle, then the measured loop runs and the result is reported as ns/op:

```
BENCH Parser.summing_speed 70.0 ns/op (1000 iterations)
```

A benchmark can never turn the suite red — it has no verdict — and it only runs under `--bench`, so it never slows an ordinary test run. `[Benchmark]` and `[Test]` on the same method is a compile error: one measures, the other judges.

16.7 Fixtures: the lifecycle hooks

A test that needs an expensive thing — a generated world, a parsed file, an open connection — should not rebuild it for every assertion. Four hooks control when setup happens:

Hook	Runs
<code>[BeforeAll]</code>	once, before the class's first test
<code>[AfterAll]</code>	once, after the class's last test
<code>[Setup]</code>	before each test of the class
<code>[Teardown]</code>	after each test of the class

All four are `public static` methods returning `void`, and there may be at most **one of each per class** — two would have no defined order, so the runner would have to silently pick one, and the compiler rejects it instead.

```
public class Census {
    private static mutable int[] cells;      // the class fixture
    private static mutable int[] scratch;    // a per-test buffer

    [BeforeAll]
    public static method buildWorld() returns void {
        Census.cells = new int[65536]();
        return;
    }

    [AfterAll]
    public static method dropWorld() returns void {
        delete Census.cells;
        return;
    }

    [Setup]
    public static method openScratch() returns void {
        Census.scratch = new int[8]();
        return;
    }

    [Teardown]
    public static method closeScratch() returns void {
        delete Census.scratch;
        return;
    }

    [Test]
    public static method values_stay_in_band() returns void {
        Test.checking("every cell holds a value in 0..6");
        mutable int i = 0;
        while (i < Census.cells.length()) {
            Test.assertBetween(Census.cells[i], 0, 6);
            i = i + 1;
        }
        return;
    }
}
```

The hooks belong to their class alone; another class in the same file is unaffected. And `[BeforeAll]` is **skipped entirely** when `--filter` selected none of the class's tests — the expensive fixture is never built for a run that was not going to use it.

16.8 Known-broken tests

A test for something that does not work yet has two bad fates: deleted, or commented out. Both make the gap invisible. `[Ignore]` is the third option — the test stays in the file and in the report:

```
[Ignore(reason: "regrowth is not implemented yet")]
[Test]
public static method forest_regrows() returns void {
    Test.fail("would fail today");
    return;
}
```

```
SKIP Census.forest_regrows -- regrowth is not implemented yet
```

It never runs, so it cannot turn the suite red, and it names itself and its reason on every run so nobody forgets it is there.

16.9 Asserting on memory

Polaron manages memory by hand, so a program can be entirely correct and still be wrong: it produces the right answer and grows by a megabyte a second. No correctness assertion can see that.

`Test.assertNoLeaks` can:

```
[Test]
public static method parsing_gives_everything_back() returns void {
    Test.checking("a parse round-trip leaves no live blocks");
    Test.assertNoLeaks(lambda() returns void {
        Document* d = Parser.parse(sample) on heap;
        delete d;
    });
    return;
}
```

It records the live-block total before and after the action and fails if it grew. Three limits worth knowing: it measures the **calling thread**, so work handed to another thread is not covered; it measures *net* bytes, so a leak exactly balanced by a matching free elsewhere in the action reads as clean; and **the assertion machinery allocates too** — `Test.checking` stores its label as a `String` — so keep `Test.*` calls out of the action being measured.

`Test.liveBytes()` exposes the same number directly, for when the question is a budget rather than a leak:

```
long before = Test.liveBytes();
this.simulateOneFrame();
long after = Test.liveBytes();
Test.checking("a frame allocates under a megabyte");
Test.assertTrue(after - before < 1048576);
```

Asserting on what a program printed

For code whose *job* is output — a report, a formatter, a serializer — the thing to assert on is what came out. `Test.captureOutput` runs an action with its printing diverted and hands back the text:

```
String out = Test.captureOutput(lambda() returns void {
    Report.write(world);
});
Test.checking("the report names its landmass count");
Test.assertContains(out, "landmasses: 5");
```

When the output is large, comparing it line by line in assertions is worse than useless.

`Test.assertMatchesGolden(actual, path)` compares it against a file of expected text and reports the **first differing line**; running with `--update-golden` rewrites that file instead of comparing, which is how an intended change gets accepted:

```
Test.assertMatchesGolden(out, "tests/golden/world-report.txt");
```

`Test.artifact(path)` names a file the test produced — a rendered image, a dump — so a failure points at the evidence rather than only describing it. `Test.tempDir()` gives a scratch directory, created on first use and deliberately *not* cleaned up: a failing test's leftovers are usually what explains it.

16.10 Testing the whole program

Under `--test` the runner is the entry point, which means your program's own `main` is no longer the entry point — it is an ordinary static method, and a test can call it:

```
[Test]
public static method drives_the_whole_program() returns void {
    string[] argv = new string[2]();
    argv[0] = "prog";
    argv[1] = "--seed=42";
    Main.main(argv);
    Test.checking("a full run leaves the world in a valid state");
    Test.assertBetween(World.landmasses(), 3, 9);
    delete argv;
    return;
}
```

This is the answer to "can a test exercise the whole program": it can, with the arguments it chooses, as many times as it likes, asserting on whatever state the run left behind. Nothing special is needed — no harness, no separate entry point, no mode flag threaded through `main`.

16.11 Running them

Command	What it does
<code>polaron test</code>	Build and run every test in the project.
<code>polaron test -- --filter <text></code>	Run only the tests whose <code>Class.method</code> name contains <code><text></code> .
<code>polaron test -- --tag <name></code>	Run only the tests carrying that <code>[Tag]</code> .
<code>polaron test -- --exclude-tag <name></code>	Run everything except those.
<code>polaron test -- --list</code>	Print the test names without running anything.

Command	What it does
<code>polaron test -- --timing</code>	Add per-test and total durations to the report.
<code>polaron test -- --fail-fast</code>	Stop at the first failure. <code>[AfterAll]</code> still runs, so a fixture is never left behind.
<code>polaron test -- --format=json</code>	Emit one machine-readable document instead of the text report.
<code>polaron test -- --bench</code>	Also run the <code>[Benchmark]</code> methods.
<code>polaron test -- --update-golden</code>	Rewrite golden files instead of comparing against them.

Under `--format=json` the whole run is a single JSON object, and each test's own printing is captured into its record rather than interleaved — which is what keeps the document well-formed:

```
{
  "tests": [
    {
      "name": "Report.printed",
      "status": "PASS",
      "ns": 1344000,
      "output": "total: 42\n"
    },
    {
      "name": "summary",
      "status": "PASS",
      "ns": 1344000,
      "output": "passed: 1, failed: 0, skipped: 0, expectedFailures: 0, benchmarks: 0\n"
    }
  ]
}
```

Everything after `--` goes to the runner; anything it does not recognize it ignores, so a project may pass its program's own flags the same way.

A **[library] project** is tested the same way. It normally builds to a `.polb` bundle with no entry point, but `polaron test` builds its sources as an executable instead — the runner supplies the entry — so a library's tests run exactly like a program's. Its tests belong to it: a program that depends on the library does not re-run them.

The report is **deterministic by default** — durations are opt-in behind `--timing` — so it can be diffed, pasted into a review, or compared against a golden file:

```
PASS Census.values_stay_in_band
SKIP Census.forest_regrows -- regrowth is not implemented yet
FAIL Census.setup_ran_for_this_test
  [scratch is fresh] expected 8, got 0
tests: 1 passed, 1 failed, 1 skipped
```

The process exit code is non-zero if anything failed, which is all a CI needs.

16.12 Malformed tests are compile errors

A test that silently does not run is worse than a test nobody wrote: the suite stays green and nobody learns anything. So the compiler rejects, rather than ignores:

- a `[Test]` that is not `public static`, or returns something other than `boolean / void`;
- a hook that is not `public static` returning `void`;
- a second `[Setup]` (or `[BeforeAll]`, `[AfterAll]`, `[Teardown]`) in the same class;
- a method marked both `[Test]` and a hook — a hook runs *around* the tests, so it cannot be one;
- a method marked both `[Test]` and `[Benchmark]` — one judges, the other measures;
- `[Ignore]` on anything that is not a `[Test]`, where it would read as "skipped" while the method runs perfectly normally;

- a `[Test]` that takes parameters without a `[Cases]` to supply them, or whose `[Cases]` source does not exist, is not static, or returns the wrong array type;
- `[Repeat(times: 0)]` or `[Benchmark(iterations: 0)]` , which would measure and judge nothing.

17. How the pieces fit together

Every chapter before this one describes a feature on its own. This one is about the seams — the places where two features meet and the result is not the sum of the two descriptions. Each section is a pair, a rule, and the failure that happens when the rule is not known.

They are here because this is where the real mistakes live. Nobody misuses `while`. What people get wrong is the third thing that follows from two decisions taken separately.

17.1 Value semantics × containers × destructors

The rule: assignment copies. A container of values holds copies. A type with a destructor must therefore be held **by pointer**, or its copies each free what the original owns.

```
// WRONG, and it compiles.
private mutable ArrayList<Controller*> tabs;           // Controller has a destructor
tabs.add(new Controller(rows, cols) on heap);         // the heap object is COPIED in, then leaks;
                                                        // the copy's destructor frees what the orig

// RIGHT
private mutable ArrayList<Controller*> tabs;
tabs.add(new Controller(rows, cols) on heap);
```

The failure is not a compile error and not a wrong number: it is a double free, which on Windows is `0xC0000409` with no output at all — the process is killed before anything it printed reaches the console. In a real editor this cost an afternoon, and the symptom was "the self-test prints nothing".

How to tell which you want: does the type *own* something — memory, a file, a GPU name, a process? Then it is held by pointer and freed by one owner. Is it a value — three floats, an id, a date? Then it is held by value and copies are free.

17.2 Destructors × ownership × the region binder

The rule: the region binder reads ownership **out of the destructor**. A class whose destructor deletes a field owns that field; a class whose destructor does not is a viewer.

That is why adding a destructor can turn a program that compiled into one that does not, and why adding one is usually the fix rather than the problem: it is the sentence that says who frees what, and the analysis was refusing precisely because nobody had said it.

[illegible]

From here `table.rows.add(row)` inside `Table` is a **handover**, not a borrow, and the binder stops asking about it.

17.3 `weak` × the region binder

The rule: a `weak T*` is emptied when its target dies, so it cannot outlive what it points at — and the binder therefore asks for **no ordering at all** on a store into one.

This is the answer whenever two objects genuinely have no lifetime relationship and one merely *watches* the other:

```
public class Stir {  
    private mutable Wild* wild;           // owned by the thing that made this  
    private mutable weak nullable Grid* ground; // the world's; nothing orders the two  
}
```

Without `weak` the binder says "*nothing orders this object against what `ground` belongs to*", and it is right: the program had not said. `weak` is the saying.

Do not reach for it to silence a diagnostic. If the holder *should* free the thing, the answer is a destructor (§17.2). `weak` is for the case where it genuinely should not.

17.4 `delegate` × interfaces × transformers

Two features look alike from the class line and are not:

A **delegate** is a part the type *has*. A **transformer** is equipment the type *gains*.

```
public class City implements Landholder {  
    private delegate Territory territory; // a city HAS ground; a Territory means something  
}  
  
public class Wild applies TFile { ... } // a Wild GAINS an index and the procedures that k
```

The test: would the part still mean something on its own? A `Territory`, a `Storehouse`, a `Rule` would. An "ArmyMember" holding one id and three methods would not — it was invented to carry a field, and that is the shape a transformer takes over.

The cost of getting it wrong is not correctness, it is fourteen hand-written forwarders that drift.

17.5 `sealed` × `match` × `Result`

The rule: `match` over a sealed type must cover every case, and a `default` is what makes a new case invisible. Put the two together with `Result` and a failure cannot be dropped:

```
public sealed enum Refusal permits NobodyLeft, NoReachableGround, TooClose;  
  
public method found(...) returns Result<City*, Refusal>;
```

A caller cannot ignore a `Result`, and the `match` that reports the refusal does not compile until every constant has an arm. **A seventh reason added next year is a compile error, not a silent `continue`** — which is the whole reason to spend a type on it.

Where to put the exhaustive `match` matters: if it is only in the counting, adding a case still compiles (an array indexed by ordinal grows by itself). Put it where the reason is **named** — the report — and the compiler stops the build at the one place a human has to write a sentence.

17.6 Generics × monomorphisation × bundles

The rule: a generic is expanded per instantiation at compile time, and the instance is named from the base name — `Stack<int>` is `Stack$int` whoever declared it.

Two consequences that only appear in real programs:

- A library and a program that both instantiate `ArrayList<String>` produce the same symbol. That is handled (`linkonce_odr`), but it is why a `.polb` skips `$`-named classes in its header.
- **Two different generic templates with the same short name collide**, because the instance is named from the base. Your own `Stack<T>` beside the library's is the one case where telling types apart by namespace is not enough.

Non-generic types sharing a short name with the standard library are fine — yours wins, and the compiler says so with a warning.

17.7 Reflection × freestanding

The rule: reflection, exceptions, `async` and the managed runtime are absent in a `freestanding` program, and the compiler refuses them there rather than letting a kernel link against a runtime it does not have.

The seam that bites is indirect: a standard-library container calls `equalsKey`, which resolves onto `Object`, which a freestanding build does not emit. That is why `Xml`'s children are a sibling chain rather than an `ArrayList<Xml>` — the list version broke every bare-metal program in the suite with a message about `equalsKey` from a type the kernel never mentions.

Writing a library that must work in both: avoid generic containers of your own declared types in the paths a freestanding program uses. Chains and arrays are safe.

17.8 Regions × ownership × `accepts`

The rule: a region is a typed arena. `accepts` / `rejects` are checked at the allocation, and the check follows the region into a field:

```
private mutable pool region termRegion;           // on a class
this.termRegion = itself.allocate(16 kilobytes);
this.terms.add(new Terminal() in region this.termRegion);
```

Two rules that catch people:

1. **A class holding a region must release it in its destructor** — the compiler says so by name (`Polaron-0803`), because a region is memory the object owns and nothing gives it back on its own.

2. **What lives in the region is freed by the region**, so anything in it whose own destructor must run has to be deleted *before* the release. A shell behind a ConPTY freed without being closed leaves a process running with nobody to talk to.

17.9 Contracts × inheritance × generics

`requires` / `ensures` are checked at run time and inherited: an override cannot demand *more* of its caller than the method it replaces. Inside a generic they are checked per instantiation, so a contract that mentions `T`'s methods is checked for each `T` that exists.

A violated contract prints the file, the line, the method, the clause **and the operands** (`left = 0, right = 0`) — a contract that only printed "requires" was a diagnostic that sent you looking for which of three clauses it was.

17.10 Persistents × `unimport` / `reimport`

The rule: `persistent` state survives the code that uses it being unloaded and reloaded. That is the pair: `unimport` takes a bundle out of the running program, `reimport` brings the version on disk back, and a `persistent` field is what is still there afterwards.

`Memo` and `EventLog` in the standard library are the two worked examples: a cache that stays warm across a reload, and an append-only log whose *reader* can change while the events do not.

17.11 The pointer types, in one table

Written	Means	Freed by
<code>T</code>	A value. Assignment copies it, deeply.	Nobody — it lives inside whatever holds it.
<code>T*</code>	A reference to one object.	Whoever the destructor says owns it.
<code>T&</code>	A reference that cannot be null and cannot be re-seated.	Same.
<code>nullable T*</code>	May be absent, and the type says so. Dereferencing null traps.	Same.
<code>weak nullable T*</code>	A view that empties when the target dies.	Never here — that is what weak means.
<code>movable T</code> / <code>unique T</code>	Ownership that transfers with <code>move</code> , and a compile error to copy.	The last owner.
<code>address</code>	A raw machine address, no type and no proof.	Nobody; this is where the guarantees stop.

17.12 A short list of things that surprise people

- `this.` is optional, and the places it is written are the places it disambiguates. A parameter named after the field it sets is the one case where it is required.
- **Assignment is not an expression.** `if (x = 5)` does not compile.
- `main` returns `void`. A program fails by calling `Exit.failed()`; there is nothing to return.
- **Every standard-library type needs an explicit `import`.** There is no automatic namespace.
- A `catch` cannot catch what a method does not declare it `throws`.
- **Integer division by zero throws** rather than being undefined, and a cast that would not fit saturates rather than wrapping — see the no-UB principle in §8.
- `step`, `call`, `release`, `union` and `onFailure` are keywords, and a local named after one of them is a syntax error in a line that otherwise looks fine.

18. Building real things

Five programs, each a shape somebody actually needs, written the way Polaron wants them written. They are not tutorials for the syntax — the chapters before this one are that. They are the decisions: what to reach for, what to leave out, and where each kind of program keeps its state.

Every one of these shapes exists in the wild today. The IDE, the world simulation and the 3D scene at the end of this chapter are real programs that build and run with the toolchain in this repository.

18.1 A command-line tool

The shape: read arguments, do one thing, say what happened, exit with a code somebody's script can read.

```
import System.IO.Console;
import System.IO.Files;
import System.OS.Args;
import System.OS.Exit;
import System.Terminal.Style;

program Wc;

public bundle Main {
  public namespace App {
    public class Main {
      public static method main(string[] args) returns void {
        if (args.length() == 0) {
          System.IO.Console.println("usage: wc <file> ... ");
          Exit.failed();
          return;
        }
        mutable int total = 0;
        for (var path in args) {
          Option<String> text = Files.tryRead(path);
          if (text.isNone()) {
            System.IO.Console.println(Style.red("cannot read " + path));
            Exit.failed();
            return;
          }
          int lines = Files.readLines(path).size();
          total = total + lines;
          System.IO.Console.printf("%8d  %s\n", lines, path);
        }
        System.IO.Console.printf("%8d  total\n", total);
        return;
      }
    }
  }
}
```

The decisions.

- `Files.tryRead` and not `Files.read`: a missing file is an ordinary outcome for this tool, so it is an `Option` rather than an exception. Reverse that for a config file the program cannot start without — there, `read`'s throw names the path and the message is already written.
- `Exit.failed()` and not a silent return. `main` returns `void`; a refusal that exits 0 is a refusal no script, build or CI can see.
- `Style` is switched off in one place when output is redirected — colour codes in a pipe are noise in somebody's log file.
- `TextTable` when the output has columns. Measuring them by hand is the same bug every time.

18.2 A library other people plug

The shape: no entry point, a compiled bundle, a version, and foreign libraries named where they belong.

```
# polaron.toml
[polaron_project]

[library]
name = "Polaron-OpenGL"
version = "1.0.1"
language_version = "1.0"
entry = "src/Gl.pol"

# The foreign libraries this code declares with `class X library Y`, mapped to the file each
# platform spells them in. The logical name is ours and lives in the source; the file name is
# platform's and lives here.
[libraries]
OpenGL = { windows = "opengl32" }
Gdi     = { windows = "gdi32" }
```

```
public class Gdi library Gdi { // which foreign library these come from
    public extern stdcall static method SwapBuffers(long dc) returns int;
}
```

The decisions.

- **The mapping travels inside the `.polb`.** A consumer runs `polaron plug <url>` and never learns that this library needs `opengl32` and four others — that was a secret it used to have to keep.
- **One type per file.** A library is read one type at a time.
- **Version it with a git tag,** because `plug` resolves `^1.0.0` against the tags: `polaron plug https://github.com/user/lib@^1.0.0` writes the exact resolved version into `polaron.lock`.
- Publish the `.polb` + `.polh` as a release asset too, so somebody can install it without `plug`.

18.3 A simulation with a window

The shape: a world that ticks, a renderer that draws it, and a hard line between them.

```

public class Sim {
    private mutable Grid* ground;           // the world
    private mutable Population* folk;       // what lives in it
    private mutable Atlas* realm;          // and what it has built
    private mutable Crew* hands;           // the threads the tick is shared between

    private method tick() returns void {
        Grazing.tick(beasts, folk, ground, ticked, dice, log);
        Walking.tick(folk, ground, ways, ticked);
        Claiming.tick(realm, ground);
        return;
    }
}

```

The decisions.

- **The simulation never draws and the renderer never decides.** The renderer *borrow*s the world: `r.takeFolk(sim.people(), art)`. A world that can only be seen through the window it happens to have is a world nothing can test.
- **Ids cross the seam; pointers stay inside a layer.** An agent holds `homeCity` as an id, and a city holds agent ids. Objects die and their slots are reused, so a pointer held across the seam names somebody else's person a minute later.
- **Regions for anything with one lifetime:** a pool of terminals, a frame's scratch, an arena of nodes. One release instead of a thousand frees, and `accepts` makes the wrong type a compile error.
- **`Parallel.forChunks` for the passes that share nothing**, with per-chunk scratch. Mark the types the hands share `Shared`, deliberately, so that marking is a sentence somebody wrote.
- **Measure the frame in phases from the first day.** A meter added later measures the world it finds rather than the change you are making.

18.4 A network service

The shape: a socket, a router, and answers that cannot be forgotten.

```

import System.Net.ServerSocket;
import System.Net.Router;
import System.Net.ServerRequest;
import System.Net.ServerResponse;

Router* routes = new Router() on heap;
routes.get("/health", lambda[] (ServerRequest* q) returns ServerResponse* {
    return new ServerResponse(ServerResponse.ok("ok")) on heap;
});

ServerSocket* door = ServerSocket.listen(8080);
while (door.isOpen()) {
    Socket link = door.accept();
    ServerRequest req = ServerRequest.parse(link.receive(65536));
    ServerResponse* answer = routes.dispatch(req);
    link.send(answer.toRaw());
    link.close();
    delete answer;
}

```

The decisions.

- **Result at every boundary that can refuse**, with a sealed enum of reasons rather than a status int. The caller cannot drop it and the reasons cannot silently grow.
- **Contracts on the parsing**, because a request is the one input a program never controls: `requires body.length() ≤ MAX` states the limit where it is enforced.
- **TLS is one call away** — `HttpClient.send` uses it for `https`, and `TlsClient.open` gives a raw encrypted stream for anything that is not HTTP.
- Reach for `Retry`, `CircuitBreaker` and `TokenBucket` in `System.App` rather than writing three counters that each get the off-by-one differently.

18.5 Bare metal

The shape: no operating system, no allocator you did not write, no runtime.

```

program Kernel freestanding;

public bundle Kernel {
    public namespace Boot {
        public class Uart {
            public static fixed address BASE = cast<address>(0x09000000);
            public static method write(char c) returns void {
                Raw.write<int>(Uart.BASE, cast<int>(c));
            }
        }
    }
}

```

Build it for the machine rather than for this one:

```
polc kernel.pol --target=aarch64-unknown-none -o kernel.ll
```


The decisions.

- `freestanding` on the `program` is what makes the refusals compile-time: exceptions, `async`, reflection, `unimport` and `Console` are errors here, named as such, instead of link failures three layers down.
- **address is where the guarantees stop**, and that is the point of having a separate type for it. A number cast to a pointer is a fixed address — the UART at `0x09000000` was there before the program started — and the region binder treats it as the root region rather than refusing it.
- `extern` with an explicit calling convention for anything the firmware provides.
- No standard-library container of your own declared types on the paths the kernel uses (§17.7).

18.6 What each shape keeps, and where

Program	State lives in	Freed by
CLI tool	Locals, for the length of one command	Scope exit
Library	Whatever the caller passes in; a library owns almost nothing	The caller
Simulation	Long-lived layer objects, arrays and regions	The <code>Sim</code> 's destructor, and the region releases
Service	Per-request objects; a connection pool that outlives them	The request loop, explicitly
Kernel	Static storage and hardware addresses	Nothing — there is nowhere to give it back to

The pattern behind the table: **Polaron has no default owner**, so every program answers the question in its own shape. What it gives you is that the answer must be written down — in a destructor, in a region, in a `move` — where the next reader can find it.

19. Final considerations

What the language is for, what it refuses, what it costs you, and what is honestly not finished. This chapter is the one to read if you are deciding whether to write something in Polaron.

19.1 What it is for

Programs where **you are responsible for the machine**: a simulation with a hundred thousand moving things, an editor that must not stutter, a service that runs for months, a kernel. Polaron is a systems language with the manners of an application language — objects, generics, contracts, reflection, a large standard library — and no garbage collector anywhere near the tick.

The bet it makes: *the object graph was never the problem*. The reason other languages reach for a collector is not that objects are hard, it is that nobody wrote down who owns what. Polaron makes you write it down — in a destructor, a region, a `move`, a `weak` — and then proves the rest.

19.2 What it refuses, on purpose

Refused	Why
Garbage collection	A pause you did not schedule is a pause in the frame you were drawing.
Exploitable undefined behaviour	Division by zero throws, a cast that does not fit saturates, an index out of range traps. A wrong answer that looks right is worse than a crash.
Implicit conversions between number types	<code>int</code> to <code>long</code> is written down. The bugs this prevents are the ones nobody sees in review.
Method overloading	One name, one method. A call site says which thing it calls without knowing the argument types by heart.
Assignment as an expression	<code>if (x = 5)</code> has never been what anybody meant.
Free functions	Every method belongs to a type. A namespace of loose functions is a class that nobody admitted to writing.
Merging two owners	Nothing in the language moves one object's contents into another behind your back.
A partial region guarantee	There is no per-file switch for the borrow analysis. Either the program is placed or it is not — a proof about part of a partition is not a proof.

Each of these is a place where the language is *less* convenient than its neighbours, deliberately, because the convenience was paid for in a failure mode nobody could see.

19.3 What it costs you

Be honest with yourself about these before starting:

- **You will write destructors.** Not many — a value type needs none — but every type that owns something needs one, and forgetting is a leak the compiler will often, but not always, catch.

- **The region binder will refuse programs that are fine.** When two objects have no stated relationship it says so, and the fix is to state one (§17.2, §17.3). That is a real cost in the first week and mostly disappears after it.
- **No overloading and no implicit conversions** means more names and more casts.
- **The ecosystem is small.** The standard library is large — 338 public types, from `ArrayList` to TLS 1.3 — but the third-party world is what you and the people around you write. `polaron plug` installs from any git URL; there is no registry yet.
- **Windows and Linux, x86-64.** ARM and macOS are on the roadmap, not in your hands today.

19.4 What is finished, and what is not

Finished — used daily, tested, and not expected to change under you: the object model, generics, regions and ownership, contracts, `Result / Option`, exceptions, `async / await` and the concurrency set, reflection and annotations, the diagnostics system, the toolchain (`build / run / test / fmt / doc / explain / plug`), the standard library documented in Part II, freestanding compilation on the compiler's side.

Working, with edges —

- **The package manager** installs, pins and reproduces, but does not resolve version conflicts: two dependencies wanting different majors of one library take whichever was installed first, silently. There is no registry, no integrity hash and no global cache.
- **Bare metal** compiles and links for a bare target; the boot path (`_start` , a linker script, a QEMU run) is not written.
- **TLS** is 1.3 only. A server that requires 1.2 refuses the connection, and the client says so.
- `Raw.sizeof` under-reports a struct with more than one nested value struct — the fields themselves are correct, but a layout budget enforced with that call could pass while being wrong.

Not started: ARM and macOS targets, an incremental compiler, a package registry, a debugger UI of its own (DWARF and `lldb-dap` work today).

19.5 How to read the rest of this book

- Coming from **C++**: start with §5 (memory), then §17 (how the pieces fit). The object model will be familiar; the region binder will not.
- Coming from **Rust**: §5 and §17.3. Ownership here is read from destructors rather than declared in types, and `weak` is the escape that a lifetime parameter would be.
- Coming from **Java or C#**: §5 first, and take §17.1 seriously — a container of values holds copies, which is the single most surprising thing about the language if you arrived from a world where every object is a reference.
- **Writing a kernel**: §11, then §18.5.
- **In a hurry**: §1, the `examples/` directory in the repository (twenty-two programs, each compiled and run by the test suite), and §18.

19.6 A closing note on the diagnostics

If you remember one thing from this reference, make it this: **when the compiler refuses something, read the whole message.** Every one of the 82 diagnostics carries *why* the rule exists, *how* to satisfy it, and *what habit prevents it next time* — and most of them were written the day a real program hit them, by the person who then had to fix that program.

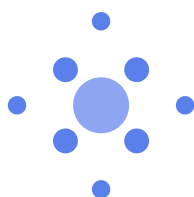
`polaron explain Polaron-1722` prints the long form of any of them from the command line. A language that refuses your program owes you the reason, and this one tries to pay that debt at the moment of refusal rather than in a chapter you would have to know to look for.

20. The mark, the colour and the icon

Polaron's visual identity, written down so that anything made for the language — a plugin, a docs site, a conference slide, an editor theme — can be made to match without asking.

The assets themselves live in `branding/` in the compiler's repository: `logo.svg`, `logo-light.svg`, `icon.svg`, `file-icon.svg`, `file-icon-light.svg`, `icon-128.png`, `icon.ico`, `file-icon.ico`.

20.1 The mark is a lattice with one thing wrong with it



A **polaron** is a real object in physics: a charge carrier moving through a crystal lattice, dragging with it the distortion it causes. Carrier and cloud, treated as one thing.

So the mark is nine points — a heavy core, four near neighbours pulled **in** toward it on the diagonals, and four further out on the axes still where the lattice put them. Near bends, far does not, and the displacement is read from that rhythm rather than from any before-and-after.

The rings are told apart by size, never by opacity. A mark that leans on transparency dies the first time it is punched into an `.ico`, stencilled into one colour, or printed.

20.2 The colours

Role	Hex	Where
Brand blue	#395fdb	The solid tile the app icon is knocked out of. The one colour to use if you use only one.
Lattice	#5a80ec	The eight outer points on a dark ground.
Core	#8ea6f2	The centre point on a dark ground.
Lattice, on the tile	#c9d6fb	The points when the ground is brand blue.
Wordmark	#e8ebf7	POLARON, on dark.
Deep	#1f3591	Shadow and the darker half of a gradient.
Ground	#13172a	The dark background the dark-ground variants are drawn for.
Muted	#5b6480 , #5f688c	Secondary strokes and disabled states.

A saturated tile with the mark opened out of it is the version that gets **found** — an app icon spends its life at 32 pixels next to a wall of other icons, and the dark tile that looked considered on a page vanished into

a dark taskbar.

20.3 The wordmark

`POLARON`, in Segoe UI Variable Display (or the platform's nearest humanist sans), **700 weight, 7 units of letter-spacing**, in `#e8ebf7` on dark or `#13172a` on light. The mark sits to its left at the cap height of the type.

Never set the wordmark in a decorative face, and never letter-space it differently in two places: the spacing is most of what makes it recognisable at a glance.

20.4 The file icon

`.pol` files get their own icon — the mark on a document silhouette rather than on a tile — so that a folder of sources reads as Polaron at a glance without every file looking like the application.

20.5 The assets are generated, not drawn

The `.ico` and `.png` files are produced from the geometry by a generator **written in Polaron** (`polaron-branding`): change `src/Mark.pol`, run it, and every raster follows.

That is not a flourish. The icons used to be made by hand, so nothing connected them to the `.svg` files beside them — and when the old mascot was removed from all five vectors it would have stayed alive inside the icons the installer ships and the marketplace shows, with no build step failing and nothing on screen to say so. A generator cannot drift.

On the previous identity. Polaron was called LDP3 and carried an amber flame called Flamo. Both are retired. Anything still amber is either old or is using the colour for its own reasons — the border of a claimed territory in a game, say, where amber is chosen because it is the one colour a green-and-blue world does not contain.

Polaron Standard Library

— Concurrency & Core

This is the "everything the language leans on" corner of the standard library. It gathers the pieces a program reaches for once it does more than compute a single value: the concurrency primitives that let work run on more than one thread, the `Result / Option` sum types that model "this can fail" and "this may be absent" without exceptions, the console/file/logging helpers for talking to the outside world, a small pseudo-random generator, the root `Object` that every class descends from, the runtime exception hierarchy, and the two iteration interfaces (`Iterator` , `Iterable`) that the collections library builds on.

All of these live in the Polaron-source prelude embedded in `src/cli/main.cpp` (the `kPreludeSource` raw string literal). There is no privileged runtime library written in C++ hiding behind them: every type below is ordinary Polaron, compiled from that prelude exactly like your own code, so the signatures shown are the verbatim declarations from the source. Where a type is genuinely a thin shell over a compiler intrinsic (a channel send, an atomic add, a `printf`), that is called out in prose — those operations are lowered directly by the compiler and have no Polaron method body to quote.

How these types fit the rest of Polaron:

- **Manual memory, no GC.** These classes are allocated with `new ... on heap` (or `on stack`) and freed with `delete` , just like any object. A `Task` , `Channel` , or `Mutex` wraps an OS/runtime handle in a `long` field; hold the wrapper for as long as you need the underlying resource.
- **Value semantics.** Assignment in Polaron is a deep copy, and to *share* one instance across threads or call sites you pass a pointer (`T*`) or reference (`T&`). The concurrency types are meant to be shared, so you almost always allocate them `on heap` and capture them into closures by value of the pointer (see the `Mutex` and `Channel` examples below).
- **Explicit imports.** Namespace visibility is enforced and `System.*` is not exempt: to use a type you must `import` it by its fully qualified name. The exact import line is given per type.
- **Immutable by default.** A field carries `mutable` only where it is actually reassigned; you will see this throughout the prelude signatures.

Thread

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Thread;`

An OS thread (spec 20.1). Holds an `Action` — the work — and its OS handle; `start()` / `join()` call the low-level thread builtins, which lower to `CreateThread` / `WaitForSingleObject` .

Public members:

- `public constructor Thread(Action* w)` — creates a thread bound to the work `w` ; the handle starts at 0 (not yet started).
- `public method start() returns void` — spawns the OS thread (via `System.Concurrency.__threadStart` , which is handed `Thread.enter` and the command) and stores its handle.

- `public method join()` returns `void` — blocks until the thread finishes (via `System.Concurrency.__threadJoin`).
-

`Task<T>`

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Task;`

The handle to an async computation that will produce a `T` (spec 20.2). An `async` method returns one of these and `await` yields the `T`. The public field `h` is the runtime `polaron_task*`.

Public members:

- `public mutable long h;` — the raw runtime task pointer, stored as a 64-bit slot.
- `public constructor Task()` — creates an empty task handle (`h = 0`); the runtime fills it in when an `async` method is invoked.

You rarely construct a `Task` by hand: an `async method` produces one for you, returning it the moment it is called (the body runs on the worker pool), and `await` blocks for its result.

```
import System.IO.Console;
import System.Concurrency.Task;

public class Main {
    public static async method sum(int n) returns int {
        mutable int s = 0;
        for (mutable int i = 1; i ≤ n; i++) { s = s + i; }
        return s;
    }
    public static method main(string[] args) returns void {
        Task<int> a = Main.sum(100);    // scheduled immediately, runs concurrently
        Task<int> b = Main.sum(200);
        System.IO.Console.printf("a=%d b=%d\n", await a, await b);
        return;
    }
}
```

`Channel<T>`

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Channel;`

A bounded blocking channel (spec 20.3): `send()` blocks while full, `receive()` blocks while empty. The element `T` is passed as a 64-bit slot (an `int` or a reference).

Public members:

- `public mutable long h;` — the runtime channel handle.
- `public constructor Channel(int capacity)` — allocates a channel with the given buffer capacity (via `System.Concurrency.__chanNew`).

- `send(T)` / `receive()` — recognized by the compiler as channel builtins (used e.g. inside `Semaphore`); they block as described above but have no prelude method declaration.

A channel is the safe way for a producer thread and a consumer thread to hand values across without sharing mutable state. Allocate it `on heap` so the same channel can be captured into a worker closure and used from `main`:

```
import System.Concurrency.Thread;
import System.Concurrency.Channel;

Channel<int> ch = new Channel<int>(4) on heap;
Action* producer = command () carries (Channel<int> out = ch) into pack returns void {
    for (mutable int i = 1; i ≤ 5; i++) { pack.out.send(i); } // blocks if the buffer is full
};
Thread t = new Thread(producer) on heap;
t.start();
mutable int sum = 0;
for (mutable int i = 0; i < 5; i++) { sum = sum + ch.receive(); } // blocks until a value arrives
t.join();
```

`atomic<T>`

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.atomic;`

A lock-free atomic cell (spec 20.6). `T` is an integer type.

Public members:

- `public mutable T value;` — the cell's current value.
- `public constructor atomic(T initial)` — initializes the cell to `initial`.
- `get` / `set` / `add` / `increment` / `compareAndSet` (and the atomic `++` / `+=` operators) — recognized by the compiler and lowered to LLVM atomic instructions; no prelude method declaration. `add` returns the post-add value (used by `CountdownLatch` / `Barrier`).

`Mutex<T>`

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Mutex;`

A mutual-exclusion lock guarding a value of type `T` (spec 20.5). The value is reached only through `synchronized (m)` using `T& x { ... }`, which holds the lock for the block.

Public members:

- `public mutable T value;` — the guarded value.
- `public mutable long lock;` — the underlying OS lock handle.
- `public constructor Mutex(T initial)` — stores `initial` and creates the lock (via `System.Concurrency.__lockCreate`).

You never touch `value` directly; the only door to it is `synchronized (m)` using `T& x { ... }`, which takes the lock for the duration of the block and binds `x` to a reference to the guarded value, so a read-modify-write cannot interleave with another thread's:

```
import System.Concurrency.Thread;
import System.Concurrency.Mutex;

Mutex<int> counter = new Mutex<int>(0) on heap;
Action* work = command () carries (Mutex<int> cell = counter) into pack returns void {
    for (mutable int i = 0; i < 100000; i++) {
        synchronized (pack.cell) using int& c { c = c + 1; }    // atomic increment
    }
};
Thread t1 = new Thread(work) on heap;
Thread t2 = new Thread(work) on heap;
t1.start(); t2.start();
t1.join(); t2.join();
synchronized (counter) using int& c {
    System.IO.Console.printf("total=%d\n", c);    // exactly 200000
}
```

Semaphore

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Semaphore;`

A counting semaphore (spec 20): `n` permits over a bounded channel. The channel's blocking send/receive do the waiting. (Named `signal` rather than `release` because `release` is reserved.)

Public members:

- `public constructor Semaphore(int n)` — creates the semaphore with `n` permits (fills a capacity-`n` channel with `n` tokens).
- `public method acquire() returns void` — takes one permit, blocking while none are free (receives a token).
- `public method signal() returns void` — returns one permit (sends a token).

CountdownLatch

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.CountdownLatch;`

A one-shot latch (spec 20): threads `waitFor()` until `n` `countDown()` calls have happened. Once the count reaches zero a token is placed in the gate and every waiter passes (take-then-return).

Public members:

- `public constructor CountdownLatch(int n)` — starts the count at `n`; if `n ≤ 0` the gate is opened immediately.

- `public method countdown() returns void` — atomically decrements the count; opens the gate when it hits zero.
 - `public method waitFor() returns void` — blocks until the gate opens, then re-posts the token so other waiters also pass. (Named `waitFor` because `await` is a keyword.)
 - `public method getCount() returns int` — returns the current count.
-

Barrier

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.Barrier;`

A cyclic-style barrier (spec 20): `n` threads `arrive()` until all have arrived, then all proceed. The `n`-th arrival releases `n` tokens; each thread (including it) takes one. Single use.

Public members:

- `public constructor Barrier(int n)` — sets the party count to `n`, with an empty arrival counter and a capacity-`n` gate.
 - `public method arrive() returns void` — records arrival; the final arrival posts `n` tokens, and each thread blocks until it can take one.
-

ReadWriteLock

- **Namespace:** `System.Concurrency`
- **Import:** `import System.Concurrency.ReadWriteLock;`

A reader/writer lock (spec 20), reader-preference: any number of readers, or one writer. Built from two semaphores plus a reader counter (the first reader takes the write lock, the last frees it).

Public members:

- `public constructor ReadWriteLock()` — creates the reader mutex, the write semaphore, and the reader counter.
 - `public method readLock() returns void` — acquires a read lock; the first concurrent reader also takes the write semaphore.
 - `public method readUnlock() returns void` — releases a read lock; the last reader releases the write semaphore.
 - `public method writeLock() returns void` — acquires the exclusive write lock.
 - `public method writeUnlock() returns void` — releases the exclusive write lock.
-

`Result<T, E>` (sealed abstract, permits `Ok`, `Err`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.Result;`

The `Result<T,E>` sum type (spec 21.2): a sealed abstract base matched with `match`. The abstract method forces a vtable so `match` can dispatch on the variant.

Public members:

- `public abstract method isOk() returns boolean` — abstract; each variant reports whether it is the success case.
-

`Ok<T, E>` (extends `Result<T, E>`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.Ok;`

The success variant of `Result<T,E>` (spec 21.2), constructed with the type args taken from the expected type at the use site (via the `Ok(x)` sugar).

Public members:

- `public final T value;` — the success payload.
 - `public constructor Ok(T value)` — wraps a success value.
 - `public override method isOk() returns boolean` — returns `true`.
-

`Err<T, E>` (extends `Result<T, E>`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.Err;`

The error variant of `Result<T,E>` (spec 21.2), constructed via the `Err(e)` sugar.

Public members:

- `public final E error;` — the error payload.
 - `public constructor Err(E error)` — wraps an error value.
 - `public override method isOk() returns boolean` — returns `false`.
-

`Option<T>` (sealed abstract, permits `Some`, `None`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.Option;`

The `Option<T>` sum type (spec 21.3): a sealed abstract base matched with `match`, with an abstract method to force a vtable for dispatch.

Public members:

- `public abstract method isSome() returns boolean` — abstract; each variant reports whether a value is present.
-

`Some<T>` (extends `Option<T>`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.Some;`

The present variant of `Option<T>` (spec 21.3), constructed via the `Some(x)` sugar.

Public members:

- `public final T value;` — the contained value.
 - `public constructor Some(T value)` — wraps a present value.
 - `public override method isSome() returns boolean` — returns `true`.
-

`None<T>` (extends `Option<T>`)

- **Namespace:** `System.Errors`
- **Import:** `import System.Errors.None;`

The absent variant of `Option<T>` (spec 21.3), constructed via the `None` sugar.

Public members:

- `public constructor None()` — the empty option.
- `public override method isSome() returns boolean` — returns `false`.

Together, `Result` and `Option` let a function report failure or absence in its return type instead of throwing. You construct the variants with the `Ok(x)` / `Err(e)` / `Some(x)` / `None()` sugar (the generic arguments are inferred from the expected type), and you take them apart with an exhaustive `match` — because the base is `sealed`, the compiler checks that you cover every variant, so no `default` arm is needed:

```
import System.IO.Console;
import System.Errors.Result;

public class Main {
    public static method parse(int n) returns Result<int, int>* {
        if (n < 0) { return Err(404); } // args inferred from the return type
        return Ok(n);
    }
    public static method main(string[] args) returns void {
        Result<int, int>* r = Main.parse(5);
        match (r) {
            case Ok(int v) { System.IO.Console.printf("ok %d\n", v); }
            case Err(int e) { System.IO.Console.printf("err %d\n", e); }
        }
        return;
    }
}
```

Console

- **Namespace:** `System.IO`
- **Import:** `import System.IO.Console;`

Console I/O (spec 2.9 / 4). The methods are recognized by the compiler and lower to libc `printf / scanf`; the class exists only so `import System.IO.Console;` resolves and the usual namespace-visibility rules require importing it before use.

Public members:

- (none declared in the prelude) — `printf`, `println`, `print`, and `read` are compiler builtins with no Polaron method declaration.

Files

- **Namespace:** `System.IO`
- **Import:** `import System.IO.Files;`

Line-oriented file helpers (spec 34.4). Layers newline handling on top of the `File` builtin so a program reads a file into a list of lines and writes a list back without joining newlines itself.

Public members (all static):

- `public static method readLines(String path) returns ArrayList<String>` — reads the whole file and splits on `\n`, dropping the trailing empty piece from a final newline.
- `public static method writeLines(String path, ArrayList<String> lines) returns void` — writes each line followed by `\n`, building the output with a `StringBuilder` (avoids $O(n^2)$ concat).
- `public static method appendLine(String path, String line) returns void` — appends `line` plus a newline to the file.
- `public static method listDir(String path) returns ArrayList<String>` — returns the directory entries, one per element (empty if not a directory); wraps the newline-separated `File.list`.

Paths

- **Namespace:** `System.IO`
- **Import:** `import System.IO.Paths;`

Path-string manipulation (spec 34.4): join segments and pull a path apart. Pure string work over `"/"`; it does not touch the filesystem.

Public members (all static):

- `public static method join(String base, String name) returns String` — joins `base` and `name` with a single `/` (handles empty base and a trailing slash).
- `public static method basename(String path) returns String` — returns the final `/`-separated component.

- `public static method dirname(String path) returns String` — returns everything before the final component (empty if there is no directory part).
 - `public static method extension(String path) returns String` — returns the text after the last `.` in the basename (empty if none).
-

Logger

- **Namespace:** `System.IO`
- **Import:** `import System.IO.Logger;`

Leveled logging to the console (spec 34): each message carries a level (debug/info/warn/error) and is printed with the logger's name only when its level is at or above the configured minimum, which starts at info. Levels are numbered debug=0, info=1, warn=2, error=3.

Public members:

- `public constructor Logger(String name)` — creates a named logger with the minimum level set to info (1).
- `public method setLevel(int level) returns void` — sets the minimum level to print.
- `public method debug(String message) returns void` — logs at DEBUG (printed only if min level $\leq$ 0).
- `public method info(String message) returns void` — logs at INFO (printed only if min level $\leq$ 1).
- `public method warn(String message) returns void` — logs at WARN (printed only if min level $\leq$ 2).
- `public method error(String message) returns void` — logs at ERROR (printed only if min level $\leq$ 3).

(The private `emit` method does the actual `Console.printf` formatting and is not part of the public API.)

Args

- **Namespace:** `System.IO`
- **Import:** `import System.IO.Args;`

Reads a program's command-line arguments (spec 34). Wraps the `String[]` handed to `main` so a program parses its options without scanning the array by hand.

Public members:

- `public constructor Args(String[] argv)` — wraps the argument array.
 - `public method has(String flag) returns boolean` — reports whether `flag` appears among the arguments.
 - `public method value(String flag) returns String` — returns the token after `flag` (empty if the flag is absent or has no following token).
 - `public method count() returns int` — returns the number of arguments.
 - `public method get(int i) returns String` — returns the argument at index `i`.
-

Random

- **Namespace:** `System.Math`
- **Import:** `import System.Math.Random;`

A deterministic PRNG (xorshift64), pure Polaron over a `ulong` state (spec 34.6 Random). `Math` itself is a compiler builtin, not a class; `Random` is the only real class in `System.Math`.

Public members:

- `public constructor Random(ulong seed)` — seeds the generator (a zero seed is bumped to 1 to avoid the fixed point).
- `public method nextInt() returns int` — advances the xorshift state and returns a non-negative 31-bit value.
- `public method nextIntMax(int max) returns int` — returns a value in `[0, max)`.
- `public method nextRange(int lo, int hi) returns int` — returns a value in `[lo, hi)`.
- `public method nextDouble() returns double` — returns a value in `[0, 1)`.
- `public method nextBool() returns boolean` — returns a coin-flip boolean.
- `public method nextGaussian() returns double` — a standard-normal $N(0,1)$ sample via the Marsaglia polar method.
- `public method nextGaussianScaled(double mean, double stddev) returns double` — a normal sample shifted/scaled to the given mean and standard deviation.
- `public static method seededNow() returns Random` — a generator seeded from the monotonic clock (a fresh, non-reproducible sequence per run).

(The private `sqrtD` and `lnD` helpers keep `Random` from depending on the `Math` builtin name and are not public.)

Object

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.Object;`

The root of the class hierarchy (spec 3.4): every class implicitly extends `Object`. `equals` defaults to identity and `hashCode` to a constant; both are virtual so subclasses (e.g. records) override them.

Public members:

- `public method equals(Object other) returns boolean` — reference-identity equality by default.
 - `public method hashCode() returns int` — returns 0 by default (subclasses override).
 - `public method equalsKey(Object other) returns boolean` — identity-based key equality, so every object is usable as a collection element/key; overridden alongside `hashCode` for value equality.
 - `toString()` — returns `String` and is added together with the `String` type (not declared inline in this prelude slice).
-

Exception (abstract)

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.Exception;`

Base for runtime exceptions (polymorphic, so it can be caught). Concrete runtime exceptions extend it and supply a message.

Public members:

- `public abstract method message() returns String` — abstract; each exception returns its human-readable message.
-

UnimportedTypeException (extends Exception)

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.UnimportedTypeException;`

Thrown when an unimported type is used (spec 30).

Public members:

- `public constructor UnimportedTypeException()` — creates the exception.
 - `public override method message() returns String` — returns `"type was unimported"`.
-

BundleNotLoadedException (extends Exception)

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.BundleNotLoadedException;`

Thrown on first use of a dynamically-loaded bundle (`--use-dynamic`) that is absent at runtime (spec 2.4); wrap the use in try/catch to run without it.

Public members:

- `public constructor BundleNotLoadedException()` — creates the exception.
 - `public override method message() returns String` — returns `"bundle not loaded"`.
-

BundleAbiMismatchException (extends Exception)

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.BundleAbiMismatchException;`

Thrown when a loaded bundle's ABI fingerprint does not match what the program compiled against (spec 2.5).

Public members:

- `public constructor BundleAbiMismatchException()` — creates the exception.
- `public override method message() returns String` — returns `"bundle ABI mismatch"`.

`ClassCastException` (extends `Exception`)

- **Namespace:** `System.Runtime`
- **Import:** `import System.Runtime.ClassCastException;`

Thrown by an invalid class downcast: `cast<Dog>(animal)` / `animal as Dog` when the object is not a `Dog` (spec 6.3). Catch it, or use `animal as? Dog` (yields null) / `animal is Dog`.

Public members:

- `public constructor ClassCastException()` — creates the exception.
 - `public override method message()` returns `String` — returns `"invalid cast"`.
-

`Iterator<T>` (interface)

- **Namespace:** `System.Collections`
- **Import:** `import System.Collections.Iterator;`

A cursor over a sequence (spec 34): reports whether another element remains and yields the current one while advancing.

Public members:

- `method hasNext()` returns `boolean` — reports whether another element remains.
 - `method next()` returns `T` — yields the current element and advances.
-

`Iterable<T>` (interface)

- **Namespace:** `System.Collections`
- **Import:** `import System.Collections.Iterable;`

Anything that can hand out a fresh `Iterator` over its elements, so a generic algorithm can walk any collection through these two interfaces (spec 34).

Public members:

- `method iterator()` returns `Iterator<T>` — returns a fresh iterator over the elements.
-

The arithmetic and reference exceptions

Four more of the runtime's own, thrown by the language rather than by a library. They are ordinary `Exception` subclasses, so `catch` and `throws` treat them like any other — what is unusual is who raises them.

Type	Raised when
<code>NullReferenceException</code>	A <code>nullable</code> reference is dereferenced while it is null.

Type	Raised when
<code>ArithmeticException</code>	The base of the two below.
<code>DivideByZeroException</code>	Integer division or remainder by zero. Checked, not undefined — see the no-UB principle in §8.
<code>OverflowException</code>	A checked arithmetic operation left the range of its type.

Polaron does not have a value that means "this went wrong": a division by zero is not a NaN travelling through the program to surface somewhere unrelated. It is an exception at the instruction that did it.

Parallel

```
import System.Concurrency.Parallel;
```

Data parallelism over a range, on the pool `Thread` uses.

- `public static method forRange(int from, int to, Action1<int>* body) returns void` — runs `body(i)` for every `i`, shared between hardware threads.
- `public static method forChunks(int from, int to, Action2<int, int>* body) returns void` — runs `body(lo, hi)` once per chunk, which is what a loop with per-hand scratch state wants: one allocation per chunk instead of one per element.

Shared (interface)

An empty interface, and the whole of what it says is **that its author thought about it**.

The region binder refuses to hand one object to two hands at once, which is right by default and wrong for the one case a worker pool is made of: a value everybody reads and nobody writes, or one whose writes are already atomic. Marking a type `Shared` is a sentence somebody has to write, so it cannot be arrived at by accident — which is the difference between this and a flag that turns the check off.

```
public class Palette implements Shared { ... } // read by every hand, written by none
```

System.Collections — Standard Library Reference

This is Polaron's general-purpose container library: the growable list, the hash and tree maps and sets, the stack/queue/deque family, priority queue, ring buffer, bitset, and the lazy `Stream` pipeline — plus the `Iterator` / `Iterable` / `Hashable` / `Comparable` interfaces they are built on. If you have reached for `ArrayList`, `HashMap`, or `TreeMap` in another language, the equivalents live here.

All types below live in the `System.Collections` namespace and ship as part of the Polaron prelude (embedded in `src/cli/main.cpp` as `kPreludeSource`). They are ordinary Polaron classes and interfaces with no runtime magic — generic containers backed by dynamic arrays, ring buffers, linked and tree nodes, and hash tables — compiled from that prelude just like your own code, so the signatures shown are the verbatim declarations from the source.

How they fit Polaron:

- **Explicit imports.** Namespace visibility is enforced, so import each type by its fully qualified name before use, e.g. `import System.Collections.ArrayList;`. The import line is given under every type.
- **Generics are monomorphized.** `ArrayList<int>` and `ArrayList<String>` are separate compiled classes; there is no boxing and no shared erased representation.
- **Manual memory.** A collection is an object like any other: allocate it with `new ArrayList<int>()` on heap and `delete` it when done. The containers manage their own backing storage internally (for example, the doubling array `delete`s the old block when it grows).
- **Value semantics for elements.** Storing an element (`add`, `put`, `set`) is an assignment, and assignment in Polaron is a copy. For a value type `T` the element is copied *into* the collection and `get` copies one back *out* — the collection owns those copies. To store shared instances instead of copies, use a pointer element type (`ArrayList<Node*>`): then the collection holds the pointers and does **not** own or `delete` the pointed-to objects — that stays your responsibility.

Key/element constraints

- `HashMap<K,V>` keys and `HashSet<T>` / `Multiset<T>` elements must implement `Hashable` (they call `hash()` / `equalsKey()`).
- `TreeMap<K,V>` keys, `TreeSet<T>` elements, and `PriorityQueue<T>` elements must implement `Comparable` (they call `compareTo()`).
- `ArrayList<T>.indexOf` / `contains` / `remove` call `equalsKey()` on the element, so `T` must be `Hashable` for those three methods.
- Primitive types (the `int` family, `String`) satisfy `Hashable` and `Comparable` through compiler builtins, so they may be used as keys/elements without boxing.

Iterator<T> (interface)

```
import System.Collections.Iterator;
```

A cursor over a sequence: `hasNext` reports whether another element remains, `next` yields the current one and advances.

Signature	Description
method <code>hasNext()</code> returns <code>boolean</code>	True while another element remains.
method <code>next()</code> returns <code>T</code>	Returns the current element and advances the cursor.

Iterable<T> (interface)

```
import System.Collections.Iterable;
```

Anything that can hand out a fresh `Iterator` over its elements, so a generic algorithm can walk any collection.

Signature	Description
method <code>iterator()</code> returns <code>Iterator<T></code>	Returns a new iterator positioned at the first element.

ArrayList<T> — implements Iterable<T>

```
import System.Collections.ArrayList;
```

A growable list backed by a dynamic array that doubles on overflow; the general-purpose sequence collection, with a full functional pipeline (spec 34).

Signature	Description
public constructor <code>ArrayList()</code>	Creates an empty list with capacity 4.
public method <code>add(T item)</code> returns <code>void</code>	Appends an element, doubling the backing array when full.
public method <code>ensureCapacity(int n)</code> returns <code>void</code>	Pre-grows the backing store to hold at least <code>n</code> elements; no-op if capacity already suffices; never shrinks.
public method <code>get(int i)</code> returns <code>T</code>	Returns the element at index <code>i</code> .
public method <code>set(int i, T item)</code> returns <code>void</code>	Overwrites the element at index <code>i</code> .
public method <code>indexOf(T item)</code> returns <code>int</code>	Index of the first element equal to <code>item</code> (via <code>equalsKey</code>), or -1 if absent.
public method <code>contains(T item)</code> returns <code>boolean</code>	True if any element equals <code>item</code> .
public method <code>removeAt(int i)</code> returns <code>void</code>	Removes the element at index <code>i</code> , shifting the tail left.
public method <code>remove(T item)</code> returns <code>boolean</code>	Removes the first element equal to <code>item</code> ; returns whether one was removed.
public method <code>clear()</code> returns <code>void</code>	Resets the list to empty (length 0).
public method <code>toArray()</code> returns <code>T[]</code>	Copies the elements out into a fresh array.
public method <code>size()</code> returns <code>int</code>	Number of elements.
public method <code>isEmpty()</code> returns <code>boolean</code>	True when the list has no elements.

Signature	Description
public method <code>forEach(Action1<T>* action)</code> returns <code>void</code>	Runs <code>action</code> on each element in order.
public method <code>filter(Predicate<T>* keep)</code> returns <code>ArrayList<T></code>	Returns a new list with the elements for which <code>keep</code> is true.
public method <code>map<R>(Mapper<T, R>* transform)</code> returns <code>ArrayList<R></code>	Returns a new list with <code>transform</code> applied to each element (may change type).
public method <code>reduce<R>(R seed, Folder<R, T>* combine)</code> returns <code>R</code>	Folds the elements left-to-right into a single accumulator starting from <code>seed</code> .
public method <code>any(Predicate<T>* pred)</code> returns <code>boolean</code>	True if <code>pred</code> holds for at least one element.
public method <code>all(Predicate<T>* pred)</code> returns <code>boolean</code>	True if <code>pred</code> holds for every element.
public method <code>count(Predicate<T>* pred)</code> returns <code>int</code>	Number of elements satisfying <code>pred</code> .
public method <code>sortedBy(Comparer<T>* compare)</code> returns <code>ArrayList<T></code>	Returns a new list ordered by <code>compare</code> (stable merge sort with insertion-sort fallback); leaves this list untouched.
public method <code>find(Predicate<T>* pred)</code> returns <code>Option<T></code>	First element satisfying <code>pred</code> as <code>Some</code> , else <code>None</code> .
public method <code>min(Comparer<T>* compare)</code> returns <code>Option<T></code>	Smallest element by <code>compare</code> as <code>Some</code> , or <code>None</code> when empty.
public method <code>max(Comparer<T>* compare)</code> returns <code>Option<T></code>	Largest element by <code>compare</code> as <code>Some</code> , or <code>None</code> when empty.
public override method <code>iterator()</code> returns <code>Iterator<T></code>	Returns an <code>ArrayListIterator<T></code> over this list.

```
import System.IO.Console;
import System.Collections.ArrayList;

ArrayList<int> nums = new ArrayList<int>() on heap;
nums.add(10);
nums.add(20);
nums.add(30);           // grows automatically past the initial capacity of 4
mutable int sum = 0;
for (mutable int i = 0; i < nums.size(); i++) {
    sum = sum + nums.get(i);
}
System.IO.Console.printf("size=%d sum=%d\n", nums.size(), sum);
```

ArrayListIterator<T> — implements Iterator<T>

```
import System.Collections.ArrayListIterator;
```

The cursor an `ArrayList` hands out: walks indices `0..size` over the list it was given.

Signature	Description
<code>public constructor ArrayListIterator(ArrayList<T> list)</code>	Binds the iterator to <code>list</code> , positioned at index 0.
<code>public override method hasNext() returns boolean</code>	True while the position is below the list's size.
<code>public override method next() returns T</code>	Returns the element at the current position and advances.

Stream<T> (abstract) — implements Iterator<T>

```
import System.Collections.Stream;
```

A lazy iterator pipeline (spec 25 / stdlib #25): a `Stream` is itself an `Iterator`, and each transform wraps the previous stream, pulling elements on demand. Nothing runs until a terminal op (`fold` / `count` / `forEach`) drives `hasNext` / `next`. The three concrete subclasses below inherit these members.

Signature	Description
<code>public abstract method hasNext() returns boolean</code>	Whether the pipeline can yield another element.
<code>public abstract method next() returns T</code>	Pulls the next element through the pipeline.
<code>public method filter(Predicate<T>* pred) returns Stream<T></code>	Wraps this stream in a <code>FilterStream</code> keeping only elements satisfying <code>pred</code> .
<code>public method map<R>(Mapper<T, R>* fn) returns Stream<R></code>	Wraps this stream in a <code>MapStream</code> applying <code>fn</code> (may change the element type).
<code>public method fold<R>(R init, Folder<R, T>* combine) returns R</code>	Terminal: folds all elements into an accumulator starting at <code>init</code> .
<code>public method forEach(Action1<T>* action) returns void</code>	Terminal: runs <code>action</code> on each remaining element.
<code>public method count() returns int</code>	Terminal: consumes the stream and returns how many elements it produced.

IteratorStream<T> — extends Stream<T>

```
import System.Collections.IteratorStream;
```

Adapts a plain `Iterator` into a `Stream` (the head of a pipeline). Build one with `new IteratorStream<T>(collection.iterator())`, then chain `filter` / `map`.

Signature	Description
<code>public constructor IteratorStream(Iterator<T> src)</code>	Wraps the source iterator <code>src</code> .
<code>public override method hasNext() returns boolean</code>	Delegates to the source iterator.
<code>public override method next() returns T</code>	Delegates to the source iterator.

FilterStream<T> — extends Stream<T>

```
import System.Collections.FilterStream;
```

Yields only the upstream elements that satisfy a predicate; caches one look-ahead so `hasNext` can skip rejected elements without losing the accepted one.

Signature	Description
public constructor FilterStream(Stream<T> src, Predicate<T>* pred)	Wraps <code>src</code> , keeping elements for which <code>pred</code> is true.
public override method hasNext() returns boolean	Advances the source past rejected elements, caching the first accepted one.
public override method next() returns T	Returns the cached accepted element and clears the cache.

MapStream<T, R> — extends Stream<R>

```
import System.Collections.MapStream;
```

Applies a transform to each upstream element as it is pulled, possibly changing the type (`T -> R`); extends `Stream<R>` so it can be chained with a different element type.

Signature	Description
public constructor MapStream(Stream<T> src, Mapper<T, R>* fn)	Wraps <code>src</code> , applying <code>fn</code> to each element.
public override method hasNext() returns boolean	Delegates to the source stream.
public override method next() returns R	Returns <code>fn</code> applied to the source's next element.

Slice<T>

```
import System.Collections.Slice;
```

A non-owning window over a `[start, start+len)` range of an array (spec 34): no copy, just a backing array and an offset. Reads go through the array's own bounds check, so there is no UB.

Signature	Description
public constructor Slice(T[] array, int start, int len)	Creates a window of length <code>len</code> starting at <code>start</code> over <code>array</code> .
public method length() returns int	Number of elements in the window.
public method get(int i) returns T	Element at window-relative index <code>i</code> .
public method set(int i, T value) returns void	Writes <code>value</code> at window-relative index <code>i</code> (into the backing array).
public method sub(int from, int to) returns Slice<T>	Returns a narrower window over <code>[from, to)</code> relative to this slice.

Signature	Description
public method toArray() returns T[]	Copies the window's elements out into a fresh array.

Stack<T>

```
import System.Collections.Stack;
```

LIFO stack backed by a doubling array (spec 34.1).

Signature	Description
public constructor Stack()	Creates an empty stack with capacity 4.
public method push(T item) returns void	Pushes <code>item</code> on top, doubling the backing array when full.
public method pop() returns T	Removes and returns the top element.
public method peek() returns T	Returns the top element without removing it.
public method toArray() returns T[]	Copies the elements out, bottom-to-top (insertion order).
public method size() returns int	Number of elements.
public method isEmpty() returns boolean	True when the stack is empty.

Queue<T>

```
import System.Collections.Queue;
```

FIFO queue backed by a doubling ring buffer (spec 34.1).

Signature	Description
public constructor Queue()	Creates an empty queue with capacity 4.
public method enqueue(T item) returns void	Appends <code>item</code> at the back, growing when full.
public method dequeue() returns T	Removes and returns the front element.
public method peek() returns T	Returns the front element without removing it.
public method toArray() returns T[]	Copies the elements out, front-to-back.
public method size() returns int	Number of elements.
public method isEmpty() returns boolean	True when the queue is empty.

Deque<T>

```
import System.Collections.Deque;
```

Double-ended queue backed by a doubling ring buffer (spec 34.1).

Signature	Description
public constructor Deque()	Creates an empty deque with capacity 4.

Signature	Description
public method addLast(T item) returns void	Appends <code>item</code> at the back.
public method addFirst(T item) returns void	Prepends <code>item</code> at the front.
public method removeFirst() returns T	Removes and returns the front element.
public method removeLast() returns T	Removes and returns the back element.
public method toArray() returns T[]	Copies the elements out, front-to-back.
public method size() returns int	Number of elements.
public method isEmpty() returns boolean	True when the deque is empty.

LinkedListNode<T>

```
import System.Collections.LinkedList;
```

Singly-linked list node (helper for `LinkedList`).

Signature	Description
public mutable T value	The node's stored value.
public mutable nullable LinkedListNode<T>* next	Pointer to the next node, or <code>null</code> at the tail.
public constructor LinkedListNode(T v)	Creates a node holding <code>v</code> with <code>next</code> set to <code>null</code> .

LinkedList<T>

```
import System.Collections.LinkedList;
```

Linked list (spec 34.1): O(1) append and O(1) head removal, built from self-referential generic pointer nodes (`LinkedListNode<T>` holding a `LinkedListNode<T>* next`).

Signature	Description
public constructor LinkedList()	Creates an empty list.
public method add(T item) returns void	Appends <code>item</code> at the tail (O(1)).
public method get(int i) returns T	Returns the element at index <code>i</code> (walks from the head).
public method removeFirst() returns T	Removes and returns the head element, freeing its node.
public method toArray() returns T[]	Copies the elements out, head-to-tail.
public method size() returns int	Number of elements.
public method isEmpty() returns boolean	True when the list is empty.

Hashable<T> (interface)

```
import System.Collections.Hashable;
```

Implemented by keys of `HashMap` / `HashSet` (spec 34). Primitive types satisfy it via compiler builtins.

Signature	Description
method <code>hash()</code> returns <code>long</code>	A hash code for this value.
method <code>equalsKey(T other)</code> returns <code>boolean</code>	Whether this value is equal to <code>other</code> for keying purposes.

Comparable<T> (interface)

```
import System.Collections.Comparable;
```

Implemented by keys of `TreeMap` / `TreeSet` and elements of `PriorityQueue` (spec 34). Primitive types satisfy it via compiler builtins.

Signature	Description
method <code>compareTo(T other)</code> returns <code>int</code>	Negative if this precedes <code>other</code> , zero if equal, positive if it follows.

HashMap<K, V>

```
import System.Collections.HashMap;
```

Hash map with open addressing (linear probing); capacity is a power of two, load factor 0.75 (spec 34.1). Keys must be `Hashable`. `get()` on an absent key returns a zero/null value — probe with `containsKey()` first.

Signature	Description
public constructor <code>HashMap()</code>	Creates an empty map with capacity 8.
public method <code>put(K key, V value)</code> returns <code>void</code>	Inserts or overwrites the value for <code>key</code> , growing at load factor 0.75.
public method <code>get(K key)</code> returns <code>V</code>	Value for <code>key</code> , or a zero/null value if absent (check <code>containsKey</code> first).
public method <code>containsKey(K key)</code> returns <code>boolean</code>	Whether <code>key</code> is present.
public method <code>getOrDefault(K key, V defaultValue)</code> returns <code>V</code>	Value for <code>key</code> , or <code>defaultValue</code> if absent (single probe).
public method <code>merge(K key, V value, Combiner<V, V, V>* combine)</code> returns <code>void</code>	Inserts <code>value</code> , or replaces the existing value with <code>combine(old, value)</code> — one probe (efficient tallying).
public method <code>remove(K key)</code> returns <code>boolean</code>	Removes <code>key</code> (backward-shift of its probe cluster); returns whether it was present.
public method <code>keyArray()</code> returns <code>K[]</code>	All keys, in arbitrary order.
public method <code>valueArray()</code> returns <code>V[]</code>	All values, in arbitrary order.
public method <code>size()</code> returns <code>int</code>	Number of entries.
public method <code>isEmpty()</code> returns <code>boolean</code>	True when the map has no entries.

```
import System.IO.Console;
import System.Collections.HashMap;

HashMap<String, int> scores = new HashMap<String, int>() on heap;
scores.put("alpha", 1);
scores.put("beta", 2);
if (scores.containsKey("alpha")) {           // get() alone can't tell absent from zero
    System.IO.Console.printf("alpha=%d\n", scores.get("alpha"));
}
int g = scores.getOrDefault("gamma", -1);    // -1 because "gamma" is absent
System.IO.Console.printf("gamma=%d size=%d\n", g, scores.size());
```

HashSet<T>

```
import System.Collections.HashSet;
```

Hash set with open addressing (spec 34.1). Elements must be `Hashable` .

Signature	Description
public constructor HashSet()	Creates an empty set with capacity 8.
public method add(T value) returns void	Inserts <code>value</code> if not already present, growing at load factor 0.75.
public method contains(T value) returns boolean	Whether <code>value</code> is present.
public method remove(T value) returns boolean	Removes <code>value</code> (backward-shift deletion); returns whether it was present.
public method toArray() returns T[]	All elements, in arbitrary order.
public method size() returns int	Number of elements.
public method isEmpty() returns boolean	True when the set is empty.

TreeNode<K, V>

```
import System.Collections.TreeNode;
```

Binary-search-tree node for `TreeMap` (self-referential generic).

Signature	Description
public mutable K key	The node's key.
public mutable V value	The value mapped to <code>key</code> .
public mutable nullable TreeNode<K, V>* left	Left child (smaller keys), or <code>null</code> .
public mutable nullable TreeNode<K, V>* right	Right child (larger keys), or <code>null</code> .
public mutable int height	AVL subtree height, used for O(log n) balancing.
public constructor TreeNode(K k, V v)	Creates a leaf node for <code>k</code> / <code>v</code> (children <code>null</code> , height 1).

TreeMap<K, V>

```
import System.Collections.TreeMap;
```

Ordered map backed by an AVL-balanced binary search tree (spec 34.1). Keys are `Comparable`. `get()` on an absent key returns a zero/null value — probe with `containsKey()` first. Navigable-key queries return a zero/null key when none qualifies.

Signature	Description
public constructor <code>TreeMap()</code>	Creates an empty map.
public method <code>put(K key, V value)</code> returns <code>void</code>	Inserts or overwrites <code>key</code> , rebalancing to keep height $O(\log n)$.
public method <code>get(K key)</code> returns <code>V</code>	Value for <code>key</code> , or a zero/null value if absent.
public method <code>containsKey(K key)</code> returns <code>boolean</code>	Whether <code>key</code> is present.
public method <code>keyArray()</code> returns <code>K[]</code>	All keys in sorted (in-order) order.
public method <code>valueArray()</code> returns <code>V[]</code>	All values, in key order.
public method <code>firstKey()</code> returns <code>K</code>	Smallest key (leftmost), or a zero/null key if empty.
public method <code>lastKey()</code> returns <code>K</code>	Largest key (rightmost), or a zero/null key if empty.
public method <code>floorKey(K key)</code> returns <code>K</code>	Largest key $\leq$ <code>key</code> , or a zero/null key if none.
public method <code>ceilingKey(K key)</code> returns <code>K</code>	Smallest key $\geq$ <code>key</code> , or a zero/null key if none.
public method <code>higherKey(K key)</code> returns <code>K</code>	Smallest key strictly $>$ <code>key</code> , or a zero/null key if none.
public method <code>lowerKey(K key)</code> returns <code>K</code>	Largest key strictly $<$ <code>key</code> , or a zero/null key if none.
public method <code>size()</code> returns <code>int</code>	Number of entries.
public method <code>isEmpty()</code> returns <code>boolean</code>	True when the map has no entries.

TreeNode<T>

```
import System.Collections.TreeSetNode;
```

BST node for `TreeSet` (self-referential generic).

Signature	Description
public mutable <code>T</code> value	The node's element.
public mutable nullable <code>TreeNode<T>*</code> left	Left child (smaller values), or <code>null</code> .
public mutable nullable <code>TreeNode<T>*</code> right	Right child (larger values), or <code>null</code> .
public mutable <code>int</code> height	AVL subtree height.
public constructor <code>TreeNode(T v)</code>	Creates a leaf node for <code>v</code> (children <code>null</code> , height 1).

TreeSet<T>

```
import System.Collections.TreeSet;
```

Ordered set backed by an AVL-balanced binary search tree (spec 34.1). Elements are `Comparable`.

Signature	Description
<code>public constructor TreeSet()</code>	Creates an empty set.
<code>public method add(T value) returns void</code>	Inserts <code>value</code> if not present, rebalancing to keep height $O(\log n)$.
<code>public method contains(T value) returns boolean</code>	Whether <code>value</code> is present.
<code>public method toArray() returns T[]</code>	All elements in sorted (in-order) order.
<code>public method size() returns int</code>	Number of elements.
<code>public method isEmpty() returns boolean</code>	True when the set is empty.

PriorityQueue<T>

```
import System.Collections.PriorityQueue;
```

Binary min-heap priority queue (spec 34.1): the smallest element (by `compareTo`) is served first. Backed by a doubling array; `add` and `poll` are $O(\log n)$. Elements are `Comparable`.

Signature	Description
<code>public constructor PriorityQueue()</code>	Creates an empty queue with capacity 8.
<code>public method add(T item) returns void</code>	Inserts <code>item</code> and sifts it up to restore the heap.
<code>public method peek() returns T</code>	Returns the smallest element without removing it.
<code>public method poll() returns T</code>	Removes and returns the smallest element, sifting down to restore the heap.
<code>public method size() returns int</code>	Number of elements.
<code>public method isEmpty() returns boolean</code>	True when the queue is empty.

Bitset

```
import System.Collections.Bitset;
```

A compact set of bit flags packed into 32-bit words (spec 34.1). Cheaper than a `HashSet<int>` for dense indices in a known range. (Non-generic.)

Signature	Description
<code>public constructor Bitset(int size)</code>	Creates a bitset holding <code>size</code> bits, all cleared.
<code>public method set(int i) returns void</code>	Sets bit <code>i</code> to 1.

Signature	Description
<code>public method clear(int i) returns void</code>	Clears bit <code>i</code> to 0.
<code>public method flip(int i) returns void</code>	Toggles bit <code>i</code> .
<code>public method get(int i) returns boolean</code>	Whether bit <code>i</code> is set.
<code>public method count() returns int</code>	Number of bits currently set (population count).
<code>public method size() returns int</code>	Total capacity in bits.

Multiset<T>

```
import System.Collections.Multiset;
```

Counts how many times each value is added (spec 34.1): a multiset over a `HashMap`. The element type must be `Hashable`, like any `HashMap` key. (Named `Multiset` rather than `Counter` so it does not shadow a user class.)

Signature	Description
<code>public constructor Multiset()</code>	Creates an empty multiset.
<code>public method add(T item) returns void</code>	Bumps <code>item</code> 's tally by one.
<code>public method count(T item) returns int</code>	Current tally of <code>item</code> (0 if never added).
<code>public method distinct() returns int</code>	Number of distinct values seen.
<code>public method total() returns int</code>	Sum of all tallies.
<code>public method keys() returns T[]</code>	The distinct values, in arbitrary order.

RingBuffer<T>

```
import System.Collections.RingBuffer;
```

A fixed-capacity circular buffer (spec 34.1): `push` appends and, once full, overwrites the oldest element; `pop` / `peek` read from the oldest end. Indices wrap with modulo, so there is no shifting.

Signature	Description
<code>public constructor RingBuffer(int capacity)</code>	Creates a buffer holding up to <code>capacity</code> elements.
<code>public method push(T item) returns void</code>	Appends <code>item</code> ; when full, overwrites the oldest element and advances the head.
<code>public method pop() returns T</code>	Removes and returns the oldest element.
<code>public method peek() returns T</code>	Returns the oldest element without removing it.
<code>public method size() returns int</code>	Current number of elements.
<code>public method isEmpty() returns boolean</code>	True when the buffer holds nothing.
<code>public method isFull() returns boolean</code>	True when the buffer is at capacity.

Signature	Description
public method capacity() returns int	The fixed maximum number of elements.

LinkedHashMap<K, V>

```
import System.Collections.LinkedHashMap;
```

A map that remembers insertion order (spec 34.1): a `HashMap` for lookup plus a list of keys in the order they were first added. `put` updates in place without reordering; `keysInOrder` walks them as inserted, which a plain `HashMap` does not promise. Keys must be `Hashable`.

Signature	Description
public constructor LinkedHashMap()	Creates an empty ordered map.
public method put(K key, V value) returns void	Inserts or updates <code>key</code> ; records first-insertion order.
public method get(K key) returns V	Value for <code>key</code> (delegates to the backing map).
public method containsKey(K key) returns boolean	Whether <code>key</code> is present.
public method size() returns int	Number of entries.
public method isEmpty() returns boolean	True when the map has no entries.
public method keysInOrder() returns ArrayList<K>	Keys in the order they were first inserted.

Stream<T> — the lazy half of the pipeline

```
import System.Collections.Stream;
```

`ArrayList`'s `map` / `filter` / `reduce` are eager: each one builds a list. A `Stream` is the same vocabulary without the intermediate lists — it pulls one element at a time, which is what makes a pipeline over a large source (or an endless one) affordable.

Type	What it is
<code>Stream<T></code>	The base: <code>next()</code> , <code>hasNext()</code> , and the operators below, each answering another <code>Stream</code> .
<code>TakeStream<T></code>	<code>take(n)</code> — the first <code>n</code> , then done.
<code>SkipStream<T></code>	<code>skip(n)</code> — everything after the first <code>n</code> .
<code>TakeWhileStream<T></code>	<code>takeWhile(pred)</code> — up to the first element that fails the test, and no further. It caches one element, because deciding to stop means having looked.
<code>toList()</code>	Back to an <code>ArrayList</code> when the result is small and wanted whole.

The three wrapper types are public because a `Stream` is extensible: a source of your own extends `Stream<T>` and the whole vocabulary applies to it.

Hashing

```
import System.Collections.Hashing;
```

- `public static method mix(long h) returns long` — the avalanche step the hashed containers use.

Exposed because a type writing its own `hash()` should mix with the same function the tables do: a hash that is merely *different* from the table's expectation clusters, and clustering is a performance bug that looks like a data problem.

Ints

```
import System.Collections.Ints;
```

Primitive-array helpers that do not go through the generic containers — no boxing, no `equalsKey` :

- `public static method sort(int[] a) returns void` — in place.
- `public static method binarySearch(int[] a, int key) returns int` — the index, or negative.
- `public static method swap(int[] a, int i, int j) returns void`.

`Arrays` beside it is the general form (`sort` , `reverse` , `shuffle` , `contains` , `indexOf`) over any array — including one of objects, which is what nothing could sort before it existed.

Standard Library Reference

— Advanced Data Structures

This slice of the Polaron standard library provides algorithmic and data-oriented types: prefix trees, graphs, heaps, disjoint sets, balanced/unbalanced trees, order-statistic sets, dynamic programming helpers, geometry/interval utilities, a small entity-component-system (ECS), and an observer/event system.

All of these types live in the embedded Polaron prelude (the `kPreludeSource` raw string in `src/cli/main.cpp`). Every `stdlib` type must be imported explicitly before use. The three namespaces covered here are:

- `System.Collections` — the algorithmic collections and structures (`Range ... SpatialGrid`)
- `System.Ecs` — the entity-component-system core (`World, ComponentStore`)
- `System.Events` — the observer/event dispatch system (`Signal, IntEvent, StringEvent`, and their handlers)

Import a type with its fully qualified name, e.g. `import System.Collections.IntHeap;`

Reach for these when the everyday containers on the `System.Collections` collections page stop being enough: when you need shortest paths instead of plain adjacency, prefix lookups instead of whole-key equality, disjoint-set merges, range queries answered in logarithmic time, or eviction once a cache is full. None of them are runtime magic — each is ordinary Polaron whose cost is exactly what its backing arrays and loops imply, and the short paragraph above each type names that cost so you can pick the right tool for the input size. Many trade pointer chasing for flat arena arrays (a `next[node * 26 + c]` link instead of a node object), which keeps allocation predictable and plays to Polaron's manual-memory model.

Notes that apply throughout: an `int[]` is a heap-backed dynamic array; `T[]` is a generic array; a `Predicate<T>*` / `Mapper<T, R>*` / `Comparer<T>*` and the rest of `System.Commands` are command types — roles a `command` satisfies structurally, so an API names the shape it will call rather than any particular command (reference 14.1); `nullable T*` is a nullable pointer; `T&` is a by-reference parameter. In the examples below, a `// →` comment shows the value a call returns.

Range

Namespace: `System.Collections` — `import System.Collections.Range;`

A first-class range value (`start..end`, `start..=end`, and `step`), iterable with `foreach` and queryable. Also exposes static helpers that materialize integer ranges as `ArrayList<int>`.

Public fields: - `public int start;` — inclusive start of the range. - `public int end;` — end bound (inclusive only when `inclusive` is true). - `public int stride;` — the step increment (`step` is a reserved keyword, so the field is named `stride`). - `public boolean inclusive;` — whether `end` is included.

Members: - `public constructor Range(int start, int end, int stride, boolean inclusive)` — build a range with an explicit step and inclusivity. - `public method size()` returns `int` — number of elements the range yields, honoring step direction and inclusivity. - `public method contains(int v)` returns `boolean` — whether `v` is one of the range's yielded values. - `public method toArray()` returns

`int[]` — materialize the range into a freshly allocated `int[]`. - `public static method upto(int n)` returns `ArrayList<int>` — the list `0, 1, ..., n-1`. - `public static method between(int start, int end)` returns `ArrayList<int>` — the list `start, ..., end-1`. - `public static method stepBy(int start, int end, int by)` returns `ArrayList<int>` — `start` up to (exclusive) `end` in increments of `by`; empty if `by ≤ 0`.

Trie

Namespace: `System.Collections` — `import System.Collections.Trie;`

A prefix tree over lowercase-letter words, stored as a flat arena (26 child links + end-of-word flag per node indexed by node number, no node pointers). A link value of 0 means "none".

Members: - `public constructor Trie()` — create an empty trie (root node only). - `public method insert(String word)` returns `void` — add a word, creating nodes as needed. - `public method contains(String word)` returns `boolean` — whether the exact word was inserted. - `public method startsWith(String prefix)` returns `boolean` — whether any inserted word has the given prefix.

Insert and lookup each cost $O(\text{length of the word})$, independent of how many words the trie holds — the reason to prefer it over a `HashSet` when you also need prefix queries. Words are restricted to lowercase `a-z`.

```
import System.Collections.Trie;

Trie t = new Trie() on heap;
t.insert("cat");
t.insert("car");
t.contains("cat");    // → true
t.contains("ca");     // → false (only whole inserted words match)
t.startsWith("ca");  // → true  ("cat" and "car" share it)
```

Graph

Namespace: `System.Collections` — `import System.Collections.Graph;`

An undirected graph on a fixed vertex set, edges kept as two parallel lists (no per-node pointers). Answers shortest hop count between vertices via BFS.

Members: - `public constructor Graph(int vertices)` — create a graph with `vertices` vertices numbered `0..vertices-1`. - `public method addEdge(int u, int v)` returns `void` — add an undirected edge between `u` and `v`. - `public method distance(int src, int dst)` returns `int` — shortest hop count from `src` to `dst`, or `-1` if unconnected.

`distance` runs a fresh breadth-first search each call, so it is $O(V + E)$ per query; for many queries from the same source, run one BFS yourself and read the distance array. Use this when edges are unweighted and you want the fewest hops (for weighted shortest paths, see `WeightedGraph`).

```
import System.Collections.Graph;

Graph g = new Graph(5) on heap;    // vertices 0..4
g.addEdge(0, 1);
g.addEdge(1, 2);
g.addEdge(3, 4);
g.distance(0, 2);    // → 2    (0 → 1 → 2)
g.distance(0, 4);    // → -1   (a different component)
```

SlotMap\

Namespace: System.Collections — `import System.Collections.SlotMap;`

A generational arena: `insert` returns a stable integer handle (slot index and generation packed into one int) that stays valid until removed, even as storage grows or slots are reused. A removed slot bumps its generation so stale handles are detected.

Members: - `public constructor SlotMap()` — create an empty slot map with initial capacity 4. - `public method insert(T value) returns int` — store a value and return its handle. - `public method containsHandle(int h) returns boolean` — whether handle `h` still refers to a live slot of the matching generation. - `public method get(int h) returns T` — the value at handle `h` (no staleness check). - `public method remove(int h) returns void` — free the slot for a live handle, bumping its generation. - `public method size() returns int` — number of occupied slots.

IntHeap

Namespace: System.Collections — `import System.Collections.IntHeap;`

A binary min-heap of integers: `push` / `pop` keep the smallest element on top, so repeated `pop` yields ascending order. The backing array grows as needed.

Members: - `public constructor IntHeap()` — create an empty heap (initial capacity 8). - `public method push(int v) returns void` — insert `v`, restoring the heap property. - `public method pop() returns int` — remove and return the minimum. - `public method peek() returns int` — the current minimum without removing it. - `public method size() returns int` — number of elements.

UnionFind

Namespace: System.Collections — `import System.Algorithms.UnionFind;`

Disjoint-set / union-find over a fixed range `0..n-1`, using union by rank with path halving.

Members: - `public constructor UnionFind(int n)` — `n` singleton sets, one per element. - `public method find(int x) returns int` — the representative (root) of `x`'s set, compressing the path. - `public method merge(int a, int b) returns void` — union the sets containing `a` and `b`. - `public method connected(int a, int b) returns boolean` — whether `a` and `b` are in the same set. - `public method groups() returns int` — number of distinct sets remaining.

With union by rank and path halving, every operation runs in near-constant amortized time (the inverse-Ackermann bound). It is the workhorse for connectivity questions: grouping equivalences, detecting cycles as you add edges, or building Kruskal's minimum spanning tree.

```
import System.Algorithms.UnionFind;

UnionFind uf = new UnionFind(6);    // 6 singletons: {0} {1} {2} {3} {4} {5}
uf.merge(0, 1);
uf.merge(1, 2);
uf.connected(0, 2);    // → true
uf.connected(0, 3);    // → false
uf.groups();           // → 4    ({0,1,2} {3} {4} {5})
```

BstNode

Namespace: `System.Collections` — `import System.Collections.BstNode;`

A node in a binary search tree, linked by nullable child pointers. `collect` does an in-order walk appending keys in sorted order. (Used to build `Bst`, but public in its own right.)

Public fields: - `public mutable int key;` — the node's key. - `public mutable nullable BstNode* left;` — left child (smaller keys), or `null`. - `public mutable nullable BstNode* right;` — right child (larger keys), or `null`.

Members: - `public constructor BstNode(int k)` — a leaf node holding key `k` (both children `null`). - `public method collect(ArrayList<int>& out)` returns `void` — in-order traversal, appending each key into the caller's list.

Bst

Namespace: `System.Collections` — `import System.Collections.Bst;`

An ordered set of ints backed by an unbalanced (genuinely pointer-linked) binary search tree. `insert` ignores duplicates; `inOrder` returns the keys sorted.

Members: - `public constructor Bst()` — an empty tree. - `public method insert(int k)` returns `void` — insert `k` (no-op if already present). - `public method contains(int k)` returns `boolean` — membership test. - `public method size()` returns `int` — number of distinct keys. - `public method inOrder()` returns `ArrayList<int>` — all keys in ascending order.

SortedIntSet

Namespace: `System.Collections` — `import System.Collections.SortedIntSet;`

A sorted set of ints on a growable sorted array with binary search: dedup insert, membership, navigable floor/ceiling, and order statistics (rank and k-th smallest). `floor` / `ceiling` return int min/max sentinels when absent.

Members: - `public constructor SortedIntSet()` — an empty set (initial capacity 8). - `public method add(int v) returns void` — insert `v`, keeping the array sorted; ignores duplicates. - `public method contains(int v) returns boolean` — membership test. - `public method rank(int v) returns int` — count of elements strictly less than `v`. - `public method floor(int v) returns int` — largest element $\leq v$, or `-2147483648` if none. - `public method ceiling(int v) returns int` — smallest element $\geq v$, or `2147483647` if none. - `public method kth(int i) returns int` — the `i`-th smallest element (0-based). - `public method size() returns int` — number of elements.

IntCounter

Namespace: `System.Collections` — `import System.Collections.IntCounter;`

A multiset / frequency counter of ints backed by a `HashMap`. Tracks the running most-common value, total, and distinct count in $O(1)$ per add; `count` returns 0 for absent keys.

Members: - `public constructor IntCounter()` — an empty counter. - `public method add(int v) returns void` — tally one occurrence of `v`. - `public method count(int v) returns int` — occurrences of `v` (0 if never added). - `public method mostCommon() returns int` — the value with the highest running count. - `public method maxCount() returns int` — the count of the most common value. - `public method total() returns int` — total number of add calls. - `public method distinct() returns int` — number of distinct values seen.

ImmutableList\

Namespace: `System.Collections` — `import System.Collections.ImmutableList;`

An immutable list: copies the source array at construction and exposes only reads, so contents can never change afterward.

Members: - `public constructor ImmutableList(T[] src, int count)` — copy the first `count` elements of `src`. - `public method get(int i) returns T` — element at index `i`. - `public method size() returns int` — number of elements. - `public method isEmpty() returns boolean` — whether the list has no elements.

EnumMap\

Namespace: `System.Collections` — `import System.Collections.EnumMap;`

A dense map keyed by enum ordinal (Polaron enums are int ordinals): values live in a flat array sized to the enum, with a parallel presence flag. $O(1)$ put/get/containsKey.

Members: - `public constructor EnumMap(int size)` — a map covering ordinals `0..size-1`. - `public method put(int ord, V v) returns void` — set the value for ordinal `ord`. - `public method get(int ord) returns V` — the value at ordinal `ord`. - `public method containsKey(int ord) returns boolean` — whether a value was set for `ord`. - `public method size() returns int` — number of ordinals with a value.

EnumSet

Namespace: `System.Collections` — `import System.Collections.EnumSet;`

A dense set of enum ordinals: a fixed-size flag array over the enum's constants, with $O(1)$ add/remove/contains and a maintained count.

Members: - `public constructor EnumSet(int size)` — a set over ordinals `0..size-1`. - `public method add(int ord) returns void` — include ordinal `ord`. - `public method remove(int ord) returns void` — exclude ordinal `ord`. - `public method contains(int ord) returns boolean` — whether `ord` is present. - `public method size() returns int` — number of present ordinals.

Fenwick

Namespace: `System.Collections` — `import System.Algorithms.Fenwick;`

A Fenwick tree / binary indexed tree: $O(\log n)$ point updates and prefix/range sums over int values. Public API is 0-based.

Members: - `public constructor Fenwick(int size)` — a tree over `size` positions, all zero. - `public method add(int i, int delta) returns void` — add `delta` to position `i`. - `public method prefixSum(int i) returns int` — sum of positions `0..i` inclusive. - `public method rangeSum(int lo, int hi) returns int` — sum of positions `lo..hi` inclusive.

SegmentTree

Namespace: `System.Collections` — `import System.Algorithms.SegmentTree;`

An iterative segment tree for range sums with $O(\log n)$ point updates, built from an `int[]`. Works for any length `n`.

Members: - `public constructor SegmentTree(int[] data)` — build the tree from the initial values. - `public method update(int i, int value) returns void` — set position `i` to `value`. - `public method query(int lo, int hi) returns int` — inclusive sum over `lo..hi`.

Reach for a segment tree when values change *and* you keep asking for range sums: both `update` and `query` are $O(\log n)$. If the data never changes, a `Fenwick` tree is lighter; if you only need range *minimums* over fixed data, `SparseTable` answers in $O(1)$.

```
import System.Algorithms.SegmentTree;

int[] data = new int[5]();
data[0] = 1; data[1] = 3; data[2] = 5; data[3] = 7; data[4] = 9;
SegmentTree st = new SegmentTree(data) on heap;
st.query(1, 3);    // → 15    (3 + 5 + 7)
st.update(2, 0);   // position 2: 5 → 0
st.query(1, 3);    // → 10    (3 + 0 + 7)
```

SparseTable

Namespace: `System.Collections` — `import System.Algorithms.SparseTable;`

A sparse table for $O(1)$ range-minimum queries over a fixed `int[]`, after $O(n \log n)$ build.

Members: - `public constructor SparseTable(int[] data)` — precompute the sparse table from the data. - `public method queryMin(int lo, int hi)` returns `int` — inclusive minimum over `lo..hi`.

WeightedGraph

Namespace: `System.Collections` — `import System.Collections.WeightedGraph;`

A weighted undirected graph on a dense adjacency matrix (`-1` means no edge). Provides Dijkstra shortest paths ($O(V^2)$) and minimum-spanning-tree weight via Prim.

Members: - `public constructor WeightedGraph(int vertices)` — a graph with no edges on `vertices` vertices. - `public method addEdge(int u, int v, int w)` returns `void` — add/replace an undirected edge of weight `w`. - `public method dijkstra(int src)` returns `int[]` — shortest-path distances from `src`; unreachable vertices stay at `1000000000`. - `public method mstWeight()` returns `int` — total weight of a minimum spanning tree (Prim's algorithm).

LruCache

Namespace: `System.Collections` — `import System.Collections.LruCache;`

A fixed-capacity LRU cache mapping `int` keys to `int` values with $O(1)$ get/put. A doubly linked list over slot arrays tracks recency; a `HashMap` finds a key's slot.

Members: - `public constructor LruCache(int capacity)` — an empty cache holding up to `capacity` entries. - `public method get(int key)` returns `int` — value for `key` (marks it most-recent), or `-1` if absent. - `public method contains(int key)` returns `boolean` — whether `key` is cached (does not change recency). - `public method put(int key, int value)` returns `void` — insert/update `key`, evicting the least-recently-used entry if full. - `public method count()` returns `int` — current number of entries.

Every operation is $O(1)$: a `HashMap` locates a key's slot and a doubly linked list over the slot arrays tracks recency, so the tail is always the entry to evict. `get` and `put` count as a use and move the entry to the front; `contains` deliberately does not, so you can probe without disturbing the order.

```
import System.Collections.LruCache;

LruCache cache = new LruCache(2) on heap;    // capacity 2
cache.put(1, 100);
cache.put(2, 200);
cache.get(1);           // → 100  (key 1 is now most-recent)
cache.put(3, 300);      // full: evicts key 2, the least-recently-used
cache.get(2);           // → -1   (evicted)
cache.get(3);           // → 300
```

Knapsack

Namespace: `System.Collections` — `import System.Algorithms.Knapsack;`

0/1 knapsack via the classic one-dimensional DP (swept high capacity to low so each item is used at most once). Static-only utility.

Members: - `public static method maxValue(int[] weights, int[] values, int n, int capacity)` returns `int` — maximum total value of items fitting within `capacity` (first `n` items).

Lcs

Namespace: `System.Collections` — `import System.Algorithms.Lcs;`

Longest common subsequence length of two strings via a flat DP table. Static-only utility.

Members: - `public static method length(String a, String b)` returns `int` — length of the longest common subsequence of `a` and `b`.

QuickSelect

Namespace: `System.Collections` — `import System.Algorithms.QuickSelect;`

Quickselect for the k-th smallest element in expected linear time (Lomuto partition, last-element pivot). Partitions the array in place.

Members: - `public static method select(int[] a, int n, int k)` returns `int` — the k-th smallest element (0-indexed) among the first `n` of `a`; reorders `a` in place.

Comparators

Namespace: `System.Collections` — `import System.Collections.Comparators;`

Comparator combinators, each a `command`; each is a `Comparer<int>` and so goes straight into `ArrayList.sortedBy`. Two of them CARRY the comparators they wrap, which is what a combinator is.

Members: - `public command naturalInt(int a, int b)` returns `int` — ascending int order. Built as `Comparators.naturalInt()`. - `public command reversed(int a, int b)` carries `(Comparer<int>* cmp)` into `pack` returns `int` — flips `cmp`. Built as `Comparators.reversed(other)`. - `public command thenComparing(int a, int b)` carries `(Comparer<int>* first, Comparer<int>* second)` into `pack` returns `int` — `first`, falling back to `second` as a tie-breaker.

MinStack

Namespace: `System.Collections` — `import System.Collections.MinStack;`

A stack that also reports its minimum in O(1) via a parallel stack of running minima.

Members: - `public constructor MinStack(int capacity)` — an empty stack that holds up to `capacity` items. - `public method push(int v) returns void` — push `v`. - `public method pop() returns int` — pop and return the top. - `public method peek() returns int` — the top without removing it. - `public method getMin() returns int` — the current minimum among all pushed items. - `public method size() returns int` — number of items.

SlidingWindowMax

Namespace: `System.Collections` — `import System.Algorithms.SlidingWindowMax;`

Sliding-window maximum in linear time via a monotonic deque of indices. Static-only utility.

Members: - `public static method maxOfEach(int[] a, int n, int k) returns int[]` — the maximum of each length-`k` window over the first `n` of `a` (returns `n-k+1` values).

Kadane

Namespace: `System.Collections` — `import System.Algorithms.Kadane;`

Maximum-subarray sum by Kadane's algorithm. Static-only utility.

Members: - `public static method maxSubarray(int[] a, int n) returns int` — largest sum of any contiguous run in the first `n` of `a` (0 if `n == 0`).

IntervalMerge

Namespace: `System.Collections` — `import System.Algorithms.IntervalMerge;`

Interval merging: sorts intervals by start and coalesces overlaps in place. Static-only utility.

Members: - `public static method merge(int[] starts, int[] ends, int n) returns int` — sort and coalesce `n` intervals in place, writing merged starts/ends into the first slots; returns the merged count. - `public static method coveredLength(int[] starts, int[] ends, int mergedCount) returns int` — total length of the merged spans (`sum of ends[i] - starts[i]`).

IntervalScheduler

Namespace: `System.Collections` — `import System.Algorithms.IntervalScheduler;`

Activity selection: the maximum number of mutually non-overlapping intervals, by the greedy earliest-finishing-time rule. Static-only utility.

Members: - `public static method maxNonOverlapping(int[] starts, int[] ends, int n) returns int` — the largest set of non-overlapping intervals among the first `n`.

BiMap\

Namespace: `System.Collections` — `import System.Collections.BiMap;`

A bidirectional map: keeps key->value and value->key in sync so either side can be looked up. `put` overwrites both directions.

Members: - `public constructor BiMap()` — an empty bidirectional map. - `public method put(K k, V v)` returns `void` — associate `k` and `v` in both directions. - `public method getByKey(K k)` returns `V` — the value mapped to key `k`. - `public method getByValue(V v)` returns `K` — the key mapped to value `v`. - `public method hasKey(K k)` returns `boolean` — whether key `k` is present. - `public method size()` returns `int` — number of entries.

MultiMap\

Namespace: `System.Collections` — `import System.Collections.MultiMap;`

A multimap: each key maps to a growable list of values. `put` appends.

Members: - `public constructor MultiMap()` — an empty multimap. - `public method put(K k, V v)` returns `void` — append `v` to the list for key `k`. - `public method countFor(K k)` returns `int` — number of values stored for `k` (0 if absent). - `public method get(K k, int i)` returns `V` — the `i`-th value stored for key `k`.

Fenwick2D

Namespace: `System.Collections` — `import System.Algorithms.Fenwick2D;`

A 2D Fenwick tree / binary indexed tree: $O(\log r \cdot \log c)$ point updates and rectangle sums over an int grid. Coordinates are 0-based.

Members: - `public constructor Fenwick2D(int rows, int cols)` — a `rows × cols` grid, all zero. - `public method update(int r, int c, int delta)` returns `void` — add `delta` at cell `(r, c)`. - `public method prefix(int r, int c)` returns `int` — sum over the rectangle `(0,0)..(r,c) inclusive. - public method rangeSum(int r1, int c1, int r2, int c2) returns int — sum over the rectangle (r1,c1)..(r2,c2) inclusive.`

DiGraph

Namespace: `System.Collections` — `import System.Collections.DiGraph;`

A directed graph on a dense adjacency matrix. `topoSort` produces a topological ordering by Kahn's algorithm (lowest-index source first); a short order means a cycle.

Members: - `public constructor DiGraph(int vertices)` — an edge-free directed graph on `vertices` vertices. - `public method addEdge(int u, int v)` returns `void` — add a directed edge `u → v`. - `public method topoSort(int[] order)` returns `int` — fill `order` with a topological ordering; returns

how many vertices were placed (< n means a cycle). - `public method hasCycle()` returns `boolean` — whether the graph contains a cycle.

SpatialGrid

Namespace: `System.Collections` — `import System.Collections.SpatialGrid;`

A uniform spatial hash grid for 2D broad-phase queries: points are bucketed by cell so `queryRect` only scans overlapping cells. Coordinates are assumed non-negative.

Members: - `public constructor SpatialGrid(int cellSize, int capacity)` — a grid with square cells of side `cellSize`, holding up to `capacity` points. - `public method insert(int x, int y)` returns `void` — add a point at `(x, y)`. - `public method queryRect(int x0, int y0, int x1, int y1)` returns `int` — count of inserted points inside the box `(x0,y0)..(x1,y1)` inclusive.

World

Namespace: `System.Ecs` — `import System.Ecs.World;`

An entity-component-system core (data-oriented): hands out integer entity ids and recycles destroyed ones. Component data lives outside the entity in `ComponentStore`.

Members: - `public constructor World()` — an empty world (initial capacity 16). - `public method createEntity()` returns `int` — allocate a new (or recycled) entity id and mark it alive. - `public method destroyEntity(int e)` returns `void` — mark entity `e` dead and recycle its id. - `public method isAlive(int e)` returns `boolean` — whether `e` is a live entity id. - `public method size()` returns `int` — number of live entities. - `public method capacity()` returns `int` — current entity-id capacity.

ComponentStore\

Namespace: `System.Ecs` — `import System.Ecs.ComponentStore;`

A sparse-set store mapping an entity id to one component of type `T`. Components live in a dense array for fast iteration; the sparse array maps entity -> dense slot. `remove` is swap-with-last. Sized for entity ids in `[0, maxEntities)`.

Members: - `public constructor ComponentStore(int maxEntities)` — a store for entity ids `0..maxEntities-1`. - `public method add(int e, T component)` returns `void` — attach component to entity `e`. - `public method has(int e)` returns `boolean` — whether entity `e` has a component here. - `public method get(int e)` returns `T` — the component attached to entity `e`. - `public method set(int e, T component)` returns `void` — replace entity `e`'s component in place. - `public method remove(int e)` returns `void` — detach entity `e`'s component (swap-with-last). - `public method size()` returns `int` — number of stored components. - `public method entityAt(int i)` returns `int` — the entity id at dense index `i` (for iteration). - `public method at(int i)` returns `T` — the component at dense index `i` (for iteration).

VoidHandler

Namespace: `System.Events` — `import System.Events.VoidHandler;`

A small object wrapping a no-argument handler function so it can be stored in an array. Used by `Signal`.

Members: - `public constructor VoidHandler(Action* f)` — wrap the handler `f`. - `public method invoke()` returns `void` — call the wrapped handler.

Signal

Namespace: `System.Events` — `import System.Events.Signal;`

An event that fires with no payload (a notification such as `onSave` / `onClose`). Keeps its handlers in a plain growable array and calls them all on `emit`.

Members: - `public constructor Signal()` — an event with no subscribers. - `public method subscribe(Action* h)` returns `void` — register a handler to run on emit. - `public method emit()` returns `void` — call every subscribed handler. - `public method count()` returns `int` — number of subscribers.

IntHandler

Namespace: `System.Events` — `import System.Events.IntHandler;`

A small object wrapping a handler taking one int argument so it can be stored in an array. Used by `IntEvent`.

Members: - `public constructor IntHandler(Action1<int>* f)` — wrap the handler `f`. - `public method invoke(int arg)` returns `void` — call the wrapped handler with `arg`.

IntEvent

Namespace: `System.Events` — `import System.Events.IntEvent;`

An event that fires with an int payload (such as `onTick(elapsed)` or `onScore(points)`).

Members: - `public constructor IntEvent()` — an event with no subscribers. - `public method subscribe(Action1<int>* h)` returns `void` — register an int-taking handler. - `public method emit(int arg)` returns `void` — call every subscribed handler with `arg`. - `public method count()` returns `int` — number of subscribers.

StringHandler

Namespace: `System.Events` — `import System.Events.StringHandler;`

A small object wrapping a handler taking one `String` argument so it can be stored in an array. Used by `StringEvent`.

Members: - `public constructor StringHandler(Action1<String>* f)` — wrap the handler `f`. - `public method invoke(String arg) returns void` — call the wrapped handler with `arg`.

StringEvent

Namespace: `System.Events` — `import System.Events.StringEvent;`

An event that fires with a `String` payload (such as `onMessage(text)` or `onError(reason)`).

Members: - `public constructor StringEvent()` — an event with no subscribers. - `public method subscribe(Action1<String>* h) returns void` — register a String-taking handler. - `public method emit(String arg) returns void` — call every subscribed handler with `arg`. - `public method count() returns int` — number of subscribers.

Quadtree / Octree

```
import System.Spatial.Quadtree;
import System.Spatial.Octree;
```

The other answer to "what is near this point". `SpatialGrid` above is a uniform grid, which is the right structure for a uniform population and the wrong one for a world where everything is in one town: give it that and it becomes a linear scan of one cell. A tree subdivides where the points are.

Type	Members
Quadtree	<code>Quadtree(centreX, centreY, halfSize)</code> , <code>insert(x, y, id)</code> , <code>size()</code> , and the range and radius queries.
Octree	The same in three dimensions: <code>Octree(centreX, centreY, centreZ, halfSize)</code> , <code>insert(x, y, z, id)</code> , <code>size()</code> .

Nodes are stored as centre and half-extent rather than min/max: it is cheaper to test, and it is what subdivision produces anyway.

`Spatial` beside them holds the growth helpers the trees are built on (`growInts`, `growDoubles`) — public because a structure of your own in the same style wants the same two.

Noise

```
import System.Spatial.Noise;
```

Gradient (Perlin-style) noise, seeded, for terrain, textures and anything that should look natural rather than random.

- `public constructor Noise(int seed)` — a permutation table from the seed; the same seed is the same field, forever, which is what makes a generated world reproducible.
- `public method at(double x, double y) returns double` — one octave, in `[-1, 1]`.

- `public method fractal(double x, double y, int octaves, double persistence)` returns double
— octaves summed with falling amplitude: the usual way to get coastlines out of it.

Standard Library Reference

— Text / Encoding / Crypto

Every type in this slice lives in the `System.Text` namespace. As with the rest of the Polaron stdlib, each type must be imported explicitly by its fully-qualified name before use (importing the namespace does not pull in its members). The import line for each type is shown under its heading.

This is where Polaron does its text work: building strings efficiently (`StringBuilder`), splitting and formatting them (`Strings`), matching and searching (`Regex` , `Kmp` , `Manacher`), walking UTF-8 (`Utf8`), turning bytes into transportable text (`Hex` , `Base64` , `Base32` , `Base58` , `Ascii85`), fingerprinting data (`Sha256` , `Sha1` , `Md5` , the CRC/Adler checksums, `Fletcher`), authenticating it (`Hmac`), and a handful of compression and classic-algorithm utilities (`Huffman` , `Lz77` , `Rle` , `Soundex` , `Calculator`).

The slice is pure Polaron built on top of the built-in `String` / `string` primitives and the raw-memory `System.Memory` builtins — there is no new runtime dependency, and nothing here calls out to a system crypto library, so the hashes are readable reference implementations rather than hardware-accelerated ones. Because `String.charAt` / `String.length` work at the byte level, the encoding, checksum and hash types treat a `String` as a byte buffer (`charAt(i)` & `255` is the *i*-th byte), and several APIs take/return `int[]` where each entry is a 0..255 byte. That byte-level view is exactly why a separate `Utf8` type exists: to recover whole Unicode codepoints from those bytes.

The cryptographic hashes match their published test vectors, so they interoperate with other tools; where an algorithm is broken for security (`Sha1` , `Md5`) the type says so and names the legacy use it is kept for. In the examples below, a `// →` comment shows the value a call returns.

Only public members (constructors, methods, static methods) are listed. Private helpers used internally by each type are omitted.

StringBuilder

```
import System.Text.StringBuilder;
```

Growable text buffer (spec 34.5). Bytes live in a raw heap buffer (`System.Memory`) that doubles on demand, so `append` is amortized O(1); `toString()` copies the accumulated bytes into an owned `String`.

- `public constructor StringBuilder()` — creates an empty buffer (initial capacity 16 bytes).
- `public method append(String s) returns StringBuilder` — appends a string's bytes (bulk memcpy); returns `this` for chaining.
- `public method appendChar(char c) returns StringBuilder` — appends a single byte; returns `this`.
- `public method appendInt(int value) returns StringBuilder` — appends the decimal text of an int (via `value.toString()`); returns `this`.
- `public method length() returns int` — the number of bytes accumulated so far.
- `public method toString() returns String` — copies the buffer into a fresh owned `String`.

Use it instead of `a.concat(b).concat(c) ...` in a loop: repeated `concat` re-copies the whole prefix each time ($O(n^2)$ total), whereas the builder appends in place and copies once at `toString()`. The `append*` methods all return `this`, so calls chain.

```
import System.Text.StringBuilder;

StringBuilder sb = new StringBuilder() on heap;
sb.append("count = ").appendInt(42).appendChar('!');
sb.length();      // → 11
sb.toString();    // → "count = 42!"
```

Strings

```
import System.Text.Strings;
```

`String` utilities the built-in `String` type does not carry as methods (spec 4): splitting, joining, replacing, padding and trimming. Each is a static helper built from the `String` primitives, so it allocates fresh owned `String`s and never mutates its input.

- `public static method split(String text, String separator)` returns `ArrayList<String>` — splits `text` on each non-overlapping `separator`; an empty separator returns the whole text unsplit.
- `public static method join(ArrayList<String> parts, String separator)` returns `String` — concatenates `parts` with `separator` between them ($O(n)$ via `StringBuilder`).
- `public static method replace(String text, String target, String replacement)` returns `String` — replaces every non-overlapping occurrence of `target` with `replacement` (empty target returns `text` unchanged).
- `public static method padLeft(String text, int width, String pad)` returns `String` — left-pads `text` with repetitions of `pad` until it reaches `width` (returns `text` unchanged if already at least `width`).
- `public static method padRight(String text, int width, String pad)` returns `String` — right-pads `text` with repetitions of `pad` until it reaches `width`.
- `public static method format(String template, ArrayList<String> args)` returns `String` — fills each `{}` placeholder in `template` with the next argument in order; extra args are ignored, a `{}` with no argument left is dropped.
- `public static method count(String text, String sub)` returns `int` — counts the non-overlapping occurrences of `sub` in `text`.
- `public static method reverse(String text)` returns `String` — returns `text` with its characters (bytes) reversed.
- `public static method capitalize(String text)` returns `String` — upper-cases the first character and leaves the rest unchanged.
- `public static method trimStart(String text)` returns `String` — removes leading whitespace only (the built-in `String.trim` does both ends).
- `public static method trimEnd(String text)` returns `String` — removes trailing whitespace only.

- `public static method isBlank(String text)` returns `boolean` — whether `text` is empty or only whitespace.
 - `public static method equalsIgnoreCase(String a, String b)` returns `boolean` — case-insensitive equality, comparing the lower-cased forms.
-

Regex

```
import System.Text.Regex;
```

A small backtracking regular-expression matcher (spec 4): literals, `.` (any), character classes `[abc]` / `[a-z]` / `[^...]`, the quantifiers `*` `+` `?`, and the anchors `^` and `$`. Pure Polaron over the `String` primitives.

- `public static method search(String pat, String text)` returns `boolean` — whether `pat` occurs anywhere in `text`; wrap the pattern in `^` and `$` to require a full match.

Matching is backtracking, so a pathological pattern like `(a*)*` has no equivalent here (there is no grouping) but a greedy quantifier over a long run can still be quadratic; keep patterns simple. There are no capture groups — `search` answers only yes/no. For plain substring search prefer `Kmp`, which is linear.

```
import System.Text.Regex;

Regex.search("a+b", "aaab");           // → true
Regex.search("c.t", "cat");             // → true   ('.' matches any one byte)
Regex.search("^[0-9]+$", "2026");       // → true   (anchored: all digits)
Regex.search("^[0-9]+$", "20x6");       // → false
```

Utf8

```
import System.Text.Utf8;
```

UTF-8 decoding (spec 4). Since `String.charAt` / `String.length` are byte-level, `Utf8` reads whole Unicode codepoints out of the byte sequence. A lead byte's high bits give the character width.

- `public static method widthAt(String s, int i)` returns `int` — the number of bytes occupied by the character starting at byte offset `i` (advance by this).
- `public static method codepointAt(String s, int i)` returns `int` — the Unicode codepoint of the character at byte offset `i`.
- `public static method length(String s)` returns `int` — the number of Unicode characters (codepoints), not bytes.
- `public static method codepoints(String s)` returns `ArrayList<int>` — every codepoint in order.

Reach for this whenever "how many characters" or "the character at position `k`" must not count raw bytes. To iterate characters, read `codepointAt(s, i)` and advance `i` by `widthAt(s, i)` rather than by 1. The decoder trusts its input: it assumes well-formed UTF-8 and does not validate continuation bytes.

```
import System.Text.UTF8;

String s = "héllo";           // 'é' takes 2 UTF-8 bytes
s.length();                   // → 6   (bytes, since String is byte-level)
Utf8.length(s);               // → 5   (Unicode characters)
Utf8.codepointAt(s, 1);       // → 233 (U+00E9, 'é')
Utf8.widthAt(s, 1);           // → 2   (advance past the 2-byte character)
```

Scanner

```
import System.Text.Scanner;
```

A cursor over text that hands out tokens (spec 4): reads words, ints and lines from an in-memory `String`, so it parses file contents or any text.

- `public constructor Scanner(String text)` — creates a scanner positioned at the start of `text`.
- `public method hasNext()` returns `boolean` — whether a non-whitespace token remains (skips leading whitespace).
- `public method nextWord()` returns `String` — reads up to the next whitespace and returns the token.
- `public method nextInt()` returns `int` — reads the next word and parses it as an int (`nextWord().toInt()`).
- `public method hasNextLine()` returns `boolean` — whether any input remains to read as a line.
- `public method nextLine()` returns `String` — reads to the next newline (consuming it) and returns the line without the terminator.

Hex

```
import System.Codecs.Hex;
```

Hexadecimal encoding (spec 4): each byte of a string becomes two lowercase hex digits, and back.

- `public static method encode(String data)` returns `String` — the lowercase hex of each byte of `data`.
- `public static method decode(String hex)` returns `String` — the bytes decoded from a hex string (case-insensitive; consumes digit pairs).

Radix

```
import System.Codecs.Radix;
```

Arbitrary-base integer conversion (spec 4), bases 2..36 using `0-9` then `a-z`.

- `public static method toBase(long n, int base) returns String` — renders `n` in the given base (with a leading `-` for negatives).
 - `public static method fromBase(String s, int base) returns long` — parses such a string back to a long (case-insensitive).
-

Base64

```
import System.Codecs.Base64;
```

Base64 encoding (spec 4): three bytes become four characters of the standard alphabet, with `=` padding on the final group.

- `public static method encode(String data) returns String` — Base64 of `data`'s bytes (standard `+ /` alphabet, `=` padded).
- `public static method decode(String data) returns String` — the bytes decoded from a padded Base64 string.

The standard (not URL-safe) alphabet, so output may contain `+` and `/`; `encode` then `decode` round-trips exactly. Encoded text is about 4/3 the size of the input. For URL- or filename-safe output without padding characters, use `Base32`, or `Hex` when readability matters more than size.

```
import System.Codecs.Base64;

Base64.encode("Man");    // → "TWFu"
Base64.encode("Ma");     // → "TWE="    (one '=' pad for the short final group)
Base64.decode("TWFu");   // → "Man"
```

Base32

```
import System.Codecs.Base32;
```

Base32 (RFC 4648) over a string's bytes: 5 bytes encode to 8 chars of the alphabet `ABCDEFGHIJKLMNOPQRSTUVWXYZ234567`, `=` padded. Uses a 40-bit long buffer.

- `public static method encode(String data) returns String` — the padded Base32 of `data`'s bytes.
 - `public static method decode(String s) returns String` — the bytes decoded from a Base32 string (stops at the first `=`).
-

Base58

```
import System.Codecs.Base58;
```

Base58 (the Bitcoin alphabet, spec 4): big-endian base-256 to base-58 with no `0 / O / I / l`; leading zero bytes are preserved as leading `1`s. Works over an `int[]` of bytes rather than a `String`.

- `public static method encode(int[] bytes, int n) returns String` — Base58 of the first `n` entries of `bytes` (each an 0..255 byte).
 - `public static method decode(String s) returns int[]` — the decoded bytes.
-

Ascii85

```
import System.Codecs.Ascii85;
```

Ascii85 / Base85 (spec 4, Adobe variant without delimiters): four bytes become five printable chars (base 85 starting at `!`); a final partial group emits one fewer char than its byte count plus one.

- `public static method encode(int[] bytes, int n) returns String` — Ascii85 of the first `n` entries of `bytes`.
 - `public static method decode(String s) returns int[]` — the decoded bytes (short final groups are padded internally with `u`).
-

Fletcher

```
import System.Codecs.Fletcher;
```

Fletcher-16 checksum (spec 4) over a string's bytes: two running mod-255 sums combined as a 16-bit value. Cheaper than CRC with good error detection.

- `public static method fletcher16(String data) returns int` — the 16-bit Fletcher checksum of `data`'s bytes.
-

Digest

```
import System.Security.Digest;
```

Non-cryptographic checksums and hashes over a string's bytes (spec 4), using unsigned 32-bit arithmetic that wraps and shifts logically. Each result is returned as an `int` (the same 32 bits reinterpreted).

- `public static method crc32(String data) returns int` — CRC-32 (reflected, the zip/png polynomial) of `data`'s bytes.
 - `public static method fnv1a(String data) returns int` — 32-bit FNV-1a hash of `data`'s bytes.
 - `public static method adler32(String data) returns int` — Adler-32 (the zlib checksum) of `data`'s bytes.
-

Sha256

```
import System.Security.Sha256;
```

SHA-256 cryptographic hash (FIPS 180-4), pure Polaron over 32-bit unsigned arithmetic (spec 4). The method is named `digest`, not `hash`, since `hash` is the `Hashable` interface method. Also provides the shared byte/hex plumbing (`toHex` , `putWord`) reused by the other hash types.

- `public static method toHex(int[] bytes, int n)` returns `String` — the lowercase hex of the first `n` entries of `bytes` (each treated as a 0..255 byte).
- `public static method digestRaw(int[] data, int len)` returns `int[]` — hashes the first `len` bytes of `data`, returning the 32 raw output bytes (used by `Hmac`).
- `public static method digest(String msg)` returns `String` — the 64-character lowercase hex digest of `msg` 's bytes.
- `public static method digestBytes(int[] data, int len)` returns `String` — the lowercase hex digest of the first `len` bytes of `data` .

`digest` is the everyday entry point: pass a message, get its 64-character hex digest. The output matches the FIPS 180-4 test vectors, so it interoperates with other SHA-256 tools. For keyed authentication (proving a message came from someone holding a shared secret) use `Hmac.sha256` rather than hashing the key and message concatenated.

```
import System.Security.Sha256;

Sha256.digest(""); // → "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
Sha256.digest("abc"); // → "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad68ce0d"

```

Hmac

```
import System.Security.Hmac;
```

HMAC-SHA256 keyed message authentication (RFC 2104), pure Polaron on top of `Sha256` . Keys longer than the 64-byte block are hashed down first.

- `public static method sha256(String key, String msg)` returns `String` — the lowercase hex of HMAC-SHA256(`key` , `msg`).

Sha1

```
import System.Security.Sha1;
```

SHA-1 cryptographic hash (FIPS 180-1), pure Polaron over 32-bit unsigned arithmetic. SHA-1 is broken for collision resistance; provided for legacy interop such as Git object ids. Reuses `Sha256.putWord` / `toHex` .

- `public static method digestRaw(int[] data, int len) returns int[]` — hashes the first `len` bytes of `data`, returning the 20 raw output bytes.
 - `public static method digest(String msg) returns String` — the 40-character lowercase hex digest of `msg`'s bytes.
-

Sha224

```
import System.Security.Sha224;
```

SHA-224 (FIPS 180-4): the SHA-256 compression with different initial hash values and a 28-byte (56 hex char) output. Reuses `Sha256.rotr / putWord / toHex`; the round constants match SHA-256.

- `public static method digest(String msg) returns String` — the 56-character lowercase hex digest of `msg`'s bytes.
-

Md5

```
import System.Security.Md5;
```

MD5 message digest (RFC 1321), pure Polaron over 32-bit unsigned arithmetic (little-endian, unlike the SHA family). MD5 is broken for collision resistance; provided for legacy interop/checksums only. Reuses `Sha256.toHex` for the final hex.

- `public static method digest(String msg) returns String` — the 32-character lowercase hex digest of `msg`'s bytes.
-

Crc

```
import System.Codecs.Crc;
```

CRC-16/XMODEM (spec 4): a 16-bit checksum over a string's bytes with polynomial `0x1021` and zero initial value. Bitwise, table-free.

- `public static method crc16(String data) returns int` — the CRC-16/XMODEM checksum of `data`'s bytes.
-

Adler32

```
import System.Codecs.Adler32;
```

Adler-32 checksum (RFC 1950 / zlib): two running sums modulo 65521 combined as `(b << 16) | a`, with `a` starting at 1. Faster than CRC but weaker; returned in a `long` so the high bit stays positive.

- `public static method checksum(String data)` returns `long` — the Adler-32 checksum of `data`'s bytes.
-

BloomFilter

```
import System.Collections.BloomFilter;
```

A probabilistic set that never misses a member but may report a false positive (spec 34.1). Two independent hashes (`Digest.fnv1a` and `Digest.crc32`) set and test bits in a fixed bit array. `mightContain` returning false is definitive; true means probably present.

- `public constructor BloomFilter(int size)` — creates a filter backed by a `size` -bit array.
 - `public method add(String key)` returns `void` — records `key` in the filter.
 - `public method mightContain(String key)` returns `boolean` — false means definitely absent; true means probably present.
-

Huffman

```
import System.Codecs.Huffman;
```

Huffman coding (spec 34.1): builds an optimal prefix code from a string's byte frequencies, stored in flat arena arrays (no self-referential pointers). Construct once over the source text, then encode/decode against that codebook. Ties are broken by lowest id, so a round-trip reproduces the input exactly.

- `public constructor Huffman(String data)` — builds the codebook from the byte frequencies of `data`.
 - `public method codeOf(int byteValue)` returns `String` — the `0 / 1` code assigned to a byte value (empty for bytes not in the source).
 - `public method encode(String data)` returns `String` — turns `data` into its concatenated `0 / 1` bit string.
 - `public method decode(String bits)` returns `String` — walks the tree over a `0 / 1` bit string to recover the original text.
-

Lz77

```
import System.Codecs.Lz77;
```

LZ77 sliding-window compression (spec 34.1): `encode` produces flattened (`offset`, `length`, `nextChar`) triples over a bounded search window (`nextChar` is -1 only at the very end); `decode` replays them, copying back-references (which may overlap, like run-length) to reconstruct the input exactly.

- `public constructor Lz77(int windowSize)` — creates a compressor with the given search-window size.

- `public method encode(String data) returns ArrayList<int>` — the flattened `(offset, length, nextChar)` triples.
 - `public method decode(ArrayList<int> tokens) returns String` — reconstructs the original text from the triples.
-

Soundex

```
import System.Text.Soundex;
```

Soundex phonetic encoding (spec 34): maps a name to a letter followed by three digits so that similar-sounding names share a code (e.g. Robert and Rupert both give R163). Vowels reset run detection; `h` and `w` are transparent between equal-coded consonants.

- `public static method encode(String name) returns String` — the 4-character Soundex code of `name` (empty string for an empty name).
-

Kmp

```
import System.Text.Kmp;
```

Knuth-Morris-Pratt substring search (spec 34.1): a failure table (longest proper prefix that is also a suffix) lets the scan never re-read text.

- `public static method indexOf(String text, String pattern) returns int` — the index of the first occurrence of `pattern` in `text`, or -1 (0 for an empty pattern).
 - `public static method count(String text, String pattern) returns int` — the number of occurrences, counting overlaps.
-

Manacher

```
import System.Text.Manacher;
```

Manacher's algorithm (spec 34.1): the length of the longest palindromic substring in linear time, over a transformed array with sentinels so odd and even palindromes are handled uniformly.

- `public static method longestPalindrome(String s) returns int` — the length of the longest palindromic substring of `s`.
-

Rle

```
import System.Codecs.Rle;
```

Run-length encoding of a string (spec 34): each run of a repeated character becomes that character followed by its decimal count, e.g. "aaabbbbc" -> "a3b4c1".

- `public static method encode(String s) returns String` — run-length encodes `s` (character then decimal run count).
 - `public static method decode(String s) returns String` — reverses the encoding, reading a character and the digits that follow as a repeat count.
-

Calculator

```
import System.Text.Calculator;
```

Evaluates an integer arithmetic expression (spec 34): `+` `-` `*` `/` with the usual precedence and parentheses, by recursive descent. Spaces are ignored; division truncates toward zero.

- `public constructor Calculator(String expr)` — creates an evaluator over the expression string `expr`.
 - `public method evaluate() returns int` — parses and evaluates the expression, returning the integer result.
-

Sha384 / Sha512

```
import System.Security.Sha384;  
import System.Security.Sha512;
```

The other two members of the SHA-2 family, with the same shape as `Sha256`: a 48- and a 64-byte digest over the 64-bit compression function. They are here because certificates ask for them by name — a chain signed with SHA-384 cannot be verified with anything else.

- `public static method ofString(String s) returns int[]` — the digest of `s`'s bytes.
 - `public static method ofBytes(int[] data, int count) returns int[]` — the digest of `count` bytes.
 - `public static method hex(int[] digest) returns String` — the usual lower-case rendering.
-

Deflate / Inflate / Gzip

```
import System.Compress.Gzip;
```

Real DEFLATE (RFC 1951) and the gzip container (RFC 1952), written in Polaron — the compression a `.gz` file, an HTTP `Content-Encoding: gzip` body and a PNG's pixel data are all made of. `Zlib` above is the third container over the same codec.

Type	Members
Deflate	<code>bytes(int[] data, int len)</code> returns <code>int[]</code> , <code>text(String s)</code> returns <code>int[]</code> — compressed, raw stream.
Inflate	<code>bytes(int[] data, int len)</code> returns <code>int[]</code> , <code>text(int[] data, int len)</code> returns <code>String</code> — back again.
Gzip	<code>compress(int[] data, int len)</code> , <code>compressText(String s)</code> , <code>decompress(int[] data, int len)</code> — the same with a gzip header, checksum and length.
Crc32	<code>ofBytes(int[] data, int count)</code> returns <code>long</code> , <code>ofString(String s)</code> returns <code>long</code> — the checksum gzip and PNG both carry.

`ByteArray` and `ByteSink`

The two helpers the codecs are written against, useful anywhere bytes are being built:

- `ByteArray.of(String s)` returns `int[]` — a string's bytes; `ByteArray.text(int[] data, int count)` turns them back.
- `ByteSink` — a growable byte buffer: `add(b)`, `size()`, `at(i)`, `toArray()`, and `repeatFrom(distance, count)`, which is the back-reference copy every LZ-family decoder needs and the one place an off-by-one silently produces plausible garbage.

Stdlib Reference — Parsing, Text Utilities, Time & JSON

This reference documents three neighbouring slices of the Polaron standard library: the text and parsing utilities under `System.Text`, the clocks and calendars under `System.Time`, and the JSON document model under `System.Json`. Together they cover the everyday "read a string, make sense of it, and stamp it with a time" work that most programs need before they get to their real logic.

Everything here lives in the Polaron prelude embedded in the compiler (`src/cli/main.cpp`, the `kPreludeSource` raw string) and is written in **pure Polaron** on top of the `System.Collections` and `System.Text` builtins. There is no hidden native magic: a `Csv` parser is an ordinary Polaron class walking a `String`, a `Json` tree is a chain of heap objects, and `Instant.now()` bottoms out in a single clock builtin. That means these classes are also worked examples of idiomatic Polaron — every one obeys the same rules your own code does (mandatory `this.`, explicit visibility, braces on every block).

A few themes run through the group:

- **System.Text** is the largest namespace: tokenizers and expression evaluators (`Rpn`, `ShuntingYard`), config and data formats (`Csv`, `Ini`, `Properties`, `QueryString`), string metrics for fuzzy matching (`Levenshtein`, `JaroWinkler`), encoders and codecs (`UrlCodec`, `VarInt`, `BitWriter` / `BitReader`), validators and checksums (`Luhn`, `Isbn`, `Validators`, `Uuid`, `Semver`), and small classics like `Roman`, `Caesar`, and `Slugify`. Most are stateless utilities exposed as `static` methods; a handful (`Ini`, `Properties`, `StateMachine`, `BitWriter`) are objects you construct and then query.
- **System.Time** models time in layers: a `Duration` is a span of milliseconds, an `Instant` is a point on the wall clock (epoch millis), and a `ZonedDateTime` is an `Instant` viewed through a fixed `ZoneOffset`. `Date` / `Calendar` handle the pure civil-calendar arithmetic, and `Stopwatch` measures elapsed time off the monotonic clock.
- **System.Json** is a tiny in-memory JSON model: `Json` is a tagged value (null / bool / number / string / array / object) you build up node by node or parse from text, then serialize back — with `JsonPointer` for RFC 6901 path lookups.

Every stdlib type requires an **explicit import** of its fully-qualified name, one class per line:

```
import System.Text.CaseConvert;
import System.Time.Instant;
import System.Json.Json;
```

Signatures below are verbatim from the prelude. Private helper methods are omitted (only public constructors, methods, and static methods are listed). Types are grouped by namespace: `System.Text` (text/parsing utilities), `System.Time` (dates and clocks) and `System.Json`.

Namespace `System.Text`

The text namespace is a grab-bag of string-shaped tools. Most are pure `static` helpers you call without allocating anything (`CaseConvert.toSnake`, `Roman.toRoman`, `Luhn.isValid`); a few are small objects that

hold parsed state (`Ini` , `Properties` , `StateMachine`) or a growing buffer (`BitWriter` , `BitReader`). Everything operates on the builtin `String` / `char` types and the `System.Collections` containers, so results come back as ordinary `String` s, `ArrayList<String>` s, or maps you can feed straight into the rest of your program.

Rpn

- Namespace: `System.Text` · Import: `import System.Text.Rpn;`
- Reverse Polish (postfix) evaluator: tokens are space-separated integers and the operators `+` `-` `*` `/` `%` , evaluated with an operand stack. Complements the infix `Calculator` / `ShuntingYard` .

Public members:

- `public static method eval(String expr) returns int` — evaluate a space-separated postfix expression and return the result on top of the stack.

ShuntingYard

- Namespace: `System.Text` · Import: `import System.Text.ShuntingYard;`
- Converts a space-separated infix expression (integers, `+` `-` `*` `/` `%` , and parentheses) to Reverse Polish, ready for `Rpn.eval` . Higher-precedence operators pop first.

Public members:

- `public static method toRpn(String infix) returns String` — convert a space-separated infix expression to a space-separated RPN string.

Template

- Namespace: `System.Text` · Import: `import System.Text.Template;`
- Fills `{name}` placeholders in a template from a map. An unknown key is left as-is, so the braces survive when there is no matching value.

Public members:

- `public static method render(String tpl, HashMap<String, String> vars) returns String` — substitute each `{key}` in `tpl` with `vars[key]` , leaving unmatched placeholders untouched.

Csv

- Namespace: `System.Text` · Import: `import System.Text.Csv;`
- Parses one line of comma-separated values into fields, honoring double-quoted fields so a comma inside quotes does not split. Quote characters are consumed, not kept. Reach for it when you are reading a single CSV record; pair it with `CsvWriter` / `CsvReader` when you need the stricter RFC 4180 escaping (doubled quotes).

```
import System.Text.Csv;
import System.Collections.ArrayList;

// The comma inside the quotes stays in one field.
ArrayList<String> fields = Csv.parse("a,\"b,c\",d");
System.IO.Console.printf("count=%d second=%s\n", fields.size(), fields.get(1));
// count=3 second=b,c
```

Public members:

- `public static method parse(String line)` returns `ArrayList<String>` — split one CSV line into its fields, respecting double quotes.

CsvWriter

- Namespace: `System.Text` · Import: `import System.Text.CsvWriter;`
- CSV row writer (RFC 4180): joins cells with commas, quoting any cell that contains a comma, quote, or newline and doubling embedded quotes. Complements the `Csv` parser.

Public members:

- `public static method buildRow(String[] cells, int n)` returns `String` — build an RFC 4180 CSV row from the first `n` cells, quoting and escaping where needed.

CsvReader

- Namespace: `System.Text` · Import: `import System.Text.CsvReader;`
- Parse one RFC 4180 CSV record, the inverse of `CsvWriter`: quoted fields may contain commas and escaped quotes (a doubled `"` becomes a single `"`). Returns the fields in order.

Public members:

- `public static method parseLine(String line)` returns `ArrayList<String>` — parse one RFC 4180 record, unescaping doubled quotes inside quoted fields.

CaseConvert

- Namespace: `System.Text` · Import: `import System.Text.CaseConvert;`
- Identifier case conversion between camelCase and snake_case/kebab-case. `toSnake` / `toKebab` insert a separator before each interior uppercase letter and lowercase it; `toCamel` uppercases the letter after each separator.

Public members:

- `public static method toSnake(String s)` returns `String` — convert to snake_case (`_` before each interior uppercase letter, lowercased).
- `public static method toKebab(String s)` returns `String` — convert to kebab-case (`-` before each interior uppercase letter, lowercased).
- `public static method toCamel(String s)` returns `String` — convert snake_case/kebab-case to camelCase by uppercasing the character after each `_` or `-`.

TextDistance

- Namespace: `System.Text` · Import: `import System.Text.TextDistance;`
- String similarity: the Levenshtein edit distance is the fewest single-character inserts, deletes, or substitutions to turn one string into another. Computed with two rolling DP rows.

Public members:

- `public static method levenshtein(String a, String b) returns int` — the Levenshtein edit distance between `a` and `b`.

UrlCodec

- Namespace: `System.Text` · Import: `import System.Codecs.UrlCodec;`
- Percent-encoding for URLs: unreserved characters (letters, digits, `-` `_` `.` `~`) pass through, everything else becomes `%XX` of its byte. `decode` reverses `%XX` escapes.

Public members:

- `public static method encode(String s) returns String` — percent-encode `s`, leaving unreserved characters intact.
- `public static method decode(String s) returns String` — decode `%XX` escapes back to their bytes.

QueryString

- Namespace: `System.Text` · Import: `import System.Codecs.QueryString;`
- URL query-string parsing: splits `"k=v&k=v"` on `&` into a map, with each key/value percent-decoded via `UrlCodec`. A key with no `=` maps to the empty string.

Public members:

- `public static method parse(String qs) returns HashMap<String, String>` — parse an `&`-separated query string into a decoded key→value map.

WordWrap

- Namespace: `System.Text` · Import: `import System.Text.WordWrap;`
- Greedy word wrapping: splits text on spaces and packs words into lines no longer than the given width, returning the lines. A single word longer than the width gets its own line.

Public members:

- `public static method wrap(String text, int width) returns ArrayList<String>` — greedily wrap `text` into lines of at most `width` characters.

Chars

- Namespace: `System.Text` · Import: `import System.Text.Chars;`

- Character classification and case conversion: the usual is-digit/letter/whitespace tests, ASCII upper/lower conversion, and `digitValue` (-1 if not a digit).

Public members:

- `public static method isDigit(char c) returns boolean` — true if `c` is `'0' .. '9'`.
- `public static method isLetter(char c) returns boolean` — true if `c` is an ASCII letter.
- `public static method isWhitespace(char c) returns boolean` — true for space, tab, newline, or carriage return.
- `public static method isUpper(char c) returns boolean` — true if `c` is `'A' .. 'Z'`.
- `public static method isLower(char c) returns boolean` — true if `c` is `'a' .. 'z'`.
- `public static method toUpper(char c) returns char` — uppercase an ASCII letter, else return `c` unchanged.
- `public static method toLower(char c) returns char` — lowercase an ASCII letter, else return `c` unchanged.
- `public static method digitValue(char c) returns int` — numeric value of a digit character, or -1 if not a digit.

Roman

- Namespace: `System.Text` · Import: `import System.Text.Roman;`
- Roman numerals: `toRoman` writes a positive integer greedily from the largest symbol down; `fromRoman` reads one back, subtracting a symbol whose value is less than the one after it.

Public members:

- `public static method toRoman(int n) returns String` — format a positive integer as a Roman numeral.
- `public static method fromRoman(String s) returns int` — parse a Roman numeral into its integer value.

Ini

- Namespace: `System.Text` · Import: `import System.Text.Ini;`
- A minimal INI / config parser: reads `[section]` headers and `key=value` lines (; and # start comments; surrounding whitespace is trimmed) into a flat `"section.key" → value` map. `get` returns the value or `""` when absent; `has` reports presence.

Public members:

- `public constructor Ini(String text)` — parse INI text into an internal `"section.key" → value` map.
- `public method get(String section, String key) returns String` — the value for `section / key`, or `""` if absent.
- `public method has(String section, String key) returns boolean` — whether `section / key` is present.

Properties

- Namespace: `System.Text` · Import: `import System.Text.Properties;`
- Java-style `.properties` parsing: flat `key=value` lines (space around `=` trimmed), with `#` and `!` comment lines skipped. Typed getters fall back to a default when a key is missing or malformed.

Public members:

- `public constructor Properties(String text)` — parse `.properties` text into a key→value map.
- `public method has(String key)` returns `boolean` — whether `key` is present.
- `public method getString(String key, String def)` returns `String` — the string value, or `def` if the key is absent.
- `public method getInt(String key, int def)` returns `int` — the value parsed as an int, or `def` if absent or malformed.
- `public method getBool(String key, boolean def)` returns `boolean` — true when the value equals `"true"`, `def` if the key is absent.

Uuid

- Namespace: `System.Text` · Import: `import System.Text.Uuid;`
- RFC 4122 UUIDs: format 16 bytes as the canonical 8-4-4-4-12 hex string, build a version-4 UUID from 16 random bytes (setting the version and variant bits), or generate one deterministically from an int seed via an inline xorshift. `isValid` checks the canonical shape. Use `v4` with bytes from `System.Security.SecureRandom` when you need real unpredictability; `v4Seeded` is for reproducible ids in tests and fixtures.

```
import System.Text.Uuid;

String id = Uuid.v4Seeded(12345);    // same seed → same UUID every run
System.IO.Console.WriteLine(id);
System.IO.Console.WriteLine(Uuid.isValid(id));    // true
```

Public members:

- `public static method format(int[] bytes)` returns `String` — format 16 bytes as the canonical 8-4-4-4-12 hex string.
- `public static method v4(int[] randomBytes)` returns `String` — build a version-4 UUID from 16 random bytes, setting the version/variant bits.
- `public static method v4Seeded(int seed)` returns `String` — generate a deterministic version-4 UUID from an int seed via xorshift.
- `public static method isValid(String s)` returns `boolean` — check that `s` has the canonical UUID shape (36 chars, dashes and hex in the right places).

Semver

- Namespace: `System.Text` · Import: `import System.Text.Semver;`

- Semantic versioning (semver.org): parse `"major.minor.patch"` (an optional leading `v` and any `-prerelease` / `+build` suffix are ignored) and compare two versions field by field. `compareTo` returns -1, 0, or 1.

Public members:

- `public constructor Semver(String v)` — parse a version string into major/minor/patch.
- `public method getMajor() returns int` — the major component.
- `public method getMinor() returns int` — the minor component.
- `public method getPatch() returns int` — the patch component.
- `public method compareTo(Semver o) returns int` — compare against `o` field by field, returning -1, 0, or 1.
- `public method toString() returns String` — render as `"major.minor.patch"`.

StateMachine

- Namespace: `System.Text` · Import: `import System.Text.StateMachine;`
- A simple deterministic finite state machine: register `(from, event) → to` transitions and drive them with `fire(event)`, which advances the current state and returns whether a transition existed. Transitions live in a `"state|event" → next` map.

Public members:

- `public constructor StateMachine(String initial)` — create a machine starting in state `initial`.
- `public method addTransition(String from, String event, String to) returns void` — register a `(from, event) → to` transition.
- `public method fire(String event) returns boolean` — advance on `event`; return true if a transition existed (and was taken).
- `public method state() returns String` — the current state.

Glob

- Namespace: `System.Text` · Import: `import System.Text.Glob;`
- Glob / wildcard matching: `*` matches any run of characters (including none) and `?` matches exactly one. Iterative with backtracking on the last `*`, so it runs in linear space.

Public members:

- `public static method matches(String pattern, String text) returns boolean` — whether `text` matches the glob `pattern`.

VarInt

- Namespace: `System.Text` · Import: `import System.Codecs.VarInt;`
- LEB128 variable-length integers: seven bits per byte, high bit set while more bytes follow, so small values take one byte. `encode` returns the bytes; `decode` reads them back.

Public members:

- `public static method encode(long value)` returns `ArrayList<int>` — LEB128-encode `value` into a list of bytes.
- `public static method decode(ArrayList<int> bytes)` returns `long` — decode a LEB128 byte sequence back to a long.

BitWriter

- Namespace: `System.Text` · Import: `import System.Codecs.BitWriter;`
- A most-significant-bit-first bit writer: pack individual bits or fixed-width fields into a byte buffer, then read them back with `BitReader`. Useful for entropy coders such as Huffman.

Public members:

- `public constructor BitWriter()` — create an empty bit buffer.
- `public method writeBit(int bit)` returns `void` — append a single bit (low bit of `bit`), MSB-first.
- `public method writeBits(int value, int count)` returns `void` — append the low `count` bits of `value`, most-significant first.
- `public method bitCount()` returns `int` — the number of bits written so far.
- `public method toBytes()` returns `int[]` — the packed bytes (ceil of bit count / 8 entries).

BitReader

- Namespace: `System.Text` · Import: `import System.Codecs.BitReader;`
- Reads bits most-significant-first out of a byte array, matching `BitWriter`.

Public members:

- `public constructor BitReader(int[] bytes)` — wrap a byte array for MSB-first bit reading.
- `public method readBit()` returns `int` — read the next single bit.
- `public method readBits(int count)` returns `int` — read the next `count` bits, most-significant first, into an int.

Colors

- Namespace: `System.Text` · Import: `import System.Text.Colors;`
- 24-bit RGB color utilities: pack/unpack channels into a `0xRRGGBB` int, hex parse/format, linear interpolation between two colors (`t` is a 0..100 percent), Rec.601 luminance and grayscale.

Public members:

- `public static method pack(int r, int g, int b)` returns `int` — pack `r / g / b` (each masked to 8 bits) into a `0xRRGGBB` int.
- `public static method red(int c)` returns `int` — the red channel of a packed color.
- `public static method green(int c)` returns `int` — the green channel of a packed color.
- `public static method blue(int c)` returns `int` — the blue channel of a packed color.
- `public static method toHex(int c)` returns `String` — format a packed color as `#rrggbb`.

- `public static method fromHex(String s) returns int` — parse a hex color (optional leading `#`) into a packed int.
 - `public static method lerp(int c1, int c2, int t) returns int` — linearly interpolate between `c1` and `c2` where `t` is a 0..100 percentage.
 - `public static method luminance(int c) returns int` — Rec.601 luminance (0..255) of a packed color.
 - `public static method grayscale(int c) returns int` — the packed grayscale color at the luminance of `c`.
-

Caesar

- Namespace: `System.Text` · Import: `import System.Security.Caesar;`
- Caesar shift cipher: rotate each letter by `n`, wrapping within its case; non-letters pass through. `decrypt` is the inverse shift; `rot13` is the classic shift of 13.

Public members:

- `public static method encrypt(String s, int n) returns String` — shift each letter forward by `n`, wrapping within its case.
 - `public static method decrypt(String s, int n) returns String` — inverse of `encrypt` for shift `n`.
 - `public static method rot13(String s) returns String` — the classic ROT13 (shift of 13).
-

Vigenere

- Namespace: `System.Text` · Import: `import System.Security.Vigenere;`
- Vigenère cipher: a repeating-key poly-alphabetic shift over letters (case preserved, non-letters skipped and not consuming key). `encrypt` and `decrypt` are inverses.

Public members:

- `public static method encrypt(String s, String key) returns String` — Vigenère-encrypt `s` with the repeating `key`.
 - `public static method decrypt(String s, String key) returns String` — Vigenère-decrypt `s` with the repeating `key`.
-

Slugify

- Namespace: `System.Text` · Import: `import System.Text.Slugify;`
- URL/filename slugs: lowercase, collapse every run of non-alphanumeric characters to a single dash, and trim leading/trailing dashes.

Public members:

- `public static method make(String s) returns String` — turn `s` into a lowercase, dash-separated slug.

Inflector

- Namespace: `System.Text` · Import: `import System.Text.Inflector;`
- English pluralization, simple rules: `-s` / `-x` / `-z` / `-ch` / `-sh` take `"es"`, a consonant+ `y` becomes `"ies"`, otherwise append `"s"`.

Public members:

- `public static method pluralize(String w) returns String` — pluralize an English word by the simple suffix rules.

Levenshtein

- Namespace: `System.Text` · Import: `import System.Text.Levenshtein;`
- Levenshtein edit distance: the minimum single-character insertions, deletions, and substitutions to turn one string into another, via two-row dynamic programming. Good for fuzzy matching and spell-checking.

Public members:

- `public static method distance(String a, String b) returns int` — the Levenshtein edit distance between `a` and `b`.

JaroWinkler

- Namespace: `System.Text` · Import: `import System.Text.JaroWinkler;`
- Jaro-Winkler string similarity in `[0,1]`: the Jaro score adjusted upward for a common prefix (up to 4 chars, factor 0.1). Good for fuzzy matching short strings like names.

Public members:

- `public static method jaro(String s1, String s2) returns double` — the base Jaro similarity in `[0,1]`.
- `public static method similarity(String s1, String s2) returns double` — the Jaro-Winkler similarity (Jaro boosted by a common-prefix bonus).

Luhn

- Namespace: `System.Text` · Import: `import System.Text.Luhn;`
- Luhn (mod-10) checksum, as used by credit-card numbers: doubles every second digit from the right (subtracting 9 when over 9); valid when the total is a multiple of 10. Non-digits are skipped.

Public members:

- `public static method isValid(String s) returns boolean` — whether the digits in `s` pass the Luhn mod-10 check.

Isbn

- Namespace: `System.Text` · Import: `import System.Text.Isbn;`

- ISBN-13 check digit: the 13 digits weighted 1,3,1,3,... must sum to a multiple of 10.

Public members:

- `public static method isValid13(String s) returns boolean` — whether `s` contains exactly 13 digits passing the ISBN-13 checksum.

TextDiff

- Namespace: `System.Text` · Import: `import System.Text.TextDiff;`
- Line-level text diff via the longest common subsequence: `common` counts shared lines in order; `removed` and `added` are the lines only in the first or second version. Uses two-row LCS DP.

Public members:

- `public static method common(String[] a, int na, String[] b, int nb) returns int` — number of lines common to both versions (LCS length).
- `public static method removed(String[] a, int na, String[] b, int nb) returns int` — number of lines only in the first version (`na` minus the LCS length).
- `public static method added(String[] a, int na, String[] b, int nb) returns int` — number of lines only in the second version (`nb` minus the LCS length).

Validators

- Namespace: `System.Text` · Import: `import System.Text.Validators;`
- Lightweight format validators: a heuristic email check (single `@`, dot in the domain, no spaces), an http/https URL check with a non-empty host, and IBAN via the ISO 7064 mod-97 test.

Public members:

- `public static method isEmail(String s) returns boolean` — heuristic email check (single `@`, no spaces, a dot after the `@`).
- `public static method isUrl(String s) returns boolean` — check for an `http://` / `https://` prefix with a non-empty host.
- `public static method isIban(String s) returns boolean` — validate an IBAN via the ISO 7064 mod-97 test (spaces ignored).

NumberWords

- Namespace: `System.Text` · Import: `import System.Text.NumberWords;`
- Spell an integer in English words, space-separated and lowercase, up to the billions (covers the full int range). Negative numbers are prefixed with `"minus"`.

Public members:

- `public static method toWords(int num) returns String` — spell `num` in lowercase English words.

Humanize

- Namespace: `System.Text` · Import: `import System.Text.Humanize;`
- Human-friendly formatting: binary byte sizes (1024-based, one decimal above bytes) and English ordinals (1st, 2nd, 3rd, 11th, 21st), handling the 11-13 `"th"` exception.

Public members:

- `public static method bytes(long n) returns String` — format a byte count as a human-readable binary size (e.g. `"1.5 MB"`).
- `public static method ordinal(int n) returns String` — format `n` as an English ordinal (`"1st"` , `"2nd"` , `"11th"` , ...).

Namespace `System.Time`

Time in Polaron is built up in small, composable pieces rather than one do-everything date-time class. A `Duration` is a length of time; an `Instant` is a point in time (epoch milliseconds); a `ZoneOffset` shifts an instant into local wall-clock fields via `ZonedDateTime`. For pure calendar math with no clock involved, `Date` and `Calendar` give you leap-year rules, month lengths, and day-of-week. `Stopwatch` is the odd one out: it reads the monotonic clock, not the wall clock, so it stays correct even if the system time is changed while it runs. All of these read the underlying clock through the `Time` builtin, which shares this namespace.

Duration

- Namespace: `System.Time` · Import: `import System.Time.Duration;`
- A span of time in milliseconds. Shares the namespace with the `Time` builtin, so `Instant.now()` can read the clock without an extra import.

Public members:

- `public constructor Duration(long millis)` — a duration of `millis` milliseconds.
- `public static method ofMillis(long m) returns Duration` — construct from milliseconds.
- `public static method ofSeconds(long s) returns Duration` — construct from seconds.
- `public static method ofMinutes(long m) returns Duration` — construct from minutes.
- `public method toMillis() returns long` — the duration in milliseconds.
- `public method toSeconds() returns long` — the duration in whole seconds.
- `public method plus(Duration other) returns Duration` — a new duration summing this and `other`.
- `public method minus(Duration other) returns Duration` — a new duration subtracting `other` from this.

Instant

- Namespace: `System.Time` · Import: `import System.Time.Instant;`

- A moment on the wall clock, as milliseconds since the Unix epoch. Subtract two instants with `since` to get a `Duration`, or push an instant forward with `plus`. Because an `Instant` is a bare epoch value it carries no time zone; wrap it in a `ZonedDateTime` when you need local calendar fields.

```
import System.Time.Instant;
import System.Time.Duration;

Instant start = Instant.now();
// ... do some work ...
Instant end = Instant.now();
Duration elapsed = end.since(start);
System.IO.Console.printf("took %d ms\n", elapsed.toMillis());

Instant deadline = start.plus(Duration.ofSeconds(30)); // 30s after start
```

Public members:

- `public constructor Instant(long ms)` — an instant at `ms` epoch milliseconds.
- `public static method now()` returns `Instant` — the current instant from the system clock.
- `public static method ofEpochMillis(long ms)` returns `Instant` — construct from epoch milliseconds.
- `public method toEpochMillis()` returns `long` — epoch milliseconds of this instant.
- `public method isBefore(Instant other)` returns `boolean` — whether this instant precedes `other`.
- `public method isAfter(Instant other)` returns `boolean` — whether this instant follows `other`.
- `public method plus(Duration d)` returns `Instant` — a new instant `d` later than this one.
- `public method since(Instant earlier)` returns `Duration` — the duration from `earlier` to this instant.

ZoneOffset

- Namespace: `System.Time` · Import: `import System.Time.ZoneOffset;`
- A fixed offset from UTC, in seconds east of Greenwich (e.g. -3h for BRT). A pure offset, not a named zone with a DST rule table; `systemDefault()` reads the machine's current offset (including any active daylight saving) from the OS.

Public members:

- `public constructor ZoneOffset(int totalSeconds)` — an offset of `totalSeconds` seconds east of UTC.
- `public static method ofSeconds(int s)` returns `ZoneOffset` — construct from seconds.
- `public static method ofHours(int h)` returns `ZoneOffset` — construct from whole hours.
- `public static method ofHoursMinutes(int h, int m)` returns `ZoneOffset` — construct from hours and minutes (minutes take the sign of the hours).
- `public static method utc()` returns `ZoneOffset` — the zero (UTC) offset.
- `public static method systemDefault()` returns `ZoneOffset` — the machine's current UTC offset, read from the OS.
- `public method totalSeconds()` returns `int` — the offset in seconds east of UTC.

- `public method id() returns String` — the ISO id: "Z" for UTC, otherwise "+HH:MM" / "-HH:MM".

ZonedDateTime

- Namespace: `System.Time` · Import: `import System.Time.ZonedDateTime;`
- A date-time at a fixed UTC offset: an `Instant` paired with a `ZoneOffset`. The wall-clock fields are the instant shifted by the offset; `toInstant` recovers the underlying UTC point.

Public members:

- `public constructor ZonedDateTime(Instant instant, ZoneOffset offset)` — pair an instant with a fixed offset.
- `public static method now() returns ZonedDateTime` — the current instant at the system default offset.
- `public static method ofInstant(Instant i, ZoneOffset off) returns ZonedDateTime` — construct from an instant and an offset.
- `public method toInstant() returns Instant` — the underlying UTC instant.
- `public method offset() returns ZoneOffset` — the fixed UTC offset.
- `public method year() returns int` — the local calendar year.
- `public method month() returns int` — the local month (1-12).
- `public method day() returns int` — the local day of month.
- `public method hour() returns int` — the local hour (0-23).
- `public method minute() returns int` — the local minute (0-59).
- `public method second() returns int` — the local second (0-59).
- `public method toString() returns String` — the ISO-8601 rendering, e.g. `2026-07-03T14:05:09-03:00`.

Stopwatch

- Namespace: `System.Time` · Import: `import System.Time.Stopwatch;`
- A monotonic elapsed-time timer: start/stop/reset accumulate high-resolution nanoseconds from the monotonic clock (`Time.nanos`), unaffected by wall-clock changes.

Public members:

- `public constructor Stopwatch()` — create a stopped stopwatch with zero elapsed time.
- `public static method startNew() returns Stopwatch` — create and immediately start a stopwatch.
- `public method start() returns void` — start (or resume) timing; a no-op if already running.
- `public method stop() returns void` — stop timing, adding the current interval to the accumulated total.
- `public method reset() returns void` — clear the accumulated time and stop.
- `public method elapsedNanos() returns long` — total elapsed nanoseconds (including the live interval if running).
- `public method elapsedMillis() returns long` — total elapsed time in whole milliseconds.

Date

- Namespace: `System.Time` · Import: `import System.Time.Date;`
- A calendar date as year/month/day: leap-year and month-length rules, conversion to and from a day number counted from 1970-01-01, day of week (0=Sunday), and date arithmetic via `addDays`. The `civil↔days` conversions use the standard proleptic-Gregorian algorithm.

Public members:

- `public constructor Date(int year, int month, int day)` — a date with the given fields.
- `public method year()` returns `int` — the year component.
- `public method month()` returns `int` — the month component (1-12).
- `public method day()` returns `int` — the day-of-month component.
- `public static method isLeap(int year)` returns `boolean` — whether `year` is a leap year.
- `public static method daysInMonth(int year, int month)` returns `int` — the number of days in that month of that year.
- `public method toEpochDay()` returns `int` — the day number counted from 1970-01-01.
- `public method dayOfWeek()` returns `int` — the day of week, 0=Sunday..6=Saturday.
- `public static method fromEpochDay(int z0)` returns `Date` — the date at day number `z0` (from 1970-01-01).
- `public method addDays(int n)` returns `Date` — a new date `n` days after this one.

Calendar

- Namespace: `System.Time` · Import: `import System.Time.Calendar;`
- Calendar arithmetic: leap years, days in a month, day of the week (0=Sunday..6=Saturday via Zeller's congruence), and day of the year.

Public members:

- `public static method isLeapYear(int y)` returns `boolean` — whether `y` is a leap year.
- `public static method daysInMonth(int y, int m)` returns `int` — the number of days in month `m` of year `y`.
- `public static method dayOfWeek(int year, int month, int day)` returns `int` — the day of week (0=Sunday..6=Saturday) via Zeller's congruence.
- `public static method dayOfYear(int y, int m, int d)` returns `int` — the ordinal day of the year for the given date.

Namespace `System.Json`

`System.Json` is a small, dependency-free JSON model: one `Json` type that is either a null, a bool, a number, a string, an array, or an object, plus a recursive-descent parser (`JsonParser`) and a JSON Pointer resolver (`JsonPointer`). You build a document by constructing nodes and wiring them together with `add` (for array elements) and `put` (for named members), and you read one back with `field`, `at`, and the `asx`

accessors. It is a document tree, not a schema binder — there is no reflection-based (de)serialization here, which keeps it usable in freestanding builds.

Json

- Namespace: `System.Json` · Import: `import System.Json.Json;`
- A JSON value. The `kind` code is 0=null, 1=bool, 2=number(long), 3=string, 4=array, 5=object. Built and read with pure-Polaron code over `System.Collections` + `System.Text`; arrays and object members are held as a sibling chain of child nodes. Numbers are `long`, so wrap integer literals in `cast<long>( ... )` when building. `field` on a missing key returns a JSON null node (never a null pointer), so lookups chain safely.

```
import System.Json.Json;

Json obj = Json.object();
obj.put("name", Json.ofStr("Polaron"));
obj.put("version", Json.ofNum(cast<long>(1)));

Json nums = Json.array();
nums.add(Json.ofNum(cast<long>(10)));
nums.add(Json.ofNum(cast<long>(20)));
obj.put("nums", nums);

String text = obj.toString();           // {"name":"Polaron","version":1,"nums":[10,20]}
Json parsed = Json.parse(text);        // round-trip back into a tree
System.IO.Console.println(parsed.field("name").asStr());    // Polaron
System.IO.Console.println(parsed.field("nums").at(0).asNum()); // 10
```

Public members:

- `public constructor Json(int k)` — create an empty node of kind `k` (0=null..5=object).
- `public static method ofNull()` returns `Json` — a JSON null value.
- `public static method ofBool(boolean v)` returns `Json` — a JSON boolean.
- `public static method ofNum(long v)` returns `Json` — a JSON number from a long.
- `public static method ofStr(String v)` returns `Json` — a JSON string.
- `public static method array()` returns `Json` — an empty JSON array.
- `public static method object()` returns `Json` — an empty JSON object.
- `public method add(Json* v)` returns `void` — append `v` as a child (array element or object member).
- `public method put(String key, Json* v)` returns `void` — set `v`'s member key to `key` and append it (object member).
- `public method kindOf()` returns `int` — the kind code (0=null..5=object).
- `public method asBool()` returns `boolean` — the boolean payload.
- `public method asNum()` returns `long` — the number payload.
- `public method asStr()` returns `String` — the string payload.
- `public method size()` returns `int` — the number of children (array elements / object members).

- `public method at(int i)` returns `nullable Json` — the `i`-th child, walking the sibling chain.
- `public method field(String key)` returns `nullable Json` — the object member named `key`, or a null `Json` node if absent.
- `public method toString()` returns `String` — the compact JSON serialization.
- `public method prettyString()` returns `String` — the indented, multi-line JSON serialization.
- `public static method parse(String src)` returns `Json` — parse a JSON string into a `Json` tree (via `JsonParser`).

JsonPointer

- Namespace: `System.Json` · Import: `import System.Json.JsonPointer;`
- JSON Pointer (RFC 6901): resolve a `"/a/0/b"` path against a `Json` tree, stepping into object members by key and array elements by index. Returns null if any step is missing or out of range.

Public members:

- `public static method resolve(Json* root, String ptr)` returns `nullable Json` — resolve the RFC 6901 pointer `ptr` against `root`, or null if any step fails.

JsonParser

- Namespace: `System.Json` · Import: `import System.Json.JsonParser;`
- Recursive-descent JSON parser (minimal: null/bool/integer/string/array/object). Usually reached through `Json.parse`, but the parser can be driven directly.

Public members:

- `public constructor JsonParser(String src)` — create a parser positioned at the start of `src`.
- `public method parseValue()` returns `Json` — parse the next JSON value at the current position.

Toml

```
import System.Formats.Toml;
```

- `public static method parse(String src)` returns `Json` — a TOML document as a `Json` tree.

Tables, dotted keys, arrays, inline tables, the four scalar kinds and comments. It answers `Json` rather than a type of its own so that everything already written against `Json` — `JsonPointer`, the getters, the printer — works on a TOML file without a second API. The toolchain's own `polaron.toml` is read this way.

Yaml

```
import System.Formats.Yaml;
```

- `public static method parse(String src)` returns `Json` — a YAML document as a `Json` tree.

The subset that configuration actually uses: nested mappings by indentation, sequences, scalars, quoted strings and comments. Not the whole specification — anchors, tags and flow-style edge cases are out — and it says so rather than half-supporting them.

Xml

```
import System.Formats.Xml;
```

A small XML tree: elements, attributes and text, with `parse` producing a node and each node answering `name()`, `text()`, `attribute(name)`, and its children.

Children are a sibling chain and not an `ArrayList<Xml>`, and `Json` next door made the same choice for the same reason: a container of the type being declared is a generic instantiation over it, that instantiation reaches `indexOf`, which calls `equalsKey`, which resolves onto `Object` — a class a freestanding build does not emit. The list version broke every bare-metal program in the suite with a message about `equalsKey`, pointing into the standard library, from a prelude type the kernel never mentions.

Polaron Standard Library — Math / Numerics / Net / Misc

This is the "everything numeric plus the outside world" half of the standard library: the mathematics and numerics of `System.Math`, the cryptography of `System.Security`, the sockets and HTTP client of `System.Net`, and a set of application-level building blocks in `System.App`, rounded out by process results (`System.OS`) and an inline test framework (`System.Test`).

Every type here is defined in the embedded prelude (the `kPreludeSource` raw string in `src/cli/main.cpp`, lines ~6340–8360) and is written in **pure Polaron** over the compiler builtins. The only exceptions are members marked `extern`, which link to a small runtime helper for things the language can't do on its own — reading the OS CSPRNG (`SecureRandom`), or talking to the network stack (`Socket`, `UdpSocket`, `Http`). Because `System.Math` leans on its own `Numerics` class for transcendental functions rather than a native math library, the scientific types work unchanged in freestanding builds.

A note on numeric style you'll see throughout: much of `System.Math` is deliberately **integer-first**. `Stats`, `Matrix`, `Polygon`, and friends compute in `int` / `long` so results are exact and reproducible, with parallel `double` types (`MatrixD`, `Vector2/3/4`, `Numerics`, `Regression`) provided where floating point is the point. Immutable value types (`BigInteger`, `Rational`, `Complex`, `Quaternion`, `Money`) return **new** objects from each operation rather than mutating in place, matching Polaron's copy-by-value model.

All stdlib types require an **explicit import**. The import names the fully-qualified type, e.g. `import System.Math.BigInteger`; or `import System.Net.Http`; . Each type below lists its namespace and the exact import line.

Signatures are reproduced verbatim from the source. Only **public** members are listed (constructors, public instance methods, public static methods, and public fields); private helpers are omitted.

Namespaces covered:

- `System.Math` — big integers, number theory, statistics, linear algebra, geometry, transcendental functions, graphics/animation math, probabilistic sketches.
- `System.OS` — process results.
- `System.Security` — secure randomness and AES.
- `System.Net` — TCP/UDP sockets and a minimal HTTP client.
- `System.App` — resilience, rate limiting, a stack VM, feature flags, pooling, money.
- `System.Test` — the inline unit-test framework.

System.Math

By far the largest namespace here, `System.Math` spans several families you can pick from by task: exact number types (`BigInteger`, `Rational`, `Complex`, `Quaternion`); number theory and combinatorics (`IntMath`, `NumberTheory`, `Sieve`, `Factorize`, `Crt`, `Combinatorics`); statistics (`Stats`, `RunningStats`, `Regression`, `Correlation`); linear algebra (`Matrix`, `MatrixD`, `Mat4`, the `Vector*` types, `GaussSolver`, `Fft`); computational geometry (`Polygon`, `ConvexHull`); graphics and animation math (`Interpolation`,

Easing , Angle); probabilistic sketches for streaming data (CountMinSketch , HyperLogLog); and the self-contained transcendental functions in Numerics that everything floating-point builds on. Nothing here shadows a user class called Math — the integer helpers deliberately live under IntMath .

BigInteger

Namespace: System.Math · **Import:** import System.Math.BigInteger;

Arbitrary-precision signed integer (spec 34). Decimal digits (0..9) are stored least-significant-first in an int[] with a separate sign flag. Supports construction from long , sign-aware add/subtract/multiply/divide/remainder, comparison, and decimal rendering. Use it when a product or sum would overflow long — a BigInteger grows its digit buffer as needed. It is an immutable value type: each operation returns a fresh BigInteger , so the operands are never modified. Note that construction is from long , so wrap literals in cast<long>(...) .

```
import System.Math.BigInteger;

BigInteger a = new BigInteger(cast<long>(123456789)) on heap;
BigInteger b = new BigInteger(cast<long>(987654321)) on heap;
BigInteger product = a.multiply(b);           // 121932631112635269 - overflows a 32-bit int
System.IO.Console.WriteLine(product.ToString());
```

- public constructor BigInteger(long value) — build from a signed 64-bit value.
- public method isZero() returns boolean — whether the value is exactly zero.
- public method add(BigInteger other) returns BigInteger — sign-aware sum (new value).
- public method subtract(BigInteger other) returns BigInteger — sign-aware difference (new value).
- public method multiply(BigInteger other) returns BigInteger — long multiplication with sign (new value).
- public method compareTo(BigInteger other) returns int — sign-aware compare: -1, 0, or 1.
- public method divide(BigInteger other) returns BigInteger — quotient truncated toward zero (zero on divide-by-zero).
- public method remainder(BigInteger other) returns BigInteger — remainder whose sign follows the dividend (zero on divide-by-zero).
- public method toString() returns String — decimal string, with a leading - when negative.

IntMath

Namespace: System.Math · **Import:** import System.Math.IntMath;

Integer math helpers (spec 34.6) on plain int s, with no floating point: gcd/lcm, factorial, primality, integer power and square root, combinatorics, modular exponentiation, and digit utilities. (Named IntMath so it never shadows a user class called Math .)

- public static method gcd(int a, int b) returns int — greatest common divisor via Euclid (handles negatives).

- `public static method lcm(int a, int b) returns int` — least common multiple (0 when either input is 0).
- `public static method factorial(int n) returns int` — $n!$ computed in an `int`.
- `public static method isPrime(int n) returns boolean` — primality by trial division.
- `public static method ipow(int base, int exp) returns int` — integer base raised to exp .
- `public static method isqrt(int n) returns int` — floor of the square root (0 for negative n).
- `public static method nCr(int n, int r) returns int` — binomial coefficient, multiplying/dividing as it goes to stay exact.
- `public static method nPr(int n, int r) returns int` — number of ordered r -arrangements of n items.
- `public static method modpow(int base, int exp, int mod) returns int` — modular exponentiation by repeated squaring (64-bit intermediates).
- `public static method digitSum(int n) returns int` — sum of the decimal digits of $|n|$.
- `public static method reverseDigits(int n) returns int` — digits of $|n|$ reversed as an integer.
- `public static method isPalindrome(int n) returns boolean` — whether n is a non-negative decimal palindrome.

Rational

Namespace: `System.Math` · **Import:** `import System.Math.Rational;`

An exact fraction kept in lowest terms with a positive denominator (spec 34.6). The constructor divides out the gcd and normalizes the sign; arithmetic returns new reduced `Rationals`.

- `public constructor Rational(int n, int d)` — build n/d , reduced and sign-normalized.
- `public method numerator() returns int` — the reduced numerator.
- `public method denominator() returns int` — the reduced (positive) denominator.
- `public method add(Rational o) returns Rational` — sum, reduced (new value).
- `public method sub(Rational o) returns Rational` — difference, reduced (new value).
- `public method mul(Rational o) returns Rational` — product, reduced (new value).
- `public method toDouble() returns double` — floating-point approximation num/den .

Complex

Namespace: `System.Math` · **Import:** `import System.Math.Complex;`

A complex number with `double` real and imaginary parts (spec 34.6). Immutable: `add`, `sub`, `mul`, and `conjugate` each return a new `Complex`. Useful anywhere a computation needs the imaginary axis — signal processing, root-finding, or feeding the `Fft`.

```
import System.Math.Complex;

Complex p = new Complex(1.0, 2.0) on heap;
Complex q = p.mul(new Complex(3.0, 4.0) on heap); // (1+2i)(3+4i) = -5 + 10i
System.IO.Console.printf("%d + %di\n", cast<int>(q.real()), cast<int>(q.imag()));
```


- `public constructor Complex(double re, double im)` — build $re + im \cdot i$.
- `public method real()` returns `double` — the real part.
- `public method imag()` returns `double` — the imaginary part.
- `public method add(Complex o)` returns `Complex` — complex sum (new value).
- `public method sub(Complex o)` returns `Complex` — complex difference (new value).
- `public method mul(Complex o)` returns `Complex` — complex product (new value).
- `public method conjugate()` returns `Complex` — the complex conjugate (new value).

Stats

Namespace: `System.Math` · **Import:** `import System.Math.Stats;`

Summary statistics over an `int[]` (spec 34.6): totals, central tendency, spread, and order statistics on a sorted copy. All results are integer-valued, which keeps them exact and reproducible; when you need fractional means or variances, accumulate `double`s through `RunningStats` (Welford) instead. Order statistics (`median` , `percentile`) sort a copy, so the input array is left untouched.

```
import System.Math.Stats;

mutable int[] xs = new int[5]();
xs[0] = 4; xs[1] = 8; xs[2] = 15; xs[3] = 16; xs[4] = 23;
System.IO.Console.printf("sum=%d mean=%d median=%d p90=%d\n",
    Stats.sum(xs), Stats.mean(xs), Stats.median(xs), Stats.percentile(xs, 90));
// sum=66 mean=13 median=15 p90=23
```

- `public static method sum(int[] xs)` returns `int` — sum of all values.
- `public static method mean(int[] xs)` returns `int` — integer arithmetic mean (0 for empty).
- `public static method min(int[] xs)` returns `int` — smallest value.
- `public static method max(int[] xs)` returns `int` — largest value.
- `public static method variance(int[] xs)` returns `int` — population variance about the integer mean (0 for empty).
- `public static method stddev(int[] xs)` returns `int` — population standard deviation (isqrt of the variance).
- `public static method median(int[] xs)` returns `int` — middle value of a sorted copy; upper-middle for an even count.
- `public static method range(int[] xs)` returns `int` — max minus min (0 for empty).
- `public static method mode(int[] xs)` returns `int` — most frequent value; ties resolve to the first to reach the top count.
- `public static method percentile(int[] xs, int p)` returns `int` — value at percentile p in $[0,100]$ by nearest-rank on a sorted copy.

Dual

Namespace: `System.Math` · **Import:** `import System.Math.Dual;`

Forward-mode automatic differentiation via dual numbers (spec 34.6): each value carries its derivative, and arithmetic propagates the chain rule. Evaluate a function with `variable(x)` and read `deriv()` for the exact derivative at `x`; `constant(c)` has derivative 0.

- `public constructor Dual(double value, double deriv)` — build a dual number (value, derivative).
- `public static method variable(double x)` returns `Dual` — a seed variable at `x` (derivative 1).
- `public static method constant(double x)` returns `Dual` — a constant `x` (derivative 0).
- `public method value()` returns `double` — the primal value.
- `public method deriv()` returns `double` — the accumulated derivative.
- `public method add(Dual o)` returns `Dual` — sum with chain rule (new value).
- `public method sub(Dual o)` returns `Dual` — difference with chain rule (new value).
- `public method mul(Dual o)` returns `Dual` — product with the product rule (new value).

Matrix

Namespace: `System.Math` · **Import:** `import System.Math.Matrix;`

A dense integer matrix stored row-major in one flat `int[]` (spec 34.6). Construction gives you a zero-filled matrix you fill with `set`; `multiply`, `add`, and `transpose` return new matrices, and `determinant` uses fraction-free Bareiss elimination so the result stays an exact integer. For floating-point linear algebra (LU-style determinants, non-integer data) use `MatrixD` instead.

```
import System.Math.Matrix;

mutable Matrix a = new Matrix(2, 2) on heap;
a.set(0, 0, 1); a.set(0, 1, 2);
a.set(1, 0, 3); a.set(1, 1, 4);
Matrix aT = a.transpose();
Matrix sq = a.multiply(a);           // ordinary matrix product (new matrix)
System.IO.Console.printf("det=%d\n", a.determinant()); // det=-2
```

- `public constructor Matrix(int rows, int cols)` — allocate a zero-filled rows×cols matrix.
- `public method rows()` returns `int` — row count.
- `public method cols()` returns `int` — column count.
- `public method set(int r, int c, int value)` returns `void` — store value at (r, c).
- `public method get(int r, int c)` returns `int` — read the element at (r, c).
- `public method multiply(Matrix o)` returns `Matrix` — matrix product (new matrix).
- `public method transpose()` returns `Matrix` — transpose (new matrix).
- `public method add(Matrix o)` returns `Matrix` — elementwise sum (new matrix).
- `public method determinant()` returns `int` — exact integer determinant via Bareiss fraction-free elimination (0 if singular; assumes square).

Polynomial

Namespace: `System.Math` · **Import:** `import System.Math.Polynomial;`

A polynomial with integer coefficients in ascending order, `coeff i` multiplying `xi` (spec 34.6).

- `public constructor Polynomial(int[] coeffs)` — build from ascending coefficients (index i is the x^i term).
 - `public method degree()` returns `int` — the polynomial degree ($\text{length} - 1$).
 - `public method coeff(int i)` returns `int` — the i -th coefficient (0 if out of range).
 - `public method evaluate(int x)` returns `int` — evaluate at x via Horner's method.
 - `public method derivative()` returns `Polynomial` — the differentiated polynomial (new value).
-

IntVector

Namespace: `System.Math` · **Import:** `import System.Math.IntVector;`

A fixed-length integer vector (spec 34.6). For floating SIMD vectors use the built-in `vec2` / `vec3` / `vec4` types instead.

- `public constructor IntVector(int[] elems)` — wrap the given elements.
 - `public method get(int i)` returns `int` — the i -th component.
 - `public method size()` returns `int` — component count.
 - `public method dot(IntVector o)` returns `int` — dot product.
 - `public method normSquared()` returns `int` — squared Euclidean length.
-

Sieve

Namespace: `System.Math` · **Import:** `import System.Math.Sieve;`

The sieve of Eratosthenes up to a limit (spec 34.6): the constructor marks composites once, then lookups are constant-time.

- `public constructor Sieve(int limit)` — sieve all integers up to `limit` inclusive.
 - `public method isPrime(int n)` returns `boolean` — constant-time primality within the sieved range.
 - `public method count()` returns `int` — number of primes found up to the limit.
-

Xorshift

Namespace: `System.Math` · **Import:** `import System.Math.Xorshift;`

A fast deterministic pseudo-random generator (xorshift32, spec 34.6): the same seed always yields the same sequence, unlike the wall-clock-seeded `Random`.

- `public constructor Xorshift(int seed)` — seed the generator (a 0 seed is bumped to 1).
 - `public method next()` returns `int` — the next raw 32-bit value.
 - `public method nextInRange(int n)` returns `int` — a non-negative value in $[0, n)$.
-

GaussSolver

Namespace: `System.Math` · **Import:** `import System.Math.GaussSolver;`

A dense linear-system solver (spec 34.6) via Gaussian elimination with partial pivoting, using `double` throughout.

- `public static method solve(double[] aug, int n) returns double[]` — solve $[A|b]$ given as a flat row-major $n \times (n+1)$ augmented matrix (overwritten in place); returns the solution vector x .

RunningStats

Namespace: `System.Math` · **Import:** `import System.Math.RunningStats;`

Online mean and variance via Welford's algorithm (spec 34.6): add samples one at a time in $O(1)$ space, then read the running statistics without keeping the data.

- `public constructor RunningStats()` — start an empty accumulator.
- `public method add(double x) returns void` — incorporate one sample.
- `public method getMean() returns double` — the running mean.
- `public method populationVariance() returns double` — population variance (0 with fewer than 1 sample).
- `public method sampleVariance() returns double` — sample variance (0 with fewer than 2 samples).
- `public method count() returns int` — number of samples seen.

NumberTheory

Namespace: `System.Math` · **Import:** `import System.Math.NumberTheory;`

Number-theory toolkit (spec 34.6): gcd/lcm, fast modular exponentiation, deterministic Miller-Rabin primality for 32-bit ints (witnesses 2, 3, 5, 7), and modular inverse.

- `public static method gcd(int a, int b) returns int` — greatest common divisor (handles negatives).
- `public static method lcm(int a, int b) returns int` — least common multiple (0 when either is 0).
- `public static method modpow(long base, long exp, long mod) returns long` — modular exponentiation by repeated squaring, in `long`.
- `public static method isPrime(int num) returns boolean` — deterministic Miller-Rabin primality for 32-bit ints.
- `public static method modInverse(int a, int m) returns int` — modular inverse via extended Euclid (-1 when a is not invertible mod m).

Crt

Namespace: `System.Math` · **Import:** `import System.Math.Crt;`

Chinese remainder theorem (spec 34.6): solve $x \equiv a[i] \pmod{n[i]}$ for pairwise-coprime moduli.

- `public static method solve(int[] a, int[] n, int k) returns long` — least non-negative x satisfying the first k congruences.

Factorize

Namespace: `System.Math` · **Import:** `import System.Math.Factorize;`

Integer factorization by trial division (spec 34.6).

- `public static method largestPrimeFactor(int num) returns int` — the largest prime factor of num.
- `public static method factorCount(int num) returns int` — count of prime factors with multiplicity.

Combinatorics

Namespace: `System.Math` · **Import:** `import System.Math.Combinatorics;`

Combinatorics (spec 34.6): factorials and binomials in `long`, Catalan numbers, and in-place permutation enumeration.

- `public static method factorial(int n) returns long` — $n!$ in a `long`.
- `public static method choose(int n, int k) returns long` — binomial coefficient (0 when k out of range).
- `public static method catalan(int n) returns long` — the nth Catalan number.
- `public static method nextPermutation(int[] a, int n) returns boolean` — advance `a[0..n)` to the next lexicographic permutation in place; false past the last.

Polygon

Namespace: `System.Math` · **Import:** `import System.Math.Polygon;`

Planar geometry over integer coordinates given as parallel `xs/ys` arrays (spec 34.6).

- `public static method area2(int[] xs, int[] ys, int n) returns int` — twice the shoelace area (doubled so integer inputs stay integer), always non-negative.
- `public static method contains(int[] xs, int[] ys, int n, int px, int py) returns boolean` — point-in-polygon by ray casting.

ConvexHull

Namespace: `System.Math` · **Import:** `import System.Math.ConvexHull;`

Convex hull of a point set (spec 34.6) by Andrew's monotone chain, given parallel `xs/ys` arrays.

- `public static method size(int[] xs, int[] ys, int n) returns int` — number of hull vertices, dropping collinear points.

CountMinSketch

Namespace: `System.Math` · **Import:** `import System.Math.CountMinSketch;`

Count-Min sketch (spec 34.1): a sub-linear frequency estimator over `depth` independent hash rows. Estimates never underestimate (and are exact when the width avoids collisions).

- `public constructor CountMinSketch(int width, int depth)` — allocate a `width×depth` counter table.

- `public method add(String key, int count) returns void` — accumulate count for a key across all rows.
- `public method estimate(String key) returns int` — estimated frequency (the minimum matching row).

HyperLogLog

Namespace: `System.Math` · **Import:** `import System.Math.HyperLogLog;`

HyperLogLog (spec 34.1): estimates distinct-element count in near-constant memory. Precision is $\log_2$ of the register count (e.g. $10 \rightarrow 1024$ registers, $\sim 3\%$ error).

- `public constructor HyperLogLog(int precision)` — allocate $2^{\text{precision}}$ registers.
- `public method add(String key) returns void` — record one observed key.
- `public method estimate() returns int` — estimated cardinality, with linear counting in the small range.

Numerics

Namespace: `System.Math` · **Import:** `import System.Math.Numerics;`

Self-contained transcendental functions in pure Polaron (spec 34.6), so scientific classes never depend on the `Math` builtin. `sqrt` is Newton's method; `ln` uses an `atanh` series; `exp`/`sin`/`cos` use range-reduction plus Taylor series; `pow` is $\exp(e \cdot \ln(b))$ for positive `b`.

- `public static method pi() returns double` — the constant π .
- `public static method abs(double x) returns double` — absolute value.
- `public static method sqrt(double x) returns double` — square root (0 for $x \leq 0$).
- `public static method ln(double x) returns double` — natural logarithm (0 for $x \leq 0$).
- `public static method exp(double x) returns double` — e^x .
- `public static method sin(double x) returns double` — sine.
- `public static method cos(double x) returns double` — cosine.
- `public static method pow(double b, double e) returns double` — b^e for positive `b` (0 when $b \leq 0$).

Interpolation

Namespace: `System.Math` · **Import:** `import System.Math.Interpolation;`

Interpolation helpers for animation and graphics (spec 34.6).

- `public static method lerp(double a, double b, double t) returns double` — linear interpolation $a \rightarrow b$ by `t`.
- `public static method inverseLerp(double a, double b, double v) returns double` — the `t` that maps to `v` (0 when $a == b$).
- `public static method clamp(double v, double lo, double hi) returns double` — clamp `v` into `[lo, hi]`.

- `public static method smoothstep(double edge0, double edge1, double x) returns double` — Hermite smoothstep between the edges.
- `public static method remap(double v, double inLo, double inHi, double outLo, double outHi) returns double` — remap `v` from one range to another.

Bits

Namespace: `System.Math` · **Import:** `import System.Math.Bits;`

Bit-twiddling helpers over 32-bit words (spec 34.6), using `uint` for logical (zero-fill) shifts.

- `public static method popcount(int x) returns int` — number of set bits.
- `public static method leadingZeros(int x) returns int` — count of leading zero bits (32 for 0).
- `public static method trailingZeros(int x) returns int` — count of trailing zero bits (32 for 0).
- `public static method isPow2(int n) returns boolean` — whether `n` is a positive power of two.
- `public static method nextPow2(int n) returns int` — smallest power of two $\geq n$ (1 for $n \leq 1$).
- `public static method reverse(int x) returns int` — full 32-bit reversal.

Vector2

Namespace: `System.Math` · **Import:** `import System.Math.Vector2;`

A 2D vector of doubles (spec 34.6). The components are public mutable fields; arithmetic returns new heap vectors.

- `public mutable double x` — the `x` component (public field).
- `public mutable double y` — the `y` component (public field).
- `public constructor Vector2(double x, double y)` — build `(x, y)`.
- `public method add(Vector2 o) returns Vector2` — vector sum (new value).
- `public method sub(Vector2 o) returns Vector2` — vector difference (new value).
- `public method scale(double s) returns Vector2` — scalar multiple (new value).
- `public method dot(Vector2 o) returns double` — dot product.
- `public method length() returns double` — Euclidean length (via `Numerics.sqrt`).

Vector4

Namespace: `System.Math` · **Import:** `import System.Math.Vector4;`

A 4D vector of doubles, e.g. homogeneous coordinates (spec 34.6). Components are public mutable fields.

- `public mutable double x` — the `x` component (public field).
- `public mutable double y` — the `y` component (public field).
- `public mutable double z` — the `z` component (public field).
- `public mutable double w` — the `w` component (public field).
- `public constructor Vector4(double x, double y, double z, double w)` — build `(x, y, z, w)`.
- `public method add(Vector4 o) returns Vector4` — vector sum (new value).
- `public method dot(Vector4 o) returns double` — dot product.

- `public method length()` returns `double` — Euclidean length (via `Numerics.sqrt`).

Easing

Namespace: `System.Math` · **Import:** `import System.Math.Easing;`

Easing curves mapping `t` in `[0,1]` to `[0,1]` for animation (spec 34.6).

- `public static method quadIn(double t)` returns `double` — quadratic ease-in.
- `public static method quadOut(double t)` returns `double` — quadratic ease-out.
- `public static method cubicIn(double t)` returns `double` — cubic ease-in.
- `public static method cubicOut(double t)` returns `double` — cubic ease-out.

Angle

Namespace: `System.Math` · **Import:** `import System.Math.Angle;`

Angle conversions (spec 34.6).

- `public static method toRadians(double deg)` returns `double` — degrees to radians.
- `public static method toDegrees(double rad)` returns `double` — radians to degrees.

Mat4

Namespace: `System.Math` · **Import:** `import System.Math.Mat4;`

A 4×4 matrix in row-major storage for 3D transforms (spec 34.6). Pairs with `Vector4`; each operation returns a new matrix.

- `public constructor Mat4()` — allocate a zero-filled 4×4 matrix.
- `public static method identity()` returns `Mat4` — the identity matrix (new value).
- `public method get(int r, int c)` returns `double` — read the element at `(r, c)`.
- `public method set(int r, int c, double v)` returns `void` — store `v` at `(r, c)`.
- `public method multiply(Mat4 o)` returns `Mat4` — matrix product (new value).
- `public method transpose()` returns `Mat4` — transpose (new value).

Fft

Namespace: `System.Math` · **Import:** `import System.Math.Fft;`

Radix-2 fast Fourier transform (spec 34.6), iterative Cooley-Tukey over parallel real/imag arrays whose length is a power of two. `inverse(forward(x)) == x`.

- `public static method forward(double[] re, double[] im, int n)` returns `void` — forward transform in place.
- `public static method inverse(double[] re, double[] im, int n)` returns `void` — inverse transform in place, divided by `n`.

Regression

Namespace: `System.Math` · **Import:** `import System.Math.Reggression;`

Ordinary least-squares linear regression (spec 34.6): fit $y = \text{slope} \cdot x + \text{intercept}$ from parallel arrays and report the coefficient of determination r^2 .

- `public constructor Regression(double[] x, double[] y, int n)` — fit the line over the first n points.
- `public method getSlope()` returns `double` — the fitted slope.
- `public method getIntercept()` returns `double` — the fitted intercept.
- `public method getR2()` returns `double` — the coefficient of determination r^2 .
- `public method predict(double x)` returns `double` — the fitted value at x .

Correlation

Namespace: `System.Math` · **Import:** `import System.Math.Correlation;`

Pearson correlation coefficient (spec 34.6).

- `public static method pearson(double[] x, double[] y, int n)` returns `double` — correlation in $[-1, 1]$ over the first n pairs (0 when a spread is degenerate).

Quaternion

Namespace: `System.Math` · **Import:** `import System.Math.Quaternion;`

Quaternions for 3D rotation math (spec 34.6): Hamilton product, conjugate, magnitude and normalize. Immutable; operations return new heap quaternions.

- `public constructor Quaternion(double w, double x, double y, double z)` — build (w, x, y, z) .
- `public method getW()` returns `double` — the scalar component w .
- `public method getX()` returns `double` — the i component x .
- `public method getY()` returns `double` — the j component y .
- `public method getZ()` returns `double` — the k component z .
- `public method magnitude()` returns `double` — Euclidean norm (via `Numerics.sqrt`).
- `public method conjugate()` returns `Quaternion` — the conjugate (new value).
- `public method mul(Quaternion o)` returns `Quaternion` — Hamilton product (new value).
- `public method normalize()` returns `Quaternion` — unit quaternion (identity when magnitude is 0).

MatrixD

Namespace: `System.Math` · **Import:** `import System.Math.MatrixD;`

A dense `double` matrix stored row-major (spec 34.6).

- `public constructor MatrixD(int rows, int cols)` — allocate a zero-filled $\text{rows} \times \text{cols}$ matrix.
- `public method set(int r, int c, double v)` returns `void` — store v at (r, c) .
- `public method get(int r, int c)` returns `double` — read the element at (r, c) .

- `public method rowCount() returns int` — row count.
- `public method colCount() returns int` — column count.
- `public method mul(MatrixD o) returns MatrixD` — matrix product (new value).
- `public method transpose() returns MatrixD` — transpose (new value).
- `public method determinant() returns double` — determinant by Gaussian elimination with partial pivoting (0 if singular).

Vector3

Namespace: `System.Math` · **Import:** `import System.Math.Vector3;`

A 3D vector of doubles (spec 34.6). Immutable; operations return new heap vectors.

- `public constructor Vector3(double x, double y, double z)` — build (x, y, z).
- `public method getX() returns double` — the x component.
- `public method getY() returns double` — the y component.
- `public method getZ() returns double` — the z component.
- `public method add(Vector3 o) returns Vector3` — vector sum (new value).
- `public method sub(Vector3 o) returns Vector3` — vector difference (new value).
- `public method scale(double s) returns Vector3` — scalar multiple (new value).
- `public method dot(Vector3 o) returns double` — dot product.
- `public method cross(Vector3 o) returns Vector3` — cross product (new value).
- `public method length() returns double` — Euclidean length (via `Numerics.sqrt`).
- `public method normalize() returns Vector3` — unit vector (zero vector when length is 0).

System.OS

ProcessResult

Namespace: `System.OS` · **Import:** `import System.OS.ProcessResult;`

The result of running a subprocess (spec 34): its captured stdout and exit code. Built by the `Process.run(cmd)` builtin, which runs the command through the shell.

- `public String output` — the captured standard output (public field).
- `public int exitCode` — the process exit code (public field).
- `public constructor ProcessResult(String output, int exitCode)` — build from output and exit code.
- `public method success() returns boolean` — whether the exit code is 0.

System.Security

SecureRandom

Namespace: `System.Security` · **Import:** `import System.Security.SecureRandom;`

A cryptographically secure random source (spec 34): 64 bits per draw from the OS CSPRNG, suitable for keys, tokens and nonces (unlike `System.Math.Random`, which is a fast PRNG). The draw links to a runtime helper via `extern`.

- `public constructor SecureRandom()` — open the secure source.
- `public method nextLong()` returns `long` — 64 random bits.
- `public method nextInt()` returns `int` — a non-negative 31-bit random int.
- `public method nextIntMax(int max)` returns `int` — a random int in [0, max).
- `public method nextBool()` returns `boolean` — a random boolean.
- `public method nextDouble()` returns `double` — a random double in [0, 1) (52-bit mantissa).
- `public method nextBytes(int n)` returns `int[]` — n random bytes as ints in 0..255.

Aes

Namespace: `System.Security` · **Import:** `import System.Security.Aes;`

AES block cipher (spec 34; FIPS-197), pure Polaron. Supports 128- and 256-bit keys (16 or 32 bytes); S-boxes are generated from the GF(2⁸) inverse plus the affine transform. `encryptBlock` / `decryptBlock` are the raw 16-byte ECB primitive; `ctr()` is the recommended stream mode (encryption and decryption are the same call). Bytes are carried as ints in 0..255.

- `public constructor Aes(int[] key)` — build a cipher from a 16- or 32-byte key (expands the round keys).
- `public method encryptBlock(int[] input)` returns `int[]` — encrypt one 16-byte block (ECB).
- `public method decryptBlock(int[] input)` returns `int[]` — decrypt one 16-byte block (ECB).
- `public method ctr(int[] data, int[] iv)` returns `int[]` — CTR-mode keystream XORed with the data; symmetric, no padding. iv is a 16-byte nonce/counter start.

System.Net

The networking namespace is thin and blocking by design: a handful of classes that wrap OS socket handles through `extern` runtime helpers. `Socket` / `ServerSocket` are the TCP pair (connect a client, or bind-listen-accept a server); `UdpSocket` / `Datagram` cover connectionless UDP; and `Http` / `HttpResponse` layer a minimal HTTP/1.1 GET client on top of `Socket`. There is no async here — a `receive` blocks until data arrives — so for concurrency, drive sockets from `System.Thread` workers. Because the transport is `extern`, these types are the one part of this reference that does not run in freestanding builds.

Socket

Namespace: `System.Net` · **Import:** `import System.Net.Socket;`

A blocking TCP socket (spec 34) wrapping an OS handle (or `-1` on failure). Build a client with `Socket.connect`; `ServerSocket.accept()` also hands back a `Socket`. Send/receive/close lower to runtime winsock helpers.

- `public constructor Socket(long handle)` — wrap an existing OS socket handle.
- `public static method connect(String host, int port)` returns `Socket` — open a client connection.
- `public method isOpen()` returns `boolean` — whether the handle is valid (≥ 0).
- `public method send(String data)` returns `long` — send bytes; returns the count sent.
- `public method receive(int max)` returns `String` — receive up to max bytes.
- `public method close()` returns `void` — close the socket.

ServerSocket

Namespace: `System.Net` · **Import:** `import System.Net.ServerSocket;`

A listening TCP server socket (spec 34): bind+listen on a port, then accept the next connection.

- `public constructor ServerSocket(int port)` — bind and listen on the given port.
- `public method isOpen()` returns `boolean` — whether the listening handle is valid (≥ 0).
- `public method accept()` returns `Socket` — block for the next connection and return its `Socket`.
- `public method close()` returns `void` — close the listening socket.

Datagram

Namespace: `System.Net` · **Import:** `import System.Net.Datagram;`

A received datagram (spec 34): its payload plus the sender's address, so a server can reply.

- `public String data` — the payload (public field).
- `public String host` — the sender's host (public field).
- `public int port` — the sender's port (public field).
- `public constructor Datagram(String data, String host, int port)` — build from payload and sender address.

UdpSocket

Namespace: `System.Net` · **Import:** `import System.Net.UdpSocket;`

A UDP socket (spec 34): connectionless datagrams. Open with port 0 for an ephemeral client port, or a fixed port to receive on.

- `public constructor UdpSocket(int port)` — open a UDP socket on the given port (0 = ephemeral).
- `public method isOpen()` returns `boolean` — whether the handle is valid (≥ 0).
- `public method send(String host, int port, String data)` returns `long` — send a datagram to an address; returns the count sent.
- `public method receive(int max)` returns `Datagram` — receive a datagram (payload plus the sender's address).

- `public method close()` returns `void` — close the socket.

HttpResponse

Namespace: `System.Net` · **Import:** `import System.Net.HttpResponse;`

A parsed HTTP response (spec 34): the raw text is kept and queried lazily.

- `public constructor HttpResponse(String raw)` — wrap a raw response string.
- `public method raw()` returns `String` — the full raw response text.
- `public method status()` returns `int` — the status code from the start line (0 if unparseable).
- `public method body()` returns `String` — everything after the blank line ("" if none).
- `public method header(String name)` returns `String` — a header field's value ("" if absent).

Http

Namespace: `System.Net` · **Import:** `import System.Net.Http;`

A minimal HTTP/1.1 client over Socket (spec 34). Request building and response parsing are pure; `get` performs the network round-trip. `get` opens a `Socket`, sends a `GET` with `Connection: close`, drains the whole reply, and hands back an `HttpResponse` you can query for the status code, body, and individual headers. It is intentionally small — one verb, no TLS, no redirects — for talking to plain-HTTP services and health-check endpoints.

```
import System.Net.Http;
import System.Net.HttpResponse;

HttpResponse res = Http.get("example.com", 80, "/");
System.IO.Console.printf("status=%d\n", res.status());
System.IO.Console.println(res.header("Content-Type"));
System.IO.Console.println(res.body());
```

- `public static method buildRequest(String verb, String host, String path)` returns `String` — format a request line with Host and Connection: close headers.
- `public static method get(String host, int port, String path)` returns `HttpResponse` — open a socket, send a GET, drain the reply, and parse it.

System.App

CircuitBreaker

Namespace: `System.App` · **Import:** `import System.App.CircuitBreaker;`

Circuit breaker (spec 34): trips open after `threshold` consecutive failures and rejects calls until a cooldown passes, then allows one trial (half-open); a success closes it, a failure reopens it. Time is passed in explicitly (milliseconds) so behavior is deterministic.

- `public constructor CircuitBreaker(int threshold, long cooldownMs)` — build with a failure threshold and cooldown.
 - `public method allow(long now) returns boolean` — whether a call is permitted at time `now`.
 - `public method recordSuccess() returns void` — record a success (closes the breaker).
 - `public method recordFailure(long now) returns void` — record a failure (may open the breaker).
 - `public method getState() returns int` — current state: 0 closed, 1 open, 2 half-open.
-

TokenBucket

Namespace: `System.App` · **Import:** `import System.App.TokenBucket;`

Token-bucket rate limiter (spec 34): tokens refill continuously at `ratePerMs` up to capacity; each acquire spends tokens if enough are available. Time is passed in explicitly (milliseconds).

- `public constructor TokenBucket(double capacity, double ratePerMs)` — build with a capacity and refill rate.
 - `public method tryAcquire(long now, int count) returns boolean` — refill to `now` and spend `count` tokens if available.
-

StackVm

Namespace: `System.App` · **Import:** `import System.App.StackVm;`

A tiny stack virtual machine (spec 34): executes a flat program of (opcode, operand) pairs against an operand stack and a small memory. Opcodes: 0 HALT, 1 PUSH v, 2 STORE a, 3 LOAD a, 4 MUL, 5 SUB, 6 JZ t, 7 JMP t, 8 ADD (jump targets are instruction indices).

- `public static method run(int[] prog, int plen, int memSize) returns int` — run the program (plen ints, memSize memory cells) and return the top of the stack.
-

FeatureFlags

Namespace: `System.App` · **Import:** `import System.App.FeatureFlags;`

Named feature flags (spec 34): enable/disable toggles by name, defaulting to off when unset.

- `public constructor FeatureFlags()` — start with all flags off.
 - `public method enable(String name) returns void` — turn a flag on.
 - `public method disable(String name) returns void` — turn a flag off.
 - `public method isEnabled(String name) returns boolean` — whether a flag is on (false when unset).
-

ObjectPool

Namespace: `System.App` · **Import:** `import System.App.ObjectPool;`

A fixed-capacity pool of reusable integer ids (spec 34).

- `public constructor ObjectPool(int capacity)` — build a pool of the given capacity.
- `public method acquire() returns int` — hand out a fresh or recycled id (−1 when exhausted).

- `public method recycle(int id) returns void` — return an id for reuse.
- `public method inUse() returns int` — count of live (unrecycled) ids.

Money

Namespace: `System.App` · **Import:** `import System.App.Money;`

Fixed-point money as integer cents (spec 34), avoiding floating-point rounding.

- `public constructor Money(long cents)` — build an amount in cents.
- `public method getCents() returns long` — the amount in cents.
- `public method plus(Money o) returns Money` — sum (new value).
- `public method minus(Money o) returns Money` — difference (new value).
- `public method times(int factor) returns Money` — scaled amount (new value).
- `public method format() returns String` — a signed dollar string like "\$12.34".

System.Test

Tests live next to the code they test, as annotated static methods. There are no `test` or `assert` keywords — it is all annotations, static methods and classes. `polaron test` (or `polc --test`) finds them, runs them, and exits non-zero if any failed.

Every annotation below may also be written `@Name`; the two spellings are identical (\$10.3).

The annotations

Namespace: `System.Test` · **Import:** `import System.Test.Test;`

Annotation	Applies to	Meaning
<code>[Test]</code>	<code>public static</code> method returning <code>boolean</code> or <code>void</code>	An inline test. Returning <code>boolean</code> makes the test its own verdict; returning <code>void</code> takes the verdict from its <code>Test.assert*</code> calls.
<code>[Ignore(reason: " ... ")]</code>	a <code>[Test]</code> method	Reported as SKIP with the reason, never run. Keeps a known-broken case visible instead of commented out.
<code>[BeforeAll]</code> / <code>[AfterAll]</code>	<code>public static</code> method returning <code>void</code>	Run once around the whole class — where an expensive fixture is built and released. Skipped entirely when <code>--filter</code> selected none of the class's tests.
<code>[Setup]</code> / <code>[Teardown]</code>	<code>public static</code> method returning <code>void</code>	Run around each test of the class.
<code>[Cases(source: "m")]</code>	a <code>[Test]</code> taking one parameter	Runs once per element of the array <code>m</code> returns, reported as <code>Class.method[i]</code> . <code>m</code> must be a <code>public static</code> method of the same class returning an array of the parameter's type.

Annotation	Applies to	Meaning
<code>[Repeat(times: N)]</code>	a <code>[Test]</code> method	Runs the whole test N times and reports one verdict, naming the runs that failed. For flakiness.
<code>[ExpectedToFail(reason: " ... ")]</code>	a <code>[Test]</code> method	Inverts the verdict: failing is expected (XFAIL), passing is a failure . Unlike <code>[Ignore]</code> , it still runs.
<code>[Tag(name: " ... ")]</code>	a <code>[Test]</code> method, repeatable	Groups tests for <code>--tag / --exclude-tag</code> .
<code>[MaxTime(ms: N)]</code>	a <code>[Test]</code> method	A passing test that took longer becomes a failure. A budget checked after the fact, not a timeout: it does not abort a hang.
<code>[Benchmark(iterations: N, warmup: M)]</code>	public static method, no arguments, returning void	Measured, never judged: warmup pass, then the timed loop, reported as ns/op. Runs only under <code>--bench</code> .

At most one hook of each kind per class: two would have no defined order. A malformed test, hook or annotation — not static, wrong return type, `[Ignore]` on something that is not a test, a hook or a benchmark that is also a test, a parameter with no `[Cases]` , a `[Cases]` source of the wrong type — is a **compile error**, not a silent no-op.


```

import System.Test.Test;

public class Census {
    private static mutable int[] cells;

    [BeforeAll]
    public static method buildWorld() returns void {
        Census.cells = new int[64]();
        return;
    }

    [AfterAll]
    public static method dropWorld() returns void {
        delete Census.cells;
        return;
    }

    [Test]
    public static method values_stay_in_band() returns void {
        Test.checking("every cell holds a value in 0..6");
        mutable int i = 0;
        while (i < Census.cells.length()) {
            Test.assertBetween(Census.cells[i], 0, 6);
            i = i + 1;
        }
        return;
    }

    [Ignore(reason: "regrowth is not implemented yet")]
    [Test]
    public static method forest_regrows() returns void {
        Test.fail("would fail today");
        return;
    }
}

```

Test

Namespace: `System.Test` · **Import:** `import System.Test.Test;`

The assertions. Each records a failure and prints what went wrong; the test's verdict is "no assertion failed". The runner resets the state around every test, so nothing bleeds between them.

Naming the check

- `public static method checking(String what) returns void` — names what the assertions that follow are checking. A bare `assertBetween(pct, 8, 22)` reports only numbers and leaves the reader guessing *which* criterion broke; with a label the failure reads `[mountain share stays in band] expected 8..22, got 31`. Cleared at the start of every test, so it costs nothing when unused.
- `public static method fail(String why) returns void` — record a failure directly, for a check no assertion covers.
- `public static method skip(String why) returns void` — give up at runtime on an unmet precondition (no GPU, no network, a fixture that could not be built). Reported as SKIP, which is neither

a pass nor a defect.

Equality and predicates

- `assertEqual(int actual, int expected)` · `assertNotEqual(int actual, int unexpected)`
- `assertEqualLong(long actual, long expected)` · `assertEqualString(String actual, String expected)`
- `assertEqualChar(char, char)` · `assertEqualBoolean(boolean, boolean)` · `assertEqualDouble(double, double)` (exact — almost always the wrong tool; reach for `assertWithin` / `assertNear`)
- `assertTrue(boolean condition)` · `assertFalse(boolean condition)`
- `assertContains(String haystack, String needle)` · `assertStartsWith(String, String)` · `assertEndsWith(String, String)`

Arrays — every one of these reports the **first differing index**, which with a thousand elements is the whole diagnosis.

- `assertEqualIntArray(int[], int[])` · `assertEqualLongArray(long[], long[])` · `assertEqualStringArray(String[], String[])`
- `assertEqualDoubleArray(double[] actual, double[] expected, double tolerance)` — element-wise; a computed array is never bit-identical.
- `assertSorted(int[] values)` — ascending, naming the first pair that breaks it.

Ranges and tolerances — most measurements of a generated or simulated system are only ever "within an acceptable band", and writing that as `assertTrue(v ≥ 8 && v ≤ 22)` throws away the numbers the report needs.

- `assertBetween(int value, int low, int high)` — inclusive.
- `assertAtLeast(int value, int minimum)` · `assertAtMost(int value, int maximum)`
- `assertBetweenDouble(double value, double low, double high)`
- `assertWithin(double actual, double expected, double tolerance)` — absolute tolerance.
- `assertNear(double actual, double expected, double relativeTolerance)` — tolerance as a *fraction* of the expected value, for quantities whose scale varies. `0.05` means "within 5%". Falls back to an absolute comparison when the expected value is zero.
- `assertThrows<E>(Action* action)` — the action must throw `E`.
- `assertDoesNotThrow(Action* action)` — and this one must not. Without it, a test that swallows an unexpected exception higher up reads as a pass.

Output — for code whose job is to produce text.

- `captureOutput(Action* action)` returns `String` — runs the action with its printing diverted and returns that text.
- `assertMatchesGolden(String actual, String goldenPath)` — compares against a file of expected text and reports the **first differing line**; `--update-golden` rewrites the file instead, which is how an intended change is accepted.
- `artifact(String path)` — names a file the test produced, so a failure points at the evidence.
- `tempDir()` returns `String` — a scratch directory, created on first use and deliberately not cleaned up: a failing test's leftovers are usually what explains it.

Memory — the assertion only a manually-managed language can offer.

- `assertNoLeaks(Action* action)` — the action must give back everything it took. Measures the **calling thread's net** live bytes, so work handed to another thread is not covered and a leak exactly balanced by a matching free reads as clean. The assertion machinery allocates too (`checking` stores its label as a `String`), so keep `Test.*` calls out of the measured action.
- `liveBytes()` returns `long` — the live heap total on this thread, for asserting an allocation *budget* rather than a leak.

The runner also calls `reset()`, `failures()`, `wasSkipped()` and `skipReason()`; they are public so a custom harness can reuse the same state, but a test does not need them.

Running them

Command	What it does
<code>polaron test</code>	Build and run the project's tests.
<code>polaron test -- --filter <text></code>	Run only the tests whose <code>Class.method</code> name contains <code><text></code> .
<code>polaron test -- --tag <name> / -- exclude-tag <name></code>	Select or skip by <code>[Tag]</code> .
<code>polaron test -- --list</code>	Print the test names without running anything.
<code>polaron test -- --timing</code>	Add per-test and total durations to the report.
<code>polaron test -- --fail-fast</code>	Stop at the first failure (<code>[AfterAll]</code> still runs).
<code>polaron test -- --format=json</code>	One machine-readable document, each test's own output captured into its record.
<code>polaron test -- --bench</code>	Also run the <code>[Benchmark]</code> methods.
<code>polaron test -- --update-golden</code>	Rewrite golden files instead of comparing.

A `[library]` project is tested the same way: `polaron test` builds its sources as an executable (the runner supplies the entry point) instead of the usual `.polb` bundle.

The report is **deterministic by default** — durations are opt-in behind `--timing` — so it can be diffed, pasted into a review, or compared against a golden file:

```
PASS Census.values_stay_in_band
SKIP Census.forest_regrows -- regrowth is not implemented yet
FAIL Census.setup_ran_for_this_test
tests: 1 passed, 1 failed, 1 skipped
```

Assert

Namespace: `System.Test` · **Import:** `import System.Test.Assert;`

Boolean assertion helpers (spec 34): each returns whether the check holds, to be fed to `TestRunner.check`.

- `public static method eq(int a, int b)` returns `boolean` — int equality.

- `public static method eqLong(long a, long b) returns boolean` — long equality.
- `public static method eqStr(String a, String b) returns boolean` — string equality.
- `public static method near(double a, double b, double eps) returns boolean` — doubles within an epsilon.
- `public static method isTrue(boolean c) returns boolean` — passes through a true condition.
- `public static method isFalse(boolean c) returns boolean` — negates a condition.

TestRunner

Namespace: `System.Test` · **Import:** `import System.Test.TestRunner;`

A minimal unit-test runner (spec 34): tally pass/fail, print each result and a summary. Lets Polaron code (and this stdlib) self-test.

- `public constructor TestRunner()` — start with no results.
- `public method check(String name, boolean cond) returns void` — tally a pass/fail for `name` and print a line.
- `public method passed() returns int` — number of checks passed.
- `public method failed() returns int` — number of checks failed.
- `public method allPassed() returns boolean` — whether every check held.
- `public method report() returns void` — print the "N passed, M failed" summary.

Retry

```
import System.App.Retry;
```

The counter beside `CircuitBreaker` and `TokenBucket` : how many attempts are left, kept as state rather than as a loop variable somebody re-derives at each call site.

- `public constructor Retry(int maxAttempts)` .
- `public method allow() returns boolean` — is there another attempt to be had. **Asked before trying**, so a `maxAttempts` of 3 means three calls and not four.
- `public method recordFailure() returns void` / `recordSuccess() returns void` — the outcome; a success resets the count.

Style / TextTable

```
import System.Terminal.Style;
import System.Terminal.TextTable;
```

What a command-line tool needs to be readable, and nothing more.

Type	Members
<code>Style</code>	<code>red</code> , <code>green</code> , <code>yellow</code> , <code>blue</code> , <code>bold</code> , <code>dim</code> ... each wrapping a string in an ANSI sequence, plus <code>enabled()</code> / <code>enable(boolean)</code> — one switch , so a program redirected to a file turns colour off in one

Type	Members
	place instead of at every call site.
TextTable	<code>column(title, rightAligned)</code> , <code>row(cells)</code> , <code>rowCount()</code> , <code>render()</code> — columns measured across every row so the output lines up.

Standard Library — Networking and TLS

```
import System.Net.Socket; · import System.Net.HttpClient; · import System.Net.Tls.TlsClient;
```

Two layers, and the second is the unusual one. The first is what every language has: sockets, an HTTP client, a URL parser, a small server-side router. The second is a **TLS 1.3 implementation written in Polaron itself** — the handshake, the key schedule, the record layer, X.509 parsing and chain validation — talking to real servers on the public internet, with no OpenSSL and no C.

That is here for the same reason the standard library is written in Polaron rather than bound to a C one: a language that cannot express its own cryptography is a language with a hole in the middle of it, and the hole is exactly where every program that matters has to go.

1. Sockets

```
import System.Net.Socket;
import System.Net.ServerSocket;
import System.Net.UdpSocket;
```

Member	What it does
<code>Socket.connect(String host, int port)</code> returns <code>Socket</code>	Opens a TCP connection.
<code>socket.isOpen()</code> returns <code>boolean</code>	Whether the handle is live.
<code>socket.send(String data)</code> returns <code>long</code>	Bytes written, or negative on failure.
<code>socket.receive(int max)</code> returns <code>String</code>	Up to <code>max</code> bytes; empty at end of stream.
<code>socket.close()</code> returns <code>void</code>	Closes it. Idempotent.
<code>ServerSocket.listen(int port)</code> / <code>.accept()</code> returns <code>Socket</code>	The listening half.
<code>UdpSocket</code> — <code>send(host, port, data)</code> , <code>receive(max)</code> returns <code>Datagram</code>	Datagrams, with the sender's address on the way back.

A `Socket` is a handle and not a stream: it says how many bytes it moved and lets the caller decide what a message is. Everything above it in this page is built from exactly these five calls.

2. `Url` — parsing, and the part everybody gets wrong

```
Url u = Url.parse("https://example.org:8443/docs/a?q=1#top");
u.scheme();           // "https"
u.host();              // "example.org"
u.port();              // 8443, or the scheme's default when absent
u.path();              // "/docs/a"
u.query();             // "q=1"
u.fragment();          // "top"
u.isSecure();          // true
u.requestTarget();     // "/docs/a?q=1" -- what goes on the request line
u.authority();          // "example.org:8443"
u.resolve("../b");     // a new Url, resolved against this one
```

`resolve` is the one worth knowing about: a relative reference against a base URL is the operation every crawler, every redirect and every HTML link needs, and hand-rolling it with string concatenation is how a program ends up fetching `https://example.org/docs/..b`.

3. HTTP

The request

```
import System.Net.HttpRequest;

HttpRequest* r = HttpRequest.post("https://api.example.org/v1/things") on heap;
r.setJson("{\"name\":\"one\"}");
r.setHeader("Authorization", "Bearer " + token);
```

Builder	Note
<code>HttpRequest.get/post/put/patch/head(String url)</code>	The verb, and the URL parsed into a <code>Url</code> .
<code>HttpRequest.remove(String url)</code>	DELETE. Named <code>remove</code> because <code>delete</code> is a keyword.
<code>setBody</code> / <code>setJson</code> / <code>setForm</code>	The body, and the content type that goes with it.
<code>setBasicAuth(user, password)</code>	Base64 credentials, written once rather than at every call site.
<code>setHeader</code> / <code>header</code>	Anything else.
<code>toWire()</code> returns <code>String</code>	The exact bytes that will go out — for logging, and for tests that assert on the request rather than on a server.

Sending it

```
import System.Net.HttpClient;

HttpResponse res = HttpClient.send(r);
if (res.status() == 200) {
    System.IO.Console.println(res.body());
}
```

Member	What it does
<code>HttpClient.send(HttpRequest* r)</code> returns <code>HttpResponse</code>	One request. https goes through the TLS stack below , with the machine's trust store.
<code>HttpClient.sendFollowing(HttpRequest* r, int limit)</code>	The same, following up to <code>limit</code> redirects.
<code>HttpClient.fetch(String url)</code> returns <code>HttpResponse</code>	A GET, for the common case.
<code>HttpClient.get(String host, int port, String path)</code>	Plain HTTP, no URL parsing.
<code>res.status()</code> / <code>res.body()</code> / <code>res.header(name)</code> / <code>res.hasHeader(name)</code> / <code>res.raw()</code>	The response, and the unparsed bytes when something needs them.

Serving

`ServerRequest`, `ServerResponse` and `Router` are the other side: parse a request off a socket, match it against a pattern, answer.

```
Router* routes = new Router() on heap;
routes.get("/things/{id}", command (ServerRequest* q) returns ServerResponse* {
    return new ServerResponse(ServerResponse.json("{\"id\": \"" + q.param("id") + "\"}")) on heap;
});
ServerResponse* answer = routes.dispatch(request);
```

Member	What it does
<code>Router.add(verb, pattern, handler)</code> / <code>.get</code> / <code>.post</code>	A route. A pattern is path segments, with <code>{name}</code> capturing one: <code>/users/{id}/posts</code> matches <code>/users/42/posts</code> and gives <code>param("id") = "42"</code> .
<code>router.dispatch(ServerRequest*)</code> returns <code>ServerResponse*</code>	The first match, in the order the routes were added — which is the rule that lets a specific path be registered before a general one. A 404 when nothing matches.
<code>Route</code>	One entry: the verb, the pattern and the handler. Held by the router; public because a router of your own wants the same record.
<code>request.param(name)</code>	A captured segment. <code>query(name)</code> reads the query string instead.
<code>ServerResponse.ok/json/notFound/of(status, body)</code>	The usual answers. <code>toRaw()</code> is the wire form.

4. TLS 1.3, written in Polaron

```
import System.Net.Tls.TlsClient;
import System.Net.Tls.TlsSession;

TlsSession* s = TlsClient.open("example.org", 443);
s.send("GET / HTTP/1.1\r\nHost: example.org\r\nConnection: close\r\n\r\n");
String page = s.receive();
s.close();
delete s;
```

Member	What it does
<code>TlsClient.open(host, port)</code>	Connects, handshakes and validates against the system trust store .
<code>TlsClient.connect(host, port, TrustStore*)</code>	The same with a trust store you chose.
<code>TlsClient.over(Socket, host, TrustStore*)</code>	Wraps a socket somebody else opened.
<code>TlsClient.systemTrust()</code> returns <code>TrustStore*</code>	The machine's roots, read once — walking every root the OS holds costs about half a second, so a client that paid it per request would be a client nobody uses twice.
<code>session.send(String)</code> / <code>.receive()</code> returns <code>String</code> / <code>.isOpen()</code> / <code>.close()</code>	The stream, encrypted.

What is implemented: TLS 1.3 (RFC 8446) with X25519 and P-256/P-384 key exchange, AES-GCM record protection, the HKDF key schedule, and full X.509: DER parsing, chain building, name checks, validity windows, basic constraints, RSA (PKCS#1 v1.5 and PSS) and ECDSA signature verification.

The pieces, for anybody who needs one on its own

Type	What it is
<code>TrustStore</code>	A set of root certificates. <code>systemTrust()</code> fills it from the OS.
<code>Certificate</code>	One X.509 certificate: <code>fromPem</code> , <code>dnsNames</code> , <code>notBefore</code> / <code>notAfter</code> , <code>isValidAt</code> , <code>isCertificateAuthority</code> , <code>isIssuedBy</code> , <code>subjectCommonName</code> , the public key, the signature and its algorithm.
<code>CertificateValidator.check(chain, host, trusted, nowSeconds)</code>	Validates a chain by throwing , naming what failed — an expired leaf, a name that does not match, a missing issuer. A boolean answer would lose which of those it was.
<code>Der</code> , <code>Oid</code>	The encoding underneath: lengths, tags, well-formedness, and the object identifiers by name.
<code>Rsa</code> — <code>verifyPkcs1</code> , <code>verifyPss</code> , <code>verifySha256</code>	Signature verification.
<code>Sha</code> — <code>sha256()</code> , <code>sha384()</code> , <code>sha512()</code> , <code>forAlgorithm(alg)</code>	The digest a certificate asks for, chosen by its own algorithm id.

Type	What it is
RsaPublicKey	A parsed RSA key: the modulus and exponent a Certificate hands back from rsaPublicKey() .
Hkdf , KeySchedule , KeyShare , Gcm , Record , Wire , ClientHello , Alert	The handshake and record layer, if you are implementing a protocol on top rather than using one.
ByteWriter	The wire builder underneath them: length-prefixed fields written in the order the protocol names them, so a handshake message is assembled rather than concatenated.
X25519 , Curve , EcPoint , Mont , U256	The curve arithmetic.
TlsException , CertificateException	What a refusal throws.

Where it fits

HttpClient uses this automatically for https , so ordinary code never names it. Reach for TlsSession directly when the protocol is not HTTP — a mail server, a message broker, a database wire protocol.

The one decision still open: whether TLS 1.2 joins it. Some server estates (and one SQL Server dialect) still require it. Until then a peer that refuses 1.3 is a peer this client cannot talk to, and it says so rather than falling back silently.

Standard Library — The System: machine, environment, files, processes, IPC

```
import System.OS.Machine; · import System.IO.Files; · import System.OS.Subprocess;
```

Everything a program needs to be a *program on a machine* rather than a computation: what hardware it is on, what the environment says, where its files are, how to start another process and talk to it, and how two Polaron programs speak to each other.

One rule runs through all of it, and it is worth stating once because it decides every signature below:

Absent is not empty. A variable that is not set, a file that is not there and a process that never started are each a different thing from a variable holding `""`, an empty file and a process that exited. Where the difference matters the answer is an `Option`, an exception, or a named boolean — never a value that could also be real data.

1. Machine — what you are running on

```
import System.OS.Machine;

int hands = Machine.threads();           // share the work between this many
if (Machine.isWindows()) { ... }
```

Member	Answers
<code>threads()</code>	Hardware threads. What a work crew is sized from.
<code>memory()</code> / <code>availableMemory()</code>	Total and free physical memory, in bytes.
<code>pageSize()</code>	The OS page size.
<code>uptime()</code>	Seconds since boot.
<code>os()</code> → <code>"windows"</code> / <code>"linux"</code> / <code>"macos"</code> ; <code>isWindows()</code> , <code>isLinux()</code> , <code>isMacOS()</code>	Which system.
<code>architecture()</code> , <code>is64Bit()</code>	Which processor.
<code>hostname()</code> , <code>user()</code> , <code>processId()</code>	Who and where.

2. Environment — variables, three ways on purpose

```
import System.OS.Environment;

Option<String> maybe = Environment.lookup("EDITOR");           // absent is a state
String editor = Environment.get("EDITOR", "vi");               // absent has a fallback
String key = Environment.require("API_KEY");                   // absent is an error
```

Member	When to reach for it
lookup(name) returns Option<String>	The variable may or may not be set, and both are ordinary.
get(name, fallback)	There is a sensible default.
require(name) throws(EnvironmentException)	The program cannot run without it — and the throw names it, so the message is not "something went wrong".
has(name), set(name, value), clear(name)	Testing and changing.
all() returns HashMap<String, String>	Everything, for a dump or a child process.
searchPath() returns ArrayList<String>, pathSeparator()	PATH, already split for the platform.

Three ways to read one variable looks like too many until the alternative is written out: one `get(name)` returning "" makes "unset" and "set to empty" the same answer, and every caller that cares has to remember which one it got.

3. Files

`Files` is the large one — the whole filesystem surface, as static methods over paths.

```
import System.IO.Files;

String text = Files.readOr("config.toml", "");
Files.write("out.txt", text);
for (var path in Files.walk("src").toArray()) { ... }
```

Reading and writing: `read` (throws `IOException` naming the path), `tryRead` (an `Option`), `readOr` (a fallback), `readLines`, `write`, `append`, `writeLines`, `appendLine`, `replace`. **Bytes:** `readInto(path, Buffer*)`, `writeBytes`, `appendBytes`. **Asking:** `exists`, `size`, `isFile`, `isDirectory`, `isReadOnly`, `modifiedAt`, `createdAt`, `accessedAt`, `isNewerThan`, `matches(pattern, path)` (glob). **Changing:** `touch`, `setReadOnly`, `rename`, `move`, `copy`, `delete`, `createDirectory`, `createDirectories`, `deleteDirectory`, `deleteRecursively`, `copyDirectory`. **Walking:** `listDir`, `walk`, `walkDirectories`, `directorySize`. **Links:** `isSymbolicLink`, `linkTarget`, `createSymbolicLink`, `createDirectoryLink`, `createHardLink`.

Streams, for what does not fit in a `String`

Type	What it is
<code>FileStream</code> — <code>open(path, mode)</code> , <code>read(path)</code> , <code>write(path)</code> ,	A handle: <code>readBytes(Buffer*, n)</code> , <code>writeBytes</code> , <code>seek</code> , <code>tell</code> , <code>flush</code> , <code>close</code> . For files bigger than memory, and for binary formats.

Type	What it is
<code>append(path)</code>	
<code>LineReader</code>	A <code>FileStream</code> as an <code>Iterator<String></code> — <code>foreach</code> over a file's lines without holding the file.
<code>TempFile</code> / <code>TempDirectory</code>	Made on construction, deleted by the destructor . <code>keep()</code> says "not this one" when a failure needs to be inspected.
<code>DirectoryWatcher</code> — <code>poll()</code> returns <code>DirectoryChanges</code>	What changed since the last poll: created, modified, removed. Polling rather than callbacks, because a callback from another thread is a rule about threads that a file API should not be imposing.

4. Workspace, disk, exit, signals

Type	Members
<code>Workspace</code>	<code>current()</code> , <code>moveTo(path)</code> , <code>homeDirectory()</code> , <code>tempDirectory()</code> , <code>executable()</code> , <code>executableDirectory()</code> . The last two are how a program finds its own assets — relative to the binary, not to wherever it was launched from.
<code>Disk</code>	<code>freeSpace(path)</code> , <code>totalSpace(path)</code> , <code>usedSpace(path)</code> .
<code>Exit</code>	<code>now(code)</code> , <code>ok()</code> , <code>failed()</code> . main returns void , so this is how a program refuses: a run that fails without an exit code is a failure no script can see.
<code>Signals</code>	<code>answer(sig, handler)</code> , <code>giveBack(sig)</code> , <code>sendToSelf(sig)</code> — Ctrl-C that shuts down cleanly rather than being killed mid-write.

5. Processes

```
import System.OS.Process;
import System.OS.Command;
import System.OS.Subprocess;

ProcessResult r = Process.run("git status --porcelain");
if (r.ok()) { System.IO.Console.println(r.output()); }
```

Type	What it is
<code>Process.run(commandLine)</code> returns <code>ProcessResult</code>	One-shot: run it, capture stdout to the end, get <code>exitCode()</code> , <code>output()</code> , <code>ok()</code> . Goes through the system shell, with everything that implies about quoting.
<code>Command</code>	The same, built from a program and its arguments rather than one string — no shell, so nothing in an argument can become syntax. Reach for this whenever any part of the command came from outside the program.
<code>Subprocess</code> — <code>start</code> , <code>startCombined</code> , <code>startVisible</code>	A child that stays open: <code>write</code> , <code>read</code> , <code>canRead</code> , <code>isAlive</code> , <code>closeInput</code> , <code>close</code> . For request/response protocols (a language server, a debugger adapter). <code>startCombined</code> merges stderr into stdout — right for a compiler, wrong for anything speaking a framed protocol.

Type	What it is
<code>Pty - start(command, cols, rows)</code>	A child on a real pseudo-terminal: <code>write</code> , <code>read</code> , <code>resize</code> , <code>isAlive</code> , <code>close</code> . What an integrated terminal needs — a shell behind a pipe does not draw.

6. Talking to another Polaron program

`import System.Ipc.*`; — cross-program calls, where the other program is another Polaron **program** rather than a library: it runs in its own process and exposes bundles.

```
ProgramHandle engine = ... ; // an open connection
BundleAccessToken* token = engine.requestAccess("audio.mixers");
if (token.granted()) { ... }
```

Type	What it is
<code>ProgramHandle</code>	A connection: <code>bundle(name)</code> , <code>namespace(name)</code> , <code>requestAccess(capability)</code> , <code>close()</code> .
<code>BundleAccessToken</code>	The answer to a request for access: <code>granted()</code> , <code>capability()</code> , <code>nonce()</code> . Access is asked for and granted, not assumed — a program exposing a bundle decides who may call it.
<code>RemoteType<T></code>	A type on the other side; <code>instantiate()</code> builds one there and hands back a proxy.
<code>IpcChannel - request(frame) throws(IpcError)</code>	The transport underneath.
<code>IpcWriter / IpcReader</code>	The frame format: <code>putInt / getInt</code> , <code>putString / getString</code> , and so on for every primitive.
<code>IpcProto</code> , <code>IpcRuntime</code>	The message kinds and the dispatcher on the serving side.
<code>IpcServer.serve(name, auth)</code>	The serving half: publishes this program under <code>name</code> and answers calls, with <code>auth</code> deciding per capability whether a caller may have it. A program that serves without an <code>auth</code> is a program that answers everybody.

Ordinary code uses `import from program ...` (see §2 of the guide) and never names these; they are here for anybody implementing a transport or debugging a frame.

Standard Library — What reflection gives you for free

```
import System.Data.Serializer; · import System.Data.Validator; · import System.Data.Services;
```

Four libraries that would be code generators, macros or annotation processors in another language, and are ordinary Polaron here because the language can **read its own types at run time**. Each is a few hundred lines, each works on any type, and none of them asks you to write a line of glue.

They are grouped on one page because they are one idea: *the type already says this; stop saying it again*.

1. `Serializer` — JSON from the fields themselves

```
import System.Data.Serializer;

[Serializable]
public class Order {
    public mutable int id;
    public mutable String customer;
    public mutable double total;
}

String text = Serializer.text<Order>(order);           // {"id":7,"customer":"Ana","total":19.5}
Json shape = Serializer.of<Order>(order);             // or the tree, to add to
```

Member	What it does
<code>Serializer.of<T>(T value)</code> returns <code>Json</code>	The value as a JSON tree, field by field, following nested objects and lists.
<code>Serializer.text<T>(T value)</code> returns <code>String</code>	The same, printed.
<code>[Serializable]</code>	Marks a type as intended for this. Not required to make it work — required to say you meant it.
<code>Schema.of<T>()</code> returns <code>String</code>	The shape rather than a value: the field names and their types, for a document, a contract test, or the other end of a wire.

No macro, no generated file, no build step. Add a field and it is serialised; rename one and the output follows; delete one and nothing anywhere still mentions it.

2. Validator — the rules live on the field

```
import System.Data.Validator;

public class Signup {
    [NotEmpty] [Length(min: 3, max: 32)] public mutable String user;
    [Email] public mutable String mail;
    [Range(min: 18, max: 120)] public mutable int age;
    [Positive] public mutable double credit;
    [Chars(allow: "0123456789-")] public mutable String phone;
}

Findings bad = Validator.check<Signup>(form);
if (bad.any()) { ... }
```

Annotation	Holds
[NotEmpty]	A string with something in it.
[Length(min, max)]	A string's length, inclusive.
[Range(min, max)]	A number's value, inclusive.
[Positive]	Greater than zero.
[Email]	Shaped like an address.
[Chars(allow)]	Every character drawn from a set.

`Validator.check<T>` walks the type's fields, reads their annotations and answers `Findings` — which names the field and the rule it broke, not a boolean. A form with three bad fields reports three findings, because "invalid" is not a message anybody can act on.

3. Services — dependency injection, no container configuration

```
import System.Data.Services;

public class Repo { ... }
public class Api {
    [Inject] public mutable Repo* repo;    // filled in by the container
}

Services* box = new Services() on heap;
box.provide<Repo>();
nullable Object made = box.build<Api>();
if (!box.complete()) { ... }           // something asked for what nobody provides
```

Member	What it does
<code>provide<T>()</code>	Registers a type the container can build.
<code>build<T>()</code> returns <code>nullable Object</code>	Builds one, filling every <code>[Inject]</code> field from what is registered.
<code>complete()</code> / <code>unwiredCount()</code>	Whether every injection point was satisfied — asked rather than assumed, because a half-wired object fails later and somewhere else.

4. `Compare` — equality and a diff, field by field

`Compare.equal<T>(a, b)` answers whether two values of any type match on every field, and `Compare.diff<T>(a, b)` says **which fields differ and what each holds** — the sentence a failing test wants instead of "expected X, got Y" over two objects nobody can read.

5. `Arena` — a container with a typed region

```
import System.Memory.Arena;

Arena* work = new Arena(64 kilobytes) on heap;    // accepts: what may live here
```

An arena is a region with the container's manners: many objects, one lifetime, one release. Its `accepts` clause is checked **on the field**, so a type the arena was not built for cannot be allocated into it — the region binder's rule, applied where a library holds the region rather than where a method declares one.

6. Persisted state — `Memo` and `EventLog`

Two `persistent` types: their contents survive a `reimport` of the code that uses them, which is what persistents are for.

Type	Members
<code>Memo</code>	A cache that outlives the code: <code>remember(key, value)</code> , <code>has</code> , <code>get</code> , <code>size</code> , <code>hits</code> , <code>misses</code> , <code>discard</code> . Reload a function and its memo table is still warm.
<code>EventLog</code>	An append-only log with a schema version: <code>append(kind, value)</code> , <code>size</code> , <code>kindAt</code> , <code>valueAt</code> , <code>version</code> , and <code>replay(combine, initial)</code> to fold it. Change the code that reads the events, keep the events.

7. Where the annotations themselves are documented

How to declare and read your own annotations, and the reflection API these are built on (`Type.of`, `fields()`, `get / set`, `annotations()`), is [§10 Compile-Time, Reflection & Universal Prefixes](#).

Standard Library — Numerical methods, statistics, and units

```
import System.Science.Roots; · import System.Science.Normal; · import
System.Memory.Units.Span;
```

Two things that look unrelated and are the same idea: **a number on its own is not a quantity**. The statistics half gives a number its distribution; the units half gives it its dimension. Both exist so that the meaning travels with the value instead of living in a variable name.

1. Units — a literal suffix that produces a type

```
import System.Memory.Units.Span;
import System.Memory.Units.Length;

Span timeout = 30 seconds;
Length far = 4 kilometres;
Speed pace = Speed.over(far, timeout);
```

Polaron's unit suffixes are `comptime` literal declarations: `30 seconds` is not an int, it is a `Span`, decided at compile time and costing nothing at run time. A method taking a `Span` cannot be handed a number of milliseconds by mistake, because a bare number is not a `Span` at all.

The suffix mechanism existed for **bytes** alone — `64 kilobytes` — and everything else a program measures went back to being a bare number, which is how a timeout in seconds reaches a parameter expecting milliseconds with nothing said.

Type	Suffixes	Reading it back
Span (time)	<code>nanoseconds</code> , <code>microseconds</code> , <code>milliseconds</code> , <code>seconds</code> , <code>minutes</code> , <code>hours</code> , <code>days</code>	<code>asNanoseconds()</code> , <code>asMicroseconds()</code> , <code>asMilliseconds()</code> , <code>asSeconds()</code> , ...
Length	<code>micrometres</code> , <code>millimetres</code> , <code>centimetres</code> , <code>metres</code> , <code>kilometres</code>	<code>asMillimetres()</code> , <code>asMetres()</code> , ...
Mass	<code>milligrams</code> , <code>grams</code> , <code>kilograms</code> , <code>tonnes</code>	<code>asGrams()</code> , <code>asKilograms()</code> , ...
Speed	built from the other two: <code>Speed.over(Length, Span)</code>	<code>asMetresPerSecond()</code> , <code>asKilometresPerHour()</code>
Turn	<code>degrees</code> , <code>radians</code> , <code>turns</code>	<code>asDegrees()</code> , <code>asRadians()</code>

The readers are named `asX()` and not `x()` deliberately: they **truncate**, and `asSeconds()` on 1500 milliseconds is 1. A method called `seconds()` would be read as "the seconds part", which is a different number.

One representation per dimension, always — a `Span` is nanoseconds, a `Length` is micrometres — so adding two of them is adding two integers, and no conversion happens where it could be lost.

2. Root finding

```
import System.Science.Roots;

double r = Roots.brent(command (double x) returns double { return x * x - 2.0; },
                        0.0, 2.0, 1e-12, 100);
```

Member	Method	When
Roots.bisect(f, lo, hi, tol, iters)	Bisection	Always converges when the bracket has a sign change. Slow and certain.
Roots.newton(f, df, x0, tol, iters)	Newton–Raphson	Fast when you have the derivative and a good start.
Roots.secant(f, a, b, tol, iters)	Secant	Newton's speed without the derivative.
Roots.brent(f, lo, hi, tol, iters)	Brent	The default. Bisection's guarantee with the secant's speed.

3. Calculus and differential equations

Member	What it does
Calculus.derivative(f, x) / secondDerivative(f, x)	Central differences, at a step chosen from the magnitude of <code>x</code> .
Calculus.integrate(f, a, b, n)	Definite integral (Simpson).
Ode.euler(f, t0, y0, t1, steps)	The simplest integrator; useful when the step is already tiny.
Ode.rungeKutta(f, t0, y0, t1, steps)	Fourth-order Runge–Kutta: the one to use.

4. Distributions

Each is `pdf` / `pmf`, `cdf`, and where it is meaningful `quantile`.

Type	Parameters
Normal	<code>pdf(x, mean, sd)</code> , <code>cdf</code> , <code>quantile(p, mean, sd)</code>
Exponential	<code>pdf(x, rate)</code> , <code>cdf</code> , <code>quantile</code>
Poisson	<code>pmf(k, mean)</code> , <code>cdf</code>
Binomial	<code>pmf(k, n, p)</code> , <code>cdf</code>
ChiSquare	<code>cdf(x, df)</code> , <code>pValue(x, df)</code>
StudentT	<code>cdf(t, df)</code> , <code>pValue(t, df)</code>

5. Hypothesis tests, optimisation, decomposition

Member	What it does
<code>Hypothesis.tTestOneSample(sample, expected)</code>	Returns the p-value.
<code>Hypothesis.chiSquareGoodness(observed, expected)</code>	Likewise.
<code>Optimize.minimize(f, lo, hi, tol, iters)</code>	Golden-section minimisation of a one-dimensional function.
<code>Optimize.descend(f, start, rate, steps)</code>	Gradient descent, where <code>f</code> answers the gradient.
<code>Decompose.lu(a, n, perm)</code>	LU decomposition in place; returns the determinant's sign-adjusted product.

6. Special functions

`Special.gamma`, `lgamma`, `beta`, `lbeta`, `erf`, `erfc`, `lowerGamma`, `incompleteBeta` — the pieces the distributions above are built from, exposed because anybody implementing another distribution needs exactly these.

7. Where the rest of the mathematics is

This page is the numerical-methods half. The algebraic half — `BigInteger`, `Rational`, `Complex`, `Matrix`, `Vec2` / `Vec3` / `Vec4`, `Fft`, `Statistics`, number theory and geometry — is in [Math, Net & Misc](#).